

Computational Cognitive Models of Adaptive Planning

by

Hao Chuan Lei

A dissertation submitted in partial fulfillment

of the requirements for the degree of

Doctor of Philosophy

Center for Neural Science

New York University

August 2026

Wei Ji Ma

© Hao Chuan Lei

All rights reserved, 2026

Thus conscience doth make cowards of us all,
And thus the native hue of resolution
Is sicklied o'er with the pale cast of thought,
And enterprises of great pith and moment
With this regard their currents turn awry
And lose the name of action.

—William Shakespeare, *Hamlet* (Act III, Scene 1)

Dedication

To Mom, Dad, and Matthew

Acknowledgments

Completing a thesis is a labor of love and, more often than not, a source of self-inflicted frustration. To that end, I have many people to thank for putting up with my antics over the course of five years of doctoral research.

First, to my advisor, Wei Ji Ma. In many ways, being a mentor is an unsung and underappreciated role, one that receives little praise in success and much criticism in failure. To the extent that my work can be considered a success, I owe that to your patience, support, and guidance. Through your example and mentorship, I have come to learn not only what it means to do good science, but also what it means to be a good scientist. Your dedication to mentorship in the lab and advocacy in the sciences underscores the deeply human part of science. Thank you for being an unwavering source of stability throughout my research journey and extending your patience to my more short-tempered moments.

Next, to my collaborators, without whom this research would not be possible. Many thanks to Daeyeol Lee for welcoming me into the world of monkey physiology and onto the Four-in-a-Row project, which has since become a cornerstone of my thesis. A special thanks to Min-Yoon Park and Eunju Shin for entrusting me with their data, which was painstakingly collected over the course of several years. The project described here is the result of an incredible team of researchers, including Norman Lam, Daniel Mittler, Rey Elise Lautenschlager, Kristian Osborne, Hexin Liang, Marianne Oemisch, Bas van Opheusden, and Veit Stuphorn. Many thanks also to Cate Hartley, Nora Harhen, and Olive Duan for opening my eyes to a whole new world of developmental psychology that I had never known before. I would also like to thank Maya Nandy for being a steadfast champion of the Treasure Hunt learning in stochasticity task. To Marcelo Mattar and Yotam Sagiv, it has been a pleasure to work with you on the complex domain of metacontrol in chess; to think that we would think so much about thinking about thinking ...

Additional thanks to the other members of the chess project, Tom Griffiths, Ionatan Kuperwajs, and Evan Russek. Thank you all for your help in turning mere ideas into full-fledged inquiry.

To my labmates, who have become integral to the texture of my life and work. In particular, Jeroen Olieslagers and Nastaran Arfaei – I couldn’t ask for a better cohort of co-conspirators. I will sorely miss our lunchtime chats about everything from eco-brutalism to Jeroen’s latest board game obsession. I would also like to acknowledge fellow lab members Yotam Sagiv, Olive Duan, Maya Nandy, Daisy Lin, Ionatan Kuperwajs, Peiyuan Zhang, Xiang Li, James Jin, Deanna Garcia, Sixuan Chen, Shucheng Li, Jieyu Li, Hsin-Hung Lee, Dongjae Kim, and Heiko Schutt. I hold that science is a deeply human endeavor, and in that regard I owe much of this work to your generosity and kindness.

To my mentors present, including those who constitute my thesis committee, Cristina Savin, Todd Gureckis, and Marcelo Mattar, for their invaluable feedback over the course of my doctoral research. Your enthusiasm and thoughtful comments have long steered the direction of the work here today. To my mentors past, Konrad Kording, Joshua Gold, Vijay Balasubramanian, and Adam Grant, for believing in me and seeing what I myself could not as a naive undergraduate, and for your continued support, guidance, and advice.

To my friends. As much as I would like to thank you all individually, I risk my acknowledgement section becoming longer than my thesis itself, which I think we can all agree would be rather embarrassing. I hope you will forgive me for my brevity. To my cohort, first and foremost, for being a constant source of joy and counsel over these years; I will always think fondly about our shared time together, from the first day we met at the strangely cultish obelisk in Central Park to our nearly three-year-long D&D campaign. To my friends in the psychology department, for accepting me as one of your own and putting up with my yapping. I hope my myriad interjections with lab small talk have not added more than a couple months to your defense timelines. To the members of the “Stooper Troupers” for being a thread of continuity that has kept me grounded since my undergraduate days; you have all seen me grow, if not wiser, at least older. To the affectionately-named “Dumpling Crew” for bringing much-needed perspective from an

adjacent field and our shared love of socializing over food. To my fellow eleventh-hour hobbyists – particularly those in running and pottery, which I picked up in my five years here. And to all who have made this city feel like home.

I would like to end by thanking my family. To Mom, Dad, and Matthew, for your unwavering belief and love. Matthew, it has been a great source of joy to see you grow into the kind, conscientious, and hilarious person you have become over the past five years, and I cannot wait to see where the next five take you. There is no greater source of pride for an older brother than to see you succeed, achieve your dreams, and carve your own path. Dad, I hope that someday I can learn to find that same balance between humor and wisdom that you carry with you so effortlessly. I hope you know how much I look up to you, as an example of what it means to live a life of grace, poise, and joy. Mom, you have been my role model for as long as I can remember. It is in your footsteps that I am here today, as the second member of our family to pursue a doctorate in a neuroscience-related field. I cannot imagine how you did it with two little monkeys to feed.

Thank you, I love you.

Abstract

Planning is remarkably useful; it allows us to use predictions about the future to guide present decisions. However, real-world planning is difficult. The future is uncertain, and planning itself requires time and cognitive resources. Adaptive planning therefore requires knowing both how to plan and whether planning is worthwhile in a given context. In this thesis, we develop computational models of how people plan and adapt to uncertain environments. First, we examine how stochasticity influences the effort people allocate toward planning. Across three forms of stochasticity, we find that people adapt by exerting less planning effort and becoming less sensitive to differences in future values. Next, we investigate a different source of uncertainty in the form of an adversarial opponent, and examine how planning is implemented in the brain. We build computational models of a rhesus macaque playing a two-player strategy game called Four-in-a-Row and find early evidence that the monkey engages in deeper-than-myopic search. We further find that search-related computations are encoded by neurons in DMPFC, DLPFC, and caudate. Finally, we take a normative approach to adversarial planning at scale by analyzing human gameplay in a massive online dataset of chess games. We develop a normative model that learns when and how deeply to search in a given position, demonstrating as a proof of principle that the halt-or-continue decision is learnable from tree structure alone. Together, these studies provide behavioral, neural, and normative accounts of planning adaptation, showing that planning is not a fixed computational process but a flexible strategy shaped by environmental uncertainty, cognitive constraints, and task structure.

Contents

Dedication	iv
Acknowledgments	v
Abstract	viii
List of Figures	xii
List of Tables	xv
List of Appendices	xvi
1 Introduction	1
1.1 Planning as Mental Computation	2
1.1.1 Computational Reasoning	3
1.1.2 Towards the Computational Mind	5
1.1.3 Reinforcement Learning	7
1.1.4 Model-Free and Model-Based Decision Making	8
1.1.5 Planning, Search, and Approximation	9
1.1.6 The Neural Basis of RL and Planning	15
1.2 Adaptive Planning	20
1.2.1 Rational Behavior, Heuristics, and Biases	20
1.2.2 Bounded, Ecological, and Resource Rationality	22
1.2.3 Formalizing Resource Rationality	24
1.2.4 Resource Rational Planning	26

1.3	Overview	27
2	Human Planning in Stochastic Environments	30
2.1	Introduction	30
2.2	Results	33
2.2.1	Unreliability Decreases Planning Effort	33
2.2.2	Volatility and Uncontrollability Decrease Planning Effort	36
2.2.3	Computational Models of Planning Effort	37
2.3	Discussion	43
2.4	Methods	47
2.4.1	Experiment	47
2.4.2	Mixed-Effects Models	50
2.4.3	Computational Models of Planning Effort	54
2.4.4	Model Fitting	57
3	Planning in a Strategy Game in Monkeys	60
3.1	Introduction	60
3.2	Results	63
3.2.1	Task	63
3.2.2	Gameplay	64
3.2.3	Gaze	68
3.2.4	Neural Activity	77
3.3	Discussion	83
3.4	Methods	88
3.4.1	Trial Structure	88
3.4.2	Behavioral Modeling	90
3.4.3	Eye Tracking and Gaze Analysis	93
3.4.4	Neural Analysis	99

4	Learned Metacontrol of Planning at Scale	104
4.1	Introduction	104
4.2	Results	109
4.2.1	Model Agnostic Analysis of Response Time	109
4.2.2	A Learned Metacontroller for Search	115
4.3	Discussion	122
4.4	Methods	128
4.4.1	Data	128
4.4.2	Planner	129
4.4.3	Metacontroller	130
4.4.4	Training	133
5	Conclusion	141
5.1	Summary	141
5.2	Limitations and Future Work	143
5.2.1	Modeling Space and Tractability	143
5.2.2	State Space and Representations	146
5.2.3	Utility and Task Alignment	147
5.2.4	Extensions and Generalization	148
5.2.5	Towards a Broader Metapanning Framework	153
5.2.6	Affordances and Learning Action Spaces	154
5.3	Closing Remarks	156
	Appendices	160
	References	235

List of Figures

1.1	Reinforcement learning diagram	7
1.2	Decision to plan or halt.	26
2.1	Task design	33
2.2	Participant behavior	35
2.3	Model illustrations	38
2.4	Model comparison	41
2.5	PC Depth-filtering Level-Mean model results	42
3.1	Four-in-a-Row game	63
3.2	Model comparison	65
3.3	Model simulations overlaid with data	69
3.4	Gaze time course	71
3.5	Gaze aligned to move	72
3.6	Zero-shot gaze NLL	75
3.7	Gaze regression	76
3.8	Gaze regression with controls	76
3.9	Recording sites and spike-triggered averages	78
3.10	Population-level coding of state value	79
3.11	Population-level coding of Tree Search beyond Myopic	80
3.12	Neural regression model comparison	82
3.13	Trial structure	89
3.14	Tree Search and heuristic value function	90

4.1	Player response times	110
4.2	Clock time vs response time	111
4.3	Player material vs response time	112
4.4	Material imbalance vs response time	113
4.5	Number of legal moves vs response time	114
4.6	Number of captures available vs response time	115
4.7	Metacontroller architecture	116
4.8	Metacontroller training	118
4.9	Regret compared to baselines	120
4.10	Regret scatterplot	121
4.11	Paired regret differences.	122
4.12	Encoder Architecture	131
4.13	Training Procedure	134
4.14	DP Halt/Continue Oracle	136
5.1	Metaplanning diagram	152
A.1	Screenshots of participant instructions and stimuli	161
A.2	Resource rationality simulated results	176
A.3	Total response time results	178
A.4	PC Depth-filtering Path model results	179
A.5	PC Depth-filtering Max model results	181
A.6	PC Depth-filtering Sum model results	182
A.7	PC Depth-filtering EV model checking	183
A.8	Model recovery for the PC Depth-filtering EV model	186
A.9	Model recovery for the PC Depth-filtering Level-Mean model	187
A.10	Model recovery for the PC Depth-filtering Max model	188
A.11	Model recovery for the PC Depth-filtering Path model	189

A.12	Model recovery for the PC Depth-filtering Sum model	190
A.13	Parameter recovery for the PC Depth-filtering Level-Mean model	192
A.14	Parameter recovery for the PC Depth-filtering Path model	193
A.15	Parameter recovery for the PC Depth-filtering Max model	194
A.16	Parameter recovery for the PC Depth-filtering Sum model	195
B.1	Parameter recovery	213
B.2	Zero-shot gaze KL divergence	216
B.3	Raw gaze regression R^2	217
B.4	Time-resolved mean ΔR^2	218
B.5	Time-resolved proportion preferring Tree Search	219
C.1	Legal moves vs. response time, excluding in-check positions	224

List of Tables

A.1	Mixed-effects model convergence diagnostics	184
B.1	M1 training protocol	198
B.2	Computer opponent heuristic parameters	199
B.3	Known issues in behavioral data	200
B.4	Monkey Four-in-a-Row dataset summary	202
B.5	Monkey Tree Search model parameters	208
C.1	Chess dataset summary	222
C.2	Metacontroller training-data funnel	231
C.3	Baseline controller decision rules	234

List of Appendices

A	Human Planning in Stochastic Environments	160
A.1	Experiment Design	160
A.1.1	General Task Instructions	160
A.1.2	Manipulation-Specific Instructions and Checks	163
A.1.3	Attention Checks	168
A.2	Algorithms	168
A.2.1	Filter Functions	169
A.2.2	Value Functions	170
A.2.3	Computing Choice Probabilities	173
A.3	Resource-Rational Simulations	173
A.3.1	Estimating Rewards Under Effort	173
A.3.2	Cost of Planning	174
A.3.3	Simulation Results	175
A.3.4	Discussion	177
A.4	Response Times	177
A.5	Model Comparison	178
A.5.1	PC Depth-Filtering Path Model	178
A.5.2	PC Depth-Filtering Max Model	180
A.5.3	PC Depth-Filtering Sum Model	180
A.5.4	PC Depth-Filtering EV Model	180
A.5.5	G/LMM Convergence	184
A.6	Model and parameter recovery	185

A.6.1	Simulation procedure	185
A.6.2	Model recovery	185
A.6.3	Parameter recovery	191
A.7	Model Heterogeneity	191
A.7.1	Robustness of Heterogeneity Analysis	196
A.7.2	Heterogeneity Results	196
B	Planning in a Strategy Game in Monkeys	198
B.1	Data	198
B.1.1	Training Protocol	198
B.1.2	Computer Opponent	199
B.1.3	List of Known Issues in Behavioral Data	200
B.2	Gameplay	201
B.2.1	Behavioral Preprocessing	201
B.2.2	Board and Value Functions	203
B.2.3	Tree Search Algorithm	205
B.2.4	Negative Log-Likelihood Estimation with Inverse Binomial Sampling	207
B.2.5	Model Fitting	208
B.2.6	Model Variants	209
B.2.7	Changes from Van Opheusden et al. (2023)	209
B.2.8	Parameter Recovery and Identifiability	210
B.2.9	The Four-in-a-Row Weight	212
B.3	Gaze	214
B.3.1	Gaze Preprocessing	214
B.3.2	Trial-Wise Validation via KL Divergence	215
B.3.3	Gaze Regression R^2	216
B.4	Neural	218
B.4.1	Neural Preprocessing	218

B.4.2	Time-Resolved Confirmation of the Paired Model Comparison	219
C	Learned Metacontrol of Planning at Scale	221
C.1	Data	221
C.2	Empirical Analyses	222
C.2.1	Board and Response-Time Variables	223
C.2.2	Legal Moves: In-Check Positions	224
C.3	Metacontroller	225
C.3.1	The Planner	225
C.3.2	Tree Encoder	227
C.3.3	Halt/Continue Values	228
C.3.4	Training	230
C.3.5	Baseline Controllers	233

1 | Introduction

“There is nothing either good or bad, but thinking makes it so.”

—William Shakespeare, *Hamlet* (Act II, Scene 2)

In William Shakespeare’s *Hamlet*, the titular prince of Denmark is tasked with avenging his father’s murder by his uncle, King Claudius. To avenge his father’s death, Hamlet concocts a plan with several components: feigning madness, staging a play to expose his uncle’s guilt, and finally confronting his uncle face to face. In principle, the structure of the plan is coherent. In practice, however, the execution is anything but. His feigned madness alienates those closest to him, the staged play deepens his isolation, and when finally presented with an opportunity to act, he hesitates. Throughout the play, Hamlet delays, revises, and second-guesses, failing to act when action is most required. His paralysis stems not from a lack of planning ability, but from an excess of deliberation.

What makes *Hamlet* enduring is that it captures a deeply human truth: planning in the real world is difficult. Effective decision making in a dynamic world involves both knowing *how* to plan and *when* to stop planning. The future is inherently uncertain, and planning – mentally simulating possible actions and their consequences – is a critical tool for navigating that uncertainty. Even in fairly straightforward settings, the number of actions and associated outcomes quickly balloons exponentially as decisions unfold over time. When combined with the complexity of the real world, effective planning demands that we adapt to changing environments, build abstract representations of the world, and make decisions in unfamiliar contexts. From this perspective, Hamlet’s predicament is unsurprising: exhaustive planning quickly becomes intractable. Yet somehow, people routinely manage to plan and act effectively, even in the face of incredible uncertainty.

In the real world, planning doesn't come for free. Simulating the future requires time, cognitive effort, and attention, all of which are limited and often consequential. In time-sensitive situations, excessive deliberation can delay or even preclude action altogether. This extends beyond Hamlet's indecision to everyday domains as well: a physician weighing a patient's treatment options, or a candidate deciding between job offers. In such cases, the cost of deliberation must be balanced against its potential benefits. In Hamlet's case, failing to do so proves fatal. Effective planning therefore requires not only strategies for generating and evaluating possible futures, but also mechanisms for regulating when and how much to deliberate. The control of deliberation, deciding when to rely on fast heuristics versus explicit simulation, is itself a core component of adaptive decision-making.

Stepping back from *Hamlet*, we are left with a set of broader questions about how people contend with a messy world. How do people (or agents more generally, biological or artificial) adapt their planning to dynamic environments? And how do they determine when planning is worth its cost? These questions arise at a time when multiple fields, including computer science, cognitive science, and neuroscience, are converging on a shared science of intelligence and flexible reasoning. Addressing them requires integrating perspectives across disciplines and across different kinds of agents, from humans and nonhuman animals to artificial systems. This thesis contributes to that effort.

1.1 Planning as Mental Computation

On the surface, the link between reasoning and computation may seem somewhat abstract. As we will see, however, the processes underlying decision making can be formalized through computational frameworks that describe how agents evaluate and act in different contexts. From this understanding, planning emerges as a computational process of selecting sequences of actions based on anticipated outcomes.

1.1.1 Computational Reasoning

The idea that cognitive processes can be carried out as computation predates modern computers by centuries. At its core lies a simple but powerful insight: the external world can be represented using discrete symbols, and the manipulation of these symbols can generate new knowledge. With the right symbols and the right rules for manipulating them, it is possible to make increasingly complex inferences about the world.

This idea traces its roots to antiquity across diverse cultural traditions. Systems such as the *I Ching* (Wilhelm, 2011) (Western Zhou dynasty, c. 1000–750 BCE), West African Ifá (Bascom, 1991) (codified c. 11th century CE, with much earlier oral origins), Pāṇini’s *Aṣṭādhyāyī* (Pāṇini, 1980; Kadvany, 2016) (c. 5th–4th century BCE), and Ramon Llull’s *Ars Magna* (Llull, 1305, 1984) (1305 CE) each developed formal symbolic frameworks capable of generating a wide range of structured outcomes from a finite set of primitives through combinatorial rules. Though differing in purpose (divination, linguistic analysis, and philosophical inquiry), these systems share a common underlying principle: they all use discrete, manipulable structures as the substrate for complex inference about the world at large. Notably, several of these systems use compact representations, mapping the world to a set of symbols, using rules to generate ever more complex descriptions of the world. These traditions anticipated a central insight that would later be formalized in philosophy: that reasoning itself might be reducible to symbol manipulation.

This view is perhaps best summarized by Thomas Hobbes’ assertion in *Leviathan* that reasoning is a form of “reckoning”: the systematic combination of consequences according to formal rules (Hobbes, 1651). In the 17th century, Gottfried Wilhelm Leibniz formalized this philosophical intuition into a precise mathematical framework. His foundational work in binary calculus represented truth and falsehood as discrete values and proposed that logical reasoning itself could be reduced to symbolic manipulation (Leibniz, 1679, 1703). Leibniz’s ambition extended further still: he envisioned a *characteristica universalis*, a universal formal language in which all human thought could be decomposed into primitive elements and recombined according to explicit rules,

such that reasoning would become, in his words, a matter of calculation. When the Jesuit missionary Joachim Bouvet sent Leibniz a diagram of the *I Ching* hexagrams in 1703, Leibniz saw the hexagrams as a striking independent confirmation of this idea: he took it as evidence that the principle of deriving complex structure from binary primitives was a universal feature of how minds could represent and reason about the world (Aiton and Shimao, 1981; Perkins, 2004; Maitre, 2022).

The mechanization of reasoning took a step forward in the 19th century with the work of Ada Lovelace, who recognized that Charles Babbage’s Analytical Engine (proposed 1837) could operate over any symbolic system, not merely numbers (Lovelace, 1843). This insight was later formalized in the 20th century by Alan Turing, whose 1936 theory of computation demonstrated that symbolic manipulation could be realized as an explicit procedure executed by a machine (Turing, 1936).

This opened up a whole new interpretation of reasoning at large. Once reasoning is cast as a sequence of symbolic operations, it becomes natural to see it as a form of computation rather than separate from it. Decision making itself can thus be framed as a generative process of repeatedly applying rules to simulate actions and evaluate their outcomes. Early *look-ahead* algorithms, developed in mid-20th century work on game-playing, formalized this idea by representing possible futures as branching trees (Shannon, 1950), transforming reasoning from static inference into a dynamic process of planning over time.

An important step in understanding how these look-ahead algorithms would work in practice was to characterize how they would behave under optimal and relatively simple conditions. A key insight came in the form of the Bellman equation, which expresses how an optimal agent would choose among different actions to maximize its expected reward (Bellman, 1957). A formal definition of the Bellman equation will follow when we introduce the concept of reinforcement learning.

1.1.2 Towards the Computational Mind

Early developments in computer science suggested that complex reasoning could emerge from the manipulation of symbolic representations. However, whether the mind itself operated according to similar principles remained unclear.

Early attempts to study learning and decision-making focused primarily on observable behavior rather than internal mental structure. Through studies of conditioning, such as Edward Thorndike’s early “puzzle box” experiments, cats were able to learn sequences of actions to open a door and obtain a food reward, demonstrating that behavior could be systematically shaped through reinforcement and association (Thorndike, 1898, 1911). Similarly, in Ivan Pavlov’s classical conditioning experiments, dogs learned to associate certain sounds with the presentation of food¹, eventually salivating in response to presented tones (Pavlov, 1906). Later, B. F. Skinner showed that animals could acquire complex action patterns through operant conditioning, where behaviors were reinforced or suppressed depending on their consequences (Skinner, 1963). These findings were remarkable because they suggested that sophisticated behavior could emerge without requiring explicit reasoning about hidden mental states. Rather than appealing to internal symbolic representations or hidden mental processes, so-called *behaviorism* instead treated the mind largely as a black box whose outputs could be understood through the lens of stimulus-response associations. Under this view, reward signals were powerful tools for shaping behavior over time.

Assertions that the mind might be representing a symbolic or condensed representation of the world began as early as the 1940s. In 1943 Kenneth Craik suggested that the mind might be capable of translating external processes into internal models, manipulating those models through internal processes, and translating the results into actions (Craik, 1967)². In the late

¹The common depiction of Pavlov conditioning dogs with a “bell” is likely apocryphal. In the cited work, Pavlov himself notes that a loudly ringing bell would be a poor experimental stimulus.

²Craik’s theory of cognition has proven to be surprisingly prescient. Unfortunately, his work remains poorly cited in part because of his untimely death at the age of 31. Warren McCulloch cites it as one of the sources of cybernetics, noting that he read it five times (McCulloch et al., 1974). Craik’s model is cited in the proposal for the influential Dartmouth Summer Research Project on AI (McCarthy et al., 2006).

1940s Tolman provided some of the earliest empirical evidence of this by showing that rodents could learn internal or “latent” representations of the world without requiring an explicit reward. When rats explored a maze without any food rewards, strict behaviorism would predict minimal learning. But Tolman observed that when a reward was introduced later, these same rats were just as able to find the reward as those who had been rewarded all along (Tolman, 1948). Additional challenges to strictly behaviorist accounts emerged in studies of language and cognition. Noam Chomsky and George Miller argued that language could not be explained purely as a cached history of linear word associations, but that it instead relied on hierarchical recursive structure³ (Chomsky, 1959; Miller and Chomsky, 1963).

This opened the door for Allen Newell and Herbert Simon to set the foundation for framing the mind as a symbolic reasoning machine. The introduction of the Logic Theorist demonstrated that a physical machine could prove mathematical theorems, a capability once thought to be uniquely human (Newell and Simon, 1956). In the years that followed, they introduced the General Problem Solver (GPS), a general-purpose model intended to mimic human problem solving across a wide variety of domains such as chess and the Towers of Hanoi (Newell et al., 1959). The GPS framed problem solving as an explicit search over a problem space, with the aim of minimizing the distance between the current state and the goal state. To make this tractable, Newell and Simon used *heuristics*, or mental shortcuts, to prevent the tree of possible paths through the search space from growing indefinitely. The idea of reasoning as bounded heuristic search is still alive and well today. They formalized these ideas in their Physical Symbol System Hypothesis (PSSH), which posited that symbolic computations were both necessary and sufficient for general intelligence, providing an explicit alternative to strictly behaviorist accounts of cognition (Newell and Simon, 1976).

While behaviorism and symbolic computation are sometimes presented as contrasting hypotheses for the mechanisms behind reasoning, they are not mutually exclusive. Modern attempts to understand reasoning rely on aspects of both: using structured symbolic representations to un-

³Chomsky noted parallels between his ideas in formal grammar with the formal rule-based Sanskrit grammar of Pāṇini, cited in the earlier section on computational reasoning (Chomsky, 1969).

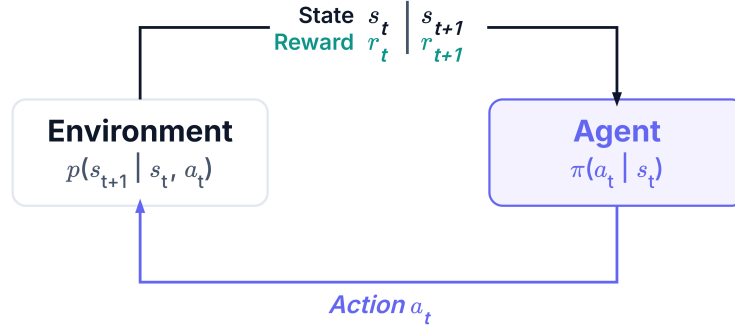


Figure 1.1: **Reinforcement learning diagram.** An agent interacts with an environment over a sequence of steps. At each step, the agent receives a state s_t , selects an action a_t , and receives a reward r_{t+1} as the environment transitions to a new state s_{t+1} . The agent’s goal is to learn a policy π that maximizes expected cumulative reward over time. Figure adapted from Sutton and Barto (1998).

derstand how the mind makes sense of the world and combining this with reward signals that inform and shape behavior relative to those internal representations. It is in this context that reinforcement learning emerges as an influential framework for understanding structured computation in planning and cognition.

1.1.3 Reinforcement Learning

Reinforcement learning (RL) provides a powerful framework for understanding decision making. At a high level, RL describes how an agent (be it an animal, human, or machine) interacts with an environment in order to maximize expected cumulative reward (Sutton and Barto, 1998). In the canonical formulation, this interaction unfolds over episodes. At each step, the environment presents the agent with a state, which encodes the relevant features of the world. The agent selects an action based on this state, after which the environment transitions to a new state and provides a scalar reward. This loop continues until a terminal state is reached, at which point the episode ends (Fig. 1.1).

A key component of reinforcement learning is the Bellman equation, which expresses the value of a state recursively in terms of immediate reward and the value of successor states, under optimal decision making. Formally, the optimal value function $V^*(s)$ for a state $s \in S$ can be

written as:

$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s' \in S} P(s' | s, a) V^*(s') \right] \quad (1.1)$$

where γ is a discount factor over future rewards and $P(s' | s, a)$ denotes the probability of transitioning to successor state s' after taking action a in state s (Bellman, 1957; Sutton and Barto, 1998).

The Bellman equation serves as a kind of normative goalpost for reinforcement learning: it tells us what optimal behavior would look like if the agent had complete knowledge of the environment. The equation emphasizes the recursive nature of optimal decision making: to evaluate the long-term consequences of an action in this state, the agent must estimate the value of successor states, which themselves depend on future actions and outcomes. Solving this equation exactly would require either full knowledge of the environment dynamics or exhaustive evaluation of future trajectories, along with the computational capacity to carry out this evaluation. In practice, these conditions are rarely met, which makes approximation necessary.

Reinforcement learning methods can therefore be understood as different strategies for approximating solutions to the Bellman equation. This might involve learning approximate value functions from cached experience, as in the case of so-called *model-free* learning, or constructing simplified models of state dynamics to guide evaluation, in the case of *model-based* learning. Across these approaches, the reward signal plays a central role in shaping behavior, driving updates to the agent's internal representations and ultimately defining a policy that maps states to actions.

1.1.4 Model-Free and Model-Based Decision Making

Both model-free and model-based decision making can be understood as different ways of approximating value through experience and internal updating in response to reward. More intuitively, they reflect different strategies for encoding past experience and using it to guide future

decisions.

Model-free learning, at its core, is about learning from repeated experience and caching the results. In its simplest form, this corresponds to maintaining a lookup table over state-action pairs. When an agent encounters a state and selects an action, it records the outcome as an entry in this table. In this sense, the agent behaves like a highly optimized filing cabinet of past experiences. When the same state is encountered again, it simply retrieves the action that is associated with the best outcome. This makes model-free decision making computationally cheap at test time, since it reduces choice to lookup and comparison. However, it is often memory intensive. As the dimensionality of the state space grows, the number of possible state-action pairs grows rapidly, a classic example of the curse of dimensionality. Without strong assumptions about how states relate to one another, this also hampers generalization: small changes in the environment can produce states that appear entirely novel.

By contrast, model-based learning involves constructing an internal representation of the environment's dynamics. A model-based agent uses this representation to simulate the consequences of different actions and evaluates them before acting. In effect, it approximates the Bellman equation through on-the-fly evaluation of possible future trajectories. This enables more flexible behavior, since the agent can reason about novel situations without having directly experienced them. However, this flexibility comes at a cost. Building and maintaining an accurate model is itself difficult, and using it requires computation at decision time. Additionally, simulation is slow, and in complex environments, evaluating all relevant futures quickly becomes intractable. Planning is often formalized in this context as search over a branching tree of possible actions and outcomes.

1.1.5 Planning, Search, and Approximation

From an algorithmic perspective, the study of planning has been deeply intertwined with graph theory, particularly with search algorithms. A graph $G = (V, E)$ is a set of nodes (vertices) $v \in V$ joined by edges $(u, v) \in E$. A search task asks for a path, edge, or node satisfying some criterion,

for example the shortest route between two cities or finding the highest-rated restaurant within a certain radius.

The earliest of these algorithms enumerate the graph exhaustively. *Depth-first search* (DFS) explores a graph like trailing a ball of twine through a maze: unspool down one corridor until you reach a dead end or a node already seen, then rewind to the last junction and try another branch, until every node has been visited (Even, 2011; Sedgewick, 2001). *Breadth-first search* (BFS) inverts the order of exploration, fanning outward from the start in rings of increasing distance, like ripples in a pond (Moore, 1959; Lee, 1961). From an implementation perspective, the two differ only in whether newly discovered nodes are expanded last-in-first-out (a stack, yielding DFS) or first-in-first-out (a queue, yielding BFS). Both forms of traversal are complete, guaranteed to find a satisfying node if one exists, but in the worst case completeness requires visiting every reachable node, with traversal complexity $O(|V| + |E|)$.

A related family of problems involves finding *shortest paths* on graphs with weighted edges, where each edge carries a cost $w(u, v)$ and the goal is the path of least cumulative weight between two nodes. Here exhaustive enumeration is wasteful: there is no use expanding a node that is any worse than the currently known cheapest route. *Dijkstra's algorithm* exploits this by repeatedly expanding the closest unsettled node and *relaxing* its outgoing edges (tightening a neighbor's tentative distance whenever a cheaper path is found), which returns optimal paths on graphs with non-negative weights (Dijkstra, 1959). The *Bellman-Ford* algorithm operates similarly, but trades efficiency for generality: relaxing every edge $|V| - 1$ times accommodates negative edge weights (Bellman, 1958; Ford Jr, 1956). Any subsequent pass that still finds an improvement reveals a negative cycle in the graph, along which “shortest” is undefined.

Both Dijkstra and Bellman-Ford are instances of a more general principle that begins to separate so-called *exhaustive* search algorithms from what we might today consider more adaptive search. Each computes a node's distance by combining one edge's cost with a neighbor's already-computed distance, a small recursive step that presumes the neighbor's value is itself optimal. Bellman (1957) named this the *principle of optimality*: an optimal solution is built from optimal

solutions to its subproblems, so a hard problem can be solved by caching and reusing the answers to smaller subproblems. This is the key to *dynamic programming*, which uses recursion to cleverly solve problems that obey this property of optimal substructure. More broadly, when the “problem” is to act well across a sequence of states (with rewards and transitions to further downstream states), the principle yields the *Bellman equation* we saw earlier, and represents the seeds of what would later become the field of reinforcement learning.

These algorithms are well-behaved when the number of possible states and edges remains limited. But for graphs of larger scale, this quickly becomes intractable. In these cases, we must rely on shortcuts, or heuristics, to narrow down the possibilities. One approach, known as A^* , takes into account knowledge imparted by heuristics and integrates it in a guided search, using $g(n) + h(n)$ to order the visitation of nodes, where $g(n)$ is the distance so far from start state u to n and $h(n)$ is a scalar heuristic evaluation over the distance from n to the target (Hart et al., 1968). As long as h never overestimates the true remaining cost, A^* is proven to find the minimum path, and often much faster than without a heuristic (Dijkstra’s algorithm is essentially A^* where the heuristic is always the scalar 0). More generally, these fall under what are known as best-first search algorithms (BeFS)⁴, a class of algorithms that expands nodes in order of some evaluation function f (Pearl, 1983), where in the A^* case $f(n) = g(n) + h(n)$.

Another option to lower the cost of search is by focusing only on promising nodes while leaving unpromising ones behind, which sometimes requires sacrificing optimality to save on computation. Beam search is a variant of this: rather than incurring the cost of visiting all neighbors, it keeps only the top k (the *beam-width*) candidates and prunes the rest at each step, ensuring that the frontier is always width k wherever the tree grows (Lowerre, 1976). Note that this is not guaranteed to converge on the optimal result; sometimes pruning too aggressively comes at the cost of overlooking the best possible option.

A different kind of pruning can reduce runtime considerably without losing the guarantee of optimality, but only in limited contexts. So far, the tasks we have discussed involve a single

⁴As if it were not confusing enough, sometimes best-first search is abbreviated BFS; here we will stick to the BeFS notation to disambiguate the two.

agent navigating down a path of their own choosing, essentially *solitaire* in graph search form. Things get much more interesting when the environment contains other agents that may act adversarially; von Neumann (1928) showed that two-player zero-sum games settle into a stable optimal value, known as the minimax value, which results from players minimizing the objective their opponent is trying to maximize. Shannon generalized this idea to chess, representing the game as an alternation between a player’s and an opponent’s choices, where the opponent’s goal is to minimize the player’s value (Shannon, 1988, 1950). These cases allow for *alpha-beta pruning* (Knuth and Moore, 1975), where a branch is pruned once it has been proven it cannot improve on an alternative the minimax search has already guaranteed. Indeed, the use of smart heuristics combined with extensive pruning can be incredibly effective; this was exactly the strategy used by Deep Blue (Campbell et al., 2002) to defeat Garry Kasparov in a game of chess in 1996.

Even with clever pruning and heuristics, tree search can be quite expensive. Deep Blue relied on heavy pruning and carefully hand-crafted heuristics to achieve competitive performance. In games like chess there is often a combinatorial explosion of states, making it expensive to guide search exhaustively; the number of legal positions in chess has been estimated to be between 10^{40} and 10^{50} (Tromp and Österlund, 2021)⁵. In Monte-Carlo tree search (MCTS), the algorithm samples paths by conducting simulations, rolling out possible outcomes iteratively until termination and averaging over rollouts (Coulom, 2007; Kocsis and Szepesvári, 2006). In the limit, MCTS provably converges to the minimax value, but doing so efficiently is a challenge. The key to efficient search is selecting the right nodes to roll out. Explore too narrowly and you may converge on an early but suboptimal answer, explore too broadly and you might never converge at all. The trick is to strike a middle ground that balances exploration with exploitation. One method to do this is the upper-confidence bound (UCB) rule, which encourages sampling of less-visited options (Auer et al., 2002); a more recent update to UCB adds a prior to this rule to bias the sampling to better

⁵The provenance of this number is somewhat convoluted; in 1950, Claude Shannon estimated the total number of unique chess *games* to be around 10^{120} , widely cited as the *Shannon number*, with around 10^{43} legal positions (Shannon, 1950). Since then attempts have been made to refine it, including an upper bound of 10^{52} by Allis (1994), 10^{46} by Chinchalkar (1996), and 10^{40} without promotions by Steinerberger (2015). A frequently cited estimate of 10^{44} comes from a Github repository which implements an unbiased weighted Monte-Carlo estimate (Tromp and Österlund, 2021). Suffice it to say that the state space is large enough for exhaustive search to be infeasible at runtime.

options upfront (Rosin, 2011). When applied to trees, the upper confidence-bound rule is known as the upper confidence bound on trees (UCT) (Kocsis and Szepesvári, 2006) and incorporating a prior is called prior upper confidence bound on trees (PUCT) (Rosin, 2011).

MCTS significantly reduces the number of samples needed to converge on the minimax value in simple domains, but it can remain prohibitively expensive when the number of states is large. Because it relies on rollouts until termination to iteratively update, it becomes especially costly when there are many steps before termination (in chess, games can easily last over a hundred moves before ending in a checkmate or a stalemate). In these cases, rather than roll out to a perfect termination condition, MCTS can be modified to stop when a search is “good enough” and evaluate the non-terminal leaf nodes once again using heuristics. Traditionally, these heuristics were either broadly applicable but fairly simple rules-of-thumb for evaluating a position, or very specific ones carefully crafted by hand. Obtaining heuristics flexible enough to operate in both broad and specific situations required statistical learning rather than hand-crafting. This was exactly the approach taken by AlphaGo, which combined Monte-Carlo tree search with neural networks trained from human games and self-play, using learned policy and value estimates to guide search via PUCT. Its policy network was trained on tens of millions of positions drawn from human expert games and then sharpened through self-play, while a separate value network learned to evaluate positions by predicting the eventual winner. Later systems like AlphaGo Zero (Silver et al., 2017b) and AlphaZero (Silver et al., 2017a) dispensed with human games entirely, learning these heuristics from scratch through self-play alone, bootstrapping by repeatedly playing until coherent strategies converged.

Retrospectively, the development of efficient and learned search traces out a compelling narrative about how to conduct search effectively in the real world. A key insight is that search is *expensive*; it is rarely worthwhile to enumerate all possible outcomes in large state spaces just to decide the next action. Instead, adaptive search relies on two main levers for improving its efficacy under limited resources. The first is *heuristics*, which provide rough but good-enough estimates of value; these can be handcrafted, as they were in early search algorithms, or learned

through optimization or self-play. The second is *efficient sampling*, smartly guiding search toward nodes that are more promising while pruning away those that aren't. These two go hand in hand: sharper heuristics steer sampling toward the nodes that matter, and the outcomes of sampling supply the experience from which sharper heuristics are learned.

Recall that these algorithms were designed to optimize performance, not to mimic the mind. While cognitive algorithms differ in important ways from those optimized for performance, the basic building blocks of effective search and the models of human planning share common motifs. It has been known for many decades that people employ heuristics in everyday decision making. In the domain of chess, grandmasters were shown to have better recall of board positions than novices (de Groot, 2008), an advantage that disappeared when the positions were arranged at random rather than drawn from real games (Chase and Simon, 1973). This suggests that people encode context-specific knowledge about the relationships between board pieces, and that they recall these representations more easily with experience. Indeed, the aforementioned Deep Blue is just as much a testament to human ingenuity as it is to machine intelligence: the hand-crafted features were meant to encode decades of human knowledge and distill it into usable rules of thumb at runtime. More recent evidence hints at the remarkable flexibility of human planning. For example, people have been shown to prune decisions early when they encounter large losses, much like the pruning rules employed by algorithms (Huys et al., 2012; Sezener et al., 2019). Computational models of human planning have likewise demonstrated that people increase their planning depth with experience, improving their ability to search deeply as they gain familiarity with a game (van Opheusden et al., 2023).

The recent rise of process-level accounts of cognitive planning offers a chance to ask how the brain might implement these search algorithms, not just what they compute. These models also situate planning within the broader study of decision making: the deliberative, model-based end of behavior, as opposed to the simpler associative and habitual choices that rely on cached, model-free value. The section that follows traces the rich literature connecting neuroscience to RL, from the value-learning machinery that supports simple choices up to the circuits implicated

in deliberation, planning, and metacognition.

1.1.6 The Neural Basis of RL and Planning

The connection between reinforcement learning and neuroscience is often regarded as a major success story for computational theories of learning. In the late 1990s, studies by Montague, Dayan, and Schultz demonstrated that phasic firing of dopaminergic neurons in the ventral tegmental area (VTA) resembled a reward prediction error signal analogous to the temporal-difference (TD) error used in reinforcement learning models (Montague et al., 1996; Schultz et al., 1997). This finding provided an early computational interpretation of dopaminergic signaling in terms of learning from prediction error.

Subsequent work extended this perspective to other components of reinforcement learning, implicating cortical and striatal circuits in representing value-related quantities such as state- and action-value functions (Kim et al., 2008; Kim and Lee, 2011; Lau and Glimcher, 2008; Cai et al., 2011; Ito and Doya, 2009). Rather than encoding scalar value alone, these regions were also found to carry richer task-relevant information. In the striatum, neural activity has been shown to reflect both prospective valuation of actions and outcome-related signals tied to chosen actions (Cai et al., 2011), while neurons in prefrontal cortex appear to encode variables such as stimulus anticipation, chosen actions, and expected rewards (Niki and Watanabe, 1979; Matsumoto et al., 2003; Barraclough et al., 2004; Leon and Shadlen, 1999). Together, these findings motivated a broader view in which corticostriatal circuits support both value learning and flexible decision-related computations.

One influential direction sought to explain these results through the lens of distinct reinforcement learning systems supporting habitual and goal-directed behavior. Early computational accounts proposed that components of reinforcement learning might be implemented in separable corticostriatal circuits, with dorsal striatum contributing to action selection and ventral striatum functioning as a critic involved in evaluating reward expectations and prediction errors (O’Doherty et al., 2004). Consistent with this view, activity in dorsal striatal subregions

was found to reflect different aspects of stimulus-response and associative learning (Haruno and Kawato, 2006).

Parallel work emphasized a broader role for frontal-striatal circuitry in flexible, goal-directed control, an idea that traces its roots to early physiological case studies. Lesion studies and neuropsychological observations of frontal lobe damage highlighted impairments in planning and executive function, including early reports such as those of Phineas Gage (Harlow, 1869; Penfield and Evans, 1935). More systematic work demonstrated that planning deficits arise in both frontal lobe patients and individuals with Parkinson's disease, implicating dopaminergic frontostriatal circuits in goal-directed behavior (Owen et al., 1990, 1995; Owen, 1997). Complementary findings in rodents showed that lesions to dorsomedial striatum and prefrontal cortex impaired the acquisition of goal-directed actions (Balleine and Dickinson, 1998; Yin et al., 2005), while human neuroimaging studies linked ventromedial prefrontal and related regions to prospective evaluation and adaptive decision making (Tanaka et al., 2004; Daw et al., 2006; Hampton et al., 2006).

Together, these findings distinguished between habitual and goal-directed systems of behavioral control, later formalized within reinforcement learning through the lens of model-free and model-based control. In this framework, ventral and dorsolateral striatal circuits were proposed to support model-free learning, whereas prefrontal, orbitofrontal, and associative cortical regions were implicated in model-based evaluation and planning (Daw et al., 2005; Gläscher et al., 2010; Daw et al., 2011). A key empirical test of this distinction was the two-step task introduced by Daw and colleagues (Daw et al., 2011), in which sequential decisions with probabilistic state transitions allowed model-free and model-based strategies to generate dissociable behavioral signatures. Model-free learners tend to repeat actions following reward regardless of transition structure, whereas model-based learners incorporate task dynamics when updating action values. Using computational modeling and fMRI, the study found that ventral striatum tracked reward prediction errors, while prefrontal and parietal regions were associated with model-based evaluation. Subsequent work further identified prefrontal representations related to reward history, strategy selection, and arbitration between model-free and model-based control (BarracloUGH et al., 2004;

Averbeck et al., 2006; Lee et al., 2014; Bartolo and Averbeck, 2020).

The idea of a world model is central to the concept of model-based control, and it motivates the question of where such a model might be encoded in the brain. While this thesis primarily focuses on the prefrontal and striatal substrates of planning, any account of model-based control would be incomplete without acknowledging where the world model itself might reside. Perhaps the simplest instantiation of a world model is a map, encoding spatial relationships, which connects back to our earlier discussion of cognitive maps (Tolman, 1948). The discovery that individual hippocampal cells fire selectively when an animal occupies particular locations, so-called *place cells* (O’Keefe and Nadel, 1978), and later that grid cells tile space in a regular lattice downstream (Hafting et al., 2005), established the hippocampus as encoding an allocentric, map-like representation of the world, grounding Tolman’s behavioral proposal in a neural substrate. This role, however, extends well beyond spatial navigation: damage to the hippocampus and surrounding medial temporal lobe produces a profound inability to form new memories (Scoville and Milner, 1957), and a parallel tradition accordingly casts the hippocampus not as a spatial map per se but as a relational memory system, of which spatial relationships are only one special case (Eichenbaum and Cohen, 1993). Consistent with this more general view, hippocampal cells encode non-spatial dimensions of a task, tiling elapsed time much as place cells tile space, so-called *time cells* (MacDonald et al., 2011). A map of space is thus one instance of a broader capacity to encode structured relationships among states of the world, be it physical or abstract.

Rather than merely storing learned or past information, the hippocampus has also been shown to encode possible futures. Early evidence came from the discovery that place cells active during navigation are reactivated during sleep, a phenomenon known as hippocampal replay, initially proposed to reflect the reactivation of past experience (Wilson and McNaughton, 1994). Such replays are not confined to rest: during wakeful deliberation they have been proposed to track outcome evaluation (Foster and Wilson, 2006), reward (Singer and Frank, 2009), and future trajectories toward goals (Singer and Frank, 2009; Pfeiffer and Foster, 2013; Pfeiffer, 2020). During vicarious trial-and-error in navigation, where an animal’s gaze darts between multiple possible

paths, hippocampal neurons sweep ahead down the alternative paths under consideration (Johnson and Redish, 2007; Redish, 2016). These findings establish the hippocampus as a substrate for prospective as well as retrospective reasoning, and as integral to implementing the world model invoked in model-based control.

Taken at face value, model-free and model-based reasoning represent two ends of a spectrum of how to represent and use experience. From a model-free perspective, perfect granularity makes generalization impossible: no two states are ever truly identical, making learning by enumeration infeasible. From a model-based perspective, perfect modeling is equally impossible: as Borges noted in *Del rigor en la ciencia*, a point-for-point map of an empire would be as large as the empire itself, and therefore useless. In both cases, abstraction is essential, either to group states that are meaningfully similar or to construct models that are useful rather than exhaustive. So far we have been discussing model-based and model-free representations as complete dichotomies. In the real world, however, the difference is much less stark: humans and animals flexibly combine both forms of reasoning depending on context (Dolan and Dayan, 2013). In the two-step task (Daw et al., 2011), for example, both rodents and humans exhibit behavior consistent with an arbitration between model-based and model-free strategies, with mixed activations of both in prefrontal and striatal regions.

These observations invite a subtler possibility than a strict dichotomy: that the brain employs representations intermediate between model-free and model-based control. One influential candidate is the successor representation, which caches, for each state, the expected discounted future occupancy of all others under the current policy (Dayan, 1993). Because these long-run predictions are stored rather than recomputed, value can be read off as their product with reward at each state, yielding flexibility to changes in reward without the cost of simulation—an intermediate that was subsequently proposed as a model of hippocampal coding (Stachenfeld et al., 2017). Human behavior bears its signature in the asymmetry between reward and transition revaluation (Momennejad et al., 2017), and successor-based algorithms reproduce a range of otherwise model-based behaviors, including latent learning and policy revaluation (Russek et al., 2017). The

successor representation thus offers one route to model-based flexibility that sidesteps explicit simulation, compressing the world model into a cached predictive map rather than querying it online.

A different route keeps the world model explicit and asks instead when it is worth querying. A normative account of memory access makes this precise: the brain ought to access precisely those memories whose reactivation would most improve future choices, formalized as the expected value of a backup (Mattar and Daw, 2018). Here planning is intertwined with the problem of deciding which computations are worth performing in the first place. Most recent models of planning integrate prefrontal cortex, striatum, and hippocampus, casting planning as a dynamic interaction between executing a plan and deciding whether planning is worthwhile. One influential proposal casts the prefrontal–striatal circuit as a meta-reinforcement learning system (Wang et al., 2018). In this account, the dopaminergic striatal system acts as a slow outer loop, adjusting the synaptic weights of the prefrontal network through reward prediction errors accumulated across many tasks; the prefrontal network, in turn, comes to implement its own fast, free-standing reinforcement learning algorithm entirely within its recurrent activity dynamics. Notably, such an agent recovers the mixture of signals found in the two-step task, offering a mechanistic account of the mixed representations described above. More recent work integrates the hippocampus into this picture by supplying the prefrontal meta-RL agent with an explicit world model over which to plan (Jensen et al., 2024). Here the recurrent prefrontal network is augmented with the ability to sample imagined action sequences, or rollouts, from its own policy, evaluating them and folding the result back into its dynamics before committing to an action. These rollouts are then interpreted as the prefrontal network interrogating the hippocampal world model through replay. Here, the agent learns not only how to plan but when to plan, deploying rollouts only when the expected improvement in behavior justifies the time spent, and in doing so reproduces the contextual variability of human deliberation times. The rollouts employed by the artificial agent closely resemble hippocampal replays recorded in navigating rodents, suggesting that replay may implement precisely this form of rollout-driven planning. These recent approaches increasingly

frame planning as a dynamic process that flexibly combines both model-free and model-based components.

1.2 Adaptive Planning

The key question, then, is not whether agents use model-free or model-based strategies, but how they balance the trade-off between them. This involves deciding when to rely on cached experience, when to engage in simulation, and how much structure to encode in the first place. From this perspective, the act of choosing when and how to plan can be viewed as a problem of resource allocation: how biological agents distribute limited computational effort across complex prospective decisions.

1.2.1 Rational Behavior, Heuristics, and Biases

Understanding how people *ought* to act in order to achieve their goals has been a central question in philosophy since antiquity. In this context, acting in accordance with reason and one's ends is often referred to as rational behavior. Traditionally, this notion of rationality has been a normative one: it specifies how an ideal agent should act rather than describing how people actually behave.

A major shift occurred in the twentieth century with the development of axiomatic models of rational choice in economics. Rather than treating rationality as a purely philosophical ideal, economists formalized it in terms of internally consistent preferences (Samuelson, 1938) and probabilistic beliefs, from which patterns of choice and even preferences could be derived. This move allowed human decision-making to be studied using mathematical structure, enabling theoretical predictions about choice behavior under uncertainty and constraint. A foundational work in this vein was the von Neumann-Morgenstern utility theorem (von Neumann and Morgenstern, 1947), which showed that any set of preferences that satisfies certain axioms (completeness, transitivity, continuity, and independence) could be represented by a utility function, a scalar-valued

function that captures the relative desirability of specific outcomes. From the expected utility function, one could compute inferences about people’s beliefs and how they translate into choice behavior, extending expected utility theory to subjective experience (Savage, 1954). These ideas laid the groundwork for understanding human behavior through the lens of rational choice; by inferring preferences and beliefs from choice behavior, one could estimate how a rational agent would act in a given situation to maximize their utility. Rather than understanding choices as piecewise observations, the utility function provided a principled way to reason about people’s choices broadly and across domains.

Expected utility theory proved useful as a normative benchmark for individual and collective decision-making, but it was clear that it was built on imperfect assumptions about how people actually behave. In the decades that followed, researchers like Daniel Kahneman and Amos Tversky demonstrated that human judgments under uncertainty often deviated systematically from the predictions of expected utility theory (Tversky et al., 1982). Rather than behaving as ideal statistical reasoners, people frequently relied on heuristics (fast and computationally efficient mental shortcuts) that produced predictable biases in decision-making. These included effects such as anchoring, representativeness (Kahneman and Tversky, 1972), and availability (Tversky and Kahneman, 1973). Further work demonstrated that human judgments were sensitive to framing (Tversky and Kahneman, 1981), salient examples, or irrelevant contextual information. Prospect theory later extended these findings by proposing that people evaluate outcomes relative to reference points and weigh gains and losses asymmetrically (Tversky and Kahneman, 1991), helping explain empirical phenomena such as loss aversion and nonlinear probability weighting (Kahneman and Tversky, 1979; Tversky and Kahneman, 1992). Note that these findings did not invalidate the idea that people could, in principle, act rationally, but rather that people’s choices often deviated systematically from normative accounts of rational behavior⁶.

⁶These ideas later contributed to dual-process theories of decision-making that distinguish between fast, heuristic-driven cognition and slower, more deliberative forms of reasoning (Kahneman, 2011). Related distinctions have also been drawn in computational neuroscience and reinforcement learning between model-free and model-based systems of behavioral control (Dolan and Dayan, 2013).

1.2.2 Bounded, Ecological, and Resource Rationality

Predating Kahneman and Tversky’s heuristics-and-biases program by nearly two decades, Herbert Simon attempted to grapple with the inherent limitations of expected utility theory. Informed by his own work on human decision making and organizational behavior, he proposed that rather than optimizing for maximum utility, people are fundamentally constrained both by their environment and by the mind’s own cognitive limitations (Simon, 1955), a framework known broadly as bounded rationality. Simon argued that people often *satisfice*, choosing actions that are good enough and thereby trading optimality for efficiency; such satisficing could account for many of the deviations between human behavior and strict rationality. But bounded rationality raised questions of its own: where did these bounds come from, and were they any less arbitrary than the seemingly innumerable biases identified in the prior literature? In some ways, bounded rationality simply shifted the burden of explaining irrational behavior upstream, from the behavior itself to the bounds that shaped it.

Simon’s ideas opened several broad paths that developed largely in parallel across cognitive science, computer science, and economics. One path investigated where the bounds came from in the first place. Gerd Gigerenzer and others showed that some of the apparent failures of probabilistic reasoning documented by Kahneman and Tversky were attenuated when the same information was presented as natural frequencies rather than conditional probabilities (Gigerenzer and Hoffrage, 1995). More broadly, simple heuristics such as *take-the-best*, while not normatively optimal, proved surprisingly effective in naturalistic environments (Gigerenzer and Goldstein, 1996). This motivated the idea of *ecological rationality*, which held that the structure of natural environments shapes and licenses the heuristics people use (Gigerenzer, 2008). On this view, a heuristic is not a deficiency but an adaptation to environmental regularities.

A second path took the bounds as given and asked a design question: given fixed computational constraints, how should an agent act to achieve its goals? This produced the program of rational metareasoning (Russell and Wefald, 1991), which treated deciding how to think as it-

self an optimization problem subject to computational cost. Within this framing one could ask whether an agent was *bounded optimal*, whether it had selected the best available algorithm for choosing its actions given the constraints imposed by its hardware and its environment. Extending this logic to cognitive architectures gave rise to computational rationality in the decision sciences (Lewis et al., 2014; Gershman et al., 2015).

A third path pursued a broader research program: deriving rational behavioral algorithms from first principles, starting from a small set of assumptions about the goals of cognitive computation. Known as rational analysis (Anderson, 1990; Chater and Oaksford, 2000), this paradigm began with the goals of a cognitive system, built formal models from those goals, and then introduced limitations as the models' predictions diverged from observed behavior. Notably, in rational analysis cognitively plausible limitations were treated as auxiliary, an invoke-as-needed correction to what was otherwise modeled as rational pursuit of a goal.

One recent attempt to draw these threads together is known as resource rationality; it proposes that people act meta-rationally, solving a resource-aware utility optimization problem, and the corresponding methodology of deriving such algorithms from assumptions about the agent and its environment is termed resource-rational analysis (Lieder et al., 2013; Lieder and Griffiths, 2020; Lieder et al., 2025). The framework inherits ecological rationality's appeal to naturalistic environments as the source of relevant constraints, while making the mechanism of heuristic selection explicit as an optimization under cost rather than leaving it to an adaptive toolbox. It shares computational rationality's search over algorithms shaped by an objective function, while widening the relevant constraints beyond architecture to include working memory, time pressure, and other cognitive costs; this borrows existing computational frameworks of bounded-optimal agents (Russell and Subramanian, 1995). Finally, it recasts rational analysis by treating those constraints as central rather than auxiliary: the objective is not to find the algorithm that maximizes utility, but the one that maximizes utility net of the cost of computation.

1.2.3 Formalizing Resource Rationality

We begin with bounded optimality and build towards resource rationality⁷. Following Lieder et al. (2025), a cognitive architecture C is the space of cognitive processes c an agent could execute, where a process might be, for example, a sequence of cognitive operations. A *cognitive policy* π is a mapping from states to cognitive processes c . A bounded-optimal agent is one whose policy π^* is the best such strategy its architecture allows (Russell and Subramanian, 1995): the policy, drawn from the admissible space Π_{C_G} under the global architecture C_G , that maximizes expected utility across the whole environment E :

$$\pi_{E,C_G}^* = \arg \max_{\pi \in \Pi_{C_G}} \mathbb{E}_{\substack{s \sim E \\ c \sim \pi(\cdot|s) \\ a \sim p(\cdot|s,c) \\ o \sim q(\cdot|s,a)}} [u(o)] \quad (1.2)$$

where s is a state drawn from the environment, c a cognitive process selected by the policy, a the action produced by that process under action kernel p , and o the outcome of that action under outcome function q , evaluated by utility function u . This requires optimizing over every state and every cognitive process, which is intractable in most realistic settings.

A more tractable tradition gives up on the whole policy and focuses instead on a *single unit* of thinking: is one act of thought worth performing? This is the *value of computation* (VOC), which weighs the improvement a computation makes to the action eventually taken against the cost of running it (Russell and Wefald, 1991). For a state s , the benefit of a computation c is how much it improves expected utility over acting straight away:

$$\Delta \text{EU}(s, c) = \underbrace{\mathbb{E}_{\substack{a \sim p(\cdot|s,c) \\ o \sim q(\cdot|s,a)}} [u(o)]}_{\text{with computation}} - \underbrace{\mathbb{E}_{\substack{a \sim p(\cdot|s,\emptyset) \\ o \sim q(\cdot|s,a)}} [u(o)]}_{\text{without computation}}, \quad (1.3)$$

⁷Earlier formulations of resource rationality, e.g. (Lieder and Griffiths, 2020), frame it as the application of boundedly optimal principles to cognitive processes. More recent work recasts resource rationality as interpolating between the general framework of bounded optimality and the specific instantiation given by the value of computation (Lieder et al., 2025). We adopt this more recent and more precise formulation throughout.

where $\emptyset$ denotes acting immediately, without any further cognition. The value of that computation is its benefit less its cost:

$$\text{VOC}(s, c) = \Delta\text{EU}(s, c) - \text{cost}(s, c). \quad (1.4)$$

That cost has a natural reading as an opportunity cost: what is given up by running c in state s rather than some other computation drawn from the full space C_G , including the possibility of no computation at all. Where bounded optimality optimizes a whole policy over an entire environment, VOC zooms all the way in, asking only about one computation in one state. Resource rationality interpolates between these two extremes. It fixes a *scope*, a set of processes $C \subseteq C_G$ and a domain $D \subseteq E$, and optimizes the policy within it:

$$\pi_{D,C}^* = \arg \max_{\pi \in \Pi_C} \mathbb{E}_{s \sim D} \left[\mathbb{E}_{c \sim \pi(\cdot|s)} \left[\mathbb{E}_{\substack{a \sim p(\cdot|s,c) \\ o \sim q(\cdot|s,a)}} [u(o)] - \text{cost}(s, c) \right] \right] \quad (1.5)$$

Widen the scope to the whole environment ($D = E, C = C_G$) and the opportunity cost falls away (with every process already inside the scope, nothing remains outside it to forgo), and we are left with the same formulation as bounded optimality. Shrink the domain to a single state s and process c , and we recover VOC.

These formulations are one-shot, but cognition and behavior both unfold over time, and the planning problems we care about are inescapably sequential. At each step, the agent selects both a physical action and a cognitive operation. Two things now evolve: the world state w_t , the true state of the environment, which physical actions move, and the mental state m_t , the agent's internal picture of the situation, which cognitive operations and incoming observations update. The mental state is what the one-shot state s becomes once thinking takes time. At each step the policy draws an action–operation pair $(a_t, c_t) \sim \pi(\cdot | m_t)$. The policy then maximizes utility net

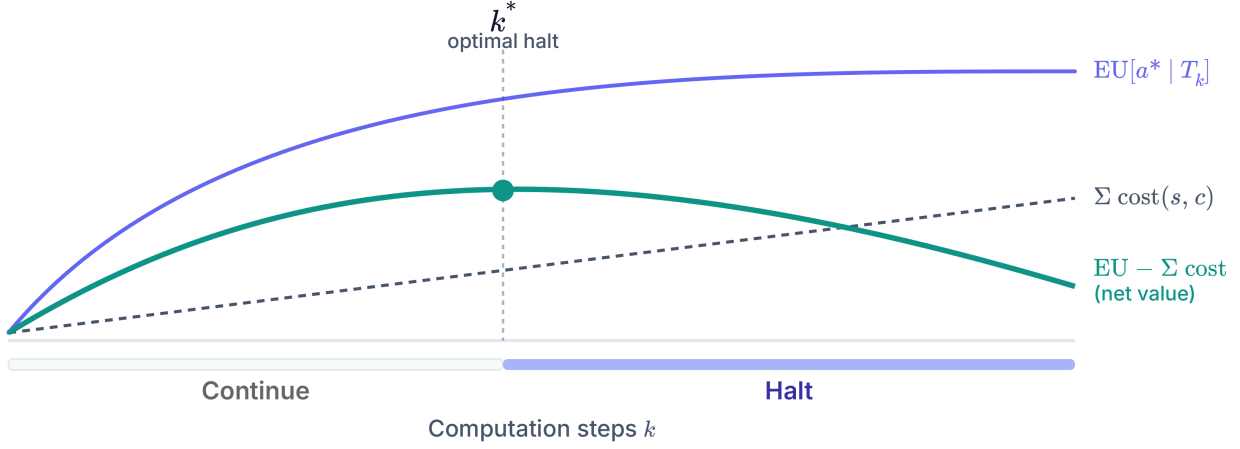


Figure 1.2: **Decision to plan or halt.** As computation proceeds, the expected utility of the best action $EU[a^* | T_k]$ rises with diminishing returns while the cumulative cost of computation grows roughly linearly. Their difference, the net value of computation, peaks at the optimal stopping point k^* : a greedy metaplanner continues planning while the marginal gain exceeds its cost ($\Delta EU > \text{cost}$) and halts once it does not ($\Delta EU \leq \text{cost}$).

of the cost spent thinking:

$$\pi_{D,C}^* = \arg \max_{\pi \in \Pi_C} \mathbb{E}_{\substack{(a_t, c_t) \sim \pi(\cdot | m_t) \\ m_t \sim p_C(\cdot | m_{t-1}, c_{t-1}, o_t) \\ (w_t, o_t) \sim p_D(\cdot | w_{t-1}, a_{t-1})}} \left[u(w_N) - \sum_{t=1}^N \text{cost}(m_t, c_t) \right] \quad (1.6)$$

where w_t is the world state, m_t the mental state, o_t an observation of the world, a_t a physical action, and c_t a cognitive operation. The kernel p_C governs how the mental state evolves under cognitive operations, p_D how the world evolves (and what it reveals) under physical actions, and N is the step at which the episode resolves. Instead of a single cost of computation, the agent now faces a sequence of costs for each step.

1.2.4 Resource Rational Planning

Yet even *this* reduced sequential policy can be hard to pin down. A more tractable reframing casts the choice of *which computation to run next, and when to stop and act* as a sequential decision problem in its own right, a metalevel Markov decision process (Hay et al., 2014), whose states are computational states, whose actions are computations, and whose objective trades the utility

of the resulting action against the cost of the thinking that produced it (Fig 1.2). Optimizing the exact metalevel objective is generally intractable, and somewhat circular: knowing whether a computation is worthwhile often requires performing it first.

In practice, then, researchers reach for approximations rather than exact solutions, and these differ tellingly in *which* value they assign to additional computation. While several approximations exist in various subfields of cognitive science, we restrict our focus to ones deployed in the domain of planning. One line expands the search tree in the directions, and to the depth, where the net value of further expansion, its information value minus the cost of computation, stays positive, pruning elsewhere (Sezener et al., 2019); this offers a normative counterpart to the reflexive, loss-triggered *Pavlovian pruning* seen in human planners (Huys et al., 2012). A related approach samples the value of information from prospective search in plan-and-uncover paradigms, weighing how much what it uncovers would improve the chosen action against the cost of uncovering it (Callaway et al., 2022). Others recast the target more generally: not the value of resolving uncertainty about the consequences of actions, but uncertainty about *which action is best*, treating each evaluation as a noisy measurement of an action’s value and sampling until the winner is clear enough (Kuperwajs et al., 2024). Consistent with predictions of resource rationality, thinking time in chess has been reported to correlate with an engine-derived value-of-computation signal (Russek et al., 2022). This motivates a closer, mechanistic look at how people allocate thinking time during planning, which we take up in Chapter 4.

1.3 Overview

Planning is costly, and an agent that deliberates over every decision wastes resources it could have spent acting. The control of one’s own deliberation is therefore a problem in its own right. In this thesis, we pay particular attention to environments where planning is computationally expensive and uncertainty makes the value of additional deliberation difficult to determine, reflecting the challenges of planning in the real world.

The real world is stochastic, making planning difficult. Intuitively, it is easy to imagine that there comes a point where an environment is so noisy that planning simply isn't worthwhile; from a resource-rational perspective this implies that there should be a graceful degradation of planning effort as stochasticity increases. However, how people implement this in real life is far from clear. In Chapter 2, we focus on how people flexibly adapt their planning effort in stochastic environments. We design a navigation task where participants are asked to choose the path in a tree of options that maximizes their overall reward by moving either left or right at each step. We conduct three manipulations to study different forms of stochasticity. In the reliability condition, the reward you expect may not be the reward you receive. In the volatility condition, the reward is subject to random change over time; outcomes further in the future are less certain than those that are nearer. In the controllability condition, the outcomes are subject to randomness, where for example pressing the button associated with moving left sometimes leads to a rightward move. We then designed and evaluated several computational models of participant behavior to understand how people adapt their planning effort to different levels of uncertainty.

Uncertainty does not always come from unknown sources in the environment; sometimes it comes from interacting with other agents whose behavior is structured but difficult to predict. In the next chapter, we focus on a particular source of uncertainty and noise, but this time structured rather than completely random: interaction with an adversarial opponent. We take a combinatorial two-player planning task studied in humans, Four-in-a-Row, and try to understand the neural mechanisms underlying planning in a nonhuman primate. In Chapter 3, we take a deep dive into how one monkey learns to strategize in a complex planning game orders of magnitude more complex than those traditionally studied in laboratory tasks. In doing so, we ask whether neural representations supporting multi-step planning and abstract feature representations can be identified in the brain, and how those representations are combined during deliberation. Connecting with the broader theme of planning adaptation and effort, this chapter is where we interrogate how planning is implemented at the neural level.

Next, we will see that choosing how much effort to allocate to planning can be made explicit as a task in itself; one that requires energy and cognitive resources. To understand how this adaptation is deployed at scale, we will turn our attention to a game that has distinctly shaped society over the course of centuries, one that has formed its own culture and sport: the game of chess. In Chapter 4, we will focus on how people allocate thinking time in chess, using a sample of millions of online games from the Lichess database, and ask what structural features of a position, such as how many reasonable options it affords, predict how long people deliberate over it. Motivated by our behavioral findings, we build a normative benchmark for the underlying metaplanning problem: a metacontroller, built from a graph neural network encoder (trained with supervised learning) and a readout head (trained with policy-gradient reinforcement learning), that learns when to halt or continue an ongoing tree search. We treat these as two distinct contributions rather than a single validated account, and discuss future directions to reconcile them.

Moving from unpredictable uncertainty to structured adversaries, spanning planning behavior, neural implementation, and normative models of metacontrol, this thesis tackles the central question of how agents allocate effort in an uncertain world while balancing the rewards and costs of planning.

2 | Human Planning in Stochastic Environments

2.1 Introduction

Planning is a cognitive process that involves thinking multiple steps ahead to achieve a goal. In the real world, planning is complicated by the fact that the environment is often stochastic. Random factors out of our control, such as shifting climate patterns (Woodruff, 2016; Zandvoort et al., 2018) or unexpected disease pandemics (Loued-Khenissi and Corradi-Dell’Acqua, 2024) can ruin even the best-laid plans.

Stochasticity in planning can manifest in various forms. Imagine you are planning a trip to an unfamiliar city and want to dine at the best restaurants, but your list of recommendations is outdated. Some restaurants may have improved or declined in quality since they were last rated. Here, the stochasticity lies in the reward. Reward stochasticity can take on different characteristics. In some cases, as with outdated recommendations or imperfect information, the environment can be described as *unreliable*. Developmental studies have suggested that reliability of future information modulates cognitive effort. For example, children and adults become more proactively engaged when information about future tasks is reliable than when it is unreliable (Chevalier et al., 2020; Munakata et al., 2023). In other cases, such as with stocks (Sharpe, 1964; Black and Scholes, 1973; Daly, 2008) or weather patterns (Angelova and Käbel, 2019; Campbell and Diebold, 2005), the reward may be known but is subject to unpredictable fluctuations over time. This dynamic form of stochasticity, which involves temporal fluctuations in reward, is sometimes described as *volatility* (Behrens et al., 2007; Mathys et al., 2011; Piray and Daw, 2020, 2021; Zylberberg et al., 2016; Story et al., 2023). Existing work on volatility has focused on how agents learn in volatile environments (Behrens et al., 2007; Mathys et al., 2011; Piray and Daw, 2020, 2021; Zylberberg et al., 2016), but does not speak to its potential effects on planning effort.

Another form of stochasticity arises in the transition structure of the environment. Imagine you're trying to get to your favorite restaurant by navigating a complex subway system. However, due to ongoing maintenance, some trains are unpredictably rerouted. Even if you plan your route carefully, a train might unexpectedly take you to a different station. In these environments, the connection between an agent's choices and the resulting future states is disrupted. This reduced *controllability* adds a unique dimension to decision-making. For example, people engage in more counterfactual reasoning when faced with more controllable scenarios compared to less controllable ones (Giroto et al., 1991; Wilkinson et al., 2015, 2010). However, it remains an open question how people change their effort in uncontrollable environments.

Despite the prevalence of stochasticity in the real world, most planning studies have either relied on deterministic tasks – such as the Tower of London (Shallice, 1982; Shallice and Burgess, 1991; Owen, 1997; Owen et al., 1990, 1995), chess (de Groot et al., 1996; Simon and Chase, 1973; Holding and Reynolds, 1982), and Four-in-a-Row (van Opheusden et al., 2023) – or used stochasticity merely as an incidental tool (Daw et al., 2011; Gläscher et al., 2010; Wunderlich et al., 2012; van Opheusden and Ma, 2019). One critical question in the study of planning is how planners allocate cognitive resources to plan ahead (Sezener et al., 2019; Kuperwajs et al., 2024; Eluchans et al., 2024). This becomes especially relevant in stochastic environments, where determining the best plan may be cognitively demanding or even computationally infeasible. Much of the existing work on how stochasticity affects planning has focused on risk preferences in planning rather than on how individuals allocate cognitive resources in stochastic environments. For example, studies show that people develop effective contingency plans to overcome spatial stochasticity (Wiener et al., 2008; Ma et al., 2021), and individuals tend to choose safer routes over riskier ones, even when shorter paths are available (Wiener et al., 2008; Ma et al., 2021). While these studies highlight the importance of stochasticity in planning, they fail to address how people manage the tradeoff between the rewards and costs of planning in stochastic environments.

A growing body of work suggests that people weigh cognitive costs against potential benefits when making decisions (Frömer et al., 2021; Lieder and Griffiths, 2020; Lebiere and Anderson,

2011). This idea is formalized in the framework of *resource rationality* (Lieder and Griffiths, 2020), which describes how to optimally manage this tradeoff. For instance, when deciding whether to plan ahead or act on prior knowledge, people evaluate the benefits of exploiting existing knowledge against the cost of getting new information (Callaway et al., 2018, 2022). This echoes the earlier notion that optimal planners should consider the “Value of Perfect Information” relative to the cost of obtaining it (Ierapetritou et al., 1996). Planning depth has also been shown to vary with task demands (Eluchans et al., 2024) and individuals’ subjective cost of cognitive effort (Felson and Lieder, 2022). Along the same lines, people often prune their decision trees when confronted with large potential losses (Huys et al., 2012), a behavior that can be understood as a strategic adjustment of planning effort based on expected value and uncertainty reduction (Sezener et al., 2019). Under resource-rational principles, we would expect stochasticity to cause a reduction in planning effort. One model formalizes this by predicting that agents plan less when outcomes are less controllable (Lieder et al., 2013). While this model replicates patterns associated with decision making under learned helplessness and anhedonia (Huys et al., 2008), it left its predictions about planning empirically untested. A related behavioral study found that participants in a multi-step farming task chose shorter-horizon crops after unexpected negative shocks, consistent with reduced planning under unreliable environments (Mitra et al., 2023). However, this finding focuses exclusively on negative outcomes, leaving open the question of whether reduced planning effort is a general response to stochasticity or specifically tied to the threat of future losses. More broadly, it remains unclear whether stochasticity alone, independent of outcome valence, drives reductions in planning effort.

In this study, we directly tested the hypothesis that people reduce their planning effort in response to stochasticity. We designed a planning task where participants chose the best path down a visualized decision tree with the aim of maximizing the number of points earned. Keeping the same task structure, we manipulated a different form of stochasticity with each of our three experiments: reliability, volatility, and controllability. This structured approach allowed us to systematically examine how different forms of stochasticity shape planning effort. Finally, we

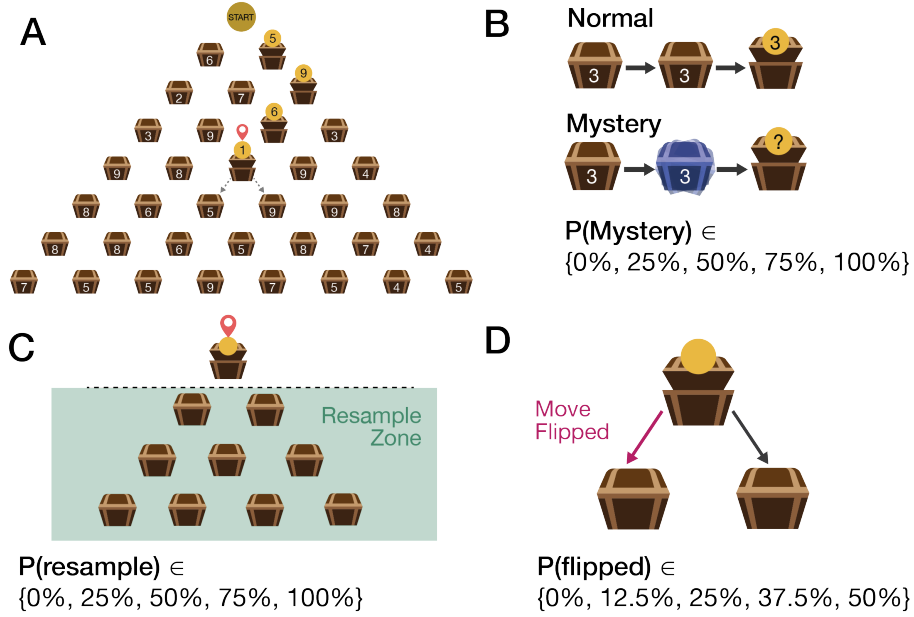


Figure 2.1: **Task design.** **A.** Each game consists of a triangular arrangement of treasure chests. At each step, participants choose either left or right while progressing down the tree (arrows here are shown only for illustration). **B.** Reliability: with probability q , the participant encounters a “mystery” treasure chest, which returns a random value between 1 and 9. **C.** Volatility: with probability q , any treasure chest in the rows below the participant’s current position is re-drawn with a new random value between 1 and 9. **D.** Controllability: with probability q , the participant’s choice (left or right) is flipped.

developed several models to capture participants’ planning processes and used the best model to estimate planning effort as a function of stochasticity level.

2.2 Results

2.2.1 Unreliability Decreases Planning Effort

We started by examining a simple form of stochasticity: reliability. Reliability refers to the consistency between an agent’s knowledge of the world and the actual state of the world. This type of stochasticity, as illustrated earlier by the example of planning with an outdated list of recommendations, arises when an agent’s information about rewards is imperfect.

To study the effect of reliability on planning, we designed an online task loosely inspired by

earlier work by Snider et al. (Snider et al., 2015). The participant was instructed to accumulate as many points as possible while navigating a branching tree of treasure chests, making a binary left-or-right decision at each branch (Fig. 2.1A). Each chest displayed a number, drawn uniformly at random from 1 to 9. For clarity, we will refer to the number written on the front of the chest as the “label”, and the number of points received upon visiting the chest as “points earned”. To manipulate reliability, we introduced two types of treasure chests: “normal” chests, where the number of points each chest was worth was equal to its label, and “mystery” chests, where the label was uninformative and the number of points earned was uniformly sampled at random from 1 to 9 (Fig. 2.1B). Participants were informed only of the overall probability (q_r) that any given chest would be a mystery chest, which we varied across five levels: 0%, 25%, 50%, 75%, and 100%.

Participants learned the task effectively, as revealed by two patterns in the data. First, participants’ choices were systematically influenced by the chest labels, reflecting attention to cues about potential rewards. Specifically, participants showed a clear preference for selecting the higher-valued chest when the difference between the labels was larger (Fig. 2.2A; GLMM, main effect $\beta = 0.39$, $\chi^2(1) = 32300.0$, $p < 10^{-10}$). The magnitude of this effect was higher for lower stochasticity levels (GLMM, interaction $\beta = -0.38$, $\chi^2(1) = 3030.0$, $p < 10^{-10}$). Second, participants earned significantly more points in more reliable environments (Fig. 2.2D; LMM, $\beta = -1.16$, $t_{99} = -19.8$, $p < 10^{-10}$), indicating that they strategically leveraged environmental reliability rather than making arbitrary choices. These results confirm that participants’ decision-making was driven by differences in reward value and that they shaped their strategies based on the level of reliability.

To assess planning effort behaviorally, we conducted exploratory analyses of response times as an empirical proxy for planning effort. This measure has been validated in other domains, such as chess and navigation, where longer thinking times are associated with more strategic processing and reflect a greater value of computation (Ho et al., 2020; Russek et al., 2022). In our analysis, we focus primarily on the thinking time preceding the first choice in each game, when people spend the most time planning their upcoming choices.

Empirical Data

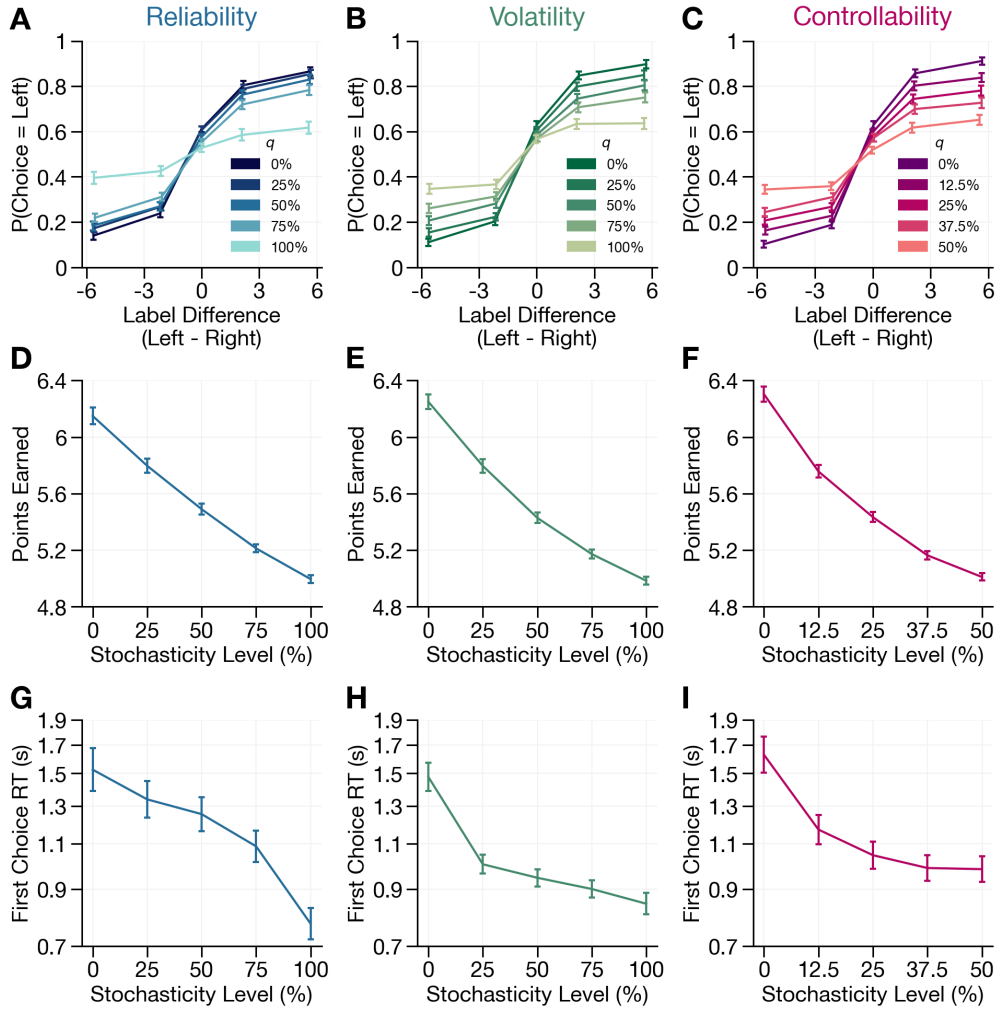


Figure 2.2: **Participant behavior** across three forms of stochasticity: reliability ($n = 100$), volatility ($n = 100$), and controllability ($n = 100$). **A–C.** Probability of leftward choice based on label difference for different stochasticity levels q . Throughout this figure, error bars represent the SEM. **D–F.** Earnings per trial across different stochasticity levels. **G–I.** first-choice response times as a function of stochasticity level, plotting on a log-scaled y-axis.

Our analysis shows that first-choice response time decreases monotonically as environmental stochasticity increases (Fig. 2.2G; LMM, $\beta = -0.63$, $t_{98} = -7.76$, $p < 10^{-10}$). In other words, as the reliability decreases, participants invest less time in planning. This supports the interpretation that under reduced reliability, the expected benefit of extensive planning diminishes, leading participants to spend less effort.

2.2.2 Volatility and Uncontrollability Decrease Planning Effort

So far, we have shown that in less reliable environments, participants spend less effort planning. However, reliability is only one example of the forms of stochasticity people encounter. Many real-world scenarios, such as planning a trip with shifting weather conditions or investing in volatile markets, feature a different kind of stochasticity: temporal fluctuation. In these scenarios, stochasticity diminishes as the future approaches, and it is often possible to change course as the future becomes more certain. For example, when crafting an itinerary for a trip, one might choose to plan specific outdoor activities only once the weather forecast becomes less volatile. In our next task variant, we examined how this temporal fluctuation, or volatility, influences planning effort.

In the volatility condition, we preserved the overall structure of the task but introduced temporal fluctuations. In this condition, a chest’s label always corresponded to its point value. To introduce volatility, we introduced a probability q_v that any chest below the participant’s current row would be resampled at each step (Fig. 2.1C). This meant that chests further away were more likely to change before being reached, modeling the intuition that distant outcomes are less stable than immediate ones. We varied volatility across five levels, $q_v = 0\%, 25\%, 50\%, 75\%$, and 100% .

As before, we first confirmed that participants effectively learned the task. Indeed, participants’ decisions were influenced by the value difference in labels (Fig. 2.2B; GLMM, main effect $\beta = 0.48$, $\chi^2(1) = 35700.0$, $p < 10^{-10}$), and this effect was modulated by stochasticity level (GLMM, interaction $\beta = -0.42$, $\chi^2(1) = 3610.0$, $p < 10^{-10}$). Furthermore, they leveraged low-volatility conditions to maximize their rewards, consistently earning more points in the low volatility conditions compared to the high volatility ones (Fig. 2.2E; LMM, $\beta = -1.26$, $t_{99} = -23.2$, $p < 10^{-10}$).

Consistent with the reliability condition, we found that as volatility increased, participants’ planning effort, as measured by first-choice response times, monotonically decreased (Fig. 2.2H; LMM, $\beta = -0.49$, $t_{99} = -8.83$, $p < 10^{-10}$). This supports the idea that participants invested less

cognitive effort when planning in highly volatile environments.

We have shown so far that stochasticity in rewards, through either unreliable or volatile environments, can drive decreases in planning effort. However, in many real-world scenarios, stochasticity can also affect the transition structure of the environment. This type of stochasticity is familiar to anyone who has learned a new skill, whether it's playing a new instrument or driving a car. Sometimes, the result of an action is far from what was intended, adding an extra layer of unpredictability to the decision-making process. In the final variant of our task, we investigated how transition noise, or controllability, impacts planning effort.

In the controllability condition, the labels accurately reflected the point value of each chest and did not fluctuate with time. However, participants could not always rely on their choices being executed as intended. With probability q_c , their chosen direction was reversed: pressing left would result in a rightward action, and vice versa. We varied q_c across five conditions: 0%, 12.5%, 25%, 37.5%, and 50%.

As in the other two conditions, participants' choices were shaped by the interaction between stochasticity level and the label differences between the immediate following options (Fig. 2.2C; GLMM, main effect $\beta = 0.41$, $\chi^2(1) = 33400.0$, $p < 10^{-10}$), which was modulated by stochasticity level (GLMM, interaction $\beta = -0.86$, $\chi^2(1) = 3700.0$, $p < 10^{-10}$). As before, participants earned significantly more points when the environment was more controllable (Fig. 2.2F; LMM, $\beta = -2.54$, $t_{98} = -22.9$, $p < 10^{-10}$). Similarly, we found a negative relationship between uncontrollability and first-choice response times (Fig. 2.2I; LMM, $\beta = -0.94$, $t_{98} = -9.44$, $p < 10^{-10}$).

Together, these findings show a consistent pattern: as stochasticity increases—whether through unreliability, volatility, or uncontrollability—people reduce their planning effort.

2.2.3 Computational Models of Planning Effort

We used computational cognitive modeling to infer the algorithms that people use to make decisions across the three tasks (Fig. 2.3), allowing us to test how planning effort adapts to environ-

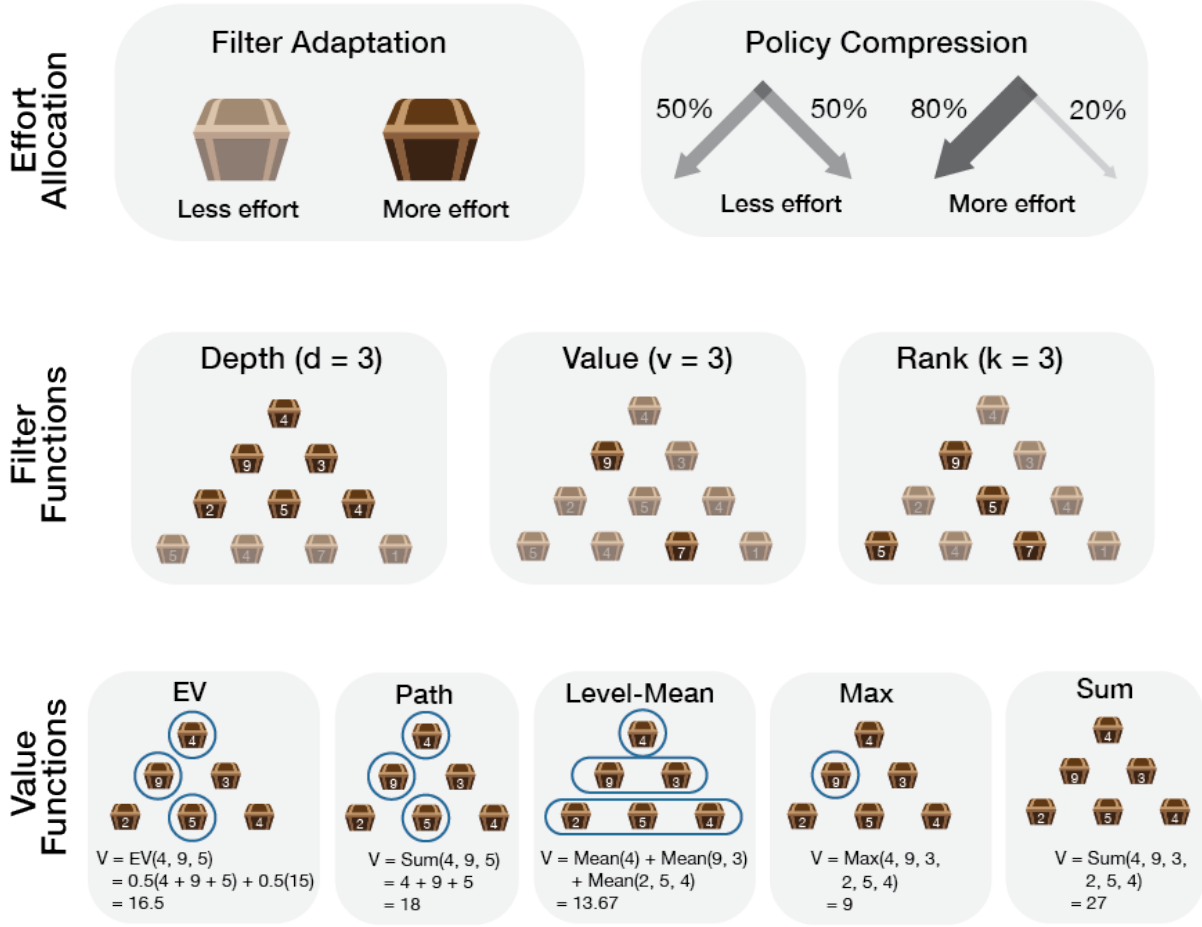


Figure 2.3: **Model illustrations.** Each model consists of a filter function and a value function to evaluate the left and right subtrees (only one subtree is shown here). The *depth filter* keeps only the top d rows. The *value filter* keeps only values $\geq 10 - v$. The *rank filter* keeps only the top k -ranked values. For each filter function, several possible value functions are available. We show the value associated with each subtree under the reliability manipulation with $q_r = 0.5$. EV denotes expected value.

mental uncertainty. Our models predicted participants’ choices at each decision point, assuming that each choice was guided by the subjective values assigned to the left and right subtrees of the decision tree. The greater the difference in these values, the more likely participants were to select the higher-valued option. We assumed conditional independence across choices, such that each decision depended only on the values of the immediately available subtrees.

Given the environment’s complexity, it is unlikely that participants evaluated all available information. Instead, we hypothesized that they employed *filtering* to focus on specific aspects of the tree. We hypothesized three main filtering approaches, grounded in task demands and cognitive plausibility: (1) **Depth filtering**: participants plan ahead only a limited number of steps, truncating their evaluation at a depth d ; (2) **Value filtering**: participants attend only to treasure chests whose values are at least $10 - v$; and (3) **Rank filtering**: participants attend only to the top k valued treasure chests, regardless of their absolute value.

After filtering, participants must assign scalar values to the left and right subtrees. We considered several *value functions* that vary in computational sophistication and cognitive effort: (1) **Expected Value (EV)**: participants compute the expected value of all paths in the subtree, fully accounting for stochasticity, an optimal but computationally demanding approach. In the volatility condition, we excluded the EV model because simulating all possible states produced an exponential explosion of outcomes, making the computation both intractable and psychologically implausible. (2) **Path**: participants treat the environment *as if* it were deterministic, evaluating all possible paths and selecting the one with the highest reward; (3) **Level-Mean**: a coarser deterministic heuristic that averages rewards at each level of the tree and sums these level-wise means, approximating subtree value without tracking specific trajectories; (4) **Max**: participants consider only the maximum-valued chest within each subtree; and (5) **Sum**: participants sum all treasure chest values within a subtree, treating it holistically.

Only the EV model is fully optimal; the other models are *determinizing* because they treat the stochastic environment as if it were certain, replacing potentially intractable expected-value calculations with deterministic surrogates. Determinization has been used in the machine learn-

ing literature to simplify otherwise intractable problems (Yoon et al., 2007; Saisubramanian et al., 2019; Saisubramanian and Zilbertsein, 2019) but rarely examined in human cognition.

Finally, we modeled the participant’s choice to move left or right as a noisy action based on the difference in the values of the filtered subtrees, with the inverse temperature (or sensitivity) of the decision parameterized by β . We also included a generic lapse rate λ to capture purely random choices.

Having defined how participants filtered and evaluated information, we next modeled *effort allocation* as a function of stochasticity level in two distinct ways, each grounded in the decision-making literature: (1) **Filter adaptation (FA)**: effort corresponds to considering more information. The exact form that this takes depends on the type of filtering in the model. For example, in the depth filtering case, increasing the depth threshold requires greater cognitive load. Thus, the filter threshold can vary across stochasticity levels while the policy sensitivity β remains constant. (2) **Policy compression (PC)**: effort corresponds to deploying more precise control over the decision process. Here, filtering thresholds remain constant across stochasticity levels, but the sensitivity parameter β varies. This view aligns with theories of rational inattention, where β parameterizes the amount of resources (in information units, e.g. bits) available to represent a specific policy; a greater β corresponds to a more complex policy and therefore a greater cognitive cost (Bhui et al., 2021; Ortega and Braun, 2013; Lai and Gershman, 2024, 2021; Gershman and Lak, 2025). We operationalize policy compression as adaptive changes in policy sensitivity (inverse temperature), abstracting away additional effects of explicit policy priors (e.g., perseveration or a default reference policy). These two formulations capture complementary notions of planning effort: one emphasizes how much of the environment is represented (filter adaptation), and the other emphasizes how sensitive the agent is to value differences (policy compression).

We fitted all models to participant behavior by minimizing the negative log likelihood (NLL). For model comparison, we used 5-fold cross-validated negative log likelihood. Our choice to use 5-fold cross-validated negative log likelihood reflects the fact that we are primarily interested in out-of-sample predictive accuracy and that some parameters have asymmetric impacts on par-

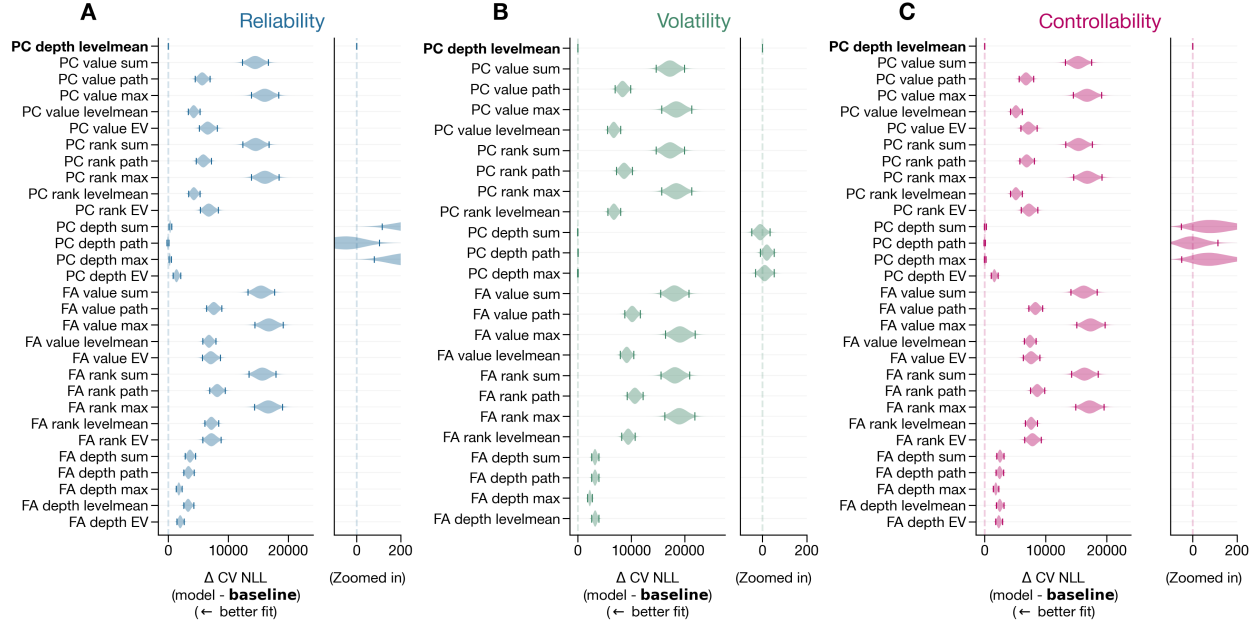


Figure 2.4: **Model comparison.** Distributions of bootstrapped ($n = 10^6$) differences between the 5-fold CV negative log likelihoods of alternative models and the baseline PC Depth-filtering Level-Mean model. Bars represent bootstrap-derived 95% CI. Each panel includes a zoomed-in inset to the right. **A.** Reliability. **B.** Volatility. **C.** Controllability.

ticular model classes.

In all conditions, the PC Depth-filtering Level-Mean and PC Depth-filtering Path models best fit participant behavior, with the PC Depth-filtering Max and Sum models also competitive in the volatility condition (Fig. 2.4). Importantly, the best-fitting models shared three key features: they all incorporated policy compression as a mechanism to modulate planning effort, relied on depth filtering to constrain the representational search space, and used determinizing value functions. The best-fitting models also provided quantitatively good fits to the behavioral trends reported earlier, specifically (1) the probability of choice as a function of label difference and stochasticity level, and (2) the points earned as a function of stochasticity level (Fig. 2.5A–F, Figs. A.4–A.6). By contrast, the EV models predict that participants should be indifferent to value differences in fully stochastic environments, a prediction that is not supported by our data (Fig. A.7).

Next, we examined how planning effort varied across conditions in the winning models, as indexed by the model’s inverse-temperature parameter, β . We found that β decreased with environmental stochasticity across all three conditions for the PC Depth-filtering Level-Mean model

PC depth levelmean

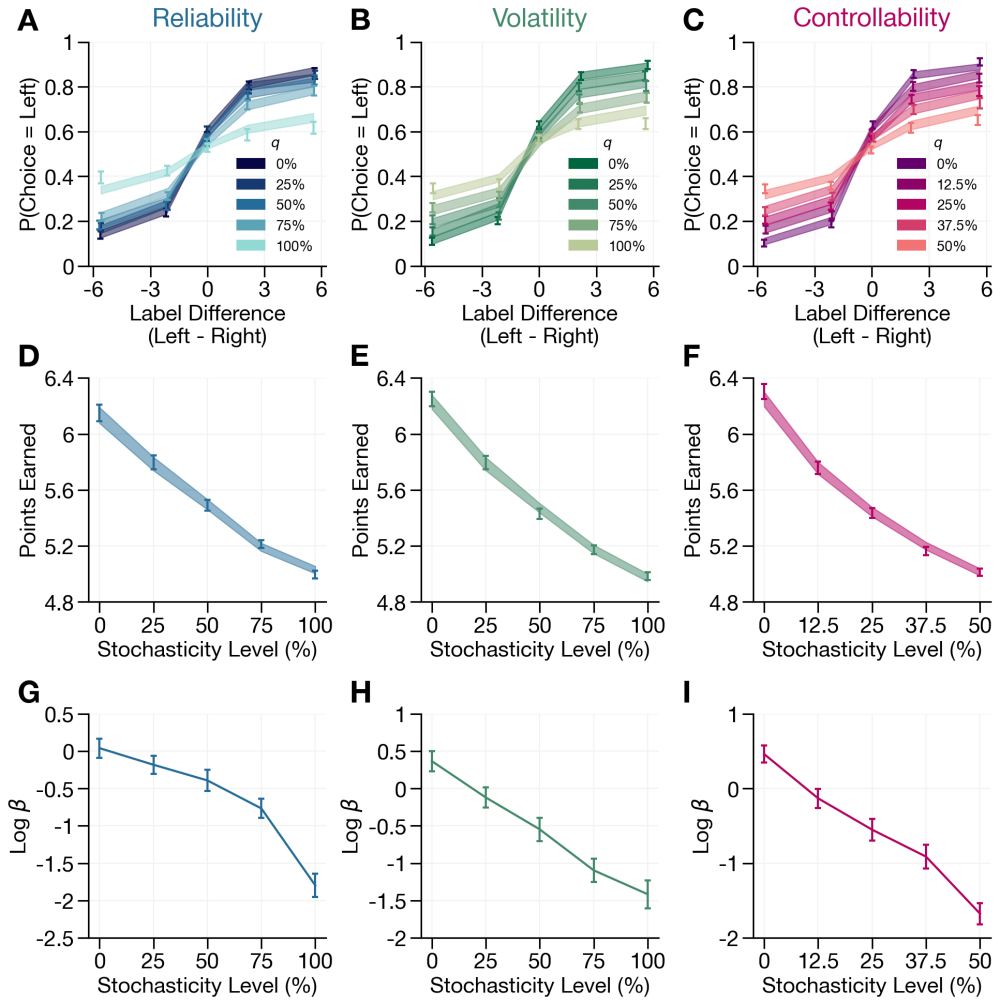


Figure 2.5: **PC Depth-filtering Level-Mean model results.** A–C. Probability of leftward choice based on label difference for different stochasticity levels q . Error bars represent the SEM for empirical data, shaded regions represent the SEM for the model. D–F. Earnings per trial across different stochasticity levels. Error bars represent the SEM for empirical data, shaded regions represent the SEM for the model. G–I. Stochasticity level versus model inverse temperature ($\log \beta$). Error bars represent the SEM for the model.

(Fig. 2.5 G–I; reliability: LMM, $\beta = -1.71$, $t_{98} = -9.1$, $p < 10^{-10}$; volatility: LMM, $\beta = -1.82$, $t_{98} = -8.7$, $p < 10^{-10}$; controllability: LMM, $\beta = -4.04$, $t_{99} = -13.7$, $p < 10^{-10}$), as well as the other competitive models (see Figs. A.4–A.6, panels G–I).

To assess participant-level heterogeneity in strategy selection, we conducted an exploratory random-effects Bayesian model comparison (RFX-BMC; §A.7). Results were not robust enough to support strong quantitative claims about individual differences in strategy use. We note that

RFX-BMC captures heterogeneity across but not within individuals, and therefore does not speak to the possibility of hybrid or mixed strategies within a single participant.

Finally, we ran resource-rational simulations to test whether optimizing reward under effort costs could reproduce the observed adaptation in planning sensitivity (§A.3). The simulated optimal $\log(\beta)$ decreased monotonically with stochasticity, demonstrating that our main effect can emerge from a simple reward-effort tradeoff.

Together, these results indicate that, regardless of the type of stochasticity, participants adopt simpler, more tractable planning strategies in stochastic environments. As stochasticity level increased, participants reduced cognitive effort via policy compression.

2.3 Discussion

In this study, we investigated how stochasticity shapes both planning strategy and planning effort. We examined how people adjust their planning depth in response to three distinct forms of stochasticity: reliability, volatility, and controllability.

Across all three forms of stochasticity, participants’ behavior was best explained by models that incorporated policy compression, filtered by depth, and used determinizing value functions. Our results show that in all best-fitting models, the policy parameter β decreased monotonically as environmental stochasticity increased. This pattern suggests that participants strategically reduced their cognitive investment by adopting simpler policies when precise value comparisons were unlikely to pay off. This strategic simplification is a core feature of policy compression, a mechanism that allows agents to balance policy complexity and reward maximization (Lai and Gershman, 2021, 2024). This tradeoff is often framed as an information bottleneck, where a greater β corresponds to a larger information budget for implementing more complex and cognitively expensive policies (Lai and Gershman, 2021; Gershman and Lak, 2025; Lai and Gershman, 2024; Bhui et al., 2021; Ortega and Braun, 2013). Policy compression has been demonstrated in humans performing complex planning tasks (Lai and Gershman, 2024), and emerging evidence suggests that

midbrain dopamine signals are responsive to policy complexity (Akam et al., 2015). Additionally, the principle of policy compression has been used to design more robust and resource-efficient artificial agents (Eysenbach et al., 2021). Our work adds to this rich domain by identifying environmental stochasticity as a key factor that drives policy simplification, a signature of policy compression.

Our findings are qualitatively consistent with predictions of resource rationality, suggesting that people weigh the cognitive costs of planning against the potential benefits when deciding whether and how far to plan ahead. Existing work using simulated planning suggests that features like stochastic transitions and reward functions might diminish the utility of model-based planning relative to habit (Kool et al., 2016). Additionally, policy compression has been directly linked to resource-rational concepts of attention allocation (Bhui et al., 2021). Consistent with this account, our resource-rational simulations yielded the same qualitative pattern as our behavioral data: optimal planning sensitivity ($\log \beta$) decreased as stochasticity increased (§A.3). An additional level of resource rationality may also operate in the selection of the model itself. In principle, an unconstrained optimal agent could compute expected future rewards directly. Under realistic computational constraints, however, resource-aware agents may instead favor simpler “good-enough” approximations that achieve favorable reward-effort tradeoffs. From this perspective, participants may first select a tractable planning strategy and then adapt the parameters of that strategy to different levels of stochasticity. Consistent with this interpretation, the best-fitting models in our study relied on simpler determinizing heuristics that treated the environment as effectively deterministic, even in settings where explicit expected-value computations remained computationally tractable in principle.

The technique of simplifying stochastic optimization by using a deterministic surrogate, known as determinization, is widely used in artificial agents and robotics as a “good enough” approximation of optimal actions (Yoon et al., 2007; Saisubramanian et al., 2019; Saisubramanian and Zilbertsein, 2019). A notable example is the FF-Replan algorithm (Yoon et al., 2007), which solves probabilistic planning tasks by assuming all nondeterministic outcomes follow the most likely re-

sult, a method called single-outcome determinization. Despite ignoring stochasticity, FF-Replan outperformed competitors in an international probabilistic planning competition. Since then, determinization principles have been widely applied to machine learning and other probabilistic planning tasks (Yoon et al., 2007; Saisubramanian et al., 2019; Saisubramanian and Zilbertsein, 2019), but to our knowledge this is the first time determinization has been proposed as a model of human planning. However, our findings are broadly aligned with work suggesting that people use simplifying heuristics to adapt to the needs of their environment (Gigerenzer, 2008; Todd and Brighton, 2016; Todd and Gigerenzer, 2012, 2007; Hertwig et al., 2022; Eluchans et al., 2024). We do not intend to suggest that participants universally ignore expected value considerations when simplifying their planning. In our task, these determinizing heuristics were both simple and well-matched to the problem structure, likely because the branching factor at each decision point was small and the overall state space was tractable. In more complex environments, where each choice may open many possible continuations or the state space grows very large, we would expect participants to adopt alternative strategies, such as pruning (Huys et al., 2012; Sezener et al., 2019; van Opheusden et al., 2023), to manage their computational load.

In a similar vein, our work underscores the limitations of traditional lottery tasks in behavioral economics, which typically focus on single-shot decisions where computing the expected value is straightforward and optimal (Allais, 1953; Kahneman and Tversky, 1984; Tversky and Kahneman, 1992; Lopes, 1983; Hertwig and Erev, 2009). However, as our findings suggest, this assumption may break down when individuals face an exponentially expanding decision space. The notion that people might adopt simpler, heuristic strategies in the face of complexity—whether due to branching possibilities or unpredictable outcomes—opens up a promising direction for future research.

Our research opens several avenues for future work. One open direction involves the study of individual differences in strategy selection and adaptation to stochasticity. The connection to resource rationality suggests that participants may themselves engage in multiple levels of strategy simplification, a phenomenon that may be better characterized at the individual level with

richer datasets or adaptive designs. Our current experimental and computational methods do not capture within-individual mixing of strategies; although hybrid strategy use remains plausible, distinguishing such models would likely require task designs and computational frameworks explicitly constructed to dissociate dynamic strategy switching at an individual level.

A distinction is often made between stochasticity (also known as first-order uncertainty, risk, or aleatoric uncertainty) and second-order uncertainty (or epistemic uncertainty) (Bland and Schaefer, 2012; Story et al., 2023; Volz and Gigerenzer, 2012). Our study primarily focuses on aleatoric uncertainty, where the randomness in the environment is irreducible. By contrast, in epistemic uncertainty, learning more about the environment through exploration or simulation can reduce the overall uncertainty of the environment. For example, paradigms like the Mouse-Lab MDP (Callaway et al., 2022, 2017) introduce costs to gathering new information about the environment. We would expect our findings to generalize to cases where it is possible *in principle* to obtain information but doing so is difficult or infeasible; in the example in our discussion of outdated recommendations, it may be possible to obtain more recent recommendations but doing so might be difficult in the moment or prohibitively costly. However, in other epistemically uncertain environments, increasing planning effort or policy sensitivity may become more valuable as the act of planning may reduce uncertainty over outcomes (Daw et al., 2005; Sezener et al., 2019). In these scenarios, we might expect people to expend more cognitive effort as the epistemic uncertainty increases, so long as the marginal value of uncertainty reduction outweighs the marginal cost (Kuperwajs et al., 2024). This may explain the discrepancy in findings between studies that show that certain kinds of uncertainty motivate more myopic behavior, while others promote greater effort (Fenneman and Frankenhuys, 2020; Callaway et al., 2018). It is possible that first-order and second-order uncertainty have competing influences on people’s preferred depth of planning, leading to an optimal effort level that balances between planning in totally uncertain environments (where the world is too stochastic to be predictable) and totally certain ones (where the world is so stable that less effort needs to be exerted to plan ahead) (Kuperwajs et al., 2024).

Another future direction is studying the developmental effects of unpredictable environments on planning effort. There is a growing body of evidence that stochastic environments can have long-lasting effects on cognitive development; studies have shown that growing up in unpredictable environments, particularly with family instability, can lead to reduced response inhibition and attentional control in children (Fields et al., 2021; Ellis et al., 2022; Davis et al., 2017). In these studies, stochasticity is often confounded with valence, making it difficult to tell whether these effects are a result of stochasticity in general or specifically the risk of adverse outcomes. Our work suggests that higher levels of stochasticity, independent of valence, during development may lead to enduring changes in how people approach future-oriented thinking and decision-making. Further research is needed to fully validate this hypothesis and understand the long-term impact of early exposure to stochastic environments.

2.4 Methods

2.4.1 Experiment

Our experiment was preregistered on the Open Science Framework (OSF; <https://osf.io/u6nfvj>), where we documented our hypotheses, experimental design, and planned analyses prior to data collection.

Participants

The Institutional Review Board of New York University approved the experimental protocol. Participants were recruited via Prolific, an online platform that connects eligible individuals with paid psychological studies. We recruited 100 participants per experiment across three experiments, resulting in a total of 300 unique participants. Participants were restricted from enrolling more than once in the study.

Eligible participants needed to be at least 18 years old, fluent in English, and based in the United States. To maintain data quality, participants were automatically excluded if they failed

two out of three attention checks, refreshed or restarted the experiment, or exited before completing the study. Prolific automatically timed out sessions exceeding 140 minutes.

Compensation

Participants received a flat payment of \$8 for completing the task, with the potential to earn additional performance-based bonuses. Before starting, they were informed that bonuses would be determined based on their performance in a randomly selected subset of 15 games. For each selected game, the participant's earned reward was compared to the average expected reward of a randomly acting agent. The difference in points was multiplied by 10 to determine the bonus amount in dollars. If the resulting bonus was negative, no bonus was given.

Procedure

Participants first completed a brief demographic survey before proceeding to the main experiment.

Participants were provided with detailed step-by-step instructions, including diagrams illustrating the task setup. They also completed practice trials at each stochasticity level, which were excluded from the final analysis. Comprehension checks were added to verify their understanding of the task design and basic understanding of probability.

Each session consisted of 10 blocks, two for each stochasticity level. Each block contained 15 games. The stochasticity level remained constant within each block and was explicitly communicated to the participants. At the beginning of each block, participants confirmed the stochasticity level by pressing the corresponding number (1 for the lowest stochasticity and 5 for the highest). To control for order effects, the sequence of stochasticity levels was randomized and mirrored in the second half of the session. The true values of the treasure chests were randomly generated from a uniform distribution ranging from 1 to 9, inclusive. The stochasticity level was presented in multiple formats, as both a proportion and percentage (e.g. "In condition 4 there is a 3 in 4 (75%) chance that a given treasure chest is a mystery chest").

Manipulations

For reliability, we introduced two types of treasure chests: “normal” chests, where the label accurately reflected the number of the points the participant would earn when visiting the chest, and “mystery” chests, where the label was irrelevant and the points were resampled uniformly at random from 1 to 9. The type of chest was revealed only after it was chosen, at which point mystery chests would turn blue, shake, and display their true value (Fig. 2.1B). Participants were not told in advance which treasure chests were “normal” or “mystery” chests. Instead, they were only given the probability that any given treasure chest would be a mystery chest (q_r). We varied this probability across five conditions, $q_r = 0\%$, 25% , 50% , 75% , and 100% . The 100% unreliability condition corresponds to an environment where every chest is a mystery chest, and the label is completely unreliable.

For volatility, we retained the same basic task structure but eliminated the concept of “mystery” chests. Unlike the reliability condition, however, the labels always accurately reflected reward values in the volatility condition. Instead, we introduced stochastic changes over time: at each turn, every chest below the current row had a probability q_v of being resampled, with its new label (and point value) drawn uniformly from 1 to 9 (Fig. 2.1C). Resampled chests were visually indicated by shaking and turning blue, providing real-time feedback about the changing labels. Although participants could not predict which specific chests would change, they were informed of the overall volatility level, q_v , which varied across five conditions: 0% , 25% , 50% , 75% , and 100% . In the 100% volatile condition, every chest below the current row was resampled at each step.

To introduce controllability into the task, we modified the basic setup from the reliability condition. Here, the values of the treasure chests are fixed at the start, with no “mystery” chests, and the labels always reflect the true point value. Instead of changing rewards, stochasticity was introduced in the player’s movement after each choice. With probability q_c , the player’s choice would be flipped—pressing left could result in moving right, and vice versa (Fig. 2.1D). A regular move was indicated by a temporary gray line pointing to the intended chest, followed by a move

to that chest. A flipped move was indicated by an animation where a line pointing to the intended chest would turn red and rotate to the flipped chest. We varied q_c across five conditions: 0%, 12.5%, 25%, 37.5%, and 50%. The 50% condition corresponds with completely random actions, regardless of choice.

Each of these manipulations incorporated a temporary visual component that indicated to the participant whether the treasure chest was being resampled (in the case of reliability or volatility) or whether the player’s choice was flipped (in the case of controllability). Importantly, participants were not required to wait until the animation completed before making their next choice, allowing them to continue acting without waiting for the visual feedback, if they chose to do so. This ensured that the manipulations introduced stochasticity without constraining choice timing.

2.4.2 Mixed-Effects Models

All reported statistics in the results section were derived from either Generalized Linear Mixed-Effects Models (GLMMs) or Linear Mixed-Effects Models (LMMs). Our choice to use GLMMs and LMMs in place of ANOVA (as suggested in our preregistration) reflects our interest in treating stochasticity as a continuous variable varying along a spectrum, rather than solely as a set of discrete experimental conditions. These models allowed us to assess the significance of effects while accounting for individual differences through participant-specific random intercepts and random slopes, which capture variation in how strongly each participant is influenced by the predictors. Mixed effects models were fit using `pymr4` (Jolly, 2018), which provides a Python wrapper around the functions `lmer` and `glmer` from R library `lme4` (Bates et al., 2015). Unless otherwise mentioned, all GLMMs were estimated using Laplace approximation and all LMMs were estimated using restricted maximum likelihood estimation (REML).

In line with best practices for mixed-effects modeling (Barr et al., 2013), we adopted a principled approach to specifying random-effects structures. All models were first fit with the maximal random-effects structure justified by the design, including by-participant random intercepts and random slopes for all predictors of interest. If the maximal model failed to converge or yielded a

singular fit, we simplified the random-effects structure stepwise. First, we removed correlations among random intercepts and slopes while retaining all random slopes. If this uncorrelated slopes model also failed, we further reduced the model to include only random intercepts. Finally, if this also failed to fit, we reduced the model to only include fixed effects. All reported results are based on the most complex model that converged reliably, adhering to best-practice recommendations for mixed-effects analyses.

GLMM: Label Difference vs. Probability of Leftward Choice

We modeled trial-level binary choices using a binomial generalized linear mixed-effects model with a logit link function:

$$\text{logit}(P(y_{ij} = 1)) = \beta_0 + \beta_1 x_{ij} + \beta_2 q_{ij} + \beta_3 (x_{ij} \cdot q_{ij}) + u_{0j} + u_{1j} x_{ij} + u_{2j} q_{ij}$$

where:

- $y_{ij} \in \{0, 1\}$ indicates whether participant j chose the left option on trial i (1 = left, 0 = right),
- x_{ij} is the label difference on that trial,
- q_{ij} represents the condition (e.g., stochasticity level),
- $u_{0j} \sim \mathcal{N}(0, \sigma_{u_0}^2)$ is a participant-specific random intercept,
- $u_{1j} \sim \mathcal{N}(0, \sigma_{u_1}^2)$ is a participant-specific random slope for x_{ij} ,
- $u_{2j} \sim \mathcal{N}(0, \sigma_{u_2}^2)$ is a participant-specific random slope for q_{ij} ,
- β_3 captures the interaction between label difference and condition.

To assess the effect of label difference on choice behavior, we conducted likelihood ratio tests by comparing nested models.

We first tested whether label difference influenced choice by comparing a main-effects model to a reduced model excluding x :

Main effects model: $\text{logit}(P(y_{ij} = 1)) = \beta_0 + \beta_1 x_{ij} + \beta_2 q_{ij} + u_{0j} + u_{1j} x_{ij} + u_{2j} q_{ij}$

Reduced model: $\text{logit}(P(y_{ij} = 1)) = \beta_0 + \beta_2 q_{ij} + u_{0j} + u_{2j} q_{ij}$

A likelihood ratio test assessed whether including x improved model fit:

$$\chi^2(df) = 2 (\log \mathcal{L}_{\text{main}} - \log \mathcal{L}_{\text{reduced}})$$

We report the main effects coefficient β_1 from the main-effects model.

Next, we tested whether the influence of label difference varied across conditions by comparing the main-effects model to a full model with an interaction term:

Full model: $\text{logit}(P(y_{ij} = 1)) = \beta_0 + \beta_1 x_{ij} + \beta_2 q_{ij} + \beta_3 (x_{ij} \cdot q_{ij}) + u_{0j} + u_{1j} x_{ij} + u_{2j} q_{ij}$

Main effects model: $\text{logit}(P(y_{ij} = 1)) = \beta_0 + \beta_1 x_{ij} + \beta_2 q_{ij} + u_{0j} + u_{1j} x_{ij} + u_{2j} q_{ij}$

The interaction effect was tested using:

$$\chi^2(df) = 2 (\log \mathcal{L}_{\text{full}} - \log \mathcal{L}_{\text{main}})$$

We report the estimated interaction coefficient β_3 .

LMM: Stochasticity vs. Points Earned

We modeled points earned per trial using a linear mixed-effects model with a fixed effect of stochasticity level and both a random intercept and a random slope for stochasticity for each participant:

$$y_{ij} = \beta_0 + \beta_1 q_{ij} + u_{0j} + u_{1j} q_{ij} + \varepsilon_{ij}$$

where:

- y_{ij} is the number of points earned by participant j on trial i ,
- q_{ij} is the stochasticity level on trial i ,
- $u_{0j} \sim \mathcal{N}(0, \sigma_{u_0}^2)$ is a participant-specific random intercept,
- $u_{1j} \sim \mathcal{N}(0, \sigma_{u_1}^2)$ is a participant-specific random slope for q_{ij} ,
- $\varepsilon_{ij} \sim \mathcal{N}(0, \sigma_\varepsilon^2)$ is residual error.

The significance of the fixed effect was assessed using a t -test with Satterthwaite-approximated degrees of freedom, and we report the coefficient β_1 .

LMM: Stochasticity vs. Log First-Choice Response Time

We modeled response time preceding the first choice of each game using a linear mixed-effects model with stochasticity level as a fixed effect and both random intercepts and random slopes for stochasticity by participant. To account for skewness in the response time distribution, response times were log-transformed prior to analysis:

$$\log(T_{ij}) = \beta_0 + \beta_1 q_{ij} + u_{0j} + u_{1j} q_{ij} + \varepsilon_{ij}$$

where:

- T_{ij} is the first-choice response time for participant j in game i ,
- q_{ij} is the stochasticity level in that game,
- $u_{0j} \sim \mathcal{N}(0, \sigma_{u_0}^2)$ is a participant-specific random intercept,
- $u_{1j} \sim \mathcal{N}(0, \sigma_{u_1}^2)$ is a participant-specific random slope for q_{ij} ,
- $\varepsilon_{ij} \sim \mathcal{N}(0, \sigma_\varepsilon^2)$ is residual error.

The fixed effect of stochasticity was evaluated using a t -test with Satterthwaite-approximated degrees of freedom, and we report the coefficient β_1 .

LMM: Stochasticity vs. Log Inverse Temperature

To assess whether log inverse temperature varied across levels of stochasticity, we fit a linear mixed-effects model to condition-level log inverse temperature estimates, including random intercepts and slopes for stochasticity:

$$y_{jk} = \beta_0 + \beta_1 q_k + u_{0j} + u_{1j} q_k + \varepsilon_{jk}$$

where:

- y_{jk} is the model-inferred log inverse temperature for participant j in condition k ,
- q_k is the stochasticity level for condition k ,
- $u_{0j} \sim \mathcal{N}(0, \sigma_{u_0}^2)$ is a participant-specific random intercept,
- $u_{1j} \sim \mathcal{N}(0, \sigma_{u_1}^2)$ is a participant-specific random slope for q_k ,
- $\varepsilon_{jk} \sim \mathcal{N}(0, \sigma_\varepsilon^2)$ is residual error.

The effect of stochasticity on log inverse temperature was evaluated with a t -test using Satterthwaite-approximated degrees of freedom, and we report the coefficient β_1 .

2.4.3 Computational Models of Planning Effort

In our task, for every trial, the participant has two possible actions: choose left or choose right. We modeled planning as a function that takes the game state s and the stochasticity level q and returns the probability that the agent will choose left.

Since the agent cannot move back up the tree, the utility of choosing left must be some value function V over the left subtree s_L , excluding the rightmost path (similarly, s_R for choosing right). The probability that the agent chooses left is the softmax over the difference between the value of a leftward choice and the value of a rightward choice. For a binary choice, the softmax function simplifies to a logistic function applied to the utility difference between left and right. Here, β

is an inverse temperature parameter that controls decision stochasticity: higher values of β lead to more deterministic choices, whereas lower values produce more random decision-making. Additionally, we include a lapse parameter λ that is a scalar $0 \leq \lambda \leq 1$, which corresponds to the probability that a participant chooses randomly between leftward and rightward actions.

$$P(a = L | s, q) = 0.5\lambda + (1 - \lambda) \text{Logistic} [\beta(V(s_L, q) - V(s_R, q))]$$

Filter Functions

It is unreasonable to assume that participants consider the entire tree of possible paths from the root. Instead, participants may focus on a subset of nodes, performing a kind of *filtering* to select only the parts of the board they consider in their plans. This raises a key question: *how do participants choose which nodes to attend to?*

We consider three broad classes of filtering strategies—*depth-filtered*, *value-filtered*, and *rank-filtered* models—each capturing a distinct way participants might limit their planning horizon or attentional scope. In all cases, a filter function F selects a subset of the state s based on a parameter f (which may correspond to depth d , value threshold v , or rank cutoff k) and sets the remaining values to zero. Given the filtered state, the probability of choosing the left action is:

$$P(a = L | s, q) = 0.5\lambda + (1 - \lambda) \text{Logistic} [\beta(V(F(s_L, f), q) - V(F(s_R, f), q))],$$

where λ is the lapse rate and β is the inverse temperature. The filter parameter f determines how much information is included in the computation of $V(s, q)$.

- **Depth-filtering.** Participants may plan top-down, considering only a subset of the tree up to depth d from the root. This *planning depth* reflects how many future steps a participant evaluates. For instance, a participant with $d = 3$ considers only the next three possible moves. We define $d \in \{1, 2, 3, 4, 5, 6, 7\}$, where $d = 1$ corresponds to greedy choice behavior.

- **Value-filtering.** Alternatively, participants may focus on high-value regions of the board, filtering out all nodes below a value threshold $\theta = 10 - v$. Values below θ are set to zero, leaving only the highest-valued regions visible for planning. Here, $v \in \{1, 2, \dots, 9\}$ parameterizes the strictness of this threshold.
- **Rank-filtering.** Finally, participants may attend only to the top k values in the board, ignoring the rest. This rank-based filtering uses relative rather than absolute thresholding, with $k \in \{1, 2, \dots, 9\}$.

Value Functions

After filtering, a value function $V(s, q)$ maps the filtered state to a scalar estimate of expected reward. We consider several possible value functions, which differ in how they approximate the utility of the filtered subtree.

- **Expected Value (EV).** The optimal model computes the maximum expected reward along any path in the filtered subtree, taking into account transition stochasticity q . In the reliability and controllability conditions, this can be computed using dynamic programming, but becomes intractable in the volatility condition where the state space grows exponentially (up to 9^{35} possible states). Because such computation is infeasible for both participants and modeling, we omit the EV model from that condition.
- **Path.** In this model, participants behave as if outcomes were deterministic, ignoring stochasticity and evaluating the sum of values along the single best path in the filtered subtree. This differs from the expected value calculation because it does not consider stochasticity when estimating value. Note that this does not return a specific *path* but rather the sum of the values along the highest-valued path in the subtree.
- **Level-Mean.** Here, participants approximate value by averaging rewards at each depth of the tree and summing these level-wise means. This provides a coarse, low-cost summary of tree structure without enumerating paths, serving as a simpler heuristic to the Path model.

- **Max.** Participants may instead focus only on the single most rewarding node in the subtree, taking $V(s, q)$ to be the maximum value of any node. This provides a computationally simple but myopic estimate.
- **Sum.** Finally, participants may sum all node values within the filtered subtree, using the total as a surrogate for its value. Like Max, this is a determinizing heuristic.

Effort Allocation

Beyond the structure of filtering and valuation, we also model how participants allocate cognitive effort across conditions. We consider two mechanisms from the planning literature:

- **Filter Adaptation (FA).** Participants may modulate how much of the state they process by adjusting the filter parameter f (e.g., d , v , or k). FA models therefore include one global lapse rate λ , one global inverse temperature β , and one filter parameter f_q per stochasticity level (5 total), for 7 parameters in total.
- **Policy Compression (PC).** Alternatively, participants may hold the filtering constant but vary their decision sensitivity β across conditions, capturing effort allocation through the sharpness of their policy. PC models include one global lapse rate λ , one global filter parameter f , and one β_q per stochasticity level, again yielding 7 parameters in total.

2.4.4 Model Fitting

We fitted all models by maximizing the likelihood of observed participant choices. Each model contained:

- **Discrete filter parameters:** d for depth-filtering, v for value-filtering, or k for rank-filtering models.
- **Continuous parameters:** the inverse temperature β (optimized in log-space, $-3 \leq \log \beta \leq 3$) and lapse rate λ ($0 \leq \lambda \leq 1$).

Model fitting proceeded in a nested fashion, executing an exhaustive search over discrete parameters within the optimization of continuous model parameters:

1. **Inner optimization:** We exhaustively evaluated all possible discrete filter parameters. For FA models, we identified the combination of f_q values across the five stochasticity levels minimizing negative log-likelihood (NLL). For PC models, we identified the single global f minimizing NLL.
2. **Outer optimization:** Using `scipy.optimize.minimize` (Virtanen et al., 2020) with L-BFGS-B (Zhu et al., 1997; Byrd et al., 1995), we optimized $\log(\beta)$ and λ to minimize NLL given the best discrete parameters. We imposed hard bounds for numerical stability and interpretability ($0 \leq \lambda \leq 1$, $-3 \leq \log(\beta) \leq 3$). FA models used a single β across all stochasticity levels; PC models allowed separate β_q values for each level.

To avoid local minima in our parameter space, we employed a multi-start optimization strategy. The policy compression models each have 6 continuous parameters (β_q for each stochasticity level q and a lapse parameter λ), while the filter adaptation models each have 2 continuous parameters (a global β and lapse λ). For each model we randomly sampled 50 starting points in the continuous parameter space, such that $-3 \leq \log(\beta) \leq 3$ and $10^{-3} \leq \lambda \leq 1$. For each start, we tracked the best-performing parameter set based on the negative log-likelihood. This approach helped ensure robustness against local optima and provided a more comprehensive understanding of the parameter landscape.

We ran this optimization procedure for each of our models using 5-fold cross-validation to capture predictive accuracy. Our choice of 5-fold cross-validation over AIC / BIC as suggested in our pre-registration reflects the fact that (1) some parameters disproportionately affect the behavior of particular model subclasses more than others, and (2) we care about the out-of-sample predictive accuracy of our models when inspecting model fit. For our across-model analysis of goodness-of-fit and model comparison, we used 5-fold CV-NLL, consistent with best practices. For our within-model analysis (i.e. extracting winning parameter values for a given model), we

re-ran the full fitting procedure for the model on the entire dataset to extract the parameter set that achieved the lowest negative-log-likelihood.

3 | Planning in a Strategy Game in Monkeys

3.1 Introduction

Chapter 2 focused on planning under environmental stochasticity, where unpredictable forces affect an agent’s ability to plan. Another important source of uncertainty arises from interactions with other agents, as in adversarial games. Here we examine planning in these settings by investigating strategic choice and deliberation in a nonhuman primate. Flexible decision making in the real world involves challenges that extend far beyond those typically studied in laboratory settings. Whereas many animal studies focus on isolated decisions with immediate outcomes, naturalistic behavior often requires reasoning across long time horizons, integrating information over extended sequences of actions, and acting effectively in novel situations despite sparse and delayed reward feedback. Planning provides a particularly compelling framework for this effort because it involves evaluating future possibilities, adapting to changing circumstances, and coordinating behavior across multiple steps, occupying a promising intersection between cognitive science and systems neuroscience. Recent computational work has developed increasingly sophisticated accounts of prospective behavior, raising the possibility of linking algorithmic accounts of planning to their underlying neural implementations. A central question is how neural systems represent potential future states, evaluate their consequences, and guide behavior in situations that involve prospective, multi-step reasoning.

Studies of instrumental learning in rodents, primates, and humans have implicated the corticostriatal loop as central to reward-driven learning. In humans, the ventral striatum has been shown to track reward-prediction errors (Rutledge et al., 2010) and the dorsal striatum tracks action values in instrumental learning tasks (O’Doherty et al., 2004). In rodents, the striatum has been implicated in monitoring signals for choice updating (Ito and Doya, 2009) and rats with

lesioned dorsomedial striatum showed deficits in forming goal-oriented actions during instrumental lever-pressing tasks (Yin et al., 2005). In nonhuman primates, caudate, putamen, and ventral striatum exhibit high heterogeneity and carry temporally discounted values (Cai et al., 2011), including dissociation between possible and chosen actions (Lau and Glimcher, 2008). Various subregions of the prefrontal cortex have also been shown to support reward-driven learning and arbitration between model-based and model-free decision making: medial PFC encodes action-reward associations supporting goal-directed selection (Matsumoto et al., 2003); dmPFC integrates past actions and outcomes to guide strategy switching (Seo et al., 2014); and dlPFC encodes past choice outcomes (Barracough et al., 2004), prospective value (Kim et al., 2008; Tanaka et al., 2004), action sequences (Averbeck et al., 2006), and preference updating during reversal learning (Bartolo and Averbeck, 2020). These findings have motivated dual-system hypotheses for the PFC and striatum, including distinguishing between habitual and goal-directed behavior (Balleine and Dickinson, 1998), model-based and model-free decision making (Gläscher et al., 2010; Daw et al., 2011), actor-critic systems (O'Reilly and Frank, 2006), and meta-reinforcement learning feedback loops (Wang et al., 2018; Jensen et al., 2024).

These frameworks provide a compelling account of the dynamic role of PFC and striatum in decision making, but they are largely constrained by task complexity. Most tasks, such as the influential two-step task (Daw et al., 2011), rely on shallow reasoning over limited state spaces (van Opheusden and Ma, 2019), limiting their generality to real-world complexities of planning. Recent computational work has begun to close this gap, developing process models for more complex planning tasks (van Opheusden et al., 2023; Snider et al., 2015; Sezener et al., 2019; Jensen et al., 2024). A key feature underlying these tasks is their exploration of vast state spaces, where the combinatorial explosion of possible sequences of actions makes it infeasible to simulate all possible futures. Here, people tend to rely on mental shortcuts, such as using pruning (Huys et al., 2012), heuristics (van Opheusden et al., 2017), and resource-aware early stopping (Sezener et al., 2019; Kuperwajs et al., 2024), demonstrating a wider range of flexible strategies than those observed in simpler tasks. What remains largely unknown is whether similar computational

principles are at play in non-human animals, and what role the corticostriatal loop plays when the state space gets genuinely large.

Our work is aligned with a general shift within the field to study behavior in more ecological contexts (Wise et al., 2023; Allen et al., 2024), which are capable of eliciting a broader range of naturalistic behaviors. Recent work in macaques has demonstrated flexible information-seeking in a Battleship-style task with latent spatial structure (Barack et al., 2023), heuristic simplification during foraging in Pac-Man (Yang et al., 2022), and compositional feature construction in drawing tasks (Tian et al., 2026). Here we build on this growing body of work with a particular focus on complex planning.

A key opportunity and challenge is to incorporate the complexities of modern planning tasks into the study of the corticostriatal loop, pairing the tools of systems neuroscience with cognitive process models of adaptive decision making. Addressing this requires a task of sufficient complexity to mirror the challenges observed in human planning. Here, we train a macaque monkey to play a strategic planning game, Four-in-a-Row, against a computer opponent. The game, previously used to study human planning (van Opheusden et al., 2023), involves 10^{16} possible states, a level of complexity unprecedented in prior animal studies. We recorded gameplay, eye movements, and simultaneous neural activity from dorsomedial prefrontal cortex (DMPFC), dorsolateral prefrontal cortex (DLPFC), and the caudate nucleus. Using computational models of behavior, we show that the monkey relies on abstract, value-ordered task features during decision making. Neurons across PFC and striatum encode abstract task features associated with prospective value, consistent with the idea that corticostriatal circuits support feature-based representations that guide decisions in large state spaces.

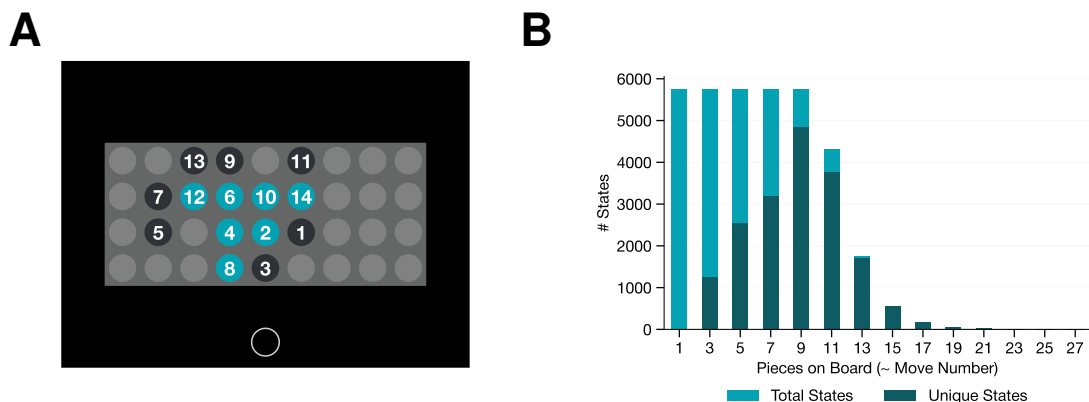


Figure 3.1: **Four-in-a-Row game.** (A) Example game board with monkey (blue) versus computer opponent (gray). Numbers indicate the order of moves. (B) Number of total versus unique states as a function of the number of pieces on the board; total states are shown in light teal, unique states in dark teal.

3.2 Results

3.2.1 Task

To study planning in large state spaces, we trained a rhesus macaque monkey to play Four-in-a-Row (4IAR), a two-player strategy game previously used to investigate human planning (van Opheusden et al., 2023). We selected Four-in-a-Row because it combines several features that make real-world planning challenging, including interaction with an adaptive opponent, the need to act effectively in novel situations, and a state space too large to support rote memorization of all possible states and action sequences. Simultaneously, the game mechanics remain simple enough to be learned without language, making it suitable for nonhuman primates.

Four-in-a-Row is a variant of Tic-Tac-Toe in which players alternate placing pieces on a 4×9 board. The objective is to be the first player to achieve four contiguous pieces horizontally, vertically, or diagonally while preventing the opponent from doing the same. If the board fills completely before either player achieves four in a row, the game ends in a draw. The game contains approximately 10^{16} nonterminal states, making it orders of magnitude larger than the planning tasks typically used in animal studies while remaining computationally tractable (van

Opheusden and Ma, 2019).

The monkey played against a computer opponent whose playing strength was intended to adjust to maintain stable performance throughout training and recording, though for part of the recorded data this adjustment mechanism was itself miscalibrated (§B.1.3). Each game consisted of a sequence of trials, with each trial comprising one computer move followed by one monkey move. At the start of each trial, the monkey moved a joystick-controlled cursor towards a hold target, holding it in place while the computer registered its move. Following a delay, the hold target was extinguished, signaling that the monkey could begin selecting a board position. The monkey confirmed its choice by holding the cursor over the selected location, after which the next trial began. Games continued until either player achieved four in a row or the board filled completely. The monkey was only rewarded at the end of each game, with a juice reward for winning and no additional reward for losing (see §B.1.3 for exceptions).

We excluded opening moves from subsequent analyses (moves with ≤ 2 pieces on the board), as the earliest stages of the game contain few strategic features from which to model decision making. A substantial fraction of the resulting board states were unique, a proportion that increased rapidly as the game progressed (Fig. 3.1B).

3.2.2 Gameplay

The monkey learned to play Four-in-a-Row effectively, completing games that took several moves to resolve (filtered dataset ply: min 10, median 11, mean 11.9, max 27), with a win rate of 50.7%. Note that the win rate is not a particularly informative metric, since the strength of the computer opponent is calibrated to the monkey’s particular level of play; it does, however, illustrate that the monkey is capable of winning the game at a rate much higher than chance. An example 14-ply game is shown in Fig. 3.1A. In the example game, the monkey initially builds a horizontal configuration but is blocked by the opponent on move 5. Rather than continuing to extend a blocked pattern, the monkey begins building a vertical configuration. After this attempt is again blocked on move 9, the monkey begins building out a separate configuration, eventually completing a

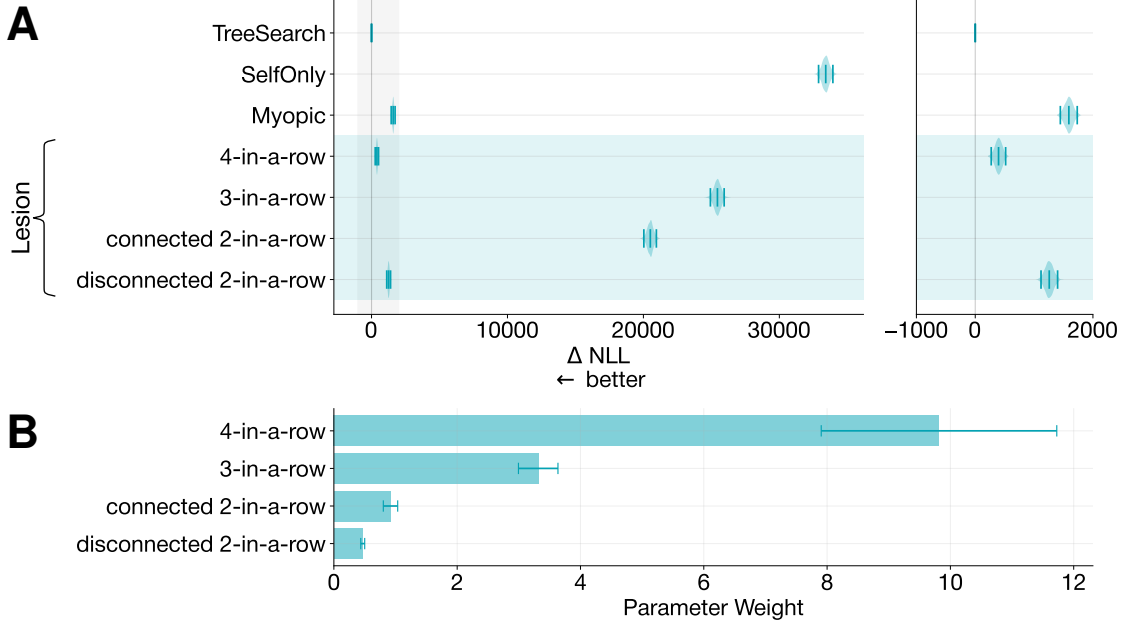


Figure 3.2: **Model comparison.** (A) Bootstrapped ($n = 10^5$) difference in negative log likelihoods between the model and the baseline (Tree Search), with shaded region expanded to the right; error bars represent bootstrap-derived 95% CI. (B) Fitted parameter weights associated with each board feature for the Tree Search model; error bars represent SEM-derived 95% CI.

horizontal 4-in-a-row and winning the game. This example illustrates flexible strategy switching contingent on board context and is consistent with an understanding of the game mechanics.

To model monkey gameplay, we use largely the same process model used to study human gameplay developed in [van Opheusden et al. \(2023\)](#). In this model, planning is operationalized as a best-first search over board states, where each board state is evaluated by a heuristic value function over features of the board. We keep parity with the original model by using the same feature set, 4-in-a-row, 3-in-a-row, connected 2-in-a-row, and disconnected 2-in-a-row, where a feature is only counted if it is unblocked (Fig. 3.14)¹. The value of each board is the sum of the weighted counts of the player’s features, where the counts are weighted by a feature weight w , which is fitted to the behavior. We consider the opponent’s feature to be opposite in value to the player’s, except instead of weighing the opponent’s pieces by $-w$, we weigh them by $-\eta \cdot w$,

¹Formally, an n -in-a-row feature is defined as an n -filled 4-in-a-row configuration, where the remainder of the positions are unoccupied. Features are enumerated by count, and the same set of occupied positions can be part of multiple instances of a feature; take the example of a horizontal 3-in-a-row unblocked on both sides; this constitutes two distinct 3-in-a-row features, one on the left and one on the right.

where η is the opponent scale. We also deploy a center-weight c , which weighs each position by its distance from the center of the board, useful to capture the heuristic that playing closer to the center affords more opportunities to win; a stopping probability γ which controls the depth of search, a pruning threshold τ which controls what positions below a threshold from the best valued option continue to be considered, and a lapse rate λ giving the probability of a random move. While the original model had an additional parameter to capture feature dropping δ , here we exclude it from the search and fix it at 0, since recoverability of the remaining parameters degraded once δ grew large (see §B.2.8 for details).

Importantly, this model class is dynamic enough to capture a wide range of flexible planning behaviors ranging from unsophisticated to deep tree search, and it has been validated on human data. We refer to the full model as the **Tree Search** model. Tuning specific parameters of this model allows us to directly compare hypotheses about the sophistication of the monkey’s deliberation. For example, setting the stopping probability $\gamma = 1$, we recover the **Myopic** model, which considers only the immediate next move rather than planning further ahead. Additionally, setting the opponent weight to $\eta = 0$ and keeping $\gamma = 1$ gives us the **Self-Only** model, a variant of the Myopic model that considers only the impact of the monkey’s own features². Finally, lesioning any feature weight w models the possibility that the monkey systematically ignored that feature when evaluating the board state.

In general, optimization over tree search is compute-intensive. To make fitting over this dataset tractable, we split the full set of ≥ 9 -ply games into 5 time-ordered cohorts (for more detail, see §3.4.2). For each cohort, we conducted 3-fold cross-validation at the level of games and fitted the models with Bayesian Adaptive Direct Search (BADs) (Acerbi and Ma, 2017) using Inverse Binomial Sampling (IBS) (van Opheusden et al., 2020) to produce an unbiased estimate of negative log likelihood for optimization, with 5 randomly initialized starts within the bounds to reduce the odds of getting stuck in local minima.

²Note that there is no satisfying equivalent for a self-only Tree Search model, because the Tree Search already requires perspective switching from the player to the opponent; setting η to 0 in that case corresponds with a player *simulating* both itself and its opponent as if neither cared about the other’s feature set, which we did not consider to be plausible enough to test in this iteration.

We compared our models using bootstrapped negative log-likelihoods (NLL) on the held-out cross-validated folds. Our results suggest that the full Tree Search model significantly outperforms the Myopic (NLL difference from Tree Search = 1,591.0, 95% CI = [1,446.2, 1,733.4]), Self-Only (NLL difference = 33,413.3, 95% CI = [32,887.0, 33,939.3]), and all lesioned model variants, suggesting that the monkey does indeed engage in deeper deliberative processes than simple myopic decision making and that it considers diverse game-relevant features (Fig. 3.2A). When we look under the hood at the relative weighting of the board features, we see a sensible ordering (Fig. 3.2B): 4-in-a-row carries the largest weight, followed by 3-in-a-row, then 2-in-a-row (connected) and 2-in-a-row (disconnected), though, as we discuss below, the 4-in-a-row weight is the least precisely estimated of the four and plays a special role. We read the gap between the two 2-in-a-row features as reflecting feature fragility: a disconnected 2-in-a-row is more brittle than a connected one, since a single block in the gap invalidates the whole configuration, whereas a block on either side of a connected pair may still leave room to build out toward a win on the other end. The stopping probability is approximately 0.54 (mean = 0.54, 95% CI = [0.52, 0.56]), corresponding to roughly 1.9 iterations of search in expectation (each iteration extending the principal variation by about one ply); this suggests that the monkey searches fairly shallowly, but does indeed perform search, consistent with the Tree Search model’s significant advantage over the Myopic baseline.

One curious note is that while 4-in-a-row carries the highest weight, the Lesion 4-in-a-row model is the best-performing of the feature lesions; removing it costs the least held-out NLL of any feature (NLL difference from Tree Search = 396.0, 95% CI = [271.6, 516.5], versus 1,257.8 for the next-cheapest lesion, disconnected 2-in-a-row, and 25,446.6 for the most costly, 3-in-a-row). The reason is a combination of the statistics of the data and a subtle feature of the Tree Search’s design. First, 3-in-a-row and 2-in-a-row configurations occur in far more positions than 4-in-a-rows, so lesioning them corrupts the evaluation of many more states, whereas the 4-in-a-row feature only matters in the comparatively rare positions where a win is actually within reach. Second, and more importantly, in the implementation of the Tree Search a winning node

is assigned the value $+\infty$ and a losing node $-\infty$. One might therefore ask why the 4-in-a-row weight has any effect at all—and the answer is that it operates not during evaluation but during *pruning*. Before a node is expanded, the search discards every candidate move whose heuristic value falls more than a threshold below the best move’s, and only the surviving subset is searched. The 4-in-a-row weight inflates the heuristic value of a winning move so that it clears this cutoff and is actually expanded, at which point its terminal $+\infty$ is realized. Without the 4-in-a-row weight, a winning move might score below the cutoff and be pruned before it is ever evaluated, leading to a missed win. The 4-in-a-row feature thus shapes the tree’s consideration set *indirectly*, by determining whether win-completing moves survive pruning, rather than by setting the value of the won position itself. These effects are consistent with results reported by van Opheusden (2019), who also showed that lesioning 4-in-a-row had small effects on model performance.

We validated the fit of our best models by showing that they could reproduce key behavioral patterns in the data, including distance to the center, proximity of moves to the monkey’s own and the opponent’s pieces, and the size of groups formed at each move. While all models qualitatively captured these trends, the Tree Search and Myopic models most closely matched the empirical data (Fig. 3.3).

We find, therefore, that M1 engages in deeper-than-myopic deliberation, and that it considers a well-ordered set of weights over the feature set of the board that obeys the relative importance of those features in the game.

3.2.3 Gaze

While the choice of individual moves provides one window onto the monkey’s planning process, the eye movements during gameplay may offer additional and independent evidence of the deliberative processes at work. For example, rodents navigating mazes have been shown to dart their eyes back and forth when choosing which direction to move, a phenomenon known as vicarious trial and error (Redish, 2016). Empirical evidence suggests that eye movements during choice reflect ongoing deliberation and information sampling (Callaway et al., 2021). Studies have also

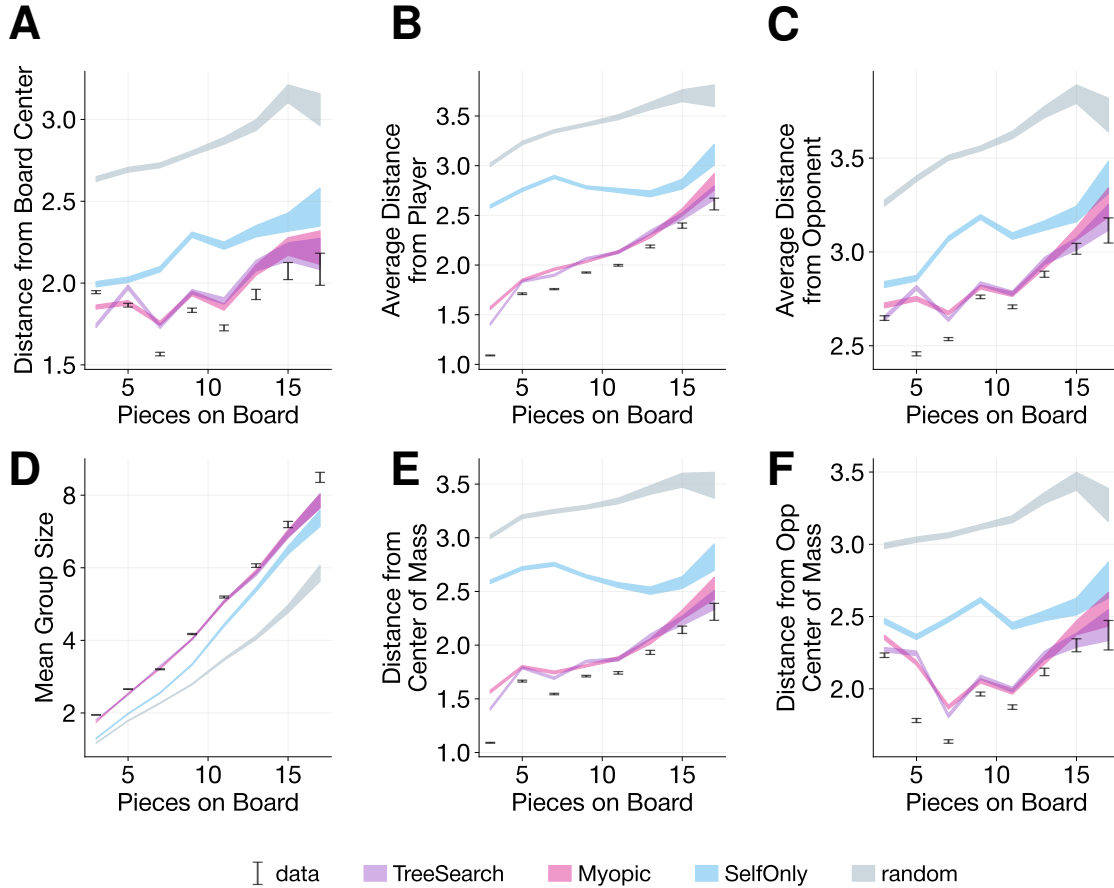


Figure 3.3: **Model simulations overlaid with data.** All simulation plots are relative to the move number (number of pieces on the board); empirical data is shown using error bars, model simulations are shown using shaded regions representing ± 1 SEM. (A) Distance between the chosen position and the center of the board. (B–C) Average distance of the chosen position from the player and opponent. (D) Mean group size of the player’s pieces. (E–F) Distance of the chosen position from the player and opponent center of mass.

demonstrated a link between gaze and computation of value (Krajbich et al., 2010). This has been demonstrated specifically in the Four-in-a-Row task, where gaze appears to trace the principal variation of the tree during gameplay (van Opheusden et al., 2023) and may be used in sequential memory encoding (Huang et al., 2023). In this section we explore the possibility of gaze reflecting a richer trace of deliberation.

Gaze Time Course

Before testing specific hypotheses about gaze, we first establish what role gaze plays in the decision. For each trial, we defined the gaze period to be the time between the moment the computer registers an action and the moment the monkey first places its piece (reach offset). During this period, the eye movements were recorded as x- and y-coordinates corresponding to pixels on the screen. We analyzed the eye fixations, periods where the velocity fell below a threshold, and classified them based on where they fell, either on the eventual move that the monkey chose, on one of the board positions, on the hold cue, or elsewhere (and if elsewhere, whether it was on- or off-screen).

We found that the eye movements reflected distinct patterns of behavior consistent with different events in the trial. From the start of the trial to the moment that the computer made its move, for example, the monkey spent much of its time fixating on the hold cue, consistent with the fact that it needed to keep its cursor steady there while the move was being executed. From the moment the computer moved to the reach onset, the monkey spent more time looking at the positions of the board; in the final phase between the reach onset and the actual moment the piece was placed, the monkey spent over 40% of its time looking at its eventual move position (Fig. 3.4).

Clearly, gaze plays a role in guiding movement towards the final choice. First, we wanted to see how this effect evolved over time, and second whether the gaze leading up to the choice depended on the kinds of features that were available in each state, including 4-in-a-row, 3-in-a-row, and 2-in-a-row. To answer the first, we centered the eye movements around the moment the

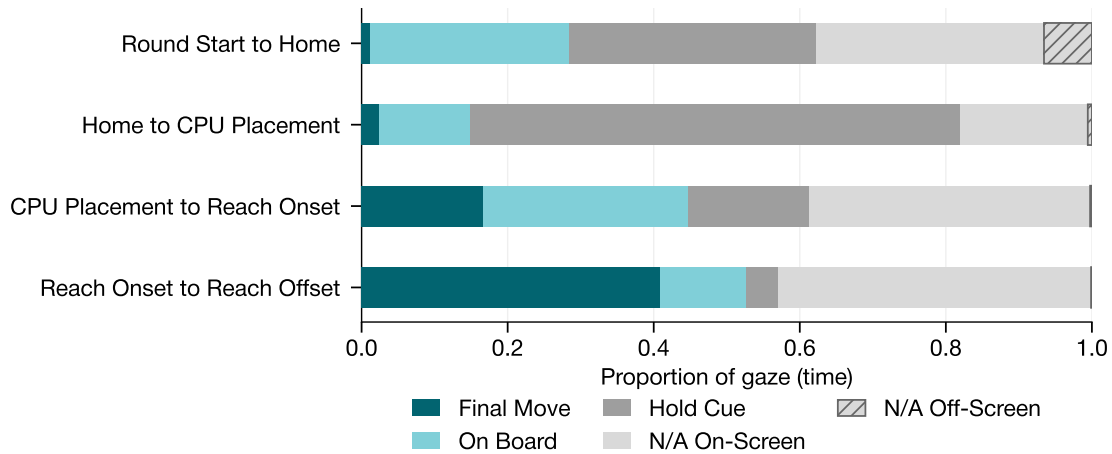


Figure 3.4: **Gaze time course.** Proportion of time spent fixating on the hold cue, the board, the eventual move, and elsewhere (on-screen and off-screen) across the trial. Time is binned by different phases of the trial, from the move start to the reach offset.

monkey made its move and examined the probability that the monkey was looking at the move it would eventually make at different time points. We found that gaze ramped up steadily from 1.5s prior to piece placement, converging maximally on the move position near the time the monkey placed its piece. This convergence of eye movements on the final move location suggests strongly that gaze is involved in choice, particularly in the motor action of directing the cursor towards the intended move.

We were interested in whether this time course had a feature-specific component, that is, whether the speed with which the monkey fixated a given configuration depended on which configurations were present. We were particularly interested in looking at the probability that, for a given feature like 4-in-a-row, 3-in-a-row, or 2-in-a-row, the monkey would look at any position belonging to that feature when it was simultaneously available and the best. That is, when did the monkey look at a 2IAR configuration when it was available and there were no 3IAR or 4IAR opportunities present? A feature was considered looked at if the monkey's gaze fell on any position associated with that feature (where there were multiple features present, the union of the positions was taken). To answer this question, we split the trials up into trials where the

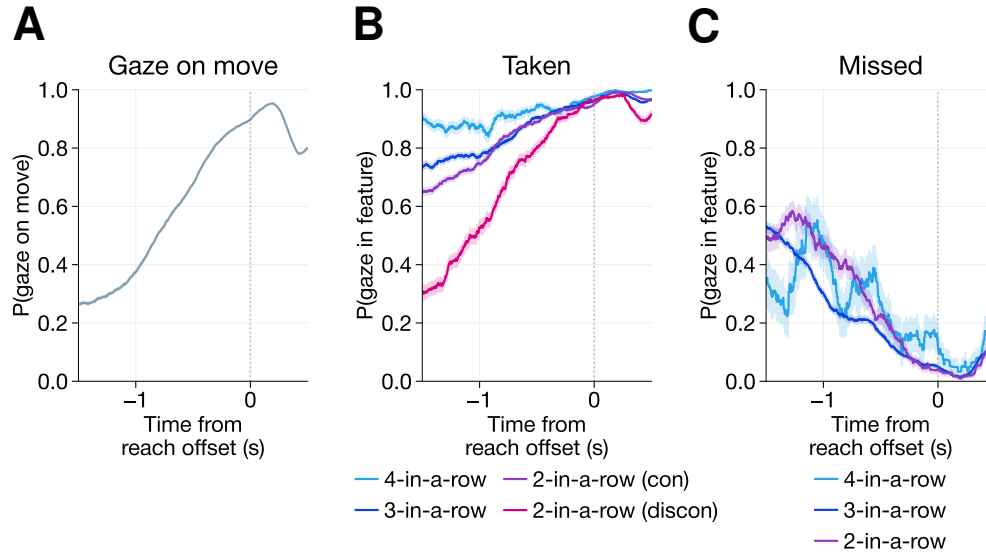


Figure 3.5: **Gaze aligned to move.** Plots are aligned to reach offset (movement completion). (A) Probability that the monkey fixates on the position where it eventually moves. (B) Probability that the monkey fixates on any position in a given feature (see legend) given that the feature was taken. (C) Same as in (B), but given that the feature was available but missed. Shaded regions represent ± 1 SEM.

monkey eventually *took* the feature of interest or missed it³.

On trials corresponding with taken features, the monkey fixated more on feature-relevant positions closer to when it placed its piece. This timing was feature-dependent, with higher-valued features converging earlier on the eventual position, in order of feature importance. This ramping pattern was notably absent in trials where the feature was missed, showing that gaze is tied to the monkey's choices. In sum, gaze is strongly related to choice behavior, and its timing is sensitive to board features. However, this analysis cannot distinguish between gaze related to information gathering, deliberation, or the final motor movement towards the final choice.

Zero-Shot Predictions

To understand whether gaze reflected deliberation beyond motor guidance, we needed to isolate a phase of each trial that would remove the motor-related process from consideration. As a first

³For taken pieces, we knew whether the monkey chose to complete a connected or disconnected 2IAR, and could then retrospectively infer whether it had been looking at that type of pattern. For missed 2IARs, no such inference was possible, since we could not know which move the monkey would have made had it noticed the available configuration.

pass, we decided to focus on the period of time between the computer move and the reach onset, during which the monkey was free to look around but had not yet moved its joystick off of the hold cue. Note that this does not completely remove the motor-related processes; for example, this period still includes pre-movement preparatory eye movements. The segmented period only removes the eye movements that co-occur with movement of the joystick itself.

Our goal was to link this period of eye movements to the underlying deliberative process. To that end, we wanted to see whether our behavioral models, fitted only on behavior, could also explain the variation in gaze between the computer's move and the reach onset. To do this, we used several predictors to relate our behavioral models to the observed gaze of the monkey on each trial. We selected the following potential predictors:

- **Principal Variation (PV).** This corresponds to a binary map over a board where a 1 represents whether or not it belongs on the best possible path through the tree. This is a valid measure only for Tree Search; in the Myopic models it corresponds exactly to the predicted move.
- **Predicted Move.** This corresponds to a binary one-hot map over the board which is 1 only where the model predicts the monkey will move.
- **Tree Nodes.** This corresponds to a uniform map over every position the search expanded, capturing the breadth of the tree rather than its single best line. Like the principal variation, this is defined only for Tree Search.
- **Feature Map.** This corresponds to a weighted sum, performed by weighing the positions associated with the feature of each board with the learned weight of that feature. In other words, for each possible 4-in-a-row configuration on the board, each position within that 4-in-a-row was weighted by the learned weight of the active feature in that configuration, which was then summed across all configurations. Intuitively, this captures something like the saliency or attention given to this feature set.

Because search is noisy, we run each predictor 100 times and sum the resulting maps into a predicted distribution over board positions. We then compare the predicted gaze distribution with the actual gaze distribution without any additional fitting (we add a small epsilon, $\epsilon = 10^{-3}$, for positions where the value is exactly 0). Because neither the principal variation nor the predicted move would ever predict moving on a presently occupied position, we remove the occupied positions from consideration by masking them out. Each predictor corresponds to a different mechanistic prediction about how the deliberation process influences the gaze: in the principal variation, the gaze follows the best path down the tree; in the predicted move, the gaze simply guides the best move; in the feature map, the gaze follows a more attentional role.

We directly compare the predicted and true gaze distributions using the cross-trial NLL, which weights trials by their number of fixations—so that fixation-rich trials, which carry denser information, count for more, rather than treating every trial equally regardless of how many fixations it contributes. We compare our four classes of contenders across two different models, the full Tree Search and the Myopic model. We find that the Tree Search principal variation is the best predictor of gaze, with the second-best being the Tree Search predicted move; somewhat unsurprising since the predicted move is simply the first segment of the principal variation and therefore highly collinear (Fig. 3.6). As a trial-wise validation of this result—which instead treats every trial equally via the KL divergence between predicted and true distributions—we find consistent results (§B.3.2).

The results from this analysis demonstrate that the Tree Search model provides the best zero-shot prediction for the gaze between the computer’s move and the reach onset. This suggests that the Tree Search model reproduces gaze trends well, even when we segment for phases of the trial that exclude direct motor action. However, this analysis explicitly avoids positions that are occupied, which may also provide rich information related to the deliberative process. Additionally, this does not rule out the possibility that the gaze during this period is simply related to preparatory motor activity.

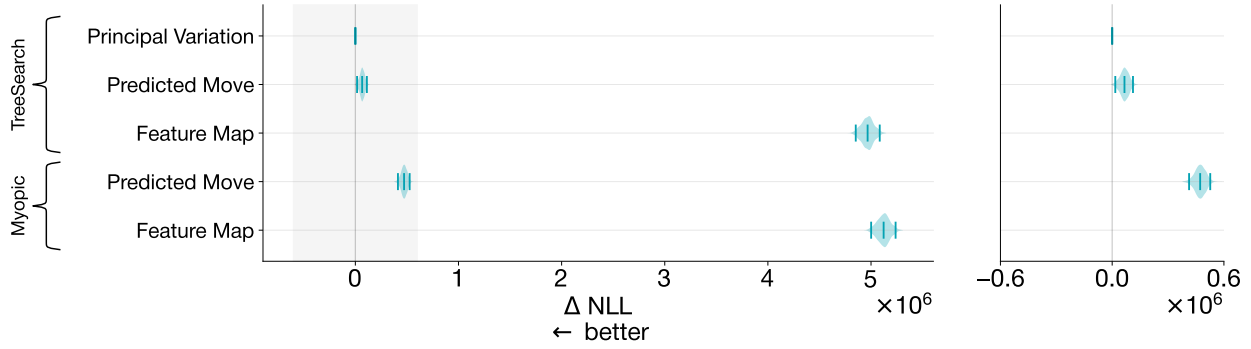


Figure 3.6: **Zero-shot gaze NLL.** Difference in negative log likelihood (weighted by the number of total fixations) between the predicted gaze distribution and the true gaze distribution for each zero-shot predictor, across two model classes. Note that occupied positions were not considered in this analysis. Error bars represent bootstrap-derived 95% CI.

Regression Models of Gaze

To tackle the first of these shortcomings, we combine our behavior-predicted-gaze features into a model to predict the number of fixations on each position, across trials. To do this, we model a linear regression where we have weights on the predicted move and feature map for both models and the principal variation as a feature for the Tree Search model. This time, rather than using only the unoccupied positions, we also included the occupied positions, allowing the feature map to soak up any variance on the occupied positions.

To conduct a head-to-head comparison of our models, we use the paired difference in R-squared across linear regressions, using 5-fold cross-validation. We use leave-one-out lesions to measure the variance captured by each predictor (Fig. 3.7). We find that, just as before, the Tree Search composite model outperforms the Myopic model, and that this difference is significant when compared to the Myopic model (paired $\Delta R^2 = 0.0052$, 95% CI [0.0045, 0.0064]). This suggests that the Tree Search model captures the gaze patterns (including those on occupied squares) better than the Myopic model but does not rule out the possibility of the gaze being related to motor preparation rather than deliberation.

To test the latter, we added nuisance variables to our regression, including a flat binary map over the monkey's pieces, a map over the opponent's pieces, and a one-hot map of the monkey's

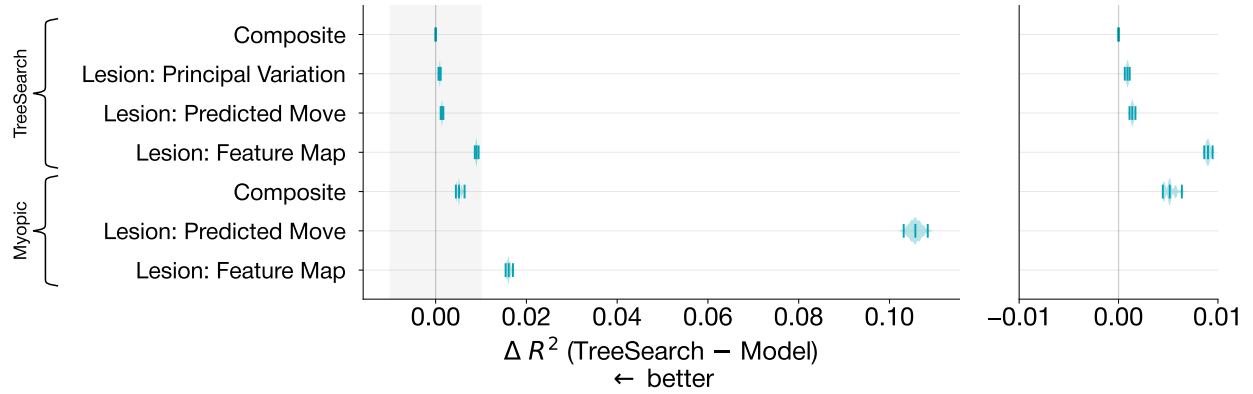


Figure 3.7: **Gaze regression.** Paired 5-fold CV R-squared differences compared to Composite Tree Search (left is better), using linear regression to predict the number of fixations with model-derived features. Composite regression involves all features, compared with leave-one-out lesions. Shaded region expanded to the right. Error bars represent bootstrapped 95% CI.

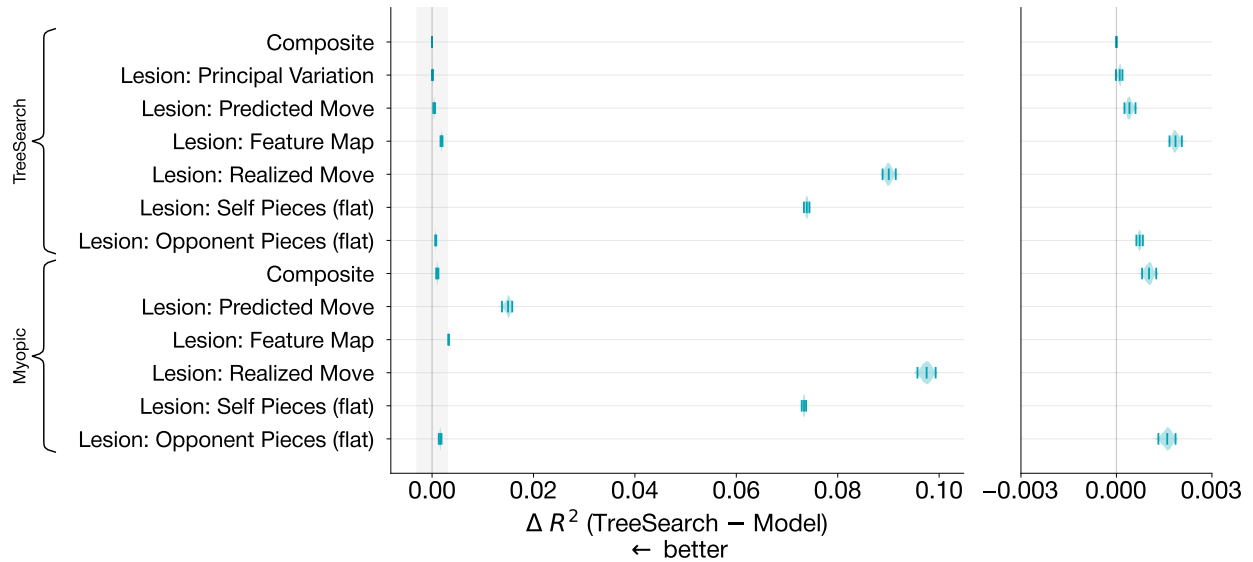


Figure 3.8: **Gaze regression with controls.** Paired 5-fold CV R-squared differences compared to Composite Tree Search (left is better) including nuisance variables (self pieces, opponent pieces, realized move), across models. Shaded region expanded to the right. Error bars represent bootstrapped 95% CI.

realized move, the move it actually chose (Fig. 3.8). The resulting R-squared of the Tree Search Composite model was $R^2 = 0.297$, 95% CI [0.295, 0.299] (for absolute R-squared comparison see §B.3.3). This allows us to isolate the difference in variance explained attributable beyond the existing board features and the intent to move to a particular location. Even with the introduction of nuisance variables, the Tree Search composite model outperforms the Myopic composite model (paired $\Delta R^2 = 0.0010$, 95% CI [0.0008, 0.0013]), suggesting that the readout of the chosen move alone cannot fully explain the difference in model performance. It is worth noting, however, that the removal of the principal variation feature is not significant (leave-one-out $\Delta R^2 = 0.00010$, 95% CI [-0.00002, 0.00019]), suggesting that much of the gaze reflects choice rather than the underlying search process. Consistent with this interpretation, the most important feature by far in either composite model is the realized move itself. This suggests that the gaze is largely related to the immediate chosen move and motor preparation. More analysis is needed to disentangle deliberative versus preparatory motor-related gaze dynamics.

3.2.4 Neural Activity

Gameplay and gaze both point to deeper-than-myopic deliberation, but neither can speak directly to the underlying implementation. Neural activity offers a more direct window: we first look at single neurons for hints of feature- and value-sensitive firing, then ask at the population level whether firing tracks model-derived value, whether that tracking survives controlling for confounds, and finally how the Tree Search and Myopic models fare head-to-head.

Neurons were recorded from three brain regions: DMPFC, DLPFC, and caudate (Fig. 3.9A), all implicated in goal-directed behavior. Each session involved simultaneous recordings of two of the three regions during active gameplay. Neural activity was recorded using NeuroNexus probes and spike-sorted with MountainSort5 followed by manual verification. We binned the spikes into 50 ms increments and investigated the firing patterns in a time window surrounding particular events of the trial.

To illustrate the diversity of feature encoding and temporal dynamics at the single-neuron

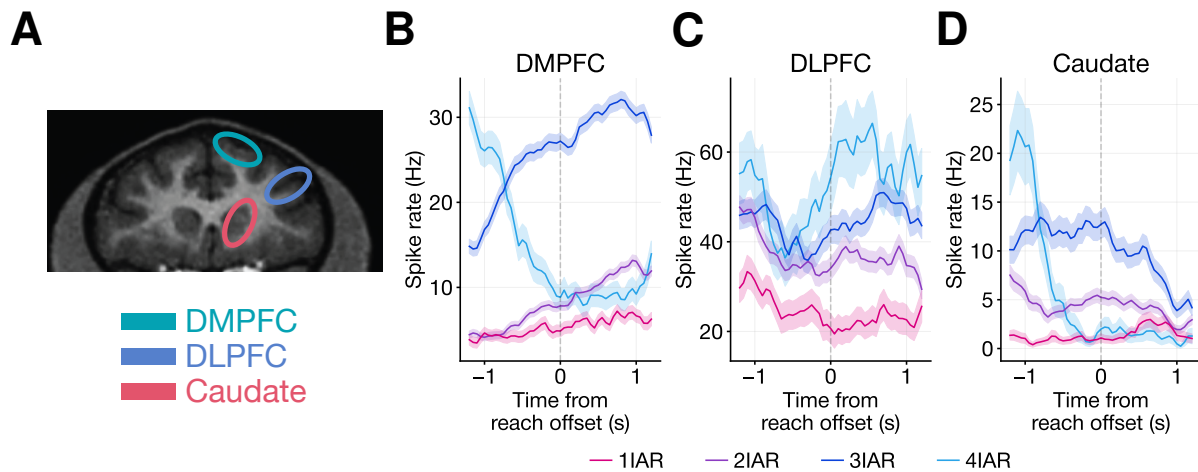


Figure 3.9: **Recording sites and spike-triggered averages.** (A) Anatomical targeting of brain regions (DMPFC, DLPFC, caudate) conducted using MRI. (B–D) Average activity across trials of an example neuron in each brain region, partitioned by the player’s maximum connection length (1IAR, 2IAR, 3IAR, 4IAR) and aligned to the moment the monkey selects a position: (B) DMPFC, (C) DLPFC, (D) caudate. Shaded regions represent ± 1 SEM.

level, we examined neurons that were particularly selective for the monkey’s maximum connection length, centered on the monkey’s reach offset (maxconn; Fig. 3.9B–D). Trials were partitioned by the monkey’s maximum connection length (4IAR, 3IAR, 2IAR, or 1IAR), and neural activity was averaged within each partition. The example DMPFC neuron exhibits ramping activity over the course of the trial, with higher firing when a 3IAR is formed and reduced activity once a 4IAR becomes certain (Fig. 3.9B), a pattern that could be consistent with sensitivity to prospective state values. By contrast, the DLPFC neuron shows graded activity that peaks approximately half a second after the monkey selects a position, with higher firing rates for greater connection values, consistent with modulation by the value of the chosen action (Fig. 3.9C). Finally, the caudate neuron displays activity whose timing and magnitude vary with feature value, with higher-valued features associated with stronger and earlier peak activity relative to movement onset; this temporal profile echoes the value-dependent ramping observed in the gaze results above (Fig. 3.9D). Notably, these neural traces are hand-picked and cannot be taken as representative of region-level dynamics; they simply serve as illustrative examples of the heterogeneity of encoding strategies across brain regions.

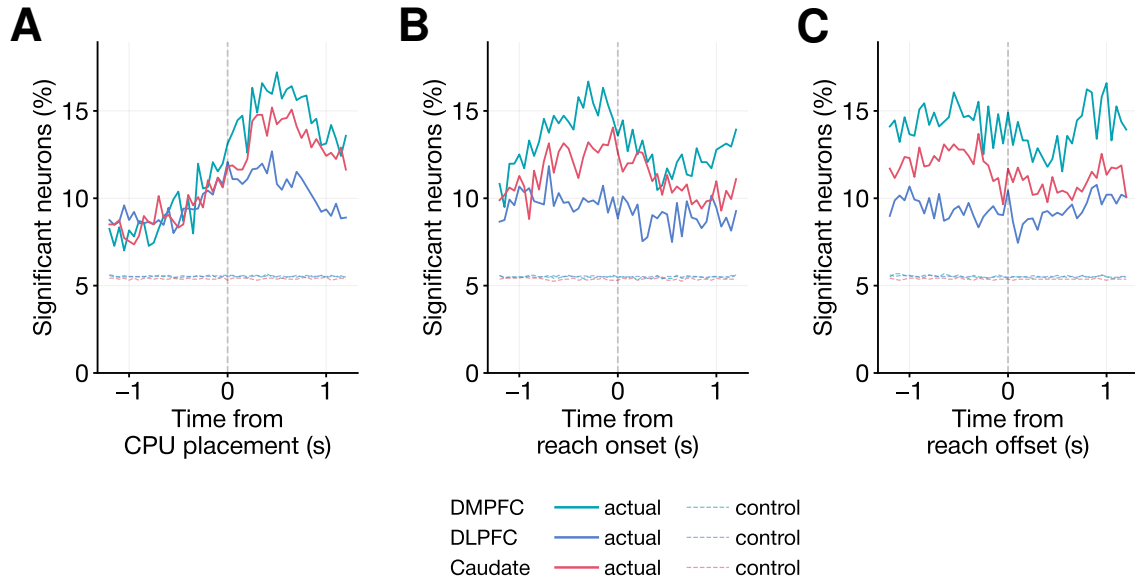


Figure 3.10: **Population-level coding of state value.** Proportion of neurons in each brain region that are significantly modulated by the value of the state, as predicted by the Tree Search model, aligned to the computer move (A), the reach onset (B), and reach offset (C). The dashed line represents the shuffled control (the null-hypothesis reference point).

Single-neuron spike-triggered averages cannot tell us alone what the population encodes, and the maximum connection length is a coarse proxy for the value of the board at any given point in time. To more directly probe population-level encoding, we computed the value of the position the monkey faced—the board after the computer’s move but before the monkey’s own—taken as the root-node value under Tree Search, averaged over 100 simulated searches for each trial. In each time bin at a given alignment, we regressed firing rate on this Tree Search value.

As a preliminary model of neural tuning to state value, we use linear regression as a model of how state value is encoded in neural firing rates. We chose linear regression because it lends an interpretable measure of variance explained for model comparison and allows for straightforward statistical analysis of nested models like Tree Search and Myopic. We note, however, that these models make often unrealistic assumptions about the nature of tuning, namely that they are linear and relatively invariant to temporal shifts; we address these limitations in the Discussion.

A neuron was said to be *significantly modulated* by the value of the state if the p -value of the regression slope was less than $p = 0.05$. We then looked, across all brain regions and trials, at the

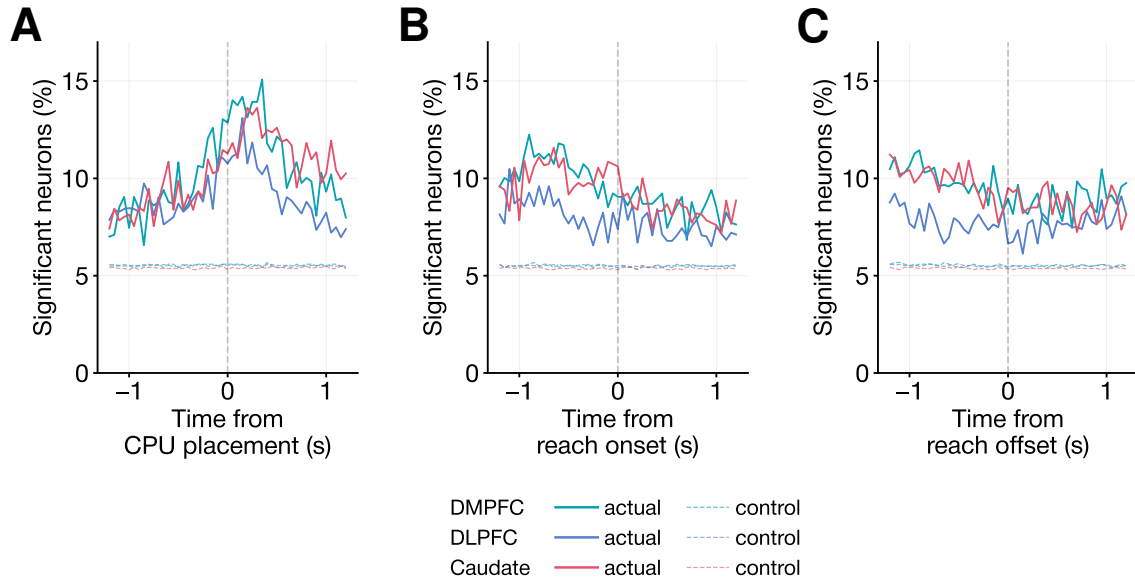


Figure 3.11: **Population-level coding of Tree Search beyond Myopic.** Same as Fig. 3.10, but for the proportion of neurons whose fit significantly improves when Tree Search value is added to a model already containing Myopic value (F-test). The dashed line represents shuffled control.

proportion of neurons in each brain region that were significantly modulated by state value. We also conducted a shuffled control to establish the actual chance level (5.4–5.5% across regions). We repeated this process by centering the neural activity around three different events: the moment the computer made its move, the moment the reach began (reach onset), and the moment the monkey made its move (reach offset).

Our results show significant state-value encoding in DMPFC, DLPFC, and caudate that falls well above chance for all three events in the trial. Notably, the proportion of significant neurons rises just after the computer makes its move, consistent with the idea that the monkey begins deliberating about its options only once the computer’s move is known. Across regions the proportion of state-value-sensitive neurons was ordered DMPFC > caudate > DLPFC at every event, peaking in DMPFC (e.g. 17.2% near when the computer places its piece versus 15.2% in caudate and 12.7% in DLPFC; Fig. 3.10).

We have shown so far that a substantial proportion of neurons in DMPFC, DLPFC, and caudate are significantly modulated by Tree Search value. However, the Tree Search value is highly correlated with low-level board properties, such as feature counts, ply, and maximum connec-

tion length, none of which are unique to Tree Search alone. Regressing all of these features out one-by-one would require enumerating a large number of hand-crafted features which could be incomplete. Instead, we take a more principled approach by using the Myopic model as a baseline to absorb the effects of shared confounding features; since the Myopic model sees the same boards as the Tree Search does and is also fit to the data, it provides a convenient baseline to control for confounding effects. The two value predictions are highly correlated (Pearson $r = 0.893$). We therefore use it not as a rival process model but as a nuisance term in a nested-model comparison: for each neuron and time bin we compare a reduced model (firing rate on Myopic value alone) against a full model that adds Tree Search value, and test by an F-test whether the addition significantly improves the fit. By asking what Tree Search value explains beyond Myopic value, we control for this strategy-independent structure and isolate the component tied specifically to deeper search. This is a statistical control, not a claim that any neuron jointly implements both models; the Myopic value is a stand-in for confounds, not a second computation.

We find that a proportion well above chance in all three regions shows this Tree-Search-specific effect, providing evidence that information beyond coarse board features is encoded (Fig. 3.11). Tellingly, this proportion peaked right around the moment when the computer made its move, which is where we might expect deliberation to actually occur.

The prior analyses suggest that the population encodes Tree Search value, and that it does so even when controlling for nuisance board variables. However, neither directly compares the Myopic and Tree Search models head-to-head. To do that, we measured, for each trial, a single firing rate averaged over the duration of the (variable-length) period between the computer's move and the monkey's reach offset, and, for each neuron, compared across trials the variance explained by Tree Search value against Myopic value ($\Delta R^2 = R_{TS}^2 - R_M^2$). We would expect the effect to be fairly small in magnitude, since the two value predictions are highly correlated (Pearson $r = 0.893$). Even so, Tree Search emerges as the dominant model in all three regions: the mean paired R^2 difference is significantly above 0 (DMPFC: 0.0035, 95% CI [0.0030, 0.0040]; DLPFC: 0.0019, [0.0017, 0.0021]; caudate: 0.0029, [0.0026, 0.0033]; Fig. 3.12A).

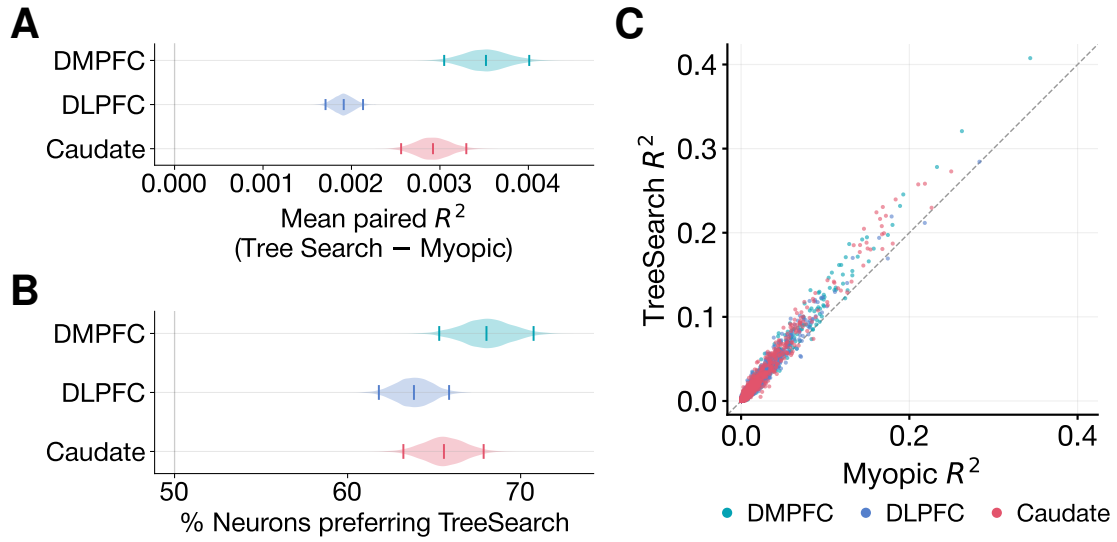


Figure 3.12: **Neural regression model comparison.** (A) Mean paired difference in R^2 (Tree Search - Myopic) per region; dashed line marks the null of no difference. (B) Percent of neurons for which Tree Search explains more variance than Myopic ($\Delta R^2 > 0$) per region; the dashed line marks the null of 50%. Error bars represent bootstrap-derived 95% CI. (C) Per-neuron R^2 (Myopic vs Tree Search), colored by region, with the unity line; points above the line favor Tree Search.

This effect is not simply driven by a small subset of neurons; the proportion of neurons for which Tree Search explains more variance than Myopic is also significantly above chance in every region (DMPFC: 68.0%, 95% CI [65.3%, 70.7%]; DLPFC: 63.8%, [61.8%, 65.9%]; caudate: 65.6%, [63.2%, 67.9%]), showing the same DMPFC > caudate > DLPFC ordering found in the state-value analysis above (Fig. 3.12B). Plotting each neuron's two R^2 values directly against one another makes the same asymmetry visible at a glance: across all three regions, points sit predominantly above the unity line, i.e. Tree Search accounts for more variance than Myopic on a neuron-by-neuron basis, not merely on average (Fig. 3.12C).

Importantly, this finding is robust to the resolution with which we aggregate firing rates. Here, our model comparison analysis is computed over the whole deliberation period rather than resolving effects across 50 ms bins. A time-resolved version of the same comparison confirms that both statistics clear their null at every one of the 49 time bins in every region and event tested (see §B.4.2).

Taken together, the population-level neural analyses suggest that regions in PFC and striatum

track Tree Search value, that this tracking persists after controlling for the Myopic model, and that Tree Search explains firing-rate variance better than Myopic when the two are compared directly, with a significantly above-chance proportion of neurons preferring Tree Search value over Myopic value. More work is needed to characterize the unique contributions of each area and their reciprocal interactions.

3.3 Discussion

The complexity of Four-in-a-Row provides a compelling opportunity to study nonhuman primate decision making in a large state space. Our results suggest that the monkey builds abstract, sensibly ordered representations of strategic game features and that these feature representations evolve dynamically over the course of a trial. Using a combination of behavioral modeling, eye movements, and neural activity, we found a largely coherent picture of how the monkey represents and uses flexible decision variables to guide behavior.

Behaviorally, choice was best captured by Tree Search: the Tree Search model significantly outperformed the Myopic model and every lesioned variant, indicating that the monkey evaluated positions beyond the immediate next move. That said, the monkey’s planning is considerably shallower than that reported in humans, and the depth we can estimate is itself shaped by opponent strength, as we discuss below. Nonetheless, board evaluations were highly structured, with the monkey consistently prioritizing higher-valued features, even distinguishing between connected and disconnected 2-in-a-row configurations (a phenomenon that was never explicitly reinforced during training). Together with this feature sensitivity, the model comparison constitutes our central behavioral evidence for tree search over simpler myopic strategies.

Gaze told a consistent story. We found that eye movements were strongly tied to choice; they guided the cursor to the eventual move, and the speed with which they converged on a configuration scaled with its feature value, ramping earliest for the highest-valued patterns. The Tree Search principal variation was the best zero-shot predictor of the pre-reach gaze distribution,

and the Tree Search composite regression explained the most variance. However, the gaze story did not give us independent verification of deliberation; the advantage of tree search in explaining variance largely disappeared when we introduced the nuisance regressors of self pieces, opponent pieces, and the realized move. The gaze finding confirms what is intuitively sensible, that fixations are important in guiding motor actions towards the intended position. Gaze is thus consistent with tree search, but it does not corroborate it beyond what choice already establishes.

The neural data corroborate this picture of search-specific computation. A substantial fraction of neurons in DMPFC, DLPFC, and caudate tracked the Tree Search value of the board. This is the value coding corticostriatal circuits are known to support in simpler instrumental settings, from action values in dorsal striatum (O'Doherty et al., 2004) to the encoding of prospective value in prefrontal cortex (Kim et al., 2008). The DMPFC showed the strongest encoding of state-value tuning, consistent with the idea that it integrates actions and outcomes for strategy switching (Seo et al., 2014). Additional analysis identified that this was true even if we controlled for local board confounds; using the Myopic model as a nuisance regressor, we found that a significant proportion of neurons in all three regions tracked this residual. The marked rise in the proportion of tuned neurons just after the computer's move further solidifies this point: this is precisely the moment at which we would expect tree search to occur, if at all. A direct comparison between the Myopic and Tree Search models demonstrated that the Tree Search model explained more variance in firing rates than the Myopic model for all three regions; this was true both in terms of the mean paired difference in R^2 and the proportion of neurons for which Tree Search explained more variance than Myopic. The ordering of DMPFC > caudate > DLPFC was also consistently found across all analyses, hinting at different roles for these regions in the deliberation process.

Together, these findings show that the monkey engages in deeper-than-myopic planning, with convergent evidence from behavior, gaze, and neural data. Corticostriatal circuits support feature-based planning in Four-in-a-Row, consistent across gameplay, deliberation, and neural encoding. The limitations below bound how far the animal's estimated search depth generalizes; they do not bear on this central comparison. We describe those limitations, how we are addressing

them, and the directions they motivate.

The data analyzed here come from a single monkey (M1), with a second monkey (M2) to follow. A few months before writing this thesis, we discovered that M1’s training had been compromised: a technical issue with the protocol made the computer opponent systematically weaker than intended, allowing the monkey to defeat it more easily than designed (see §B.1.3 for the specific issues and the date ranges over which each was present). In our current dataset, many games were won by either player at the first available opportunity, which we filtered from analysis. The deeper consequence is that the distribution of board states the monkey encountered was itself shaped by opponent strength, skewed toward non-forcing positions where extended search confers little advantage. This matters for interpretation because a player’s depth of planning may depend in part on the strength of the adversary: against an opponent that rarely contests a position, shallow evaluation is not a failure of planning but an efficient response to it. Opponent strength therefore shapes both the planning depth that is *worth* deploying and our ability to estimate the depth the animal is capable of. These pull apart under a stronger opponent, yielding a clear prediction: if planning depth is a fixed capacity, it should be insensitive to opponent strength; if it is calibrated to demand, it should scale with it. To adjudicate this, both M1 and M2 are presently undergoing additional training under a more robust opponent, which fixes the existing technical issues and is designed to block more often to encourage longer games.

A separate concern involves idiosyncratic behavior, driven by the task design, training, or inter-subject differences. It has been suggested that the monkey might be optimizing for *reward rate* rather than per-game reward, which could reduce its willingness to plan: an over-trained monkey might play quickly and myopically, winning by attrition against an opponent that occasionally lapses. Our retraining protocol is designed to address this; it explicitly drives the opponent to play more aggressively, making wins by attrition more difficult to secure. We are also considering retraining protocols where the monkey is rewarded for achieving a tie (filling up the board), or where the monkey receives greater rewards for longer games, encouraging greater deliberation. In light of these proposed changes, our present setup would, if anything, lead us

to underestimate rather than overstate the monkey’s planning capacity. Zooming out from the payoff structure of the trial to the training procedure, the staged curriculum, which builds from simpler to more complex configurations, might also play a role in how the monkey comes to value the board. An open question is whether such training installs pattern-specific responses or the extent to which the monkey abstracts transferable structure that generalizes beyond trained cases, a distinction our current data are not yet positioned to settle cleanly. One way to study this would be to train humans on a nonverbal curriculum matched to the monkey’s and ask whether the same systematic deviations emerge; convergent deviations across species would point to the curriculum’s structure rather than the species’ limits as the source. Another potential limitation arises from inter-subject differences. Even with additional training, monkey behavior can be strikingly idiosyncratic, and there is no guarantee that both animals will converge on the same planning strategies. Given the density of our dataset, hundreds of games per day and thousands over many months, we are well positioned to characterize both inter-subject differences and within-subject drift over time, and it would be especially exciting if the underlying neural signals encoded an individual animal’s idiosyncrasies.

Several of the proposed changes to data collection converge on an additional set of downstream analyses. Our early neural analyses show promising results that invite deeper investigation into the latent dynamics at play; however, they come with important limitations in the underlying modeling assumptions. Our current analysis uses linear regression over an event-derived time window to relate firing rates to model-derived value. While this constitutes a simple first pass approach, it makes a few rigid assumptions; the first that neural firing rates are linearly modulated by state value, and the second that such tuning properties are stable over time. Future work is needed to loosen these assumptions and incorporate more realistic accounts of neural firing properties. We intend to apply models that better match the noise assumptions inherent to neural firing rates, such as Poisson GLMs. Further, we anticipate that many neurons will have more complex tuning properties relative to state value and investigating these will require fitting more flexible models, such as nonlinear deep encoding models.

Our current analysis primarily focuses on the convergent roles of the three brain regions DLPFC, DMPFC, and caudate in encoding state value, but does not yet disambiguate between their roles in encoding specific variables and dynamics of the search process as it unfolds. A separate set of analyses is needed to understand how different brain regions encode more granular task variables. Representational similarity analysis (Kriegeskorte et al., 2008) offers one such approach, whereby we partition trials by the presence or absence of specific features and comparing within- and across-group neural representations. To address the temporal dynamics of population-level encoding, we also plan to apply dimensionality-reduction methods such as LFADS (Sussillo et al., 2016) or switching linear dynamical systems (Linderman et al., 2016) to recover latent dimensions that evolve over the trial and to relate them to task variables derived from the Tree Search model: state value, the value of chosen features, and the value of planned features.

Our work attempts to bridge the gap between process-level cognitive algorithms of planning and their underlying behavioral and neural substrates. To that end, what we have provided thus far is a promising starting point for relating neural population coding to cognitive manipulation of variables such as state value and tree search, demonstrated in a nonhuman primate. Looking forward, an important direction will be to connect these findings to emerging accounts of the prefrontal-striatal loop, which has been proposed to function as an actor-critic system (O'Reilly and Frank, 2006) or, alternatively, as a meta-reinforcement learning feedback loop (Wang et al., 2018). Though more work needs to be done to validate this connection, we believe that this provides a promising proof of principle for the potential of using complex cognitive games to study higher cognition in animal models, giving insight into the implementation-level mechanisms of search and complex decision making. More broadly, our work is aligned with a shift in the field towards more naturalistic and expressive tasks, ones which are capable of eliciting the complex and often unexpected behaviors we observe in the real world.

3.4 Methods

The monkey (M1) was trained in a staged curriculum spanning joystick control, pattern completion on partially filled boards, free play, and deliberate blocking, before playing full games against an adaptive computer opponent; our analyses focus on behavior recorded in this last stage, played without human intervention. The opponent was based on the same Tree Search heuristic used to describe human behavior (van Opheusden et al., 2023), with fixed feature weights over board patterns (2-, 3-, and 4-in-a-row configurations) and a trial-wise block rate that adapted to the monkey’s play while those weights stayed constant. The full training curriculum and opponent parameters are given in §B.1; a subset of recording sessions were later found to have been collected under a weakened version of the opponent (§B.1.3).

3.4.1 Trial Structure

To play the game, the monkey used a joystick to control a cursor on the screen. Each game consisted of several trials. On each trial the computer moved first and the monkey moved second. Behavior was logged with eight timestamped events per trial. On complete trials, events occurred in the fixed order below (Fig. 3.13):

1. **Trial start.** The trial began when the base target appeared at the bottom center of the screen.
2. **Home arrival.** The monkey was required to move the cursor to the base target and hold it for 700 ms.
3. **CPU placement.** The opponent’s move was registered on the board while the monkey continued to hold the base target (required hold duration before this event). The computer’s move was registered on the board instantaneously.
4. **Go cue.** After an additional hold period (500-700 ms), the base target was extinguished, permitting the monkey to leave the center.

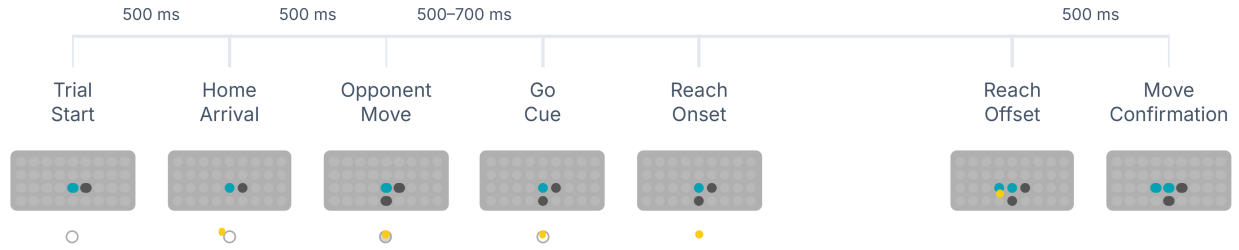


Figure 3.13: **Trial structure.** Each trial proceeds through a fixed sequence of events: the monkey holds the central home target, the computer registers its move, a go cue releases the hold, and the monkey reaches to select and confirm its piece (reach onset, reach offset, and move confirmation). Labels above the axis give approximate inter-event intervals; the boards below show the pieces accumulating over the trial.

5. **Reach onset.** The monkey left the base target (movement onset).
6. **Reach offset.** End of the outbound reach, when the cursor arrived at the chosen board location.
7. **Monkey placement.** The monkey's piece was registered on the board after maintaining the cursor at the target (task requirement: 150 ms hold; error code `registBroke` if registration fails).
8. **Endgame confirmation.** On game-ending trials only, a final delay period followed the completion of a game; this event was logged after an additional ~ 500 ms hold following the winning move.

Incomplete trials could result from cases where the monkey failed to register a move following the computer move; in these cases, the monkey was forced to return to the base target and try again. We merged these trials into *pseudotrials* in which we merged the events through the CPU placement with the reach onset to monkey placement.

We analyzed gameplay recorded from 2023-02-20 to 2024-05-01. For behavioral modeling, we restricted the dataset to games in which at least nine pieces were placed (the *total* set: 5,764 games, 32,868 trials). Within this set, we excluded trials on which the computer completed a 4-in-a-row (leaving the monkey no move to model) and opening moves (moves with ≤ 2 pieces

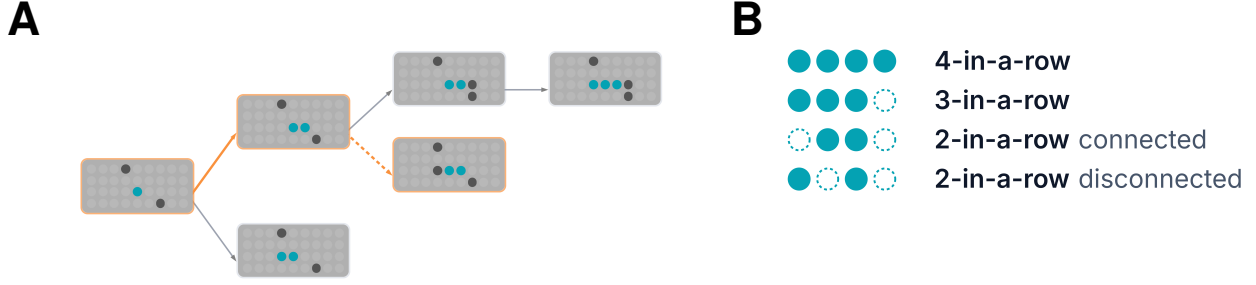


Figure 3.14: **Tree Search and heuristic value function.** (A) Example tree search expansion over board states, with the principal variation (the chosen branch) highlighted. (B) Heuristic features scored by the value function $V(s)$: 4-in-a-row, 3-in-a-row, and connected and disconnected 2-in-a-row. Solid circles mark pieces that must be present; dashed circles mark positions that must remain empty for the pattern to count. Each multi-cell group is additionally instantiated in horizontal, vertical, and diagonal orientations.

on the board, which carry few strategic features). Table B.4 reports the dataset at each filtering stage.

3.4.2 Behavioral Modeling

Effective planning in Four-in-a-Row requires identifying game-relevant board features and appropriately weighting their importance. We developed models of monkey choices using variants of the tree search code developed for modeling human planning (van Opheusden et al., 2023) (Fig. 3.14A).

All models rely on a *value function* $V(s)$, which assigns a scalar value to a board state s . The value function is defined as a weighted sum of board features,

$$V(s) = \mathbf{w}^\top \boldsymbol{\phi}(s), \quad (3.1)$$

where $\boldsymbol{\phi}(s)$ is a vector of feature counts and $\mathbf{w}$ are the corresponding weights. The fitted models used four template groups—4IAR, 3IAR (connected and disconnected 3-in-a-rows sharing one weight), connected 2IAR, and disconnected 2IAR (Fig. 3.14B)—each instantiated in horizontal, vertical, and diagonal orientations. A pattern counts toward $\boldsymbol{\phi}(s)$ only if the positions required for completion remain empty. In the implementation, each template group has a self weight and

an opponent weight; the opponent weight equals the self weight scaled by η .

In the **Tree Search** model, the value function guides an iterative best-first search through a decision tree. Starting from the current state, the model repeatedly selects a node to expand, evaluates successor states with $V(s)$, and backpropagates values. Search depth is controlled by a stochastic stopping rule: the number of expansions is drawn from a geometric distribution with parameter γ , equivalent to terminating the search after each expansion with probability γ (so $\gamma = 1$ yields a single, myopic expansion). Search breadth is limited by pruning moves whose values fall more than τ below the best available move. Fixed-variance Gaussian *value noise* $\mathcal{N}(0, 1)$ is added to the value of each candidate move, and with probability λ (the lapse rate) the model substitutes a uniformly random legal move for the search’s preferred move. A feature-dropping mechanism, which independently drops each feature group with probability δ , is available but excluded from the search and fixed at $\delta = 0$ in all production fits (Table B.5). Absent a lapse, the model selects the root action with highest backed-up value.

We compared seven models fit by cross-validation, all referenced to the full Tree Search baseline (with all parameters fitted; bounds in Table B.5). Three were variants of the search policy: Tree Search; **Myopic** ($\gamma = 1$; a single, shallow expansion); and **Self-Only** ($\gamma = 1$, $\eta = 0$; opponent feature weights zeroed). The remaining four were *lesion* models, each removing one of the four feature groups (4IAR, 3IAR, connected 2IAR, or disconnected 2IAR) from the value function to quantify that feature’s contribution.

Choice Likelihood

In general, the probability assigned by the Tree Search model to a particular move has no closed-form expression. The model contains several stochastic components, including value noise and random lapses, such that the probability of selecting an action depends on the outcome of a randomized search process. Computing the likelihood of an observed choice would therefore require marginalizing over all possible realizations of this stochastic process, which is intractable in practice. We therefore used inverse binomial sampling (IBS), an unbiased estimator of log

likelihood that is well suited for stochastic simulators (van Opheusden et al., 2020), matching the protocol used in the human Four-in-a-Row study (van Opheusden et al., 2023). IBS repeatedly samples the model until the simulated action matches the observed action, using the number of simulations required to obtain a match to construct the trial log likelihood L_i :

$$L_i = \begin{cases} 0 & \text{if } K_i = 1, \\ -\sum_{k=1}^{K_i-1} \frac{1}{k} & \text{if } K_i > 1, \end{cases} \quad (3.2)$$

where K_i is the number of attempts to sample the model before the predicted move matches the observed choice.

Model Fitting

We fit all models by maximizing the log-likelihood of observed choices under the IBS likelihood estimator described earlier. Because the objective function is stochastic, computationally expensive, and lacks analytical gradients, we used Bayesian Adaptive Direct Search (BADS) (Acerbi and Ma, 2017; van Opheusden et al., 2023), a derivative-free optimization algorithm designed for noisy objective functions. BADS combines a local Gaussian-process surrogate model with direct-search steps within a trust region, enabling efficient optimization under noisy function evaluations. Model parameters were constrained by hard and soft bounds (Table B.5). BADS was run with uncertainty handling enabled to account for noise introduced by IBS, with an absolute mesh tolerance of 1×10^{-3} , function tolerance of 10^{-7} , and a 10,000-evaluation ceiling that the mesh-tolerance criterion typically reached well before. Because IBS yields a stochastic estimate of the log-likelihood, we varied the number of IBS replicates per trial over the course of optimization, using few replicates during early, exploratory iterations and increasing them for later refinement, to balance computational cost against estimation precision (see §B.2.4 for the full replicate schedule). To prevent the risk of getting lost in local minima, each fit used five random restarts; the winning restart was selected by re-evaluating each restart’s training log-likelihood at high IBS precision (50 replicates averaged over five evaluations), with held-out test loss then

computed using the winning parameters only.

To reduce computational burden and account for potential changes in behavior over time, data were partitioned into five equal-sized chronological cohorts at the game level. Model fitting was performed independently within each cohort, using 3-fold cross-validation at the level of games. Overall model performance was quantified as the total cross-validated negative log-likelihood, summed across all folds and chronological cohorts.

Model Checking Simulations

To assess whether fitted models captured structured aspects of behavior beyond choice accuracy, we ran forward simulations from model states and compared summary statistics to empirical data, including distance of the chosen position to the board center, mean connected group size, and distances to the player and opponent pieces and centers of mass (Fig. 3.3). For each of the three model variants (Self-Only, Myopic, Tree Search), we drew a single stochastic move from the fitted model at each held-out test-fold board across all 15 cross-validation folds (5 cohorts $\times$ 3 folds), yielding one simulated move per trial in the training set (27,083 simulated moves per model). Simulated and empirical moves were then scored with the same summary statistics and grouped by move number (piece count); plotted values are the mean $\pm$ 1 SEM across trials within each group.

3.4.3 Eye Tracking and Gaze Analysis

Pseudotrials and trials with invalid behavioral event codes for the Go Cue or Reach Offset were excluded from gaze analyses. Eye position was recorded continuously at approximately 500 Hz and stored as timestamped (x, y) samples (milliseconds since session start). Gaze was aligned to behavioral events by nearest-sample indexing, with a small calibration offset applied to correct display alignment. Samples outside the game display were excluded. Traces were median-smoothed before fixation analysis.

Fixation Detection

Fixations were identified in three steps:

1. **Threshold.** Instantaneous eye velocity was estimated from the smoothed trace using a five-point central difference. Samples exceeding a fixed velocity threshold were labeled saccades; both saccades and invalid samples were excluded from fixation detection.
2. **Group.** Contiguous non-saccadic samples were grouped into candidate fixation events. Groups shorter than a minimum duration (40 samples) were discarded. Each surviving group was summarized by the mean gaze location of its samples.
3. **Map.** Samples were assigned to one of the 36 board positions on the 4×9 grid. For fixation histograms, assignment used the nearest position centroid. For event-aligned analyses, assignment required gaze to fall within the on-screen target region for that position (the circular affordance shown during gameplay). Samples outside the board or outside any target region were left unassigned.

Event-Aligned Time Course

For the feature-based gaze analyses (Fig. 3.5), eye data were aligned to reach offset over a window from 1.5 s before to 0.5 s after the event, restricted to non-trivial turns lasting more than 1.5 s from the computer's move to reach offset. Within this window we computed the probability that gaze occupied either (i) the position the monkey eventually played, or (ii) any position belonging to a feature the monkey could complete in one move: 4-in-a-row (**4IAR**), 3-in-a-row (**3IAR**), or 2-in-a-row (**2IAR**), detected with the same template formalism as the behavioral model.

Because lower-valued features often co-occur with higher-valued ones, a trial counted toward a feature only when that feature was the highest one available (e.g., a 3IAR trial is one in which a 3IAR, but no 4IAR, could be completed). Trials were labeled *taken* if the monkey's move completed the feature and *missed* if the feature was available but the monkey moved elsewhere.

For taken trials, the completed 2-in-a-row could be resolved post hoc into connected and disconnected patterns and labeled accordingly; for missed trials this distinction is not recoverable, since the monkey’s intended pattern is unknown, so the 2-in-a-row variants were pooled.

Predictor Maps

Each behavioral model is converted into a prediction about gaze by a *predictor map*, a vector $m \in \mathbb{R}^{36}$ over the board positions that marks where, under that model, the monkey should look. All maps are read straight from the model fit to choices; none is tuned to the eye data. We considered four:

- **Predicted move.** A one-hot map on the position the model selects, which over repeated searches becomes the model’s distribution over chosen moves.
- **Principal variation (PV).** A map over the positions on the best root-to-leaf path of the search tree, the line the model would play out. It is defined only for Tree Search; the Myopic and Self-Only models expand the root once and so reduce it to the predicted move. It can be split by depth, PV_d (with $d = 0$ the immediate move, $d = 1$ the opponent’s reply, and so on), where $\sum_d PV_d = PV$.
- **Feature map:** a static, search-free map of which positions carry the board’s weighted features. Where the value function collapses the board into a single number by summing each feature’s count times its fitted weight, the feature map spreads that same quantity over space. Every feature the monkey holds—an unblocked n -in-a-row of some group g (4IAR, 3IAR, or connected/disconnected 2IAR)—occupies a 4-in-a-row *parent* window W_f (its filled positions and the empty ones that would complete it), and it deposits the magnitude of its group’s fitted weight $|w_g|$ on every position of that parent window. Summing over all features present, each position s receives

$$m^{\text{feat}}(s) = \sum_{\substack{f \in \mathcal{F}_{\text{self}} \\ s \in W_f}} |w_{g(f)}| + \eta \sum_{\substack{f \in \mathcal{F}_{\text{opp}} \\ s \in W_f}} |w_{g(f)}| + c \nu_s, \quad (3.3)$$

where $\mathcal{F}_{\text{self}}$ and $\mathcal{F}_{\text{opp}}$ are the features held by the monkey and the opponent, $g(f)$ is the group of feature f , the opponent’s contributions are scaled by the opponent weight η , and $c v_s$ is the value function’s center bias. A position therefore carries more weight the more features it belongs to and the more valuable those features are, with no search involved.

The search-based maps (predicted move, PV, tree nodes) are accumulated over $N = 100$ stochastic searches per trial; the feature map is computed once from the fitted weights.

Zero-shot Feature Predictions

For each trial, analysis was restricted to the *deliberation window*, from the computer’s move to reach onset, so as to exclude the motor reach. Fixation histograms counted gaze samples in which gaze occupied each board position. Occupied positions were excluded because the zero-shot model predictions can never place gaze on an already-occupied position, so scoring them would penalize the models for gaze they cannot predict. Trials with fewer than 50 total gaze samples (summed over the 36 positions), or with no sample on any empty position, were excluded. Empirical distributions were normalized and compared to model predictions using the cross-trial NLL (Fig. 3.6), with a trial-wise KL divergence validation reported in §B.3.2.

Predicted gaze distributions were derived without refitting to the eye data (zero-shot evaluation), read directly from each model’s fit to choices. We compared two behavioral models—the full Tree Search and the Myopic model ($\gamma = 1$)—each represented by the predictor maps of §3.4.3. Before scoring, every map was smoothed with a uniform floor ($\varepsilon = 10^{-3}$) so that the divergence stayed finite where a model placed no mass, then renormalized over the empty positions.

Gaze Model Comparison

We scored each predictor map against the observed gaze in two complementary ways. For a trial i , let S_i be its empty board positions, $n_i(s)$ the observed fixation count on position $s \in S_i$, and $p_i(s)$ the predictor map (floored and renormalized over S_i).

The first score is the *cross-trial NLL*, which weights every fixation equally, so that fixation-rich trials count for more,

$$\text{NLL} = - \sum_{i=1}^T \sum_{s \in S_i} n_i(s) \log p_i(s). \quad (3.4)$$

Lower NLL indicates a better fit, and differences between models were assessed by bootstrapping over held-out trials ($n = 10^5$). A trial-wise variant that instead weights every trial equally, using the KL divergence between the predicted and observed gaze distributions, is reported as a robustness check in §B.3.2.

The second score is a held-out *regression* that asks how much of the gaze each map explains. Pooling over all 36 positions of all trials, we predicted the raw per-position fixation count $n_i(s)$ by ordinary least squares from the (per-trial renormalized) predictor maps,

$$n_i(s) = \beta_0 + \sum_k \beta_k \tilde{m}_i^{(k)}(s) + \varepsilon_{i,s}, \quad (3.5)$$

where:

- $n_i(s)$ is the observed fixation count on position s in trial i ,
- $\tilde{m}_i^{(k)}(s)$ is the (per-trial renormalized) value of predictor map k at position s ; the index k ranges over a model's components—for Tree Search, the depth-resolved PV_d terms together with the feature map; for the Myopic model, the predicted move and the feature map,
- β_0 is the intercept and β_k the coefficient on map k ,
- $\varepsilon_{i,s}$ is residual error.

We report the variance explained (R^2) on games held out under a game-level cross-validation split.

Unlike the distributional (NLL) score, the regression keeps the occupied positions. To test whether gaze reflects motor preparation rather than deliberation, we extended Eq. 3.5 with nuisance regressors—a flat map of the monkey's own pieces $\text{Self}_i(s)$, a flat map of the opponent's

pieces $\text{Opp}_i(s)$, and a one-hot map of the monkey’s realized move $\text{Move}_i(s)$ —appended to the same sum,

$$n_i(s) = \beta_0 + \sum_k \beta_k \tilde{m}_i^{(k)}(s) + \gamma_1 \text{Move}_i(s) + \gamma_2 \text{Self}_i(s) + \gamma_3 \text{Opp}_i(s) + \varepsilon_{i,s}. \quad (3.6)$$

where, in addition to the terms of Eq. 3.5:

- $\text{Move}_i(s)$ is a one-hot indicator of the monkey’s realized move,
- $\text{Self}_i(s)$ and $\text{Opp}_i(s)$ are flat indicator maps of the monkey’s and opponent’s pieces,
- $\gamma_1, \gamma_2, \gamma_3$ are their respective coefficients.

The composite comparison across models (Fig. 3.7) is scored under Eq. 3.5; the ablation (Fig. 3.8) is scored under Eq. 3.6.

To compare models rather than features, we take the paired difference in held-out R^2 between the two full (Composite) regressions—the model-vs-model analogue of the leave-one-out test below, and of the neural comparison of §3.4.4 applied here to gaze rather than firing rate:

$$\Delta R_{\text{model}}^2 = R_{\text{full}}^2(\text{Tree Search}) - R_{\text{full}}^2(\text{Myopic}), \quad (3.7)$$

where each R_{full}^2 is the held-out R^2 of Eq. 3.5 (or Eq. 3.6, for the nuisance-controlled version) fit with all of that model’s own predictor maps included. This is the *Composite* bar reported in Figs. 3.7–3.8, tested against $H_0 : \Delta R_{\text{model}}^2 = 0$ by the same held-out bootstrap ($n = 10^5$) described below.

We quantify each map’s contribution to either regression by its *leave-one-out* ΔR^2 , the drop in held-out R^2 when that map alone is removed from the sum: a nested comparison of

$$\begin{aligned} \text{Reduced model:} \quad & \text{Eq. 3.5 (or 3.6), map } k \text{ omitted} \\ \text{Full model:} \quad & \text{Eq. 3.5 (or 3.6), map } k \text{ included} \end{aligned} \quad (3.8)$$

testing $H_0 : \Delta R^2_{\text{held-out}} = 0$ against $H_1 : \Delta R^2_{\text{held-out}} > 0$, where $\Delta R^2_{\text{held-out}}$ is the full model’s held-out R^2 minus the reduced model’s. As with the NLL score above, significance was assessed by bootstrapping over held-out trials ($n = 10^5$), so that a map’s ΔR^2 —and, in the ablation model, the deliberative maps’ contribution net of the nuisance regressors—can be read against this null.

3.4.4 Neural Analysis

Recording and Preprocessing

Using 64-channel NeuroNexus probes, we recorded the neural activity of the monkey as it played against a computer opponent, focusing on the dorsomedial prefrontal cortex (DMPFC), dorso-lateral prefrontal cortex (DLPFC), and caudate nucleus. Brain regions were targeted using MRI (Fig. 3.9A). We chose these areas because existing work suggests that they play specialized roles in mediating model-based and model-free representations of reward, state-action-value, and strategic behavior (Cai et al., 2011; Abe and Lee, 2011; Barracough et al., 2004; Seo et al., 2014). Neural activity was simultaneously recorded from two of the three regions per session, for a total of 67 sessions. Spikes were sorted using MountainSort5 and sorting was manually verified prior to analysis (DMPFC: 1127 neurons, DLPFC: 2136 neurons, caudate: 1658 neurons, across all sessions).

Spike Alignment and Binning

Encoding analyses were run at three alignment events: the computer’s move, marking deliberation onset, reach onset, and reach offset. For each event and trial, spikes were counted in non-overlapping bins of width $\Delta t = 50$ ms over a window of $[-1200, +1200]$ ms relative to alignment (49 bins). Trials with invalid alignment times or insufficient data in the recording window were excluded.

Encoding Analysis

Pseudotrials and trials with invalid behavioral event codes (e.g. missing go cue or reach offset labels) were excluded from neural analyses.

Throughout this section, a neuron that never fired at all in a given time bin or window, with a constant zero spike count across every trial, is deemed *inactive*, and is handled differently depending on the analysis type. For the significance-testing analyses (percent significant), an inactive neuron/bin has no evidence of a relationship and is assigned $p = 1$: it is trivially not significant, but still counted in the population, rather than being dropped. For the paired model comparison (percent preferring Tree Search), an inactive neuron/bin leaves R^2 undefined for *both* models at once (they share the same total sum of squares), so it is excluded from both the numerator and denominator instead of being arbitrarily attributed to one model.

Example neurons (Fig. 3.9A–C) were selected as legible exemplars of feature-graded firing and are illustrative rather than part of the population analysis; trials were partitioned by the monkey’s maximum contiguous piece count (4IAR / 3IAR / 2IAR / 1IAR) on the post-move board.

We conduct several tests to understand the nature of search-specific coding: first regressing firing rate directly on state value (§3.4.4); then repeating the test on a regressor that uses the Myopic model as a nuisance regressor (§3.4.4); and finally comparing the two models’ explanatory power head-to-head within the same neuron (§3.4.4).

State-Value Regression

The regressor x_i used throughout this section is one of two model-derived quantities, each evaluated on the board the monkey faced on trial i (after the computer’s move, before the monkey’s) with parameters fixed to their mean fitted values across cross-validation folds and chronological cohorts. Because the search is stochastic—the value noise ($\sigma = 1$) and the geometric iteration budget both vary from run to run—each quantity is the *expected* value over $N = 100$ searches with value noise enabled, for parity with the fitted model. Writing $r_{i,n}^{\text{TS}}$ and $r_{i,n}^{\text{M}}$ for the negated, monkey-perspective root values of the n -th Tree Search and Myopic run (larger is better for the

monkey), the averaged state values are $V_i^{\text{TS}} = \frac{1}{N} \sum_n r_{i,n}^{\text{TS}}$ and $V_i^{\text{M}} = \frac{1}{N} \sum_n r_{i,n}^{\text{M}}$; the Tree Search value V_i^{TS} is the first regressor.

For each neuron j and each 50 ms time bin t , we fit the simple linear regression

$$y_i^{(j,t)} = \beta_0 + \beta V_i^{\text{TS}} + \varepsilon_i, \quad (3.9)$$

where:

- $y_i^{(j,t)}$ is the spike count of neuron j in time bin t on trial i ,
- V_i^{TS} is the Tree Search state value on trial i , as defined above,
- β_0 is the intercept and β the slope on Tree Search value,
- $\varepsilon_i \sim \mathcal{N}(0, \sigma_\varepsilon^2)$ is residual error.

This regresses the binned spike count $y_i^{(j,t)}$ on trial i against Tree Search value, and tested $H_0 : \beta = 0$ against $H_1 : \beta \neq 0$ (two-tailed $p < 0.05$, ordinary least squares); a neuron that never fired in a given bin has no such evidence and is considered *not significant*. The population summary for each region is the fraction of its neurons with a significant slope in that bin (Fig. 3.10). The search-specific comparison (Fig. 3.11) instead tests a nested pair of models, as described below.

We established a baseline with a permutation null. Within each session we shuffled the regressor across trials and re-ran the identical slope test, repeating this 1000 times; a single shuffle was shared across all neurons and time bins within each permutation, preserving the correlation structure of the simultaneously recorded population. The chance band in the figure is the 95th percentile of the resulting distribution of significant fractions, evaluated per region; a region's curve was treated as evidence only where it exceeded this band. We did not apply a per-neuron multiple-comparisons correction. The inference is at the population level, explaining whether more neurons track the regressor than chance allows, rather than a claim about any individual neuron, and the permutation null accounts for the number of neurons tested and their shared regressor under the threshold.

State value alone cannot distinguish search-specific computation from correlated low-level board properties, so we used the Myopic model (which is fit to the same data and sees the same boards) as a nuisance term to absorb this shared structure. For each neuron j and time bin t we fit two nested models across trials:

$$\begin{aligned} \text{Reduced model: } y_i^{(j,t)} &= \beta_0 + \beta_1 V_i^{\text{M}} + \varepsilon_i \\ \text{Full model: } y_i^{(j,t)} &= \beta_0 + \beta_1 V_i^{\text{M}} + \beta_2 V_i^{\text{TS}} + \varepsilon_i \end{aligned} \tag{3.10}$$

where:

- V_i^{M} and V_i^{TS} are the Myopic and Tree Search state values on trial i ,
- β_1 is the coefficient on Myopic value and β_2 the coefficient on Tree Search value,
- $\varepsilon_i \sim \mathcal{N}(0, \sigma_\varepsilon^2)$ is residual error.

We tested whether the full model's reduction in residual sum of squares was greater than chance via an F -test of $H_0 : \beta_2 = 0$ against $H_1 : \beta_2 \neq 0$,

$$F = \frac{(\text{RSS}_{\text{reduced}} - \text{RSS}_{\text{full}})/\text{df}_1}{\text{RSS}_{\text{full}}/\text{df}_2}, \quad \text{df}_1 = 1, \text{df}_2 = n - 3. \tag{3.11}$$

where:

- $\text{RSS}_{\text{reduced}}$ and RSS_{full} are the residual sums of squares of the reduced and full models,
- $\text{df}_1 = 1$ is the numerator degrees of freedom, the one additional parameter β_2 ,
- $\text{df}_2 = n - 3$ is the denominator degrees of freedom, with n the number of trials and 3 the number of fitted parameters in the full model $(\beta_0, \beta_1, \beta_2)$.

Paired Model Comparison

The analyses above test each regressor's slope in isolation and so cannot say whether Tree Search value is favored over Myopic value *within* the same neuron. To compare the two models head-

to-head, for each neuron j we fit the single-predictor regression of Eq. 3.9 separately for $x_i = V_i^{\text{TS}}$ and $x_i = V_i^{\text{M}}$ and computed each model’s in-sample R^2 against the *same* total sum of squares (which depends only on the spike counts, not on the regressor), so the two R^2 values share an identical denominator and their difference,

$$\Delta R_j^2 = R_{\text{TS}}^2(j) - R_{\text{M}}^2(j), \quad (3.12)$$

is a clean, apples-to-apples subtraction. Two statistics summarize ΔR^2 within each region: the mean across neurons (null 0) and the proportion of neurons with $\Delta R^2 > 0$ (null 0.5); both are reported with a 95% bootstrap CI with 10,000 samples.

The reported version of this comparison collapses each trial’s spikes into a single firing rate over the full turn—from the computer’s move to reach offset—rather than binning within it, so each trial contributes one data point (its Tree Search/Myopic value as x , its firing rate as y) and each neuron contributes one ΔR^2 pooled across its trials. This is visualized as a bootstrap violin, the same construction used for the gaze scores above (Fig. 3.6), generalized to draw from pre-computed per-region resamples and to test the proportion statistic against the null, since each region has a different number of neurons and is therefore resampled independently at its own n (Fig. 3.12).

We also computed a time-resolved version of the same comparison, indexing $\Delta R_{j,t}^2$ by 50 ms time bin as in the earlier encoding analyses and bootstrapping per bin exactly as above. We report this only as a confirmatory check (§B.4.2), not as the primary result. Because an inactive neuron is excluded from both the numerator and denominator independently at each time bin, the active population can differ in membership and size across time bins, so an apparent time course would conflate within-neuron dynamics with turnover in active neurons.

4 | Learned Metacontrol of Planning at Scale

4.1 Introduction

Chapters 2 and 3 traced a progression from human planning in stochastic environments to monkeys planning against a strategic opponent. In this chapter, we return to the human domain but continue the strategic planning thread. Here we focus on human planning in adversarial settings at scale, asking not just how people plan, but how they know when to stop. We frame this as a metacontrol problem, thinking about when it is worthwhile to think.

Planning in the real world requires considerable amounts of cognitive effort, pushing us to the bounds of our own cognitive capacities. This is perhaps most evident in chess, a game which has served for centuries as a marker of human intellect and as a window into the strategic reasoning capabilities of the mind. This sentiment is echoed by Herbert Simon’s assertion that chess might be a suitable *Drosophila* for psychology, a model task for the study of behavior, flexible enough to capture naturalistic behavior but constrained enough to study experimentally (Simon and Chase, 1988).

The psychology of chess traces its origins as far back as de Groot (1946), who designed an experiment in which players were shown board positions and asked to recall the pieces. Grandmasters were much more adept at recalling positions than novices, demonstrating a link between exposure, expertise, and memory. Later attempts by Chase and Simon (1973) showed that this board-recognition advantage applied only to chess positions that could be realistically achieved through gameplay and vanished when applied to randomly generated boards, suggesting that better players were more able to create “chunks” of representations that related to their experience with the game. These two perspectives largely centered on the perception and sense-making of the board state rather than the deliberation process; later work has demonstrated that grandmas-

ters and international masters are more effective at recognizing patterns on the board and, more tellingly, perform search faster (Connors et al., 2011) and more deeply (Campitelli and Gobet, 2004).

In parallel with the cognitive study of chess, significant advancements have been made in designing chess-playing algorithms that mimic, compete with, and even surpass human gameplay. Chess engines trace their roots to Shannon (1950), who proposed an algorithm capable of playing chess comprising a tree search with a heuristic value function. An important shift occurred in 1996 when Deep Blue (Campbell et al., 2002), an algorithm developed via hard-coding evaluations combined with deep search, defeated Garry Kasparov in a highly televised game¹. Since then, chess engines, particularly those that rely on learned policies and values combined with some form of iterative search like MCTS, have continued to improve, consistently outperforming human play (Silver et al., 2017a). Some recent algorithms have foregone planning entirely, reaching grandmaster-level play without any inference-time search at all (Ruoss et al., 2024). Following the success of chess engines, a hybrid approach has emerged combining chess engines with human gameplay; researchers have trained models to predict moves directly without search, as in Maia (McIlroy-Young et al., 2020, 2022). Additional methods that regularize search over human-learned policies and values have been shown to improve both strength and alignment with human choices (Zhang et al., 2024).

Chess engines face few of the real limitations people do when playing chess; they can train on millions of games via self-play and search deep into the future quickly and cheaply. Human thinking is far more costly, and accounting for it requires considering the cognitive load associated with planning. A model of human thinking would require an agent to self-monitor the costs and benefits of additional planning at decision time. Such a system would serve as an arbiter between fast and slow reasoning (Kahneman, 2011; Dolan and Dayan, 2013). *Fast* (System 1, model-free) reasoning acts quickly on pattern-matching and learned heuristics; *slow* (System 2, model-based) reasoning constructs and queries an internal world model to simulate possible

¹Despite losing the first game, Kasparov went on to win the match. He was later defeated in a rematch against an upgraded version of the algorithm, the creatively named *Deeper Blue*, in 1997.

futures before acting. This lineage of thinking relates to normative accounts of metareasoning (Russell and Wefald, 1991) and resource rationality (Lieder and Griffiths, 2020), which hold that an agent should treat its own computations as actions to be chosen for their expected benefit net of their cost. Extended to the domain of planning, these so-called *metaplanning* frameworks focus on how to plan optimally when planning carries an implicit cost.

Many metaplanning accounts focus on the problem of *selection*: at each step, the agent must choose which of several candidate nodes to expand, and each expansion is costly. In typical Monte-Carlo Tree Search (MCTS) (Coulom, 2007), this choice is treated as a multi-armed bandit, balancing exploration of infrequently visited nodes against exploitation of high-value ones, as in UCT (Kocsis and Szepesvári, 2006). The normative literature refines this picture: Hay and Russell (2011); Hay et al. (2012) recast the metalevel decision as a selection problem rather than a bandit one, since the agent pays for computation but is scored only on its chosen action; Sezener et al. (2019) selects nodes by their value of information, later extended beyond myopic estimates (Sezener and Dayan, 2020); and Kuperwajs et al. (2024) frames search direction as the choice that most reduces uncertainty about which action is best.

A separate but complementary approach focuses on the problem of *stopping*. One question of particular interest in a game like chess, where enumeration of all states is infeasible, is how people choose to continue versus stop planning. Stopping is a compelling metacontrol problem in part because, unlike selection, which operates on latent nodes in a tree of thought, more planning translates to more time spent thinking, which is directly observable. Why do people spend more time on some board positions than others? And does that effect vary across different levels of ability? Sigman et al. (2010) studied how players allocate thinking time across a game, showing that thinking time peaks in the midgame relative to the opening and endgame, consistent with the fact that the midgame is often the most computationally demanding (more pieces on the board and fewer stereotyped openings). It has recently been shown that people’s thinking time reflects the value of computation; across millions of games on the Lichess online dataset, moves where it was advantageous to think longer (as determined by deeper versus shallow thinking on

Stockfish 14, an algorithmic chess engine) also corresponded to longer response times in players (Russek et al., 2022). This study establishes at the computational level that human time allocation is sensitive to the value of computation, using the engine as an offline normative yardstick, yet what remains is an algorithmic account: a controller that derives the stopping decision from the structure of the search itself, at decision time. A similar study developed Allie (Zhang et al., 2024), an algorithm that could predict inference-time search in humans; however, it did so with no considerations of the implicit cognitive costs and benefits of planning itself; rather than being a normative account of when stopping is optimal, it remains largely descriptive.

Implementations of so-called *adaptive compute time* have been successful in the field of machine learning. Existing work in recurrent architectures introduced learned halting using co-activation constraints and tunable pondering costs (Schmidhuber, 2012; Graves, 2017), establishing the vocabulary for what would later be formalized as a metacontrol problem. More recent work makes the metacontrol framing explicit. Hamrick et al. (2017)’s imagination-based metacontroller learns, under an explicit computational cost, both which of a pool of predictive experts to consult and when to stop imagining and act, a learned halt decision motivated directly by human metareasoning, though applied to one-shot problems with no search tree. The Imagination-Based Planner (IBP) (Pascanu et al., 2017) extended this to sequential decision-making: a manager decides at each step whether to act or to continue imagining, and if the latter, which previously imagined state to expand, under a resource cost that may itself grow with time. Notably, IBP’s submodules are trained separately but simultaneously and on-policy, so the manager co-adapts to a controller and world model still changing beneath it, and is never trained to gate a fixed planner. Perhaps the most flexible realization of the meta-planning problem is Thinker (Chung et al., 2023). Rather than expanding its tree by a fixed rule, its agent learns a policy over model-interaction actions, navigating and expanding a learned world model and thereby making dynamic, node-level decisions about *where* to search. An extension of Thinker, Dynamic Thinker (Wang et al., 2025), explicitly added a halt option to Thinker’s action space.

Here we propose a model that uses the topology and values of an existing search tree to de-

cide whether to continue or halt the planning process; such a model would bridge the existing metacognition literature with the implementations proposed in the adaptive compute time literature. Existing implementations like IBP, Thinker, Dynamic Thinker, and extensions of this flavor of work have gotten us partway there but share several important limitations. For the most part, these algorithms are trained end-to-end; action selection and stopping are optimized together as one monolithic policy, learned over millions of environment steps and therefore prohibitively expensive for tasks like chess. In our implementation, selection is delegated to a frozen evaluator queried through a standard search, while a separately trained readout head makes the decision of when to halt or continue. Second, the tree representation is summarized through a set of hand-crafted summary hints rather than learned directly. Here, we want the tree topology and value representation to be learned rather than prescribed. In this work, we propose a metacontroller that maintains knowledge of the structure of search, via a graph neural network (GNN) operating over the explicit topology of the tree. A GNN is the natural answer to the problem of variable-sized trees encountered by the prior work on metaplanning, consuming a tree of arbitrary size and shape without collapsing it to the root or the most recent node. We draw on existing work adapting GNN approaches trees, such as TreeLSTMs (Tai et al., 2015) and those which incorporate attention over nodes, as in TreeGRUs / TreeBiGRUs (Kokkinos and Potamianos, 2017).

This chapter takes a twofold approach to understanding how people approach metaplanning in chess. First, we take a bottom-up approach, showing that people’s own thinking time in chess is related to structural features of the board that might correlate with the value of computation. In doing so, we establish a set of descriptive behaviors related to thinking time allocation, which we return to in the Discussion as a candidate empirical target for future metacontrol models. Separately, we take a top-down approach, demonstrating the feasibility of training a metacontroller on chess positions: building a tree, encoding it with a graph neural network, and deciding whether to continue or halt. We train this metacontroller on a reinforcement learning target as a proof of principle that the halt/continue decision is learnable, and benchmark its performance by comparing it with baseline models against a Bellman-optimal oracle. The result is a controller

that decides *when* planning is worth its cost rather than only *how* to plan, and that connects to an existing chess engine without the need for retraining. The implications of these models extend beyond just the chess domain; adaptive compute time is an important challenge in state-of-the-art reasoning and language models (Snell et al., 2024). Work in this chapter is ongoing; we leave the question of whether the metacontroller’s learned value-of-computation signal predicts human thinking time as a near-term future goal.

4.2 Results

Our results address two complementary goals of our project that lay the groundwork for future reconciliation. Our first goal is to characterize how people actually allocate thinking time in chess and whether their behavior is consistent with using thinking time as a constrained resource. Our second goal is to demonstrate that a normative agent can be trained with the specific objective of deciding how long to think based on the evolving search tree, and compare the results of such a model with simpler heuristic models in minimizing regret. Together these analyses move us towards a computational account of metaplanning in chess, in which people consider whether to continue or halt based on the value and topology of their existing planning tree.

4.2.1 Model Agnostic Analysis of Response Time

We begin by characterizing how human players allocate their thinking time across games before introducing our computational model. Specifically, we ask which observable properties of a chess position are associated with longer or shorter deliberation, providing a descriptive account of thinking time in chess. For our analysis, we selected games from the Lichess dataset with 10-minute time controls and no time increment (600+0) played between October and December 2023, restricting to games where both players were sufficiently strong ($\text{Elo} \geq 2000$).

Response times were approximately log-normally distributed (min: 1 s, med: 5.0 s, max: 596 s; Fig. 4.1A, B). We therefore use log-response time as the dependent variable throughout this sec-

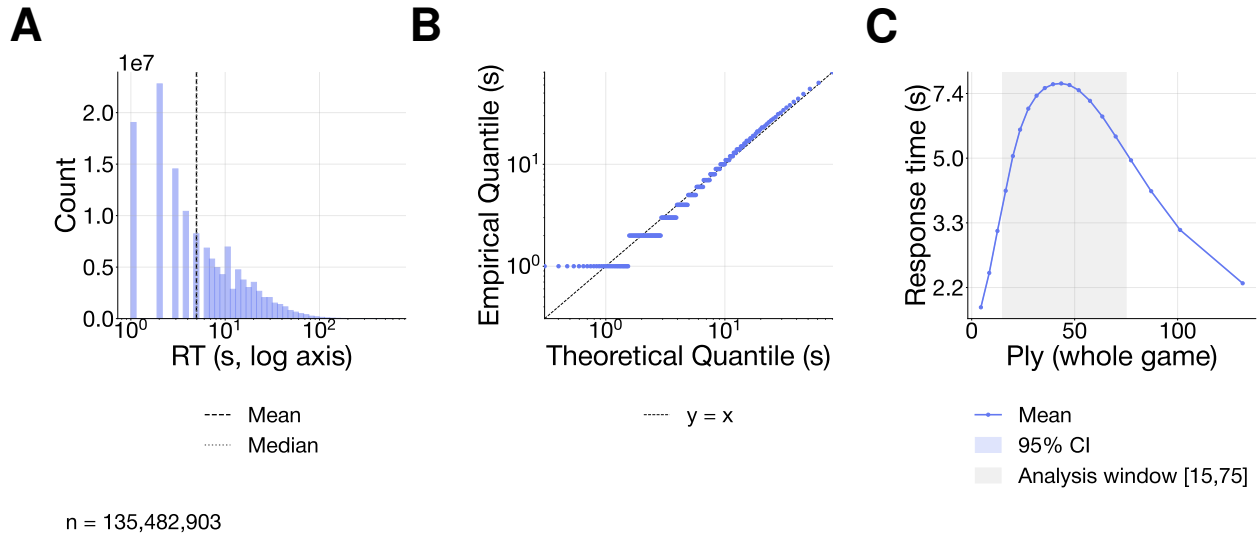


Figure 4.1: **Player response times.** (A) Histogram of response times (log-scaled RT axis). (B) Normal quantile-quantile plot of log-response time. (C) Response time as a function of ply across the whole game, separated by ply quantiles. RT is plotted on a log-transformed axis.

tion. Our first observation is that deliberation varies systematically over the course of a game. Response time follows a characteristic inverted-U profile as the game progresses: players move quickly during the opening, spend progressively longer thinking through the middlegame, and then respond more rapidly again in the endgame (Fig. 4.1C). This pattern is consistent with the changing computational demands of different phases of chess, as established in (Sigman et al., 2010). Opening positions are heavily rehearsed and often follow established theory, allowing rapid responses based on memory. As games transition into novel middlegame positions, players must increasingly evaluate candidate continuations rather than recall established lines. Later in the game, both the reduction in remaining pieces and the familiarity of some common endgames reduce the need for extended deliberation. Because these opening and endgame effects might disproportionately affect overall response time, we restrict all subsequent analyses to plies 15–75, following Russek et al. (2022).

Having restricted our ply range to reduce opening and endgame effects, we next examined how players adapt their deliberation to features of the game. Remaining clock time exhibited an inverted-U relationship with response time: players responded more quickly when they had

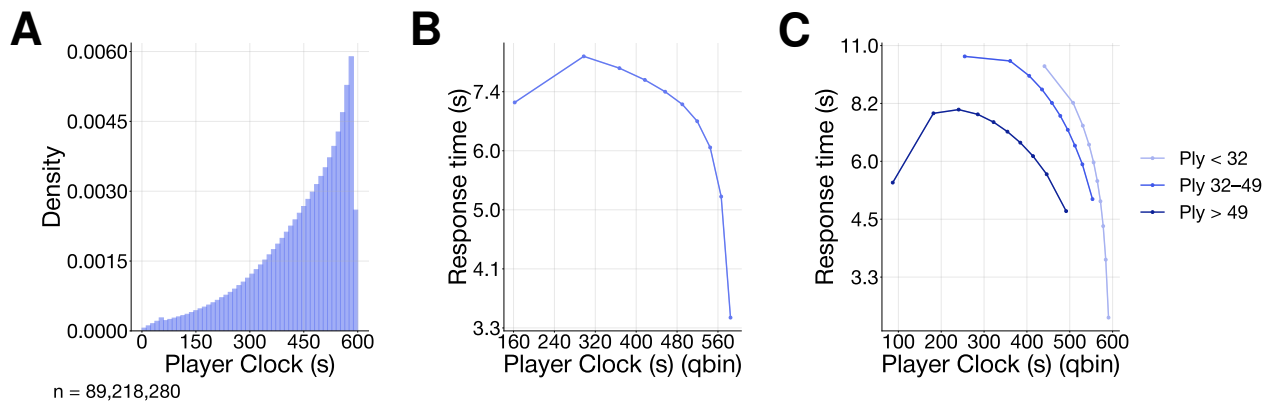


Figure 4.2: **Clock time vs response time.** Clock time is split by quantile bins. RT is plotted on a log-transformed axis. (A) Histogram of clock time remaining, from the perspective of the player to move. (B) Clock time remaining plotted against response time (mean $\pm$ SEM) (C) Same as (B) but separated by ply tertiles.

either very little or nearly all of their time remaining. This raw relationship is difficult to interpret directly, however, because remaining clock time is mechanically coupled to game progression. Early in the game, players who move quickly also happen to have a lot of time left, while later in the game, mounting time pressure forces increasingly rapid decisions. Clock time and game stage are therefore inextricably intertwined; splitting by ply tertile reveals that the clock-pressure effect (faster responses as time runs low) strengthens in later plies, while the early-game relationship instead reflects the confound with game stage (Fig. 4.2).

We have established that the stage of the game, including the ply and clock pressure, play important roles in shaping response time. We next asked whether properties of the board predict how long players think. One measure of the available resources is board material, which refers to the weighted sum of the values of the pieces on the board, providing a coarse measure of heuristic value. In chess, the standard weight of each piece is 1 for pawns, 3 for bishops and knights, 5 for rooks, and 9 for queens. Material provides a strategy-independent description of a given position, allowing us to understand how players respond to different states without making deeper assumptions about implementation.

The player's own material can reflect the number of options available to the player in terms of ability to affect the outcome of the game (Fig. 4.3). Looking at the aggregate trend, we see an

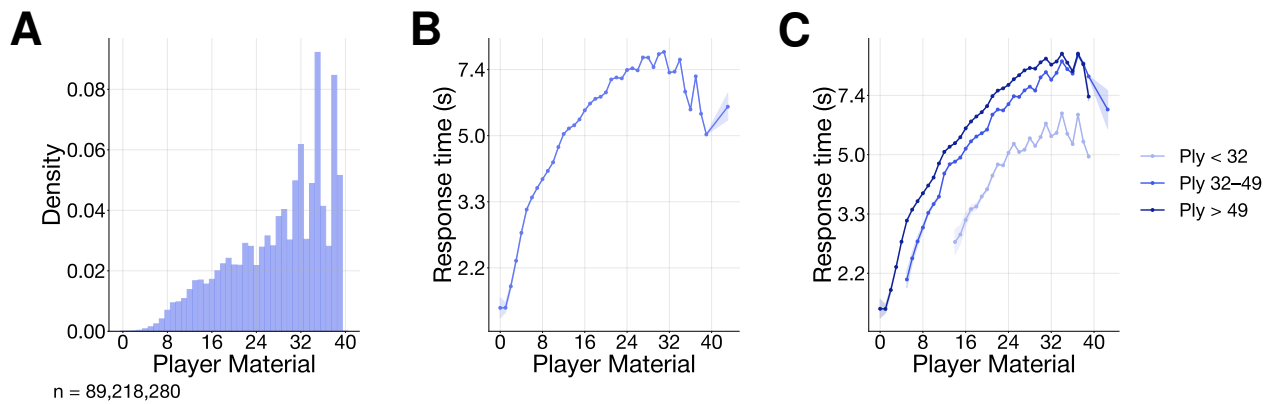


Figure 4.3: **Player material vs response time.** Player material is weighted by piece (pawn = 1, knight = bishop = 3, rook = 5, queen = 9). RT is plotted on a log-transformed axis. (A) Histogram of player material. (B) Player material plotted against response time (mean $\pm$ SEM) (C) Same as (B) but separated by ply tertiles.

inverted-U shape, although the span of the range is relatively small, which is somewhat difficult to interpret (Fig. 4.3B). Dividing by ply tertiles gives a much better view of the underlying effect (Fig. 4.3C). Player material is strongly related to the game stage; the raw material available to a player typically dwindles as the game progresses. This unequal distribution of material across plies leads the aggregate effect to wash out; opening moves where players have plenty of material are therefore confounded with midgame moves where players have *retained* their material over the game. The aggregate inverted-U shape therefore hides the fact that within each ply tertile, the response time tends to be positively related to the player’s own material.

Another important metric is to see how far ahead or behind a given player is; we can measure this approximately by looking at the difference in the material count from the player and the opponent (Fig. 4.4). We would expect that the closer to 0 the material difference is, the “tighter” the game would be, and the more players would be incentivized to spend more time thinking carefully. In cases where the imbalance is large, it is less worthwhile spending more time thinking since the game is heavily tipped in favor of one player versus another. We find that players indeed spend more time when the material imbalance is low, leading to a peak in response times around 0 that persists even after controlling for ply (Fig. 4.4C). Importantly, this peaked curve appears not to be perfectly symmetric around zero. This implies that the effect differs based on who is

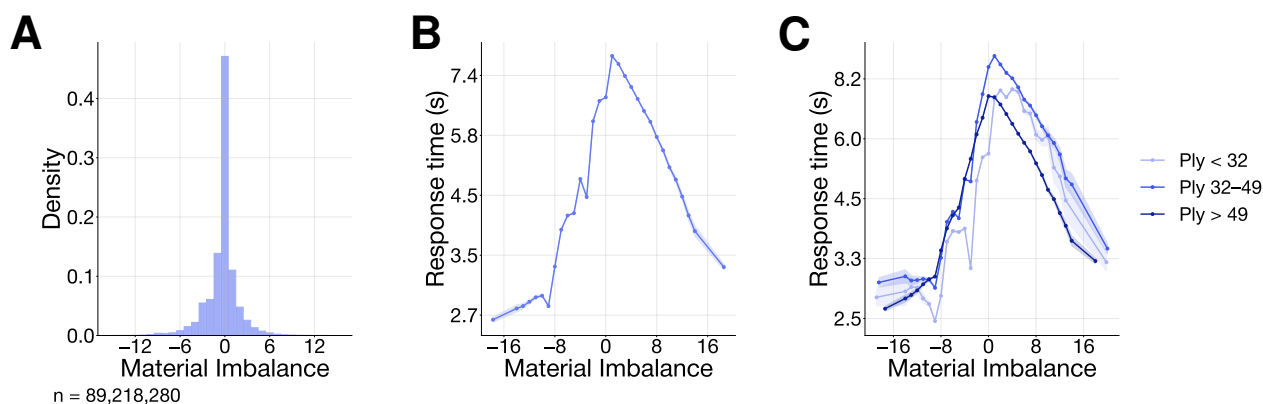


Figure 4.4: Material imbalance vs response time. Material imbalance is defined as the difference in material count between the player and the opponent (positive corresponds to the player being ahead). RT is plotted on a log-transformed axis. (A) Histogram of material imbalance. (B) Material imbalance plotted against response time (mean $\pm$ SEM) (C) Same as (B) but separated by ply tertiles.

in the lead; for positive material imbalance, the player-to-move is in the lead, and we see that the peak of the curve is just slightly *past* parity, meaning that players tend to spend the most time thinking when they are slightly ahead of their opponent, but not by much. This also makes sense because a small lead can be easily lost, but can offer a player massive upside if used to their advantage; the goal of a player in that scenario is to drive a wedge and expand their relative material advantage as much as possible, whereas when the game is very imbalanced, additional thinking might not translate to realized gains.

Beyond looking at the static features of the board, we also wanted to understand what actions the board affords, starting with the number of legal moves in a given position (Fig. 4.5). Intuitively, positions with only one or two viable options leave little room for deliberation, while positions with many legal alternatives require distinguishing among competing possibilities. Response time increased monotonically with the number of legal moves, suggesting that players devote additional deliberation to positions offering a larger space of candidate actions. We also observed a non-monotonic bump at around 1–9 legal moves, which survives ply control (Fig. 4.5B,C). Inspection of the underlying positions revealed that this effect was largely driven by boards where one or both players were in check, which disproportionately contribute to positions with few legal actions (§C.2.2). Excluding these positions removes the artifact (Fig. C.1). Both the number

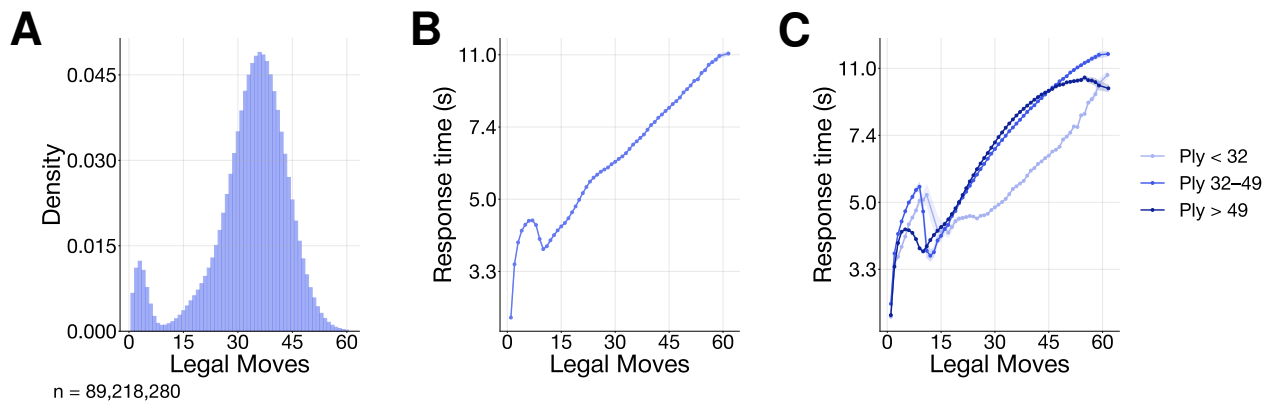


Figure 4.5: **Number of legal moves vs response time.** RT is plotted on a log-transformed axis. (A) Histogram of legal moves. (B) Legal moves plotted against response time (mean $\pm$ SEM) (C) Same as (B) but separated by ply tertiles.

of legal moves and the player's material serve as a proxy for the player's flexibility and their ability to influence the game, either through the value of their existing pieces or the degrees of freedom for choice. It follows that both make similar predictions for the direction of influence on the response time; the more raw material and available choices, the more people tend to spend time responding.

A more targeted look at the available moves looks at the downstream consequences of those moves (Fig. 4.6). We decided to focus on the number of moves in which a capture could be made; that is, the number of captures available for the player to move. We found that, as the number of captures available increased, so did response times. This trend is monotonic and does not change direction across different ply tertiles (Fig. 4.6C), suggesting that it is a robust effect that is independent of game stage. This has an intuitive normative explanation; players actively involved in deliberating between several noisy options will need more time to refine which action is best. Just as in the case of legal moves, the same consideration applies to captures, which require more planning to resolve action value discrepancies.

Collectively, these results suggest that humans allocate computation according to latent structural properties of the search problem rather than uniformly across positions. Many of these trends qualitatively follow intuitive ideas about when additional planning is worthwhile, such as the idea that having more available captures could benefit from additional deliberation. This

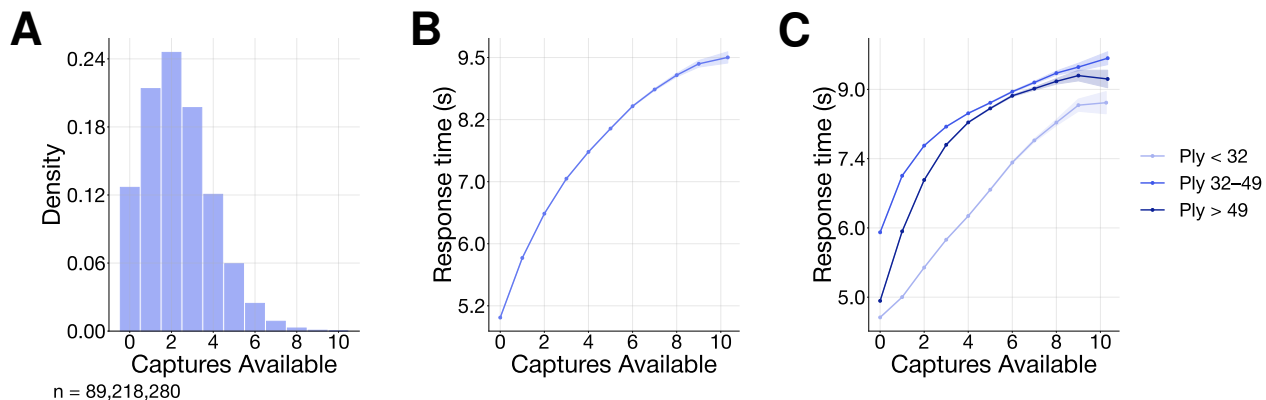


Figure 4.6: **Number of captures available vs response time.** RT is plotted on a log-transformed axis. (A) Histogram of captures available. (B) Captures available plotted against response time (mean $\pm$ SEM) (C) Same as (B) but separated by ply tertiles.

points to the broader possibility that people might be allocating cognitive resources such as response time in a normative manner. The remainder of the chapter focuses on designing a controller capable of normatively allocating resources at compute time.

4.2.2 A Learned Metacontroller for Search

Having established descriptive trends in human response time, we next ask whether a normative controller can learn when continued planning is worthwhile. Unlike the preceding analyses, which are entirely model-agnostic, this section introduces and evaluates a learned metacontroller capable of deciding whether to continue planning or halt. Here we provide a broad sketch of the construction of the metacontroller.

Metacontroller Architecture

Our metacontroller takes its inspiration from existing tree-based metacontrol such as the Imagination Based Planner (Pascanu et al., 2017) and the Thinker (Chung et al., 2023). Rather than operating directly over board states as a chess engine might, the metacontroller operates over partially completed search trees. The root of the tree always represents the current real-world state, while the remaining nodes represent the planning process considered so far. The construction of that tree is outsourced to a low-level planner that operates independently of the metacontroller. At

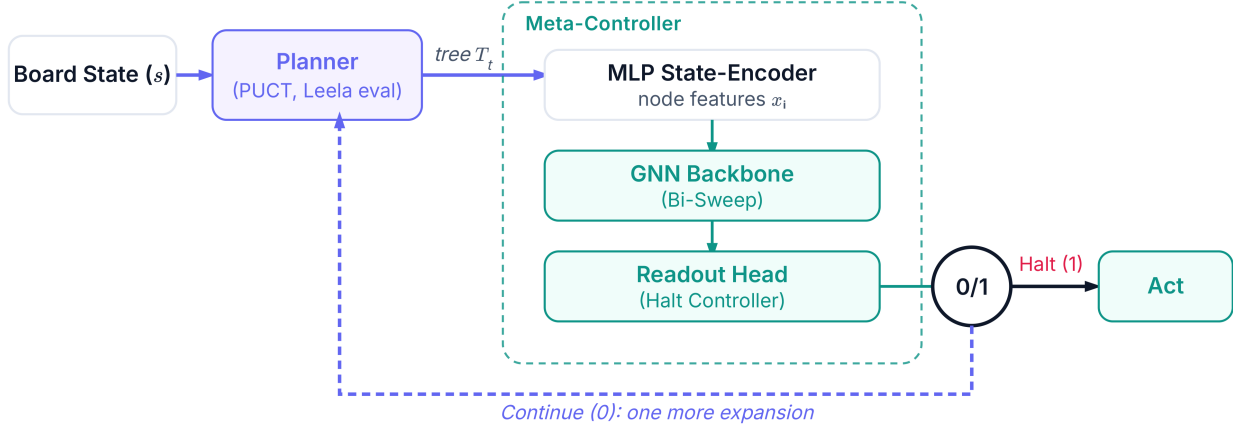


Figure 4.7: **Metacontroller architecture.** The planner, a Leela-driven PUCT search, expands a search tree T_t from the current board state, and the metacontroller decides whether to halt or continue. Each tree node is encoded by an MLP, a graph neural network embeds the tree across upward and downward sweeps, and a readout head over the root produces a scalar halt–continue advantage $\hat{A}_t$. On halt, the agent commits to the planner’s current best root move; on continue, the tree is returned to the planner for one further expansion, closing the metacontrol loop.

each step, the metacontroller takes the partially completed tree and decides whether to continue or halt. If the metacontroller chooses to halt, the agent takes the best action, corresponding to the highest-valued action from the root of the tree. If the metacontroller chooses to continue, the agent calls the planner, which computes one more expansion in the tree (Fig. 4.7).

The tree expansion policy is decoupled from the metacontrol process. In our implementation, a separate module, the planner, expands the tree using PUCT: it selects the next leaf to expand via a score balancing backed-up value against an exploration term weighted by a move prior and visit count, expands its children, queries the Leela chess engine to evaluate them, and backpropagates the value back to the root. Leela itself performs no search; it is queried purely as a one-shot evaluator, supplying the heuristic value and move prior the planner’s PUCT selection rule depends on. The planner therefore corresponds to a PUCT search tree with Leela as its evaluator.

The objective of the metacontroller is simple: to maximize expected game outcome while minimizing the costs of thinking. This can be formulated as a reinforcement learning problem, where the reward is determined by the expected outcome under the selected action while the cost reflects the accumulated cost of thinking. Broadly speaking, the metacontroller consists of two

parts. The first is an encoder, which is responsible for taking in the planning tree and learning a vector representation that can later be used to decide to plan or halt. We operationalize this as a GNN that passes messages in an upward sweep from children to parents followed by a downward sweep from parents to children. The second is a readout head, which takes the encoded vector and returns a policy over continuing or halting; we implement this as a nonlinear multi-layer perceptron (MLP).

Training

In principle, it is possible to train both modules end-to-end using the reinforcement learning objective, but in practice the sparsity of the reward signal makes it difficult for such an architecture to learn meaningful representations. The goal of the graph neural network is to estimate the value of further search from the current partial tree without completing the remainder of the search. We therefore train our network in parts.

We begin by building trees on a subsampled set of board positions from the dataset we used to analyze human behavior. Trees were constructed iteratively using the planner module, up to 96 expansions each. The tree was built stepwise; at each expansion t , the order and win-draw-loss probabilities of expanded nodes were saved in partial trees T_t which were used for training. Each node u in the partial tree T_t was represented by the feature tuple $\mathbf{x}_u^{(t)}$ (value, win-draw-loss, and WDL variance, formalized in §4.4.3). Importantly, the node representation is not fixed across time; it evolves with the tree expansion. As the tree evolves under episode expansions t , the representation of every existing node therefore continues to reflect the updated values according to the planner. Because the planner is deterministic, we only need to run this process once; we consider the fully expanded tree under 96 expansions to be the *oracle tree*. We excluded trees whose best move under the oracle was discovered early, abandoned in favor of another candidate, and then re-discovered later in the search. These *early discovery* cases (shallow search stumbling onto the right answer by accident, losing it, and only later re-validating it was correct) provide a poor signal for when search is worthwhile (See §4.4 for details).

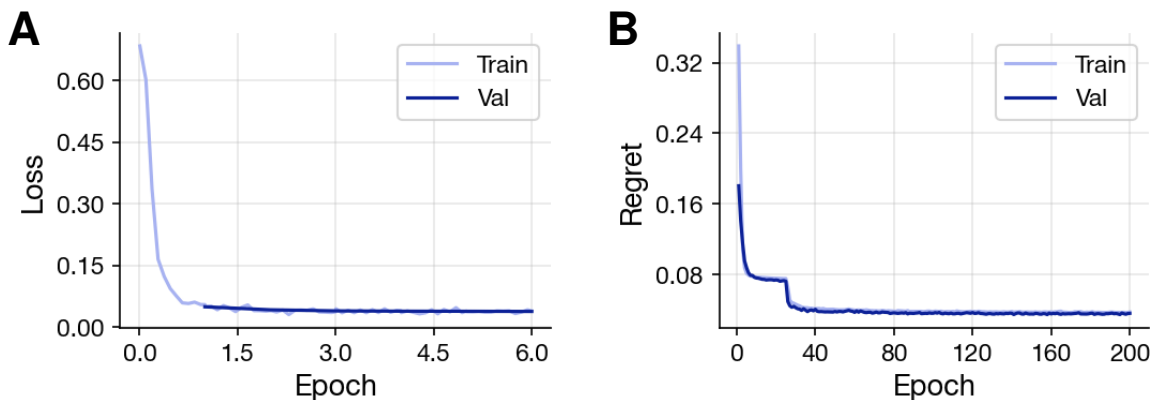


Figure 4.8: **Metacontroller training.** Training (light) and validation (dark) curves over epochs. (A) Encoder training loss (weighted cross-entropy) over epochs. (B) Readout head training regret over episodes.

We use a GNN, specifically a bidirectional Tree GRU (TreeBiGRU) (Kokkinos and Potamianos, 2017). Unlike traditional graph neural networks which involve several rounds of message passing to propagate information through a distributed graph, the Tree biGRU architecture involves a two-pass sweep in topographic order, first from the leaves to the root and then from the root to the leaves. To encode the tree structure and predict the value of further search from an arbitrary step t to a later step $t + k$. In principle, k is a flexible hyperparameter; here we choose $k = 12$. For each board position comprising the root of a planning tree, we sample a random time slot t uniformly between 1 and $96 - k$, and take the partial trees at T_t and T_{t+k} . For each edge $(u, v) \in T_t$, where u is the parent and v is the child, the encoder is trained to take u, v and predict v 's win-draw-loss estimate under T_{t+k} , using cross entropy loss. Using this surrogate supervised target allows the network to learn to pass value information from children to parents implicitly, which is a useful representation to encode. In other words, the network is trained to predict the downstream effects of an additional k steps of planning without performing search itself. Not every edge accrues the same amount of new search in the window $[t, t + k]$; a child near the root may be revisited by many later expansions while a child deep in the tree may see none, so we weight the cross entropy loss by the child v 's change in visit count from time t to $t + k$.

The MLP readout head is then separately trained to maximize its reward over each episode, where an episode consists of iteratively building up a tree from the initial root FEN string. At

each step t , the readout head encounters a partial tree T_t and decides if it wants to continue or halt. The tree T_t is then passed through the frozen weights of the encoder, resulting in a root representation z_t which is concatenated to the number of steps so far t . The advantage of continuing at a given step is the relative value of continuing compared to halting, minus the cognitive cost of continuing (we assume a per-step cost λ , so the cumulative cost grows linearly in t). The closed-form advantage at each step can be calculated via dynamic programming, and the per-instance regret can be calculated by comparing this closed-form value with an oracle that has hindsight knowledge of the value of continuing (the opportunity cost of either planning too long or not planning enough). The readout head learns via policy-gradient reinforcement learning to minimize expected regret over the episode, which is simply the dot-product of the probability of halting at stop t with the instance regret at each step (equivalently, since the episode’s hindsight-optimal return is fixed, this is the same objective as maximizing the expected return described formally in §4.4.4).

During training, our encoder learned to minimize weighted cross-entropy loss over several epochs (Fig. 4.8A). The validation loss tracks closely with the training loss, demonstrating that the model is not overfitting. After freezing the encoder, our readout head, trained over 20 epochs, learned to steadily reduce regret over several episodes (Fig. 4.8B). The training regret smoothly decreases over episodes. The validation regret, which is computed from a greedy continue / halt decision, starts off at a plateau corresponding with a fixed stopping point and then eventually descends to meet the training curve, demonstrating that the policy-gradient training is also capable of minimizing regret using the encoding of the partial tree. In the following subsection, we evaluate the resulting metacontroller against several baseline models.

Evaluation

To evaluate our metacontroller, we constructed several baseline models to assess how much the metacontroller was able to learn with our two-phase training protocol. All models can be broadly construed as metacontrollers in the sense that they decide when to stop or continue, but vary in

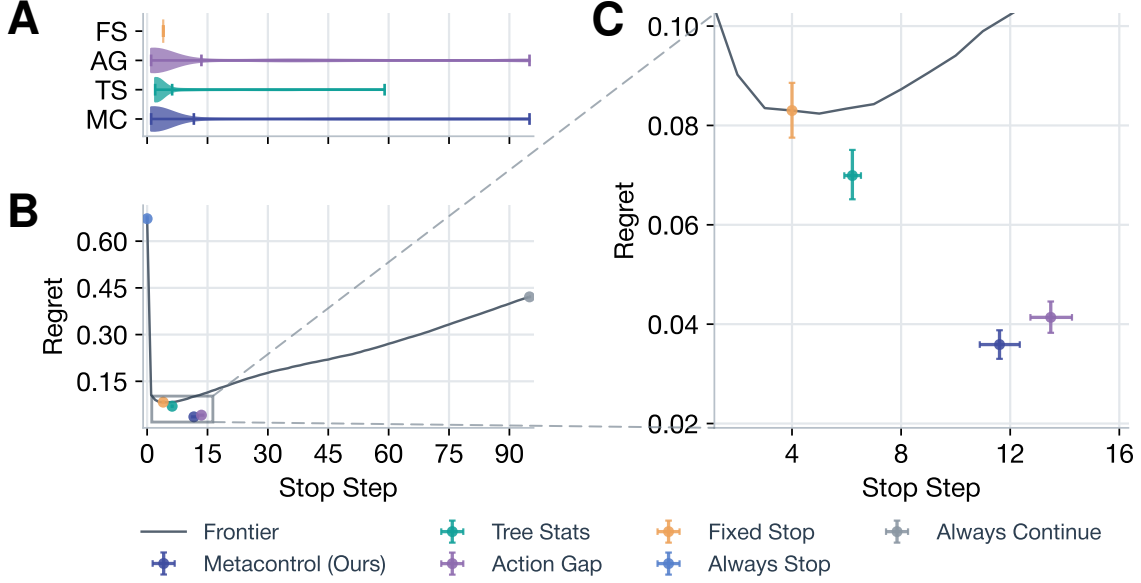


Figure 4.9: **Regret compared to baselines.** Plotted average regret under a linear-cost schedule ($\lambda = 0.005$). (A) Per-controller stop distributions (violin) for Fixed Stop (FS), Action Gap (AG), Tree Stats (TS), and the Metacontrol readout (MC); error bars mark the mean $\pm$ bootstrapped 95% CI. (B) Mean regret and mean expansions used before halting for each model, with (C) showing the boxed region enlarged. Error bars throughout represent bootstrapped 95% CI.

their level of sophistication. We consider **Always Stop**, a model which halts immediately, and **Always Continue**, a model which continues until it is forced to halt. We then consider **Fixed Stop**, a model which sets a single constant for how many steps to halt, across the board, where the number of steps is selected via grid search over the training data. Additionally, we consider a model **Tree Stats**, which sees the number of nodes, the maximum depth, and the maximum width of the partial tree T_t , as well as the number of steps taken. Finally, we consider a model **Action Gap** which takes the tuple (action gap, steps taken) and produces a policy over continuing versus halting, where the action gap is defined as the difference between the best and second-best action values at time t . The last two models are trained using policy-gradient reinforcement learning with an MLP to minimize regret.

As an illustration of how well the metacontroller readout learns compared to baseline agents, we first analyze the regret of all our fitted models and baselines under a linear-cost schedule ($\lambda = 0.005$, Fig. 4.9). We establish a fixed-stop frontier: the curve of regret achieved by every

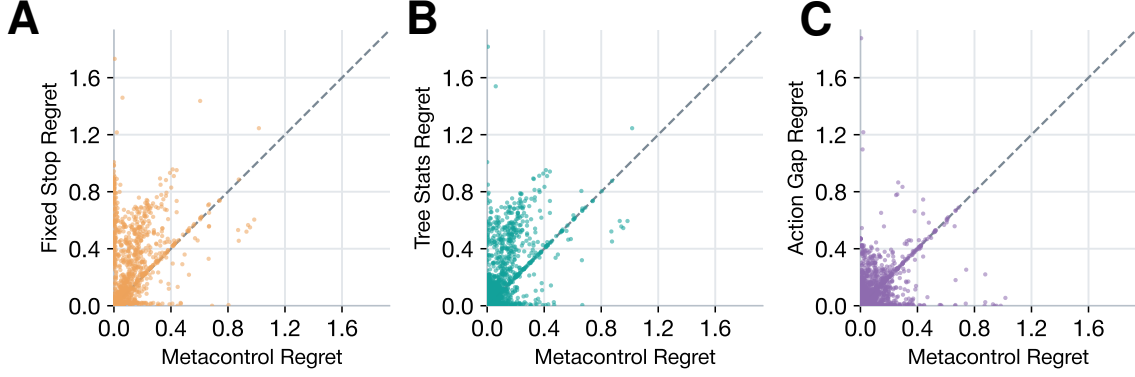


Figure 4.10: **Regret scatterplot.** Per-episode regret, metacontroller vs. each baseline, for $\lambda = 0.005$. Each point is one validation position; the dashed line is the identity, above which the metacontroller outperforms the alternative model. (A) Metacontroller vs. Fixed Stop. (B) Metacontroller vs. Tree Stats. (C) Metacontroller vs. Action Gap. All panels plotted under the $\lambda = 0.005$ linear-cost schedule.

possible constant stopping point κ , evaluated on the training set. Fixed Stop is marked as the single point on this frontier achieving the lowest regret on the training, κ^* ; its validation regret was 0.083, 95% CI [0.078, 0.089]. The Tree Stats benchmark significantly outperforms the Fixed Stop benchmark and falls below the frontier (regret 0.070, 95% CI [0.065, 0.075]), demonstrating an additional edge over the fixed stopping horizon. The Action Gap benchmark significantly outperforms Tree Stats (regret 0.041, 95% CI [0.038, 0.045]), suggesting that the current best and second best actions are a better proxy for the value of continuing and halting net the cost than any of the individual tree statistics. Our metacontroller readout head was competitive or better than all baseline models under the prescribed cost regime ($\lambda = 0.005$; regret 0.036, 95% CI [0.033, 0.039]).

A more direct way to evaluate both models is to compare regret between the metacontroller and each baseline model on each of the validation positions (Fig. 4.10). The scatterplot demonstrates that on most episodes, the metacontroller outperforms both the Fixed Stop and Tree Stats baselines, with most points lying above the identity line; the Metacontroller versus Action Gap difference is less visually distinguishable.

We conduct a comparison of the paired differences in regret between the metacontroller and each baseline model (Fig. 4.11), which allows us to investigate the robustness of our finding across

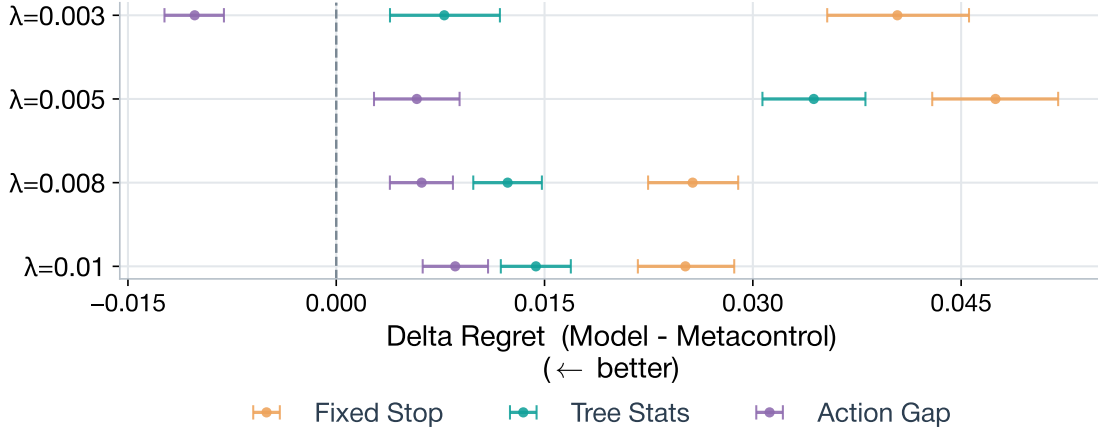


Figure 4.11: **Paired regret differences.** Mean Δ regret (baseline model minus metacontroller) across the per-episode paired differences, for each baseline (indicated by color) under different cost λ ; positive values favor the metacontroller. Error bars represent bootstrapped 95% CI ($n = 10,000$).

different cost regimes λ . The metacontroller readout significantly outperforms all baselines where $\lambda = 0.005$, including Action Gap (paired difference +0.0055, 95% CI [+0.0025, +0.0084]); notably this is true in most but not all cost regimes (see Discussion for the $\lambda = 0.003$ exception). Ultimately, the metacontroller outperforms Action Gap, the strongest baseline, indicating that the learned tree encoding supplies information beyond the myopic root value gap. Whether this advantage stems specifically from tree topology, as opposed to the richer set of root-level statistics the encoder has access to, remains an open question for future work.

4.3 Discussion

This chapter focuses on how chess players decide when additional planning is worthwhile. We approached this question from two complementary perspectives. First, we characterized how human thinking time varies as a function of the current board position using a model-agnostic analysis of large-scale behavioral data. Second, we developed a normative metacontroller capable of deciding whether additional planning is worth its computational cost. In our current approach, we separate the data, which are used to establish behavioral signatures of how people think in

chess, and the model, which establishes a normative baseline that will eventually be fit on human behavior. We take each in turn, considering what it establishes and what it cannot, before asking what it would take to bring the two together.

Our descriptive analyses demonstrate that thinking time is not allocated uniformly in a game but instead depends systematically on the context in which planning occurs. Response time differs across ply and leftover clock time, demonstrating a dependence on game stage, consistent with existing accounts of response time in chess (Sigman et al., 2010). Additionally, response times were sensitive to board features: players devote more time to positions in which they retain more material, where the material difference is small, and where more legal moves and captures are available. Although each of these board features captures a different aspect of the current position, together they suggest that players allocate additional deliberation when the board affords both greater flexibility and greater opportunity for planning.

These findings come with a few limitations, which follow from the nature of the measure and the data. First, response time does not disambiguate between time spent thinking and time spent processing perceptual changes on the board. We anticipate that this perceptual contribution is relatively small; the visual configuration of the board changes only modestly from move to move, whereas the planning implications can differ dramatically. Nevertheless, the planning process itself may consist of several stages, ranging from identifying a consideration set to deliberating among candidate actions, and our current pipeline collapses these processes into a single measure. A dedicated eye-tracking experiment could instead distinguish perceptual processing, shallow search, and deeper deliberation, providing a richer characterization of the planning process. This interpretation is consistent with existing studies which establish that experts use parafoveal processing to encode chunked configurations before planning, and that experts fixate less frequently and more decisively than novices (Charness et al., 2001). Second, and more generally, establishing direct relationships between board features and response time remains difficult because many predictors covary. As we have shown, variables such as player material, legal moves, captures, and game stage evolve together over the course of a game. This dependence is an in-

herent consequence of analyzing naturally occurring games rather than independently sampled board positions. Consequently, while these descriptive analyses establish regularities in behavior and provide useful benchmarks for future models, they are less suitable for identifying the independent contribution of individual board features; chess puzzles or controlled experimental manipulations would provide a promising avenue for disentangling these effects. Ultimately, the goal is to understand mechanistically how cognitive effort can be computationally rationed in real time, motivating our second approach.

In this study, we developed a normative model of adaptive planning based on metacontrol. We directly apply this metacontrol in a resource-rational fashion (Lieder et al., 2025), training a metacontroller to continue or halt based on a learned value of computation compared to a linear cost in thinking time. Our controller combines a graph neural network encoder with a readout head that decides whether planning should continue or halt. We demonstrated that a metacontroller, trained with a combination of supervised learning for the encoder and reinforcement learning for the readout, outperforms reasonable baseline models under most cost regimes studied. Rather than reasoning directly over board positions, the model reasons over the evolving planning process itself, allowing it to estimate whether additional computation is worthwhile before committing further search.

We consider this a proof of concept that a metacontroller in chess can be feasibly trained and deliver better halting outcomes than established baselines. We cannot yet attribute the metacontroller’s edge over Action Gap specifically to tree topology rather than to the richer root-level statistics the encoder also has access to; isolating the topological contribution would require a baseline reading the full vector of root-child values and visit counts, essentially matching the input Thinker exposes to its agent, which we leave to future work. The benefit is not universal, however, and the exceptions are informative. Under certain linear-cost schedules (e.g. $\lambda = 0.003$), the metacontroller in fact underperforms relative to the Action Gap baseline, suggesting that additional work is needed to understand the conditions under which metacontrol is most effective. The Action Gap baseline is a telling comparison because it is precisely the classical myopic

metareasoning policy for halting in tree search (Russell and Wefald, 1989), so underperformance describes cases where a myopic fallback outperforms our more complex metacontroller. Preliminary evidence suggests that this discrepancy reflects an ill-conditioned reward landscape rather than a regime in which myopic stopping is genuinely superior, though establishing this will require characterizing the stability of training across multi-starts and a wider range of cost parameterizations.

Two additional limitations bound the scope of our findings. First, our cost function is linear in the number of expansions, which is likely an oversimplification. In perceptual decision making, for example, the subjective cost of accumulating evidence has been shown to increase over time; when task difficulty is not known in advance the optimal stopping bound collapses rather than remaining fixed (Drugowitsch et al., 2012). Extending our metacontroller to an explicitly nonlinear time cost is a natural next step. Second, there are degenerate regions of the cost landscape that lead all models to predict that one should always halt or always continue, and in these regions all models make similar predictions. This has direct implications for downstream comparisons with human behavior: under several conditions the predictions of our baselines converge, making them difficult to tease apart. Isolating the specific regimes and positions in which the Fixed Stop, Tree Stats, Action Gap, and Metacontroller make genuinely different predictions is an important next step.

Taken together, these two approaches are complementary but not yet integrated: the descriptive analyses identify the situations in which people appear to invest additional computation, while the normative model shows, independently, that a halt/continue decision of this kind can be learned at all. Whether human deliberation implements something like the normative meta-control signal remains an area of ongoing work.

As it stands, the current model can only be compared with human behavior qualitatively, by demonstrating that, within a narrow band of loss functions, it can in principle replicate the trends observed in people. This comparison must contend with the fact that human response times in chess are not Markovian in the position: they are autocorrelated across successive moves and

vary systematically with game stage (Sigman et al., 2010), an effect large enough that our own analyses required conditioning on ply to recover board-feature effects at all. A likely contributor is that players cache intermediate plans rather than regenerating them from scratch at each move. Importantly, this is a constraint on the state, not an obstacle in principle: our metacontroller conditions on the evolving search tree rather than the underlying position, and persisting that tree across moves, as engines regularly do for efficiency, would introduce exactly the kind of cross-move dependence that plan caching implies. Beyond qualitative comparison, a more robust quantitative account would involve fitting the parameters of the reinforcement learning objective to observed response times, minimizing the discrepancy between model predictions and behavior. These parameters could then be independently constrained through experiments designed to probe the subjective costs associated with additional deliberation and working-memory demands, thereby narrowing the range of plausible parameter values. These experiments would provide the missing bridge between the descriptive and normative perspectives developed throughout this chapter. We view this as an ambitious but promising direction for future work.

While difficult, one thing that makes the bridge easier to build is the explicit modularity of our design. Here we use Leela with PUCT (Rosin, 2011) to evaluate positions and set the prior for search, weighing backed-up value against a prior- and visit-count-weighted exploration term rather than greedily expanding the highest-valued node alone; whether adding full Monte-Carlo rollouts to leaf evaluation, as in Monte-Carlo Tree Search (Coulom, 2007), would help further remains open. Several substitutions would make the planning module more cognitively plausible and a better fit to player behavior: one could add noise to intermediate evaluations, Pavlovian pruning (Huys et al., 2012), and lapse rates. It is also possible to swap in models trained directly to predict human moves, such as Maia (McIlroy-Young et al., 2020), or variants trained on particular subpopulations of players as in Maia-2 (McIlroy-Young et al., 2022).

Our work builds on existing frameworks developed by metaplanning agents in machine learning such as IBP (Pascanu et al., 2017) and Thinker (Chung et al., 2023), which have laid the groundwork for feasible metacontrol of search. We see this particular instance of chess metacontrol as

combining the planning-as-navigation framing proposed by Thinker / Dynamic Thinker (Chung et al., 2023; Wang et al., 2025), the functional modularity of Muppidi et al. (2026), and the structural encoding of the search tree using graph neural networks. Our baseline ladder isolates what each ingredient contributes. Fixed Stop is beaten by Tree Stats, showing that state-dependent stopping outperforms a single constant budget. Tree Stats is in turn beaten by Action Gap, showing that coarse summary statistics of the tree’s shape are the wrong features: a myopic read of the root value gap is more informative than node counts, depth, and width. Finally, the metacontroller outperforms Action Gap, indicating that the learned tree encoding carries information beyond that myopic value gap, though isolating how much of this gain is specifically topological, rather than a matter of the richer root-level statistics the encoder also sees, awaits a baseline with access to the full root-child value and visit vector. Functional modularity, by contrast, is not an ablated component but a design commitment: decoupling the metacontroller from a frozen planner is what made training tractable in a domain like chess where learning a planner from scratch is infeasible.

We see this work as having implications beyond chess alone. Chess offers a valuable sandbox for studying metacontrol: rich enough to demand nontrivial planning, yet constrained enough that both the planning process and its costs can be measured directly, earning its keep as a model organism for the broader study of reasoning. Because our metacontroller operates over the evolving planning process rather than the contents of any particular domain, the underlying planner can be replaced with a substantially more powerful search or inference procedure without retraining the controller itself. This separation of concerns is what makes this approach portable, and it connects directly to a problem that has become acute in contemporary machine learning, where systems can increasingly trade test-time computation for performance: knowing when to stop deliberating. The same halting question we study in chess recurs in any system that trades intermediate computation for accuracy, including large language models (Snell et al., 2024) and other learned reasoning systems: deciding how long a chain of thought should run before committing to an answer, how many candidate solutions to sample, how deep to expand a tree of interme-

diate steps, or how many tool calls an agent should make before it acts. Indeed, a rise in recent work showing adaptive compute in LLMs demonstrates that combining value-of-computation with test-time-compute can be remarkably effective (Damani et al., 2024; Snell et al., 2024), and that LLMs themselves may be capable of such inference (Manvi et al., 2024). We hope this work motivates the study of adaptive allocation of computation as a fundamental problem in intelligent systems. Rather than treating planning as a monolithic computation, we argue that effective reasoning emerges from the interplay between knowing how and when to plan.

4.4 Methods

4.4.1 Data

We sourced our human chess gameplay data from a massive online corpus of games from the Lichess database. For our analysis, we kept games with only 600+0 time control (ten minutes per player, no time increment), where both players had Elo at least 2000, from October to December 2023. We filtered out games with the following qualities: (1) games where the response time was listed as negative (possible in rare cases of server lag), (2) games involving “berserking” where players could forfeit time in exchange for greater payoff in tournament games, and (3) games where players granted extra time to the opponent.

This yielded 1,971,698 games and 145,142,731 recorded moves, of which 135,482,903 had non-zero response time and formed the analysis set; each observation is one player’s move from a single board position, pooled across both colors. We use response time as a proxy for thinking time, noting the drawback that it implicitly bundles perceptual processing and motor preparation. As Lichess records response times only to integer-second resolution, we excluded moves logged at exactly zero seconds. Zero-second moves correspond to pre-moves and moves too fast to register a full second of deliberation, and their inclusion would distort the low tail of the distribution. Because raw response times are heavy-tailed and approximately log-normal, we use *log* response time as the dependent variable in our analyses, labeling axes in seconds on a log-

arithmetic scale. For each move we recorded a small set of board-structural features: the number of legal moves available (decision width, or branching factor), the game stage (ply), the player’s material (pawns included: pawn = 1, knight = bishop = 3, rook = 5, queen = 9, kings excluded), and the player-to-move’s remaining clock time. Where analyses are segmented by game stage, ply tertiles were fixed *a priori* from the ply-15–75-windowed distribution (split at 32, 49 plies) and applied as constant boundaries so that the segmentation is identical across figures. Board states were represented in Forsyth-Edwards Notation (FEN) and processed using the python-chess package.

4.4.2 Planner

Our metacontroller is decoupled from the underlying planning process: a separate module we call the *planner*, which generates trees at each step t of expansion beginning with the root. The planner queries Leela Chess Zero (lc0) purely as a one-shot position evaluator, evaluating exactly one node per call, so Leela itself performs no search of its own; all tree construction (selecting where to expand, attaching children, and backpropagating values) is carried out by the planner, not by Leela. Roots correspond to chess positions, encoded as FEN strings, and expanded via PUCT (Predictor + Upper Confidence bound applied to Trees) (Rosin, 2011): at each expansion step, the planner selects the already-created leaf maximizing $Q + c_{\text{puct}} \cdot P \cdot \sqrt{N_{\text{parent}}} / (1 + N_{\text{child}})$, where Q is the child’s backed-up value, P its move prior from Leela’s policy head, and N visit counts ($c_{\text{puct}} = 1$, unvisited children start at $Q = 0$); the planner then evaluates every legal move at that leaf with Leela, attaches them as new children, and backpropagates the value to the root, flipping sign at each ply.

Leela evaluates each newly-expanded position exactly once, at the moment it is added to the tree, yielding a scalar value estimate V , a WDL probability vector ω , and a scalar var_{ω} summarizing uncertainty in ω ; together these initialize the node’s representation. As search continues, this representation is revised by backpropagation whenever a descendant is expanded, not by any further queries to the engine, so a node’s value and WDL at a later expansion step generally

differ from Leela’s original evaluation (§4.4.3).

Each tree was restricted to a maximum depth of 30 and grown to exactly 96 internal nodes; each continue decision triggers exactly one such planner expansion (leaf selection via the PUCT score, evaluation via Leela, node expansion, and value backpropagation to the root).

4.4.3 Metacontroller

The metacontroller consists of two parts: an encoder graph neural network (GNN), instantiated as a bidirectional Tree GRU network (Kokkinos and Potamianos, 2017) over the tree T , and a readout head which operates on the root embedding and outputs a scalar halt–continue advantage $\hat{A}_t$ (positive favors continue).

In its broadest formulation, at expansion step t , the metacontroller takes a snapshot of the current plan formulated as a search tree T_t , where $S_t \equiv \text{Root}(T_t)$ is the current state in the “real world”, and decides whether to halt or continue search.

Formally, Π is a policy over the tree, $\Pi(\cdot \mid T_t)$, a distribution over {halt, continue}: at inference the agent halts greedily, at the first step where continuing is no longer favored; during training, rather than sampling an action from this distribution, its continue-probability is used to compute the policy’s expected return in closed form from the recorded search trace (§4.4.4). For clarity, we separate the metacontroller policy Π , which operates over trees, from the planner policy π , which operates over states (§4.4.2).

If the metacontroller halts, the agent commits to the planner’s current best root move (in our implementation, a greedy choice over the root children’s backed-up Q -values). If it continues, the existing tree T_t is passed back to the planner for one further expansion and the planner returns T_{t+1} , which is fed back into the metacontrol loop.

Encoder

In this section, we describe how the graph neural network encodes the nodes of the planning tree (Fig. 4.12). Our formulation closely follows the bidirectional Tree GRU, also known as the

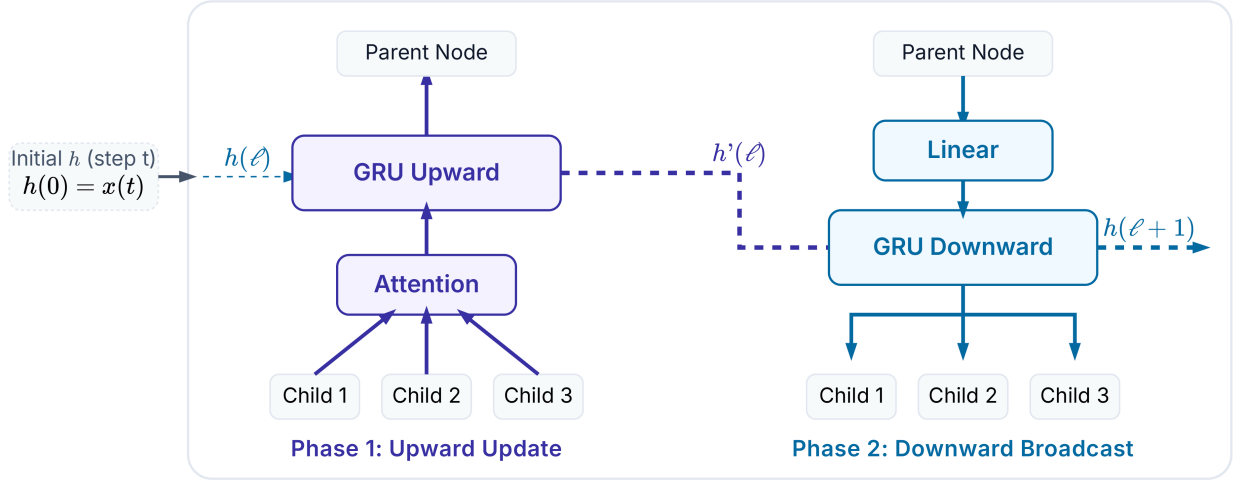


Figure 4.12: **Encoder Architecture.** The network encodes the nodes of the tree in two message-passing sweeps, constituting a TreeBiGru (Kokkinos and Potamianos, 2017): an upward sweep (leaves to root) in which each parent aggregates its children via multi-head attention, and a downward sweep (root to leaves) in which each child is updated from its parent through a linear map. Both sweeps refresh node hidden states with GRU updates, advancing the representation from $h^{(\ell)}$ to $h^{(\ell+1)}$ over one message-passing round (not to be confused with the expansion step t).

TreeBiGRU (Kokkinos and Potamianos, 2017). At expansion step t , the network encodes the tree T_t . Each node u is represented by the feature tuple $\mathbf{x}_u^{(t)} = (V_u^{(t)}, \omega_u^{(t)}, \text{var}_{\omega,u}^{(t)})$, where $\omega_u^{(t)} = (\omega_{u,\text{win}}^{(t)}, \omega_{u,\text{draw}}^{(t)}, \omega_{u,\text{loss}}^{(t)})$ is the WDL vector. $\mathbf{x}_u^{(t)}$ is u 's backed-up value and WDL as currently known at expansion step t , and it changes every time an expansion occurs anywhere in u 's subtree, since PUCT backpropagates the new leaf's value up through every ancestor on its path to the root. Consequently, the same node u , encoded at two different steps may have different representations. A multi-layer perceptron (MLP) maps this tuple to the initial hidden state,

$$h_u^{(0)} = \text{Embed}(\mathbf{x}_u^{(t)}) \in \mathbb{R}^d \quad (4.1)$$

To avoid confusion, we denote the message-passing round index ℓ , which is distinct from both the expansion step t and the lookahead length k introduced above. Message passing runs for $\ell = 0, \dots, L - 1$, where each round updates node representations from $h^{(\ell)}$ to $h^{(\ell+1)}$; in our experiments a single pass $L = 1$ is sufficient. Each round consists of an upward pass (children to parents), which produces intermediate representations h'_u , followed by a downward pass (parent

to children) that completes the round at $h^{(\ell+1)}$.

The upward pass from the perspective of node u consists of aggregating child messages with slot embeddings, consolidating their messages using multi-head attention (MHA), and propagating the information forward through a GRU cell. Children are assigned slot indices $s_{u,v} \in \{0, 1, \dots\}$ by sorting child moves in UCI lexicographic order. Each slot receives a sinusoidal encoding $\phi(s_{u,v}) \in \mathbb{R}^d$. The slot-conditioned child representation is

$$\psi_{u,v}^{(\ell)} = h_v^{(\ell)} + \phi(s_{u,v}) \quad (4.2)$$

The node u supplies the query and each child supplies keys and values derived from $\psi_{u,v}^{(\ell)}$. For node u and each $(u, v) \in T_t$,

$$\mathbf{q}_u = W_q h_u^{(\ell)} \quad \mathbf{k}_{u,v} = W_k \psi_{u,v}^{(\ell)} \quad \mathbf{v}_{u,v} = W_v \psi_{u,v}^{(\ell)} \quad (4.3)$$

$$m_u^{(\ell)} = \text{MHA}(\mathbf{q}_u, \{\mathbf{k}_{u,v}, \mathbf{v}_{u,v} : (u, v) \in T_t\}) \quad (4.4)$$

$$h_u'^{(\ell)} = \text{GRU}(m_u^{(\ell)}, h_u^{(\ell)}) \quad (4.5)$$

Here MHA denotes multi-head attention with scaled dot-product scores and a softmax over children v with $(u, v) \in T_t$.

The downward pass follows the same general structure as the upward pass, but with a single parent instead of several children. In the downward pass, each node v with parent u , $(u, v) \in T_t$, receives a linear projection of the downward message,

$$m_v'^{(\ell)} = W_{\text{down}} h_u'^{(\ell)} \quad (4.6)$$

$$h_v^{(\ell+1)} = \text{GRU}(m_v'^{(\ell)}, h_v'^{(\ell)}) \quad (4.7)$$

Unlike conventional GNNs, which require many message-passing rounds for information to traverse long distances, the acyclic tree structure permits exact propagation from leaves to root during the upward sweep and from root to all descendants during the downward sweep (Kokkinos

and Potamianos, 2017). In practice, upward passes are executed in reverse topological order and downward passes in topological order. Consequently, a single upward-downward round ($L = 1$) allows information to span the entire search tree, as is often done in belief propagation (Pearl, 1982).

Readout

The readout head takes the root representation after L message-passing rounds, $z_t \equiv h_r^{(L)}$ (where r denotes the tree’s root node), together with the number of expansions taken so far, and predicts the advantage of committing to the current best root move versus allocating one additional expansion of search,

$$\hat{A}_t = \text{MLP}(z_t, t) \tag{4.8}$$

Where $\hat{A}_t > 0$ favors continue and $\hat{A}_t \leq 0$ favors halt at inference. The precise target used to define this advantage is introduced in the following section.

4.4.4 Training

Given infinite compute time and resources, the optimal metacontroller would be trained end-to-end with a reinforcement learning objective. However, the sparsity of the scalar reward objective makes this training onerous. Instead, a more principled way of training the network splits the objective in two: a supervised auxiliary objective pretrains the encoder, and a direct, closed-form policy-gradient objective then trains the readout head to minimize expected regret under a stochastic stop policy (Fig. 4.13).

Training Dataset

To train our model, we subsampled from the same quality-filtered corpus of human games used to analyze behavioral trends (§4.4). We drew a seeded random subsample of 200,000 root FEN

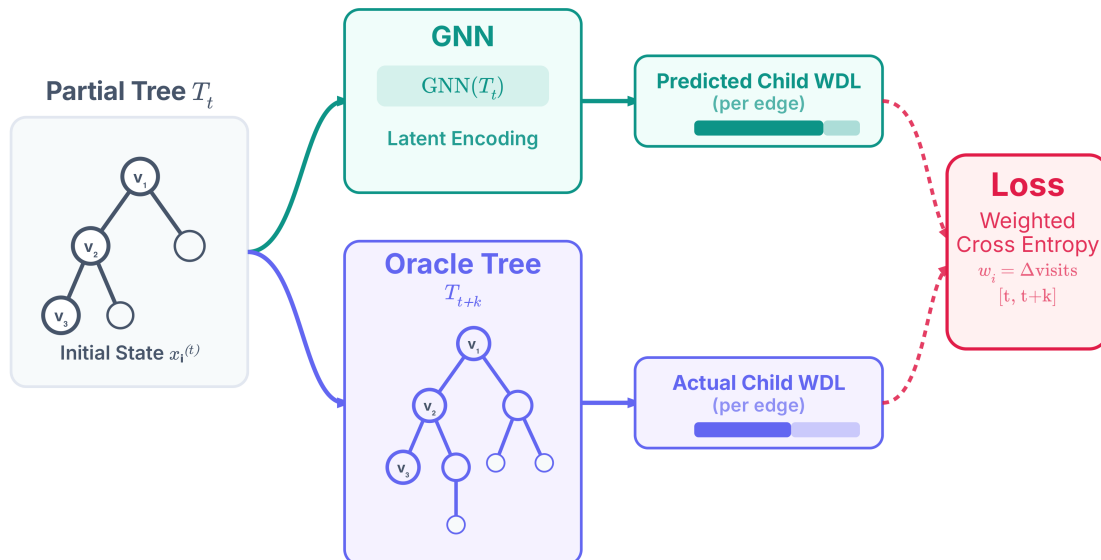


Figure 4.13: **Training procedure.** The encoder is pretrained to predict the downstream consequences of k steps of search. The input is a partial tree T_t and the target is the k -steps-ahead consolidated WDL target for each edge in the tree.

positions and ran the planner (PUCT search with Leela as evaluator) for 96 steps on each, so all trees were grown to 96 total internal (non-leaf) nodes. The resulting trees were then inspected for quality; the *oracle* best action was the highest-valued action under the 96-step tree. Trees where this best action was discovered at an early step, abandoned afterwards, and then re-discovered corresponded to false *early discoveries*, essentially a shallow tree stumbling onto the right answer by accident and only later re-validating that it was the best action. We removed these *early discovery* cases, as they serve as a poor signal of when search is worthwhile, and further restricted the survivors to the ply-15–75 window used throughout this chapter, leaving 80,011 root positions; these were split 80/20 by FEN into 64,009 training and 16,002 validation trees. §C.3.4 (Table C.2) gives the full filtering funnel.

Saved trees served two purposes: to train the encoder and to train the readout head. For the encoder, we stored the planner’s per-step node states as input (initialized by Leela, then revised by backpropagation as the tree grows; §4.4.2) and the search-consolidated win-draw-loss probabilities (which would serve as the supervised target) for each edge. For the readout head, we needed to store the order of expansions so that the construction of the tree could be simulated at

runtime. At each step, the partial tree’s root-action values were stored to calculate the advantage via dynamic programming. Concretely, these snapshots were recorded as *oracle traces*: after each expansion step $t \in \{0, \dots, N\}$, the partial tree T_t , the best root move a_t , and the Q -values of all root children. Here T_t always denotes a search tree (not a time index) and N is the maximum number of PUCT expansions (here, $N = 96$), so the completed teacher tree is T_N . These traces support offline halt/continue learning without re-running the engine.

Encoder Training

The goal of encoder pretraining is to learn a tree representation that supports downstream meta-control without relying on hand-coded search statistics. Rather than supervising on the fully completed tree T_N , we supervise on partial trees: for each tree we sample a snapshot step $t \sim U[1, N - k]$ and fix a lookahead length k (we use $k = 12$), and ask the encoder to predict, from the partial tree T_t , each child’s WDL distribution k expansions later, under T_{t+k} . Supervising this way (rather than against the omniscient, fully-searched tree) keeps the training signal restricted to information the controller could plausibly have access to during an ongoing search, and lets a single teacher tree yield many training examples at different points along its own trajectory.

Concretely, from the parent’s step- t encoded state $h_u^{(L)}$ (obtained by running the encoder on T_t) and the slot embedding $\phi(s_{u,v})$, a slot-conditioned decoder predicts child v ’s WDL distribution as of step $t + k$,

$$\hat{\omega}_{u,v} = \text{softmax}\left(\text{MLP}_{\text{dec}}\left(\left[h_u^{(L)}; \phi(s_{u,v})\right]\right)\right) \quad (4.9)$$

where $\hat{\omega}_{u,v}$ lies on the probability simplex and $\bar{\omega}_{u,v}$ denotes the teacher target obtained by visit-weighted consolidation of backpropagated ω along edge (u, v) over the window $[t, t+k]$ (the WDL estimate as it stood k expansions after t , not the fully-searched tree’s final value), and distinct from the per-node ω_u used as an encoder input feature (§4.4.3), since $\bar{\omega}_{u,v}$ is a consolidated statistic over an edge and a time window rather than a single node’s raw evaluation. For each supervised

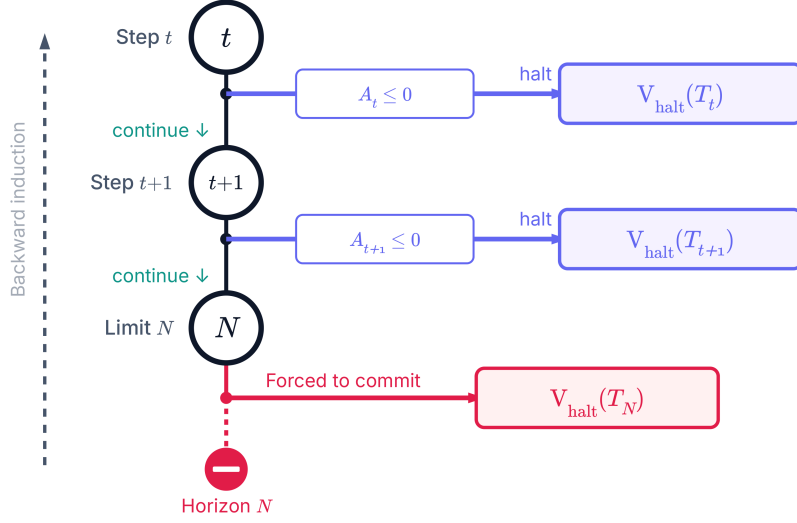


Figure 4.14: **DP Halt/Continue Oracle.** Backward dynamic-programming pass computing the oracle stopping values $V_{\text{halt}}(T_t)$ and $V_{\text{cont}}(T_t)$ from the completed teacher tree T_N , which anchor the regret used to train the readout head.

edge we minimize cross-entropy,

$$\mathcal{L}_{u,v} = \text{CE}(\hat{\omega}_{u,v}, \bar{\omega}_{u,v}) \quad (4.10)$$

the per-edge cross-entropy loss. We minimize the average of $\mathcal{L}_{u,v}$ over all edges in T_t , weighted by each edge’s increase in visit count $\Delta N_{u,v}$ over $[t, t+k]$ (how many times that edge’s WDL estimate was actually refined by a backup in the lookahead window), rather than by its subtree size: a single expansion at a high-branching-factor descendant adds many new leaves to a child’s subtree while backing up its own value only once, so a raw subtree-size weight would overstate how well-supported that child’s consolidated target actually is. Examples were packed into batched shards with a train/validation split; the encoder and decoder were optimized jointly with Adam (Kingma and Ba, 2017) for 6 epochs, and the checkpoint with lowest validation loss (cross-entropy minus target entropy) was retained for controller training.

Readout Head Training

Once the encoder was pretrained, we trained the readout head on oracle traces while keeping the encoder frozen. Throughout, T_t denotes the search tree after t expansions, T_N the completed teacher tree, N the maximum number of expansions, $n_t \equiv |T_t|$.

The central challenge is assigning credit to early stopping decisions. We treat evaluations under the completed teacher tree T_N as ground truth: T_N is not fully realized until all N expansions have been carried out, so at step $t < N$ the controller cannot observe T_N directly. Rather than asking whether a move looked best under the partial tree T_t , we ask whether committing to the step- t greedy move would ultimately have been justified given the information revealed by the full search. Concretely, root-child Q -values $Q(a, T_t)$ come from the trace at each snapshot, but halt values are scored against T_N (Fig. 4.14).

At step t , the greedy commit move is $a_t^* = \arg \max_a Q(a, T_t)$. We write $V_{\text{halt}}(T_t)$ and $V_{\text{cont}}(T_t)$ for the optimal halt and continue values at that snapshot; these are stopping values, not MCTS action- Q estimates. At the terminal expansion step,

$$V_{\text{halt}}(T_N) = Q(a_N^*, T_N) \quad (4.11)$$

$$V_{\text{cont}}(T_N) = 0 \quad (4.12)$$

For $t < N$,

$$V_{\text{halt}}(T_t) = Q(a_t^*, T_N) \quad (4.13)$$

$$V_{\text{cont}}(T_t) = \max(V_{\text{halt}}(T_{t+1}), V_{\text{cont}}(T_{t+1})) - c_t \quad (4.14)$$

where $c_t \equiv \lambda$ is the per-expansion continue cost, constant in the linear-cost schedule used throughout (§C.3.3), so the total cost accrued by step t is simply λt . Thus $V_{\text{halt}}(T_t)$ is the value of halting and playing a_t^* as judged by the final tree, while $V_{\text{cont}}(T_t)$ is the better of stopping at $t+1$ or continuing optimally thereafter, minus one expansion cost. Because $V_{\text{cont}}(T_t)$ depends on

$V_{\text{halt}}(T_{t+1})$ and $V_{\text{cont}}(T_{t+1})$, it is a Bellman recursion on future stopping decisions and can only be resolved by working backward from T_N ; this backward pass is the dynamic program of Fig. 4.14. It defines the oracle advantage

$$A_t \equiv V_{\text{cont}}(T_t) - V_{\text{halt}}(T_t) \quad (4.15)$$

where $A_t > 0$ favors continuing. A_t is not a regression target: the readout head is never fit to it directly. Instead, as described in the following paragraph, A_t supplies the per-step halting reward against which the readout’s stochastic halt/continue policy is optimized by policy gradient. At inference time, the controller halts greedily, at the first t where $\hat{A}_t \leq 0$. If the trace reaches the expansion limit N before $\hat{A}_t$ ever crosses zero, $V_{\text{cont}}(T_t)$ is replaced by a fixed timeout value at that final step.

Readout optimization. An episode begins at the root of a search tree, a single board position; at each step the readout head reads the encoder’s embedding of the current partial tree together with the number of expansions taken so far, and outputs the scalar $\hat{A}_t$ introduced above. Halting commits to the current best root move; continuing expands the tree by one more node.

Training combines two distinct quantities from the same oracle recursion, which are easy to conflate because both are built from $V_{\text{halt}}(T_t)$ but play different roles. $\hat{A}_t$ is a *marginal* quantity: is one more expansion worth it from here? Because it compares continuing against halting *at the same step*, the cost already spent reaching T_t is sunk on both sides of that comparison and cancels out— $\hat{A}_t$ never needs to know it. The quantity that does need to know it is the *episode-level* payoff, $g(t)$: the total return the episode would earn if it stopped exactly at step t , which depends on everything spent getting there,

$$g(t) \equiv V_{\text{halt}}(T_t) - \sum_{s < t} c_s, \quad (4.16)$$

the value of the move committed to at t , net of every continue cost already paid. In short, $\hat{A}_t$

decides *whether* to keep going at each step; $g(t)$ scores *what you'd get* if the episode ended there.

Because $\hat{A}_t$ is trained by gradient descent, it cannot simply threshold into a hard halt/continue choice, which has no gradient to learn from; instead, it is passed through a temperature-scaled sigmoid to give the probability of continuing at that step, $p_t = \sigma(\hat{A}_t/\tau)$, with τ annealed from an initial value down to 1 over training, keeping p_t away from the sigmoid's saturating region early on, where its gradient with respect to $\hat{A}_t$ would otherwise vanish. These per-step continue probabilities induce a distribution over which step the episode actually stops at—continuing at every step before t , then halting at t :

$$P(a_t = \text{halt}) = \left(\prod_{s < t} p_s \right) (1 - p_t). \quad (4.17)$$

Because the recorded search already reveals $g(t)$ for every candidate stopping step, the expected return of this stochastic policy is just the probability-weighted average of $g(t)$ over stopping times,

$$\mathbb{E}[\text{return}] = \sum_t P(a_t = \text{halt}) g(t), \quad (4.18)$$

computed directly from the recorded trace rather than by sampling rollouts. Training minimizes the resulting expected regret, formally $\text{regret} \equiv V^*(T_0) - \mathbb{E}[\text{return}]$, where $V^*(T_0) = \max(V_{\text{halt}}(T_0), V_{\text{cont}}(T_0))$ is the hindsight-optimal return from the same recursion, the gap between this quantity and the best return achievable with full hindsight of the completed search.

Baseline Models

We constructed the following baselines:

- **Always Stop.** A model which always halts immediately at $t = 0$.
- **Always Continue.** Continues until it is forced to stop at the maximum number of expansions.

- **Fixed Stop.** A model which continues if $t \leq \kappa$ or halts otherwise, where κ is a scalar fit by grid search.
- **Tree Stats.** Given T_t , takes the tuple (t, n_t, h_t, w_t) : steps taken, node count, max depth, and max width.
- **Action Gap.** Given T_t , takes (t, Δ_t) , where $\Delta_t = Q(a_{\text{best}}, T_t) - Q(a_{\text{second-best}}, T_t)$ is the value gap between the best and second-best root moves.

The first two baselines don't require any training, as they involve fixed policies of either always continue or always halt. Fixed Stop has a single scalar parameter, the stopping step κ , which is fitted on the training set using grid-search:

$$\kappa^* = \arg \min_{\kappa} \frac{1}{|\mathcal{D}_{\text{fit}}|} \sum_{i \in \mathcal{D}_{\text{fit}}} \text{regret}_i(\kappa). \quad (4.19)$$

The resulting κ^* is then applied uniformly to every held-out episode; this corresponds to the single point plotted for Fixed Stop against the fixed-stop frontier (§4.4.4).

Tree Stats and Action Gap are learned readout heads with the same architecture as our meta-controller's advantage head (an MLP with two hidden layers of width 64), but reading hand-crafted features in place of the encoder's embedding $h_r^{(L)}$. Tree Stats takes (t, n_t, h_t, w_t) , the step count together with the number of nodes, maximum depth, and maximum width of T_t ; Action Gap takes (t, Δ_t) , where $\Delta_t = Q(a_{\text{best}}, T_t) - Q(a_{\text{second-best}}, T_t)$ is the value gap between the current best and second-best root moves under the partial tree. Both feature sets are z-scored using the fit split's mean and standard deviation before training. Unlike the deployed readout head, no temperature anneal is applied to the stop probability ($\tau = 1$ throughout training). All baselines are evaluated equivalently, using the greedy hard stopping rule ($\hat{A}_t \leq 0$ halts) on the held-out set.

5 | Conclusion

5.1 Summary

Planning is hard. In real-world situations, it requires time and cognitive resources, both of which are in short supply. This dissertation explores how humans and artificial agents adapt their planning strategies to changing environments while managing the limited resources available for deliberation. These chapters span a wide range of approaches by design: across the agent that plans (human, macaque, and artificial network), across the uncertainty it confronts (the irreducible noise of a stochastic world and the structured resistance of an adversary), and across the substrate in which we seek an answer: what the agent chooses, what its neurons encode, and what an optimal controller would do.

In Chapter 2, we studied how humans adapt to stochasticity in their environment beyond their control. Using three different forms of environmental stochasticity, our work demonstrates that people systematically reduce their planning effort in response to stochasticity. Specifically, we find that they engage in determinization, treating their environment as if it were deterministic, and then reduce their sensitivity to value in a manner consistent with policy compression and rational inattention. The key idea here is that when the environment becomes sufficiently stochastic, planning is no longer worthwhile, so people take mental shortcuts and simplify how they act relative to future outcomes.

In Chapter 3, we took a step towards a different form of uncertainty; rather than focus on irreducible uncertainty in the form of random noise from the environment, we focused on a situation where the agent, in this case a rhesus macaque, faced an adversarial computer opponent in a two-player game called Four-in-a-Row. This form of planning required additional adaptation to a structured opponent. We found evidence from thousands of games played by a single monkey

that its behavior was most consistent with a tree search model, suggesting that it engaged in deeper-than-myopic search when making decisions. This finding should be read as preliminary; the opponent it played against was weaker than anticipated (§B.1.3), and M1 is currently undergoing retraining. Behaviorally, its choices reflected knowledge of abstract features and their relative importance in the game. Gaze told a similar story to our behavioral findings, but largely because fixations tracked closely with move intention. Neurally, we found that DMPFC, DLPFC, and caudate track state value and the component of tree search value that exceeds simple myopic evaluation, and that a direct head-to-head comparison confirms that Tree Search explains neural firing rates better than the Myopic model across all three regions. While further work needs to be done on this front, including retraining with a corrected opponent and extending to a second animal, this provides an exciting foray into understanding at a granular level how neural activity in these regions supports adaptive planning.

In Chapter 4, we looked at adaptive planning in chess at scale, using the Lichess dataset to study adaptive cognition both mechanistically in humans and from a normative resource-rational lens in deep neural networks. First, we characterized the player behavior, analyzing the response times as a proxy for thinking time in each board state. Next, we framed the metaplanning decision as a navigation over an existing planning tree. We trained a metacontroller based on a graph neural network architecture, which would take in an incomplete planning tree, encode it, and pass it to a readout controller which would decide whether to continue planning or stop. Here we are limited in what we can validly claim, and keep the two halves of the chapter separate rather than presenting them as a single validated account. First, people’s thinking time in chess is systematically predicted by structural correlates of a position’s features, such as the number of legal moves available or the material on the board. Second, our metacontroller effectively learns to allocate search adaptively from a learned encoding of the evolving search tree; this establishes the halt/continue decision as learnable and gives us a normative benchmark for planning under time costs. Whether human deliberation implements something close to the same value-of-computation signal our controller is trained on is an open question, and one that we are

actively investigating.

What these chapters share is a concern with planning under circumstances the agent does not control, a feature common in the real world but often absent from laboratory tasks. Across these chapters, planning proves highly contextual: how much an agent plans depends on the unpredictability of its environment (Chapter 2), the abstract features of the state it is evaluating (Chapter 3), and the explicit cost of computation (Chapter 4), echoing the problems agents face in naturalistic settings. In the following section, we focus on the limitations of the current work and how the broader research program may extend into neighboring domains in the future.

5.2 Limitations and Future Work

In this thesis, we have attempted to cover a broad range of phenomena relating to adaptive planning under resource constraints. In keeping with the topic of constraints, it is worth discussing the ways in which this work itself is limited and setting guideposts for future work. In some of these cases, the proposed extensions are already underway, while others are left as avenues for future exploration by readers who have proven patient enough to read this far.

5.2.1 Modeling Space and Tractability

One shared challenge in any study of planning is that the cognitive process is opaque to the experimenter: we cannot directly observe it unfold, only build computational models of it using behavioral artifacts. Moreover, the choices that these models are fit on are often sparse; the readout of a planning tree culminates in a single action. To that end, the process models we choose to attribute to planning agents in the real world are an approximation of what is actually going on under the hood. Though planning is not unique in this regard, it remains challenging to study in part because so many cognitive steps stand between thought and a single observable choice. Reconciling planning from a reinforcement learning paradigm also involves a level of assumption about aligning the goals of the agent to the stated goal of the task, a challenge that

appears in both the design of the task and the corresponding modeling effort. Here we discuss some downstream consequences of these challenges.

The space of plausible planning models is poorly constrained in general. The models we have discussed in these chapters are by no means exhaustive; one can easily think of model variants or even entire model classes that have been ruled out for brevity and tractability. In Chapter 2, we enumerate a wide range of models that we consider broadly plausible and the heuristics that could be reasonably applied to each condition. Even then, one could consider additional ways of solving the navigation problem and intermediate simplifications. Similarly, in Chapter 3, we rely on nested versions of models that are directly inspired by cognitive algorithms developed for the particular task of Four-in-a-Row but do not rule out the possibility of other planning models in explaining monkey behavior. In Chapter 4, we make a simplifying assumption that people's planning process can be approximated by one particular chess engine. In cases of novel tasks such as the navigation task in the stochasticity experiment, we deliberately tried to cover a range of models ranging from unsophisticated and blunt (such as the max heuristic) to more expressive and theoretically optimal (such as the expected value). In domains such as Four-in-a-Row and chess we build on existing precedent to establish the reasonable scope of models. The established model of human gameplay sets us up to make comparisons with nested models spanning a range of sophistication. Similarly, in chess we use well-established game engines in our analysis. Where possible, we have addressed these concerns by conducting robust model comparison with a suite of plausible alternative models and assessing goodness-of-fit using out-of-sample NLL; we use this as a way of ruling out particular model instances and in some cases entire model classes. We provide additional evidence of model validity by showing that it replicates key behavioral trends through model checking, and where feasible by comparing internal model-derived metrics with exogenous readouts from the planning process such as response times (in the case of Chapter 2 and Chapter 4), eye movements (Chapter 3), and neural activity (Chapter 3).

All our models rely on assumptions that make fitting tractable, and a few of these are worth interrogating. The most notable is the Markovian assumption, in which the present planning

decision depends only on the current state, forcing the tree to be reconstructed from scratch at every step. This is standard in the decision sciences, largely because it allows likelihoods to be estimated over individual decisions rather than entire trajectories; the latter is infeasible in the large state spaces we study, where a given trajectory may appear only once or twice in an entire dataset. The cost of this convenience is a lack of *thought permanence* in our models; mental states can persist from one decision to the next, yet current tree search models discard that persistence entirely, reconstructing the tree from scratch at every step. Encoding the persistence of mental states across planning is an important direction for future work, and existing work in our lab and others has begun to examine the role working memory plays in shaping planning dynamics. The same simplification applies to our metacognitive models: in Chapter 4 the Markovian assumption falls on thinking steps rather than individual trials, since the graph neural network re-encodes the tree afresh each time it decides to plan or halt.

A related assumption is our general preference for simple process models over more flexible, heavily parameterized ones — a preference that matters most in Chapters 2 and 3, which target specific dimensions of planning such as policy compression and stopping probability. By simplicity we mean not just parameter count but identifiability: whether those parameters can be recovered without trading off against one another. In these cases our models are cautiously restrained; we favor models which have a handful of parameters that are recoverable and cognitively motivated over models that may be more flexible but harder to interpret. This certainly leaves something on the table in model details; a concrete example is the existence of the generic lapse rate, which posits that participants may make uniformly random moves, an assumption that may be less plausible in a game like Four-in-a-Row where attention may have a greater spatial component. A similar line of reasoning factored heavily in our decision to leave out the feature dropping probability (setting it to zero), a parameter that was once present in the original Four-in-a-Row model but led to poor recovery of relevant model weights. Understanding these nuances would certainly help derive more effective models for describing behavior, but a line in the sand must be drawn for what constitutes a “reasonably” good approximation for behavior. This thesis

has several instances where such tradeoffs were made, and in those instances we have tried to be clear about the reasoning behind our choices. In Chapter 4, we take a dual approach; we keep the descriptive modeling of the human behavior localized to a set of distinct game features (move ply, number of legal moves, etc.) but allow our normative model to remain relatively unconstrained. In part, this was a deliberate choice to see where a maximally flexible model might converge under a set of simple metacognitive assumptions. Many of these models ultimately reflect, and are limited by, assumptions about the level of abstraction at which cognition should be described. We turn next to those representational assumptions.

5.2.2 State Space and Representations

Beyond the choice of model parameters and architectures, assumptions have also been made at the representational level. In all of our tasks we invoke a particular state representation that we consider to be flexible enough to describe the cognitive level construct that people manipulate when planning ahead. In Chapter 2, for example, we choose to represent the board as a map of values over the subtree of the participant’s current path, and the after-state of each decision as the subtree derived from moving left versus right. This is, of course, informed by the task structure, but one could also imagine including the irrelevant *unvisitable* nodes as distractors in an otherwise structured state space. In Chapter 3, we represent the board as a vector of positions occupied by either the monkey or the computer opponent; in reality, we know that the visual display also consists of the screen, the hold cue, and the space on the board between discrete positions. Where relevant, such as in the case of the gaze analysis, we invoke this broader understanding of the screen layout, whereas in choice modeling we deliberately choose to simplify it. One could go a step further and suggest that the screen be represented in its entirety in pixel space, with RGB values for each pixel. This is not a strawman suggestion; some reinforcement learning agents explicitly encode the perceptual nature of the task, such as (Mnih et al., 2013). Since we are concerned primarily with the manipulations of representations downstream of a stable state representation, we abstract ourselves away from learning the perceptual encoder from

scratch. Instead, we focus on a level of explanation that sits above the pixel-space representation and within the domain of board positions, n-in-a-row features, and establishing and preventing threats from the opponent. This allows us to explain behavior in these terms, but also comes at a cost. Indeed, a sufficiently expressive state encoding can sometimes do the work of planning outright: an unblocked 3-in-a-row can be read either as a single *double threat* feature requiring no lookahead, or as two separate 4-in-a-rows available next move. In Chapter 4, our choice of the FEN representation is in keeping with common chess notation, but the choice to encode this using Leela-derived metrics relies on assumptions that the engine provides reasonable proxies for human board representations, an assumption we would like to address in future work. Here, we have tried carefully to balance the level of representation with the space of questions we are trying to answer. We focus on the manipulation of concrete cognitive concepts such as boards, positions, and spaces to the extent that they provide useful abstractions for understanding what people, animals, and agents do with those concepts (Ho et al., 2019; Niv, 2019). An exciting line of work focuses on how representations are encoded into construals that can be cognitively manipulated (Ho et al., 2020). We make this connection more explicit in our proposed future work on affordances §5.2.6. Representations define what information is available for planning, but they are only one component of decision making. An aligned utility function is critical to establishing an association between states and action, which is our next focus.

5.2.3 Utility and Task Alignment

Rewards, and more broadly utilities, are central to how we think about feedback in decision making; they are the glue that binds states to associated actions. In the stochasticity task in Chapter 2, participants are asked to maximize the number of points they gain as they navigate through the board. We motivate them to do so by telling them that their bonus will be proportional to their performance on a randomly drawn subset of the games, but their true motivation to engage in their task (and therefore their true value function) is unknown. This is even more true in the case of nonhuman primates in Chapter 3; the fact that the monkey was willing to engage in the

game at all given the sparse reward schedule was already a feat of experimental engineering. To keep the monkey engaged in the task, it was rewarded with juice only when it won the game (see §B.1.3 for exceptions). However, this doesn't preclude the very real possibility that it was motivated by additional factors, including novelty, intrinsic motivation for winning, or simply trying to maximize reward over several episodes. We are addressing the latter by retraining the monkey against a more aggressive CPU so that the monkey cannot win by attrition. Scalar utilities become especially thorny in the case of metacontrol, where cognitive costs are often hidden and must be inferred. In Chapter 2, we find this to be a limiting factor in making our model truly resource rational, as it would require making several implicit assumptions about the shape of the cost function relative to the number of nodes and the size of the tree, which we had not empirically tested. We conducted a simulated, albeit simplified, version of this analysis in §A.3 and demonstrated that it qualitatively reproduced our results. In Chapter 4, we parameterize the cost explicitly in terms of the size of the tree and the number of expansions, both of which have grounding in psychological concepts. In these studies, we made few attempts to directly infer the exact cost parameter from behavior alone; future work directly measuring or deriving the cost of cognition under different contexts will be invaluable to testing concrete predictions of resource rationality.

5.2.4 Extensions and Generalization

Throughout this work we have studied adaptive planning across a range of task domains and agents, with a consistent focus on the role of uncertainty and cognitive constraints. A natural question is whether our findings generalize beyond the particular domains in which they were studied. Several opportunities remain to extend this work in different dimensions: across agents, from individual differences to population-level structure; toward shaping behavior, through causal intervention and curriculum design; and across tasks, through new manipulations, forms of uncertainty, and parameterizations of cost.

The first dimension concerns the agent itself. The models in this thesis largely describe behav-

ior at the group level, but a central question is how far a given signature of planning is a general property of the group versus an idiosyncrasy of the individuals within it. If the latter, we are also interested in how an individual's experience contributes to their own idiosyncratic behavior. In Chapter 2 we found that participants as a whole tend to determinize, treating stochastic environments as if they were deterministic and compressing their policies. How do these tendencies arise within individuals, and how might they vary with past experience? Early exposure to unpredictable environments can impair the development of cognitive function later in life (Davis et al., 2017). However, the directional impact of unpredictability in childhood is mixed; some studies suggest that it reduces exploration (Xu et al., 2023), while others show that it increases risky behavior (Simpson et al., 2012; Doom et al., 2016) and impulsivity (Frankenhuis et al., 2016). In an ongoing collaboration with the Hartley Lab at NYU, we examine whether early-life unpredictability and stress shape present adaptability to stochastic environments, in a preregistered study (<https://osf.io/svt54/overview>). Our central hypothesis is that people who faced high early-life unpredictability will adapt their planning effort to stochasticity less readily. Participants drawn from the Hartley Lab database completed the Questionnaire of Unpredictability in Childhood (Glynn et al., 2019) and the Perceived Stress Questionnaire (Levenstein et al., 1993), were sampled to span a broad range of scores, and then completed the controllability task of Chapter 2; as of writing, data have been collected and preliminary analysis is underway. The same question of individual versus group structure also applies to chess: in Chapter 4 we characterize broad trends across players, but a longer-term goal is to fit these models to individual players and ask whether stylistic differences in gameplay can be predicted, which would both illuminate how these styles arise and inform the design of more human-like engines. In the nonhuman primate work the question of individual differences is sharper still; our work so far involves a single animal. In Chapter 3 we study how one rhesus macaque learns to plan; establishing whether our findings generalize requires a second subject, and training for M2 is already underway. Both animals are now being retrained against a stronger computer opponent, after we discovered that the opponent in the original data collection was weaker than intended (§B.1.3);

while this does not affect the validity of our modeling results, the new phase has already shown encouraging signs, including longer games and more sustained blocking. M2 is needed for separating what is general to the species from what is idiosyncratic to M1, and the stakes are highest precisely where variability is greatest: gaze, which can fall anywhere on the screen, and neural activity, which is high-dimensional and noisy. With two subjects we can begin to ask whether specific patterns of activity generalize across animals.

The second dimension concerns intervention: rather than observing planning as it occurs, we can shape it, and in doing so test what is malleable. One clear opportunity is in the curriculum. We suspect that in M1 the staged training curriculum plays a strong role in shaping the representations the animal acquires, consistent with evidence that the order in which experiences are presented affects what is learned (Mi and Summerfield, 2026; Dumbalska et al., 2026). This raises a question we can address directly: what representations does a curriculum install, and which of them transfer? One way to ask this in humans is to train participants on a staged, nonverbal curriculum matched to the one used for the monkeys, and test whether the same systematic patterns emerge, which would help separate what the curriculum imposes from what the task affords. The relationship between skill in Four-in-a-Row and transfer to related planning tasks has been studied in humans (Lin and Ma, 2024), but this kind of cross-task generalization has not been examined at the neural level in nonhuman primates, where curriculum, representation, and transfer could be studied together. Intervention also has a clinical face. Planning is altered in a range of psychiatric conditions, with anxiety and obsessive-compulsive disorder both associated with characteristic deviations in how people deliberate (Gillan et al., 2016; Seow et al., 2021; Sharp et al., 2023); structured planning games of the kind developed in this thesis may offer a way to make those deviations measurable, and potentially a way to act on them. Used diagnostically, such games could index how an individual allocates and adapts cognitive effort; used interventionally, the same curriculum and manipulation tools that shape planning in healthy participants might help retrain it. Understanding how training and context reshape planning is thus a first step not only toward a science of behavior change but toward its application.

Finally, we discuss moving beyond our current suite of tasks, holding the agent fixed and instead varying the structure of the problem it faces. One place to do this is in the kind of uncertainty the task presents. The stochasticity we study in Chapter 2 is sometimes referred to as *aleatoric* uncertainty, meaning that it is irreducibly random. This is often contrasted with *epistemic* uncertainty, or uncertainty that can in principle be reduced by, for example, learning or gaining new information (Bland and Schaefer, 2012; Story et al., 2023; Volz and Gigerenzer, 2012). In the real world, the distinction between aleatoric and epistemic uncertainty is much fuzzier than meets the eye; what might at first appear to be an irreducibly random pattern might in fact contain hidden structure. Understanding how people change their behavior in this transition from apparently aleatoric to realized epistemic uncertainty has broad implications for the study of learning and curriculum design. Importantly, resource rationality makes predictions about how people ought to adapt in this case (Kuperwajs et al., 2024); in truly stochastic environments additional effort provides little benefit, whereas in learnable conditions planning can reduce uncertainty and lead to better rewards (Daw et al., 2005; Sezener et al., 2019). We have designed a preregistered study (<https://osf.io/bzjy8/overview>) to test this hypothesis by embedding a rule inside the stochastic patterns of the board, a rule that participants must learn on their own and that is not explicitly told to them in advance. We expect a gradual increase in planning effort, as measured by the inverse temperature β and corroborated by response times, as participants learn the hidden pattern of the game. This same boundary raises a further question about how planning changes as the task grows more or less learnable. In our studies we observe a graceful degradation of planning effort within a particular model class, the heuristic planners of Chapter 2 and the tree search of Chapter 4, but it remains an open question whether this gradual adjustment holds everywhere, or whether there is a point beyond which participants abandon one class of planning strategy for a simpler one altogether. The graceful degradation we observe may itself be specific to the relatively low-stakes settings we are able to probe through online experiments; future experiments might vary the kinds of representational costs imposed on participants such as memory or thinking time. The monkey planning work in Chapter 3 provides

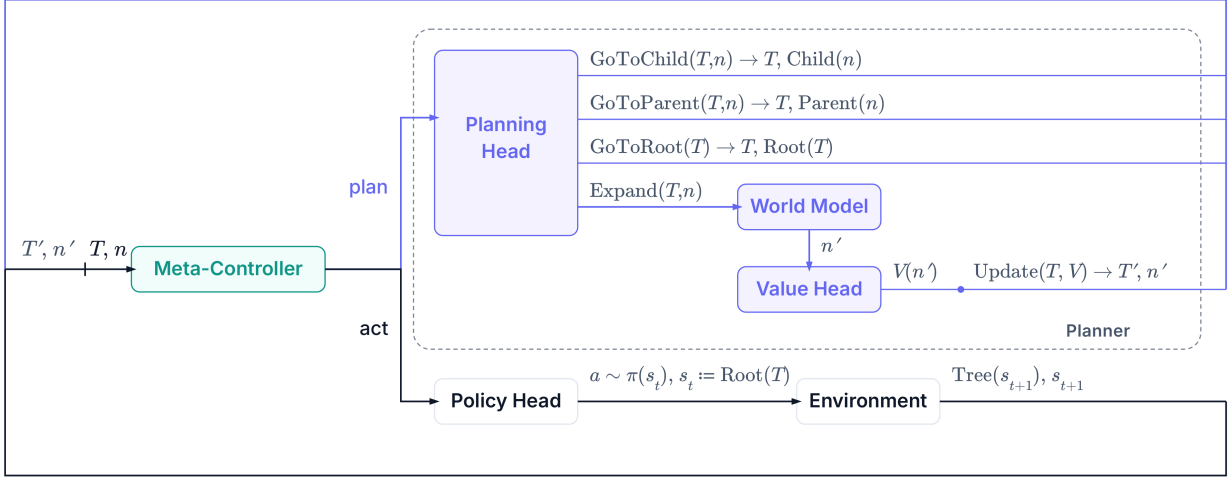


Figure 5.1: **Metaplaning diagram.** A closed-loop framework that generalizes the metaplaning setups of the Imagination Based Planner (Pascanu et al., 2017) and Thinker / Dynamic Thinker (Chung et al., 2023; Wang et al., 2025). We use a similar formulation in Chapter 4, focusing on the learned metacontroller.

an exciting scaffolding for extensions to related tasks. In our current study it is difficult to know whether the monkey considers the computer opponent an adversarial agent with its own goals and intent, or simply a structured part of the environment. In the future, experimental setups that make explicit that the monkey is facing another intelligent agent, for example by having two monkeys play against each other, could provide valuable insights into theory of mind. The task can also be scaled towards different board sizes and toward incomplete information. Extending our findings to games such as poker would allow us to combine our treatment of stochasticity in Chapter 2 with the concepts of metaplaning developed in Chapter 4. The modular setup of the metaplaner allows us to ask questions about more complex reasoning tasks; by swapping out the planning process for a modern large language model, one might imagine extending this to a wide array of tasks that are more grounded in the real world, a step toward the inductive goal of Newell and Simon’s General Problem Solver (Newell et al., 1959). It is this possibility that we discuss in the next subsection.

5.2.5 Towards a Broader Metaplanning Framework

In that spirit, the metacontroller of Chapter 4 is one instance of a broader class of metareasoning agents (Russell and Wefald, 1991) that operate over *trees* in place of individual states. Two ideas are common to this class: planning as tree-structured navigation rather than fixed-depth rollout, and explicit resource constraints on computation (memory, latency, depth). Both are a consequence of the meta-level MDP formulation (Hay et al., 2012; Callaway et al., 2022) and have recently been instantiated in deep reinforcement learning (Pascanu et al., 2017; Chung et al., 2023; Wang et al., 2025). A third idea, that the meta-level controller can itself be *modular*, decoupled from the planner so that a standard RL agent is retrofitted with cognitively inspired control, has only recently been taken up (Muppidi et al., 2026). Modularity itself is therefore no longer the open question; the harder one, and the one the Chapter 4 metacontroller takes up, is deciding *dynamically*, as deliberation unfolds, whether continued search is worth its cost. An agent can commit a compute budget up front from the current state, or it can decide moment to moment, from within the search tree as it grows, whether to halt or continue. Only the latter lets the structure of the search itself govern how long the agent thinks.

Here we adopt the general framework of planning-as-mental-navigation with computational costs, echoing the framework introduced by the Imagination-based Planner (IBP) (Pascanu et al., 2017) and more explicitly laid out in Thinker (Chung et al., 2023) and Dynamic Thinker (Wang et al., 2025). Broadly, the framework is a natural extension of reinforcement learning from individual states to trees (Fig. 5.1). The tree represents the agent’s imagined space of options and its root the current state of the world; acting reads a policy from the root, while planning proceeds by constructing and navigating the tree, expanding leaves, moving between nodes, and back-propagating their estimated values toward the root. Crucially, the objective penalizes not only poor actions but the resources their computation consumes, depth, memory, and latency, so that cognitively efficient strategies are favored instead of imposed. The agent is rewarded for thinking well and cheaply.

While both IBP and Thinker set the explicit framework of metaplanning as internal navigation over a tree, in each the meta-level controller remains bound to the particular planner it was trained with. In IBP the world model, manager, and controller are trained simultaneously and on-policy but under separate objectives; the manager therefore co-adapts to a controller and model that are themselves still changing, and never learns to gate a fixed planner (Pascanu et al., 2017). In Thinker and Dynamic Thinker, selection and stopping are instead folded into a single policy trained end-to-end (Chung et al., 2023; Wang et al., 2025). The modularity we advocate breaks this binding: because the metareasoning process is decoupled from the planner, the Chapter 4 model can replace the planning submodule wholesale with an existing chess engine, which is what makes the problem tractable in a domain where training a metaplanner from scratch is infeasible; components of interest can be fixed or learned while the rest are borrowed from pre-trained models. Building an end-to-end, psychologically plausible metaplanner then becomes a Ship-of-Theseus exercise, swapping out engineered components one at a time for more cognitively principled ones. With a grounded and modular framework for understanding how planning can be represented as individual atomic cognitive “actions”, we can begin to understand how resource constraints shape the representations and motor outputs that support and ground these cognitive processes.

5.2.6 Affordances and Learning Action Spaces

Throughout this thesis, the agent’s representation of the world has been treated as given. In Chapter 2, the board is represented as a subtree of possible rewards available to the participant; in Chapter 3, the board is represented as a vector of positions occupied by the player or opponent; in Chapter 4, the chess boards are represented as FENs and then passed through the chess engine before being processed by the metacontroller. Similarly, the action space has been treated as fixed. The action space in Chapter 2 is a simple left/right split; in the others, moves are either placements (Chapter 3) or rearrangements of pieces (Chapter 4) on an existing board. Of course, this is a major simplification. In the real world, the choice of how to represent the environment and how

to translate decisions into motor actions is remarkably flexible. Everyday actions are an intricate symphony of perception, decision making, and action dynamically interacting to produce flexible behavior, a process that we often take for granted. As a swimmer, I am reminded of the starting dive, a motion I have practiced countless times before getting it right. To a trained athlete, the dive is a single action: on the instruction to take the ready position, the chain of grasping the block, pushing off, and forming a streamline on contact with the water comes almost instinctively. To a novice, the same action appears as a sequence of effortful primitives, each demanding deliberate control, assembled slowly and unreliably. What expertise supplies is not new primitives but a *chunk*, a binding of those movements into one fluid motion that can be deployed as a whole.

The relationship between features of the environment and the meta-actions they support is what is called an *affordance*: a learned mapping from what is perceived to what can be done (Gibson, 1979; Tucker and Ellis, 1998; Khetarpal et al., 2020). In the example of a swimmer’s starting dive, an effective start not only requires the correct chaining of actions but also the association of those actions to a particular set of perceptual cues: the position of the block, the auditory ready cue, the sound of the starting buzzer. A central open question, and the one this thesis gestures toward, is how such affordances are acquired, how a system moves from a space of primitives to the structured, compact repertoire of actions that effective behavior requires, and how that repertoire is in turn realized in motor output. Recent work offers a proof of concept that this is empirically addressable: macaques trained on a drawing task assemble novel figures compositionally from a repertoire of previously acquired stroke primitives, and this compositional structure can be read from motor and premotor neural activity (Tian et al., 2026). While traditional RL frameworks sidestep the problem of discovering action spaces, assuming a well-converged perceptual system and a well-defined action space, the next step of this research program is to understand how these components themselves are learned. Existing frameworks such as hierarchical RL (Botvinick et al., 2009; Manoury et al., 2019b,a; Nica et al., 2022; De Wit et al., 2017; Costales et al., 2022) typically impose abstraction through architectural design rather than probing how abstraction might emerge from cognitive constraints themselves. One intriguing possibility is that cognitive

demands might serve as a bottleneck to drive particular learned representations; in this view affordance learning is an emergent property of cognitive constraints instead of a top-down imposed architectural choice. This would, in effect, be a translation of a resource rational framework to the neural level, connecting the (cognitive) task demands to their (neural) representational substrates. Under this view, chaining action primitives into meta-actions (so-called *action chunking*) arises as a response to the combinatorial explosion of chained action sequences and cognitive bottlenecks such as time pressure and limited memory. Affordances, which link features of the environment to policies over meta-actions, may emerge as a sparse representational solution to resource constraints. This program is a direct extension of the work in this thesis: where the preceding chapters asked how planning is controlled over a fixed set of actions, it asks how the action set itself is built, and whether its structure can be read as a resource-rational response to the demands the agent faces. This is the direction I intend to take up in my postdoctoral work.

5.3 Closing Remarks

“What a piece of work is a man! how noble in reason!
how infinite in faculty! in form and moving how
express and admirable! in action how like an angel!
in apprehension how like a god!”

—William Shakespeare, *Hamlet* (Act II, Scene 2)

Hamlet serves both as a cautionary tale and an exploration, perhaps even a celebration, of the depth and complexity of the way we as humans interact with the world. Far from being an indictment of Hamlet’s inability to act, what we are faced with as readers is the sense of how miraculous it is that we as people are able to act at all.

As researchers of the mind, our day job involves gradually chipping away at that great mystery and in doing so better understanding the machinery that allows us to make dynamic decisions in a messy world. In this thesis I have grappled with questions surrounding this adaptation, how to adapt to a world that is both within and outside of our control. In Chapter 2 we tackled how ir-

reducible randomness shapes planning behavior, finding that people reduce their planning effort by compressing their policies in favor of simpler, determinizing decision making. In Chapter 3 we extended that sense of randomness to a structured opponent, asking how monkeys could make decisions in never-before-seen states and build abstract features to interact with another agent. Finally, in Chapter 4 we investigated this extension of structured-uncertainty-as-opponent to the domain of chess, building machine learning models capable of metacontrol.

A through-line in this work is the importance of understanding the pressures of the environment on cognitive computation, particularly for multi-step planning and reasoning. We have come a long way from the ideas of humans as potentially optimal or purely heuristic-based, towards a more coherent study of behavior in the context of constraints. The adaptive, cost-sensitive control of deliberation has been a recurring motif of this work; resource rationality has offered one formal lens onto it, and under that lens much progress has been made in understanding not only how people act in certain situations, but also why it makes sense for them to do so. Just as normative theories of efficient coding have been a fruitful scaffolding for understanding sensory integration in neuroscience, resource rationality might serve as a foray into a kind of “efficient coding” for behavior, perhaps informing the kinds of algorithms that are implemented on the cognitive level as well as further down the hierarchy into neural ensembles and populations. It remains an open scientific question whether we can connect the cognitive rewards and costs, which for now live largely in well-defined symbolic and discrete state spaces, to the underlying neural mechanisms, which appear to rely on much more stochastic and continuous biological substrates. Doing so would be a great service to unifying the computational study of the mind across Marr’s levels of analysis (Marr, 2010), bringing the computational, algorithmic, and implementation level explanations into alignment, informed by the cognitive, environmental, and energy constraints of biological agents on all levels.

Of course, just as the pendulum swings one way, so it also swings back. Far from being a single coherent lens from which behavior can be inferred from rewards and constraints, what this body of work demonstrates is the remarkable diversity of the strategies that people and

animals use to deal with the unpredictable, uncontrollable, and sometimes unreasonable demands of the environment: meta-strategies such as policy compression, feature abstraction, and dynamic metacontrol. It appears that, rather than relying on a single strategy to reduce effort, a wide range of options are at our disposal when deciding when to plan, and by how much. Perhaps this is not a surprise; an open question in resource rationality is how agents can *a priori* intuit the rewards and costs of computation without doing the computation itself (in our case, how does one know whether it's worth planning unless one has already planned and deemed it not worth doing?) In this sense, maybe the diversity of options reflects a kind of lossy patchwork quilt of strategies that approximate what is "good enough" for a given situation in place of some singular normative "should". Though it remains unclear just how much we can squeeze out of the normative principles of resource rationality, what is abundantly evident is the deep influence of the environment, including the structure and rules governing the world and other agents, in shaping cognitive representations, strategies, and meta-strategies in the real world.

Today we are better positioned than ever before to tackle some of the challenges raised by this call towards a more naturalistic and ecologically grounded study of the mind across different levels of abstraction. Recent years have seen the rise of a wide variety of experimental techniques: high resolution multi-electrode arrays capable of simultaneously recording thousands of neurons from multiple brain regions; whole-brain non-invasive recording techniques that are being refined to a level of spatiotemporal resolution once thought to be accessible only via implants; and causal manipulations in the form of channelrhodopsins that allow us to probe the mind at cellular levels. Simultaneously, our tasks have gotten more complex and our modeling tools better-equipped to understand them; with the help of scalable gradient-driven optimization techniques like artificial neural networks, we are now able to connect the dots of the mind like never before. And of course I would be remiss not to mention the recent advancements in generative machine learning, particularly in the domain of transformers and large-language models, which have already begun to serve both as models of the mind and so-called *in-silico* models of cognition, though the relationship with biological minds remains yet unclear.

I want to end with a few personal notes. I find that as scientists, particularly in the psychological and neural sciences, we are often drawn to study our own shortcomings. In some ways, this echoes Polonius' double-edged admonition in *Hamlet*: "This above all: to thine own self be true". To that end I find myself no exception. In my own day-to-day decision making, I struggle greatly with over-planning and a deep discomfort with ambiguity and uncertainty. This tendency has followed me for much of my life and played no small part in my slight obsession with understanding my own mind and others'. It has been a privilege to spend the past few years understanding how people, animals, and even machines accomplish effortlessly what I sometimes struggle to do on my own. I have spent a long time deliberating how to conclude this, the culmination of five years of my life's work. I suppose that, in keeping with the core theme of my thesis, it's best not to overthink it.

Appendix A | Human Planning in Stochastic Environments

A.1 Experiment Design

Each participant was given a set of general instructions (Fig. A.1) and a set of manipulation-specific instructions based on the version of the task they were playing (Reliability, Volatility, or Controllability). Participants played the task in “blocks,” and were told at the start of each block what stochasticity level was associated with that block (Fig. A.1F). Each participant completed 10 blocks of 15 games each, for a total of 150 games. Blocks were presented in random order in the first half of the experiment and then the order was reversed in the second half to mitigate order effects on attention and gameplay. Prior to each game, participants were instructed to place their hands in a “ready” position to begin (Fig. A.1E). All experiment procedures, code, and instructions are openly available on GitHub at: https://github.com/WeiJiMaLab/human_planning_in_stochastic_environments.git.

Interactive demonstrations of the task are also available online for each manipulation:

- Reliability (<https://pirate-treasure-9e3ba.web.app/?type=demoR>)
- Volatility (<https://pirate-treasure-9e3ba.web.app/?type=demoV>)
- Controllability (<https://pirate-treasure-9e3ba.web.app/?type=demoT>).

A.1.1 General Task Instructions

Participants were guided through the task via a series of interactive instruction screens (Fig. A.1A). They were first informed that the experiment would last approximately one hour, with a base pay

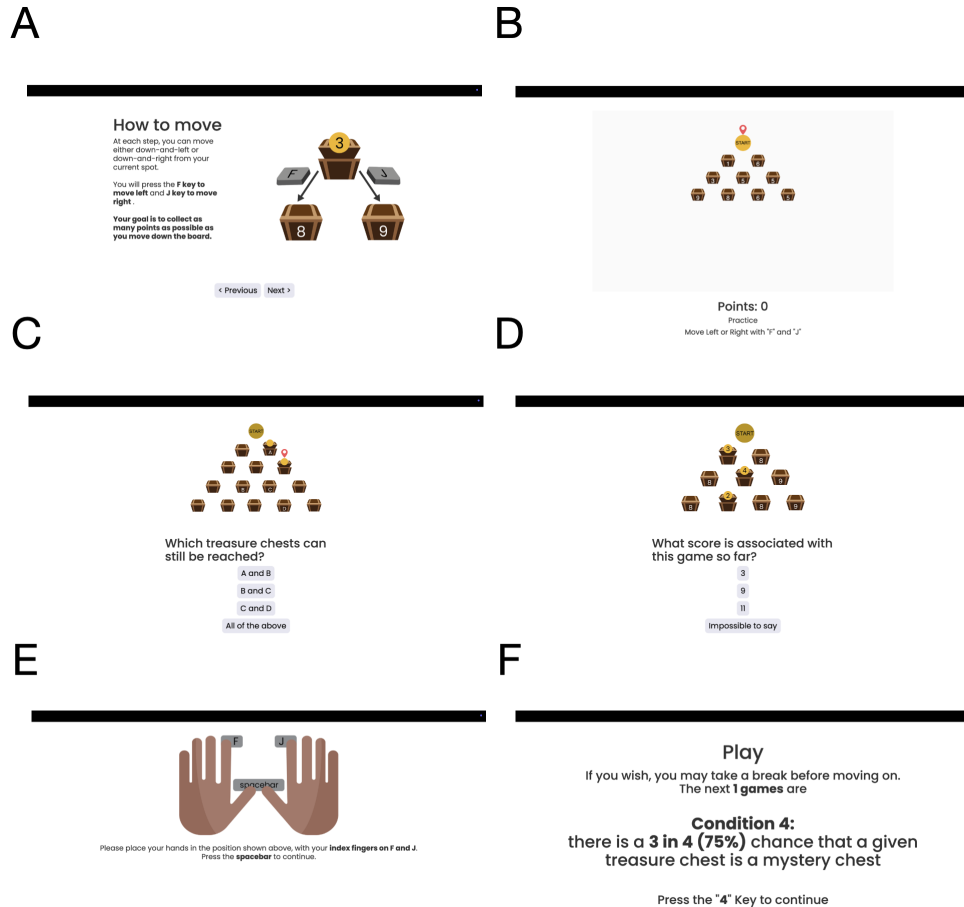


Figure A.1: Screenshots of participant instructions and stimuli. A. Example instruction slide. B. Example practice trial. C–D. Example comprehension checks. E. Hand placement instructions prior to the start of the game. F. Example block instructions; the number of games per block is 1 in the demonstration above but 15 in the real experiment.

of \$8, and that bonus payments would be based on performance: “You will receive a bonus payment based on your performance on these games. The bonus will be calculated from a random sample of the games you play.” Participants were also warned that dropping out early or ending the experiment prematurely would result in no payment, and that attention checks were included; failing two checks would end the experiment with no compensation.

For the game mechanics, participants were told they were explorers seeking treasure, with the goal of earning as many points as possible. Each treasure chest was labeled with a number from 1 to 9, representing the points it contained, with higher numbers yielding more points. At each step, participants could move down-and-left or down-and-right from their current position, pressing the F key to move left and the J key to move right. Finally, participants completed a mini-version of the game as a practice round, which was not scored, allowing them to familiarize themselves with the movement mechanics and the overall task.

Participants completed one practice game (Fig. A.1B) using a simplified 4-row version of the task with no stochastic manipulations. This allowed them to experience movement mechanics, collect points, and see visual feedback. Practice trials were excluded from the main analysis.

Comprehension checks

After practice, participants completed four multiple-choice comprehension questions. If the participant failed to answer all comprehension checks correctly, they would be taken back to the beginning of the general instruction block.

1. Goal of the game: “What is the goal of the game?”

- Collect the most points (**correct**)
- Collect the least points
- Finish the game as fast as possible

2. Movement keys: “Which of the following statements is correct?”

- The J key is for moving left and the F key is for moving right
- The arrow keys are used to navigate the board
- The F key is for moving left and the J key is for moving right (**correct**)

3. **Reachable treasure chests:** (See Fig. A.1C) “Which treasure chests can still be reached?”

- A and B
- B and C
- C and D (**correct**)
- All of the above

4. **Score calculation:** (See Fig. A.1D) “What score is associated with this game so far?”

- 3
- 9 (**correct**)
- 11
- Impossible to say

A.1.2 Manipulation-Specific Instructions and Checks

Participants were randomly assigned to one of three manipulations: **Reliability**, **Volatility**, or **Controllability**. Within their assigned manipulation, participants first received instructions describing the relevant task mechanics and stochasticity. They then completed a practice round at each stochasticity level on a full-sized board, allowing them to experience the manipulation-specific mechanics. After practice, participants completed multiple-choice comprehension questions to ensure understanding. If any question was answered incorrectly, participants repeated the entire manipulation-specific instruction block, including the practice rounds, until all comprehension questions were answered correctly.

Reliability

Participants were informed that treasure chests could be either **normal** or **mystery**. Normal chests contain the displayed number; mystery chests contain a **random number between 1 and 9**. Chest type is **completely random** and independent of location, value, or past choices. The five stochasticity conditions were presented along with their corresponding levels. Participants then completed the following comprehension checks:

1. In condition 3 there is a 50% chance that a given chest will be a mystery chest. If there are 10 treasure chests in total ...
 - Exactly 5 chests will be mystery chests
 - Fewer than 5 chests will be mystery chests
 - More than 5 chests will be mystery chests
 - It is impossible to know (**correct**)
2. In condition 1 there is a 0% chance that a given chest will be a mystery chest. If there are 10 treasure chests in total ...
 - 10 chests will be mystery chests
 - 0 chests will be mystery chests (**correct**)
 - 5 chests will be mystery chests
 - It is impossible to know
3. A treasure chest is more likely to be a mystery chest if ...
 - It has a large number written on the front
 - Its neighbors are also mystery treasure chests
 - The last treasure chest you visited wasn't a mystery treasure chest
 - None of the above (**correct**)

4. You see an unopened treasure chest with a 5 written on the front. When you visit it, you discover it is a mystery chest. The number of points you get ...

- must be equal to 5
- must not be equal to 5
- Can be any number between 1 and 9 (**correct**)
- None of the above

Volatility

Participants were informed that some treasure chests lie in an **earthquake zone** where they might shake and receive a new value between 1 and 9. Shaking is **completely random** and independent of past shakes, chest value, or location. Participants were shown the five stochasticity conditions and their corresponding levels, followed by comprehension checks:

1. In condition 3 there is a 50% chance that a given chest in the earthquake zone will shake.

If there are 10 treasure chests in the earthquake zone ...

- Exactly 5 chests will shake
- Fewer than 5 chests will shake
- More than 5 chests will shake
- It is impossible to know (**correct**)

2. In condition 1 there is a 0% chance that a given chest in the earthquake zone will shake. If there are 10 treasure chests in the earthquake zone ...

- Exactly 10 chests will shake
- Exactly 0 treasure chests will shake (**correct**)
- Around 5 treasure chests will shake
- It is impossible to know

3. On a given move, a treasure chest is more likely to shake if ...
- It has a large number written on the front
 - Its neighbors also shake
 - It did not shake in the previous move
 - None of the above (**correct**)
4. One of the treasure chests in the earthquake zone is shaking. If it had a 3 written on the front before it started shaking, the value of the treasure chest after shaking is ...
- Not equal to 3
 - Smaller than 3
 - A random value between 1 and 9 (**correct**)
 - None of the above
5. Which treasure chests are in the earthquake zone?
- A only
 - B and C
 - B, C, and D (**correct**)
 - D only
 - All of the above

Controllability

Participants were told that occasionally their moves might be **flipped** (pressing left could move them right). Flipped moves are **completely random**, independent of past moves or chest values. Participants were presented with the five stochasticity conditions and their corresponding levels, followed by comprehension checks:

1. In a flipped move, if you press the F (left) key ...
 - You will move left
 - You will move right (**correct**)
 - You will move up
 - It is impossible to know
2. In condition 3 there is a 25% chance that a given move will be flipped. For every 4 moves ...
 - Exactly 1 move will be flipped
 - Fewer than 1 move will be flipped
 - More than 1 move will be flipped
 - It is impossible to know (**correct**)
3. In condition 1 there is a 0% chance that a given move will be flipped. For every 4 moves ...
 - No moves will be flipped (**correct**)
 - All moves will be flipped
 - Half of the moves will be flipped
 - It is impossible to know
4. A move is more likely to be flipped if ...
 - The previous move was flipped
 - The treasure chests in the future have high values
 - You move left
 - None of the above (**correct**)

A.1.3 Attention Checks

To ensure participants remained engaged and followed instructions, attention checks were embedded within the task. Each participant encountered 3 total attention checks throughout the experiment. Failure of two attention checks resulted in termination of the experiment.

During these trials, participants were explicitly told to press only the spacebar when prompted. Responses were recorded, and any deviations from the instructed key press were used to assess attentiveness. This procedure ensured data quality by identifying participants who were not adequately attending to the task.

A.2 Algorithms

We represent each state s from an egocentric perspective using an (i, j) coordinate system, where i represents the number of rows from the current position and j represents the number of rightward steps. For example, the player's immediate leftward choice would be to the treasure chest at $(1, 0)$, and the immediate rightward choice would be to the treasure chest at $(1, 1)$. For all unreachable board states (including undefined positions beyond the last row of the game), the value of $s_{i,j}$ is set to 0.

The left subtree of the current state corresponds to the lower-left portion of the matrix, specifically $s_{i+1:\text{end},j:\text{end}}$ with all entries above the diagonal set to zero. Similarly, the right subtree corresponds to the lower-right portion, $s_{i+1:\text{end},j+1:\text{end}}$. These subtrees effectively capture all future potential rewards contingent on the player's initial directional choice.

A.2.1 Filter Functions

Algorithm 1 Depth Filter

Require: s : state, array of shape (n, m)

Require: d : depth threshold

```
1:  $s' \leftarrow s$ 
2: for  $i = 0$  to  $n - 1$  do
3:   for  $j = 0$  to  $m - 1$  do
4:     if  $i > d$  then
5:        $s'_{i,j} \leftarrow 0$  ▷ zero out rows deeper than  $d$ 
6:     end if
7:   end for
8: end for
9: return  $s'$ 
```

Algorithm 2 Value Filter

Require: s : state, array of shape (n, m)

Require: v : value threshold

```
1:  $s' \leftarrow s$ 
2: for  $i = 0$  to  $n - 1$  do
3:   for  $j = 0$  to  $m - 1$  do
4:     if  $s'_{i,j} < 10 - v$  then
5:        $s'_{i,j} \leftarrow 0$  ▷ zero out values below  $10 - v$ 
6:     end if
7:   end for
8: end for
9: return  $s'$ 
```

Algorithm 3 Rank Filter

Require: s : state, array of shape (n, m)

Require: k : rank threshold ($k \in \{1, \dots, 9\}$)

- 1: $L \leftarrow \{s_{i,j} : 0 \leq j \leq i < n\}$ ▷ Lower-triangular board entries
 - 2: $U \leftarrow$ sorted distinct values of L in descending order
 - 3: **if** $k > |U|$ **then**
 - 4: **return** s ▷ No further masking when k exceeds the number of score levels on the board
 - 5: **end if**
 - 6: Let $t \leftarrow$ the k -th largest distinct value in U ▷ Top k treasure score levels
 - 7: $s' \leftarrow \text{where}(s \geq t, s, 0)$ ▷ Mask out elements below the k -th score level
 - 8: **return** s'
-

A.2.2 Value Functions

Algorithm 4 Helper Functions for Tree Operations

- 1: **function** LEFT_SUBTREE(t)
 - 2: $l \leftarrow \text{zeros_like}(t)$ ▷ Initialize empty subtree
 - 3: $l_{i,j} \leftarrow t_{i+1,j}$ for all i, j where $i \geq j$ ▷ Copy left subtree elements where row $\geq$ column
 - 4: **return** l
 - 5: **end function**
 - 6: **function** RIGHT_SUBTREE(t)
 - 7: $r \leftarrow \text{zeros_like}(t)$ ▷ Initialize empty subtree
 - 8: $r_{i,j} \leftarrow t_{i+1,j+1}$ for all i, j where $i \geq j$ ▷ Copy right subtree elements where row $\geq$ column
 - 9: **return** r
 - 10: **end function**
-

Algorithm 5 Path Value Algorithm

Require: s' : filtered state, array of shape (n, m)

- 1: $t \leftarrow \text{zeros_like}(s')$ ▷ Table for dynamic programming
 - 2: $t_{n-1,*} \leftarrow s'_{n-1,*}$ ▷ Initialize last row with values from s'
 - 3: **for** $i = n - 2$ **down to** 0 **do**
 - 4: **for** $j = 0$ **to** i **do**
 - 5: $t_{i,j} \leftarrow s'_{i,j} + \max(t_{i+1,j}, t_{i+1,j+1})$ ▷ Best path value
 - 6: **end for**
 - 7: **end for**
 - 8: **return** $t_{1,0} - t_{1,1}$ ▷ Decision value: left vs right subtree
-

Algorithm 6 Sum Value Algorithm

Require: s' : filtered state, array of shape (n, m)

- 1: $l \leftarrow \text{LEFT_SUBTREE}(s')$
 - 2: $r \leftarrow \text{RIGHT_SUBTREE}(s')$
 - 3: $v_{\text{left}} \leftarrow \sum l$ ▷ Sum all values in left subtree
 - 4: $v_{\text{right}} \leftarrow \sum r$ ▷ Sum all values in right subtree
 - 5: **return** $v_{\text{left}} - v_{\text{right}}$ ▷ Decision value: left vs right subtree
-

Algorithm 7 Max Value Algorithm

Require: s' : filtered state, array of shape (n, m)

- 1: $l \leftarrow \text{LEFT_SUBTREE}(s')$
 - 2: $r \leftarrow \text{RIGHT_SUBTREE}(s')$
 - 3: $v_{\text{left}} \leftarrow \max(l)$ ▷ Maximum value in left subtree
 - 4: $v_{\text{right}} \leftarrow \max(r)$ ▷ Maximum value in right subtree
 - 5: **return** $v_{\text{left}} - v_{\text{right}}$ ▷ Decision value: left vs right subtree
-

Algorithm 8 Level-Mean Value Algorithm

Require: s' : filtered state, array of shape (n, m)

- 1: $l \leftarrow \text{LEFT_SUBTREE}(s')$
 - 2: $r \leftarrow \text{RIGHT_SUBTREE}(s')$
 - 3: $v_{\text{left}} \leftarrow 0, \quad v_{\text{right}} \leftarrow 0$
 - 4: **for** $i = 0$ to $n - 2$ **do**
 - 5: $v_{\text{left}} \leftarrow v_{\text{left}} + \frac{1}{i+1} \sum_{j=0}^i l_{i,j}$ ▷ Row-wise mean over valid subtree entries ($j \leq i$)
 - 6: $v_{\text{right}} \leftarrow v_{\text{right}} + \frac{1}{i+1} \sum_{j=0}^i r_{i,j}$
 - 7: **end for**
 - 8: **return** $v_{\text{left}} - v_{\text{right}}$ ▷ Decision value: left vs right subtree
-

Algorithm 9 EV Value Algorithm (Reliability)

Require: s' : filtered state, array of shape (n, m)

Require: q : stochasticity level of state s

Require: E : expected value parameter

```
1:  $t \leftarrow \text{zeros\_like}(s')$  ▷ Table for dynamic programming
2:  $t_{n-1,*} \leftarrow q \cdot s'_{n-1,*} + (1 - q) \cdot E$  ▷ Initialize last row
3: for  $i = n - 2$  down to  $0$  do
4:   for  $j = 0$  to  $i$  do
5:      $t_{i,j} \leftarrow q \cdot s'_{i,j} + (1 - q) \cdot E + \max(t_{i+1,j}, t_{i+1,j+1})$ 
6:   end for
7: end for
8: return  $t_{1,0} - t_{1,1}$  ▷ Decision value: left vs right path
```

Algorithm 10 EV Value Algorithm (Controllability)

Require: s' : filtered state, array of shape (n, m)

Require: q : stochasticity level of state s

```
1:  $t \leftarrow \text{zeros\_like}(s')$  ▷ Table for dynamic programming
2:  $t_{n-1,*} \leftarrow s'_{n-1,*}$  ▷ Initialize last row
3: for  $i = n - 2$  down to  $0$  do
4:   for  $j = 0$  to  $i$  do
5:      $v_{left} \leftarrow q \cdot t_{i+1,j} + (1 - q) \cdot t_{i+1,j+1}$  ▷ Left choice expected value
6:      $v_{right} \leftarrow (1 - q) \cdot t_{i+1,j} + q \cdot t_{i+1,j+1}$  ▷ Right choice expected value
7:      $t_{i,j} \leftarrow s'_{i,j} + \max(v_{left}, v_{right})$ 
8:   end for
9: end for
10:  $v_{left} \leftarrow q \cdot t_{1,0} + (1 - q) \cdot t_{1,1}$  ▷ Left choice at root
11:  $v_{right} \leftarrow (1 - q) \cdot t_{1,0} + q \cdot t_{1,1}$  ▷ Right choice at root
12: return  $v_{left} - v_{right}$  ▷ Decision value: left vs right choice
```

A.2.3 Computing Choice Probabilities

Algorithm 11 Probability of $a = L$

Require: s : state array s of shape n, m

Require: `FILTER_FN`: filtering function to apply

Require: `VALUE_FN`: value function to evaluate options

Require: $\log(\beta_q)$: (log) inverse temperature parameter

Require: λ : lapse rate

```
1:  $s' \leftarrow \text{FILTER\_FN}(s)$  ▷ Apply filter to get filtered state
2:  $\Delta \leftarrow \text{VALUE\_FN}(s')$  ▷ Compute value difference using appropriate value function
3:  $p_{\text{left}} \leftarrow 0.5\lambda + (1 - \lambda)\sigma(\Delta, \beta_q)$  ▷ Apply sigmoid and lapse (take exponent of  $\log(\beta_q)$ )
4: return  $p_{\text{left}}$ 
```

A.3 Resource-Rational Simulations

While we did not explicitly fit a resource-rational model in our main analyses, it is helpful to consider how the framework relates to our work. Resource rationality predicts that people choose planning effort to maximize the trade-off between its benefits (higher expected reward) and costs (greater cognitive effort) (Lieder and Griffiths, 2020; Lieder et al., 2025). Here we use simulations to show how one of our winning mechanisms, policy compression with Depth-filtering Level-Mean planning, arises naturally from an optimization process which considers the tradeoff between getting higher expected rewards and the cost of increasing policy sensitivity. We demonstrate that this trade-off naturally yields our main result, that optimal planning effort (inverse temperature β) decreases as stochasticity increases.

A.3.1 Estimating Rewards Under Effort

Let β denote the inverse temperature, which we interpret as a measure of planning effort, and let $q \in [0, 1]$ denote the stochasticity level.

For each stochasticity level q , with planning depth fixed at $d = 3$, we simulated the winning PC Depth-filtering Level-Mean algorithm on $N = 10,000$ randomly generated boards per

experimental condition. Because β spans several orders of magnitude, we varied it in log-space: we sampled 20 values of $\log(\beta)$ linearly between -2 and 2 (i.e., $\log(\beta) \in [-2, 2]$), corresponding to $\beta \in [e^{-2}, e^2]$, and interpreted β as the variable reflecting the sensitivity or effort invested in planning. We define:

$R(\beta, q; d=3)$ = expected reward from Depth-filtering Level-Mean planning under stochasticity q .

Two general patterns emerged (Fig. A.2A–C). First, for a fixed β , as stochasticity q increases, the expected reward $R(\beta, q)$ decreases. Higher stochasticity diminishes the benefit of effort. Second, for a fixed q , increasing β improves performance but saturates eventually, reflecting diminishing marginal returns of additional planning effort. The rate of saturation differs somewhat across conditions: in the volatility condition, rewards saturate earlier than in the reliability condition, reflecting compounding stochasticity across planning steps.

A.3.2 Cost of Planning

We modeled the cognitive cost of planning as proportional to effort:

$$C(\beta) = \alpha\beta$$

where $\alpha > 0$ scales the cost of additional effort. We allow for the value of α to vary across individuals.

The *net utility* of planning with sensitivity β under stochasticity q is then:

$$U(\beta, q) = R(\beta, q) - C(\beta) = R(\beta, q) - \alpha\beta.$$

Without any cost term, the optimal solution would be to exert as much effort as possible (except when stochasticity is maximal, where all β yield equal expected reward). Introducing the cost term makes the optimal level of effort dependent on q (Fig. A.2D–F).

To capture variability in cognitive cost across participants, we assumed:

$$\alpha = \max[0, a], \quad a \sim \mathcal{N}(\mu_\alpha, \sigma_\alpha), \quad \mu_\alpha = 5, \quad \sigma_\alpha = 3$$

We chose these parameter values for illustrative purposes; the choice of the specific parameters does not affect the qualitative conclusions.

We simulated $N_{\text{part}} = 100$ participants by drawing one α per participant. For each simulated participant and stochasticity level q , we computed:

$$U(\beta, q) = R(\beta, q) - \alpha\beta, \quad \log(\beta) \in [-2, 2],$$

and converted them to choice probabilities using a softmax:

$$P(\beta \mid q, \alpha) = \frac{\exp(U(\beta, q))}{\sum_{\beta'} \exp(U(\beta', q))}$$

We then simulated $N_{\text{trials}} = 100$ trials for each (participant, q) pair by sampling β^* from $P(\beta \mid q, \alpha)$. This produced a distribution of planning effort choices across participants and stochasticity levels, which we aggregated for analysis.

A.3.3 Simulation Results

The simulated chosen effort $\beta^*(q)$ decreased monotonically with stochasticity q , consistent with the behavioral data (Fig. A.2G–I). Across all three task variants, increasing stochasticity reduced the marginal benefit of investing in more precise value discrimination, leading resource-rational agents to select lower $\log(\beta)$ when uncertainty was greater. These simulations demonstrate that a simple reward–cost trade-off is sufficient to recover the key empirical pattern: when higher planning effort yields diminishing returns due to environmental stochasticity, agents optimally reduce planning sensitivity. This analysis illustrates that the policy compression mechanism observed in behavior is qualitatively consistent with a normative account in which planning effort

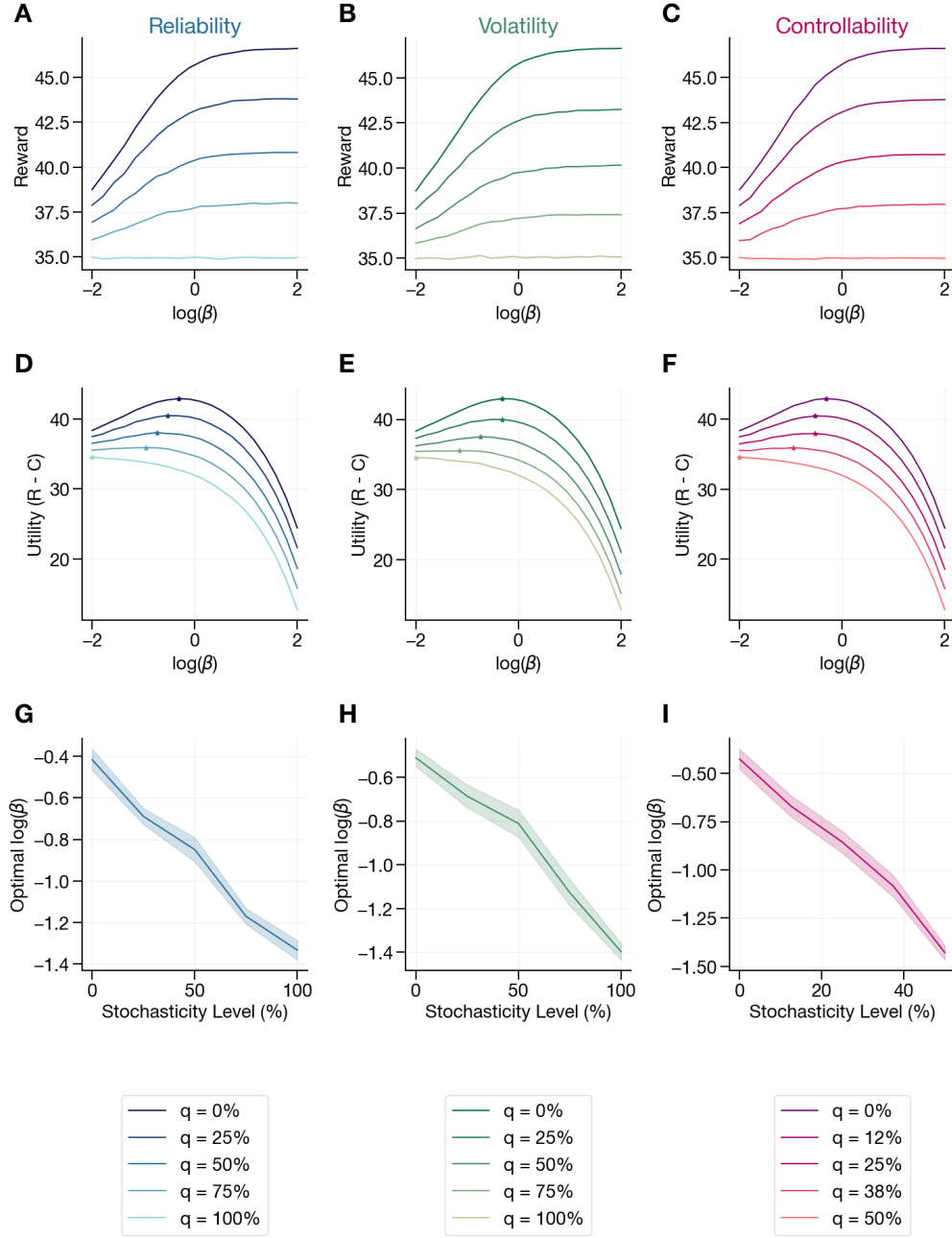


Figure A.2: **Resource rationality simulated results.** (A - C) Simulated expected rewards as a function of $\log(\beta)$, for each stochasticity level q , averaged over 10,000 board examples. (D - F) Example utility functions (here $\alpha = 3$) and their corresponding optimal $\log(\beta)$ value (denoted by stars). (G - I) Optimal $\log(\beta)$ as a function of stochasticity level, shaded region represents the SEM, for 100 simulated participants with 100 samples each.

is allocated adaptively.

A.3.4 Discussion

In these simulations, we approximated the expected reward $R(\beta, q)$ for different levels of planning effort β and stochasticity q by sampling from the task space. Participants are unlikely to explicitly compute such a function; instead, this exercise serves an illustrative purpose. It shows that an agent trading off reward and cognitive cost would rationally reduce planning sensitivity as stochasticity increases.

We modeled cost as linear in β ,

$$C(\beta) = \alpha\beta,$$

reflecting that maintaining a sharper policy requires greater precision and cognitive effort. Other cost functions could be considered, but our aim here is simply to demonstrate that the qualitative pattern observed in data can arise naturally from a reward-effort tradeoff.

Taken together, these simulations indicate that the observed decrease in policy sensitivity under greater uncertainty is consistent with a resource-rational perspective: when the environment becomes less predictable, it is adaptive to expend less computational effort.

A.4 Response Times

In the main text, we focused on first-move response times as a measure of cognitive effort, because participants spend the bulk of their planning before making their first move. However, because it is also possible for participants to deliberate mid-game, we also examined log total response times. Consistent with the first-move analysis, stochasticity level had a significant negative effect on total reaction time across all three conditions (Fig. A.3; Reliability: LMM, $\beta = -0.52$, $t_{98} = -7.97$, $p < 10^{-10}$; Volatility: LMM, $\beta = -0.21$, $t_{99} = -3.95$, $p = 1.5 \times 10^{-4}$; Controllability: LMM, $\beta = -0.41$, $t_{98} = -4.17$, $p = 6.5 \times 10^{-5}$). These results reinforce our main conclusion that higher stochasticity reduces planning-related response times.

Empirical Data

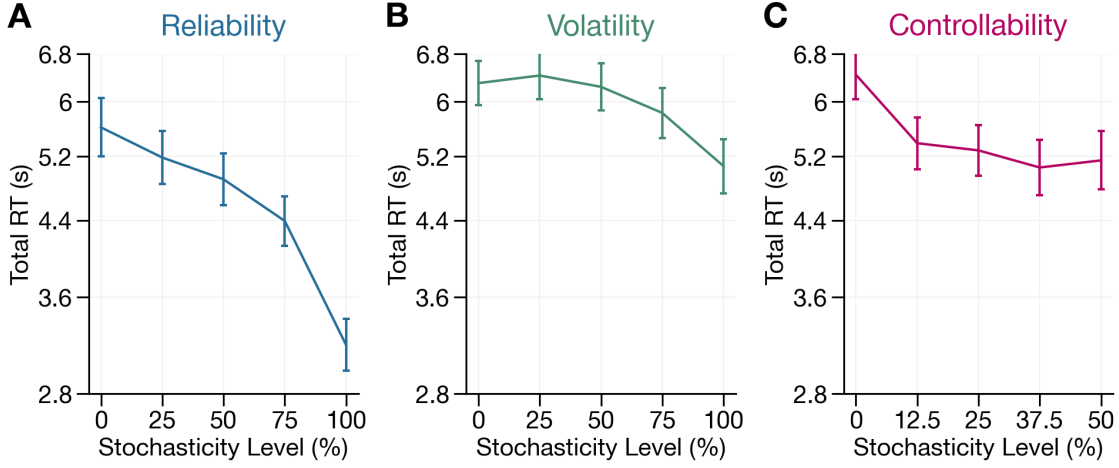


Figure A.3: **Total response time results.** A–C. Total response times as a function of stochasticity level, plotting on a log-scaled y-axis.

A.5 Model Comparison

In the main text, we reported results for the Policy Compression (PC) Depth-filtering Level-Mean model, which best captured participant behavior across all three conditions. We also found that the PC Depth-filtering Path model performed comparably to the PC Depth-filtering Level-Mean model in all stochasticity conditions, and that the PC Depth-filtering Max and PC Depth-filtering Sum models were competitive specifically in the volatility condition. We demonstrate that for each competitive alternative model, the same qualitative conclusions hold as in the main text. Finally, we illustrate why the PC Depth-filtering EV model fails to capture participant behavior in the cases where the expected-value computation is tractable (reliability and controllability).

A.5.1 PC Depth-Filtering Path Model

The PC Depth-filtering Path model also outperformed other models in the model comparison. Similar to the PC Depth-filtering Level-Mean model that we highlight in the main text, this model effectively captures the trends in participant data (Fig. A.4A–F). Also similar to the PC Depth-filtering Level-Mean model, we found that increasing stochasticity level decreased $\log(\beta)$ across

PC depth path

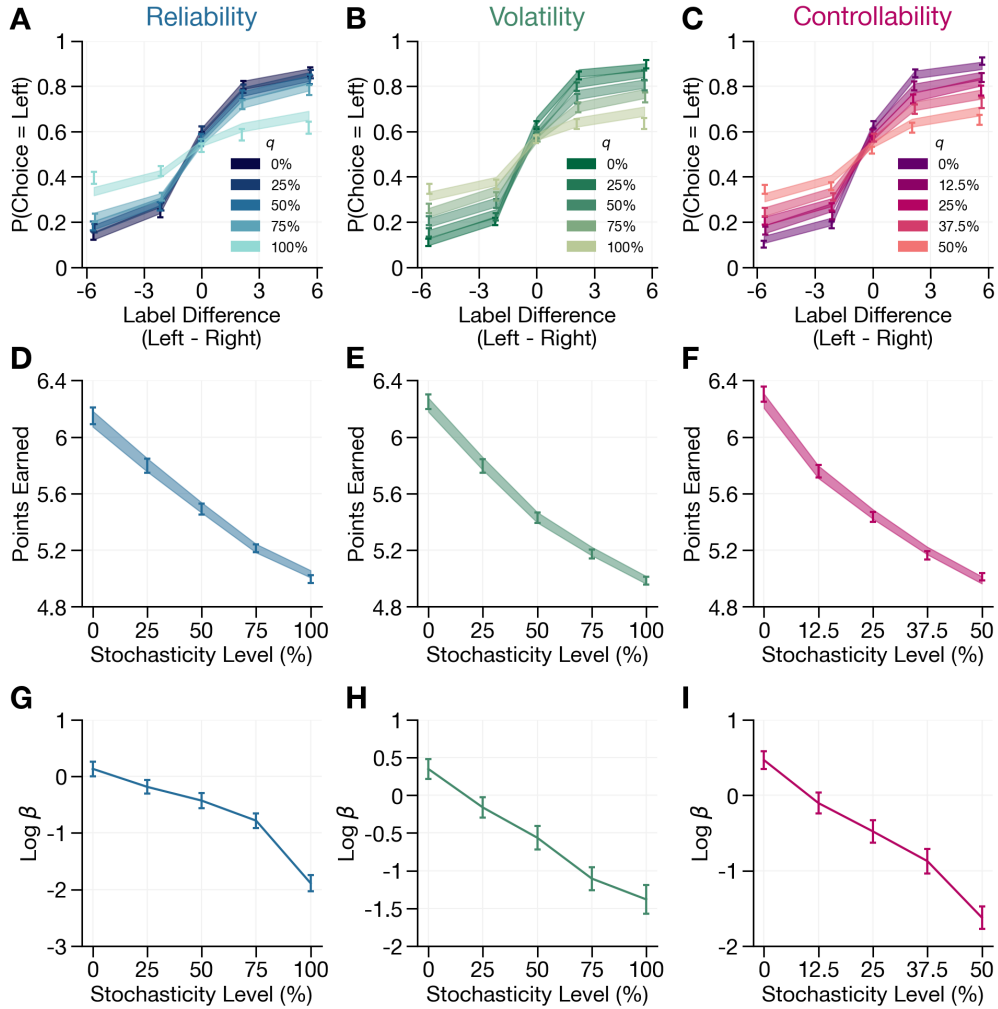


Figure A.4: PC Depth-filtering Path model results. A–C. Probability of choosing left based on the difference between the immediate next values, segmented by stochasticity level q . Error bars represent the SEM for empirical data, shaded regions represent the SEM for the model. D–F. Earnings per trial across different stochasticity levels. Error bars represent the SEM for empirical data, shaded regions represent the SEM for the model. G–I. Stochasticity level versus model inverse temperature ($\log \beta$). Error bars represent the SEM for the model.

all three stochasticity types (Fig. A.4G–I; Reliability: LMM, $\beta = -1.86$, $t_{98} = -11.1$, $p < 10^{-10}$; Volatility: LMM, $\beta = -1.76$, $t_{98} = -8.34$, $p < 10^{-10}$; Controllability: LMM, $\beta = -3.96$, $t_{98} = -11.8$, $p < 10^{-10}$).

A.5.2 PC Depth-Filtering Max Model

The PC Depth-filtering Max model was competitive in the volatility condition. We found that in the volatility condition, increasing stochasticity level decreased $\log(\beta)$ (Fig. A.5H; LMM, $\beta = -1.87$, $t_{98} = -9.77$, $p < 10^{-10}$).

A.5.3 PC Depth-Filtering Sum Model

The PC Depth-filtering Sum model was competitive in the volatility condition. We found that in the volatility condition, increasing stochasticity level decreased $\log(\beta)$ (Fig. A.6H; LMM, $\beta = -1.72$, $t_{98} = -8.35$, $p < 10^{-10}$).

A.5.4 PC Depth-Filtering EV Model

Here we illustrate why the PC Depth-filtering EV model falls short relative to the determinizing alternatives. Recall that the EV value function is only tractable in the reliability and controllability conditions; in these conditions it is informative to look at how the model describes the relationship between (greedy) label difference and probability of moving left. Looking at the model checking plots (Fig. A.7), we can see that in the highest stochasticity level (100% in reliability and 50% in controllability), the model predicts that participants would be indifferent to the label differences, having roughly chance level of choosing left for all label differences. Instead, what we find is that participants consistently continue to factor label difference into their decisions despite the fact that it is suboptimal, suggesting that they are determinizing rather than implicitly calculating the expected value of each path.

PC depth max

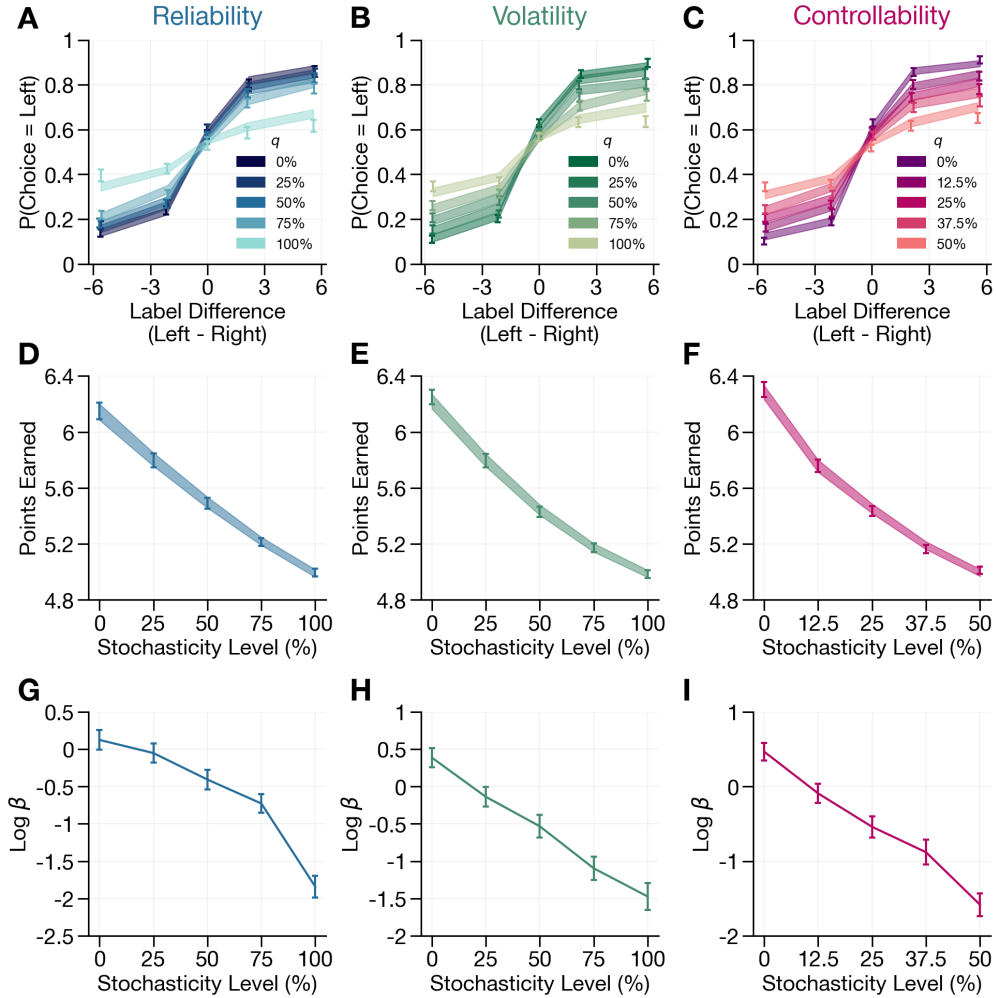


Figure A.5: PC Depth-filtering Max model results. A–C. Probability of choosing left based on the difference between the immediate next values, segmented by stochasticity level q . Error bars represent the SEM for empirical data, shaded regions represent the SEM for the model. D–F. Earnings per trial across different stochasticity levels. Error bars represent the SEM for empirical data, shaded regions represent the SEM for the model. G–I. Stochasticity level versus model inverse temperature ($\log \beta$). Error bars represent the SEM for the model.

PC depth sum

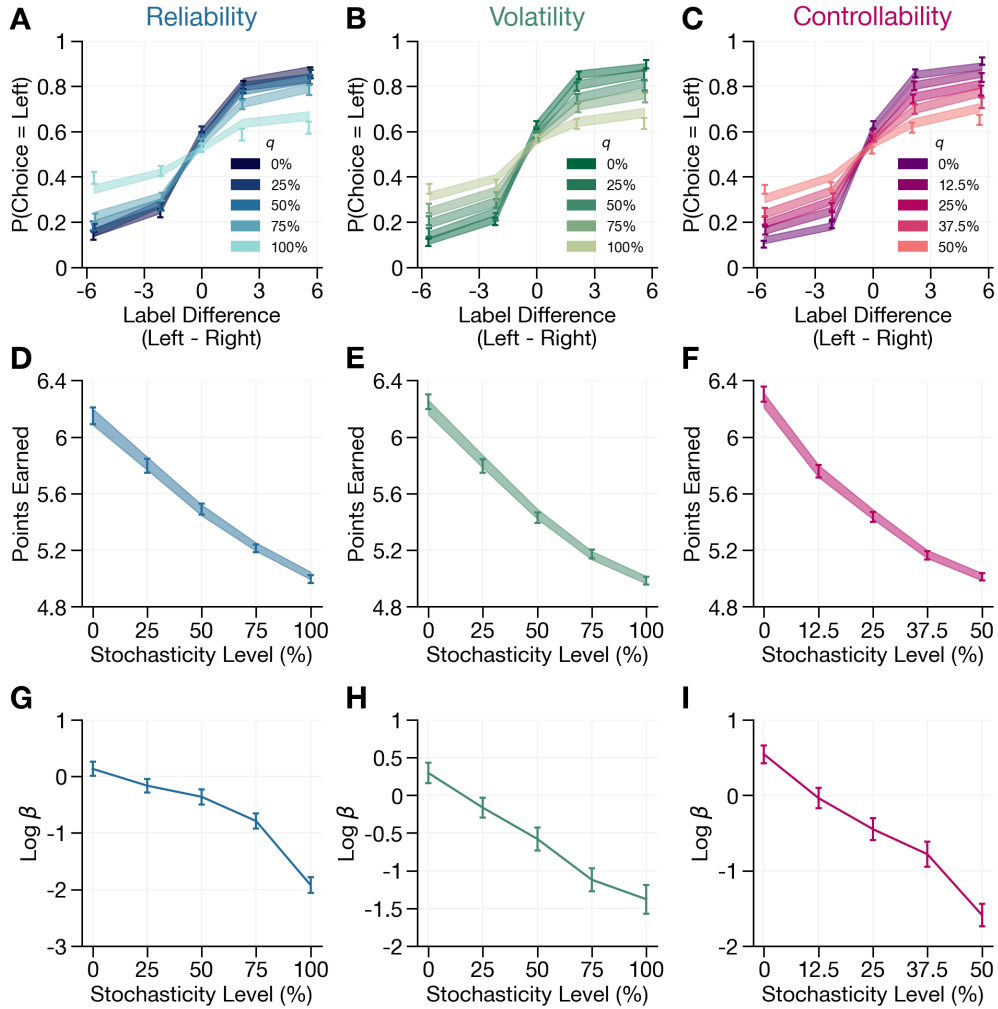


Figure A.6: PC Depth-filtering Sum model results. A–C. Probability of choosing left based on the difference between the immediate next values, segmented by stochasticity level q . Error bars represent the SEM for empirical data, shaded regions represent the SEM for the model. D–F. Earnings per trial across different stochasticity levels. Error bars represent the SEM for empirical data, shaded regions represent the SEM for the model. G–I. Stochasticity level versus model inverse temperature ($\log \beta$). Error bars represent the SEM for the model.

PC depth EV

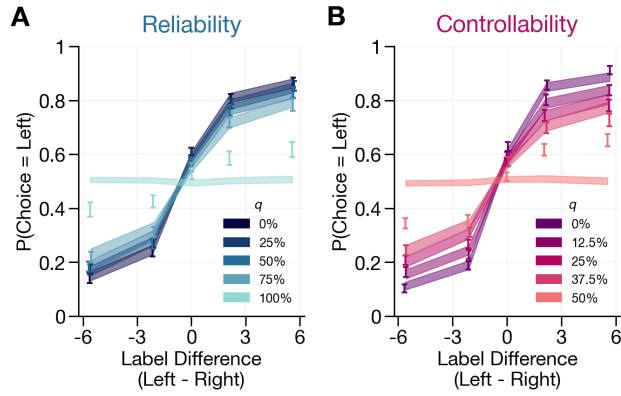


Figure A.7: PC Depth-filtering EV model checking. Probability of choosing left based on the difference between the immediate next values, for different stochasticity levels q . A. Reliability. B. Controllability.

A.5.5 G/LMM Convergence

Model Name	Stochasticity Type	Variable	Formula / Status
Empirical	Controllability	y = P(left) x = label diff	$y \sim x*s+(1+x+s\text{--}participants):Converged$
		y = log(firstRT)	$y \sim s+(1+s\text{--}participants):Converged$
		y = log(totalRT)	$y \sim s+(1+s\text{--}participants):Converged$
		y = points	$y \sim s+(1+s\text{--}participants):Converged$
	Reliability	y = P(left) x = label diff	$y \sim x*s+(1+x+s\text{--}participants):Converged$
		y = log(firstRT)	$y \sim s+(1+s\text{--}participants):Converged$
		y = log(totalRT)	$y \sim s+(1+s\text{--}participants):Converged$
		y = points	$y \sim s+(1+s\text{--}participants):Converged$
	Volatility	y = P(left) x = label diff	$y \sim x*s+(1+x+s\text{--}participants):Converged$
		y = log(firstRT)	$y \sim s+(1+s\text{--}participants):Converged$
		y = log(totalRT)	$y \sim s+(1+s\text{--}participants):Converged$
		y = points	$y \sim s+(1+s\text{--}participants):Converged$
PC depth levelmean	Controllability	y = log(beta)	$y \sim s+(1+s\text{--}participants):Converged$
	Reliability	y = log(beta)	$y \sim s+(1+s\text{--}participants):Converged$
	Volatility	y = log(beta)	$y \sim s+(1+s\text{--}participants):Converged$
PC depth max	Controllability	y = log(beta)	$y \sim s+(1+s\text{--}participants):Converged$
	Reliability	y = log(beta)	$y \sim s+(1+s\text{--}participants):Converged$
	Volatility	y = log(beta)	$y \sim s+(1+s\text{--}participants):Converged$
PC depth path	Controllability	y = log(beta)	$y \sim s+(1+s\text{--}participants):Converged$
	Reliability	y = log(beta)	$y \sim s+(1+s\text{--}participants):Converged$
	Volatility	y = log(beta)	$y \sim s+(1+s\text{--}participants):Converged$
PC depth sum	Controllability	y = log(beta)	$y \sim s+(1+s\text{--}participants):Converged$
	Reliability	y = log(beta)	$y \sim s+(1+s\text{--}participants):Converged$
	Volatility	y = log(beta)	$y \sim s+(1+s\text{--}participants):Converged$

Table A.1: **Mixed-effects model convergence diagnostics.** In all cases, s refers to the stochasticity level. Other variables are defined in the column ‘Variable’.

All models converged without singular fits. As detailed in the Methods, we used maximal random-effects structures and only simplified if necessary; here, all maximal models converged successfully (Table A.1).

A.6 Model and parameter recovery

A.6.1 Simulation procedure

We conducted simulations using synthetic data to test whether our fitting procedures could accurately recover both the true model and the underlying parameters.

Synthetic participant parameters were drawn randomly: depth $d \in \{1, \dots, 7\}$, rank/value $k, v \in \{1, \dots, 9\}$; lapse $\lambda \sim U[10^{-3}, 0.5]$; and $\log(\beta) \sim U[-2, 0.5]$ (Policy Compression models: one $\log(\beta_q)$ per stochasticity level; Filter Adaptation models: a single global $\log(\beta)$), spanning the range observed in the empirical data. To ensure a realistic recovery regime, we generated exactly 100 participants for each generating model and stochasticity variant, with the same per-variant trial structure as in the experiment: 7 decisions per game, 30 games per stochasticity level, across 5 stochasticity levels.

Fitted continuous parameters used the same bounds and multi-start procedure as in Methods ($-3 \leq \log \beta \leq 3$, $10^{-3} \leq \lambda \leq 1$; 50 random restarts per continuous-parameter fit).

A.6.2 Model recovery

Model recovery is a common technique used to test model identifiability. In our case, several models provided comparably good fits across reliability, volatility, and controllability. For instance, the PC Depth-filtering Path and Level-Mean models were best fitting in all three forms of stochasticity, while the PC Depth-filtering Sum and Max models were also competitive under volatility.

We then fitted the synthetic data using our model fitting procedure to see whether the ground truth models were recoverable. Consistent with our model fitting procedure for our real data, we conducted five-fold cross-validation to establish the best-fitting model for each synthetic dataset, using the cross-validated negative log likelihood (sum of held-out fold NLLs) as our measure of goodness-of-fit. A model was said to be recoverable when bootstrap distributions of ΔCV NLL

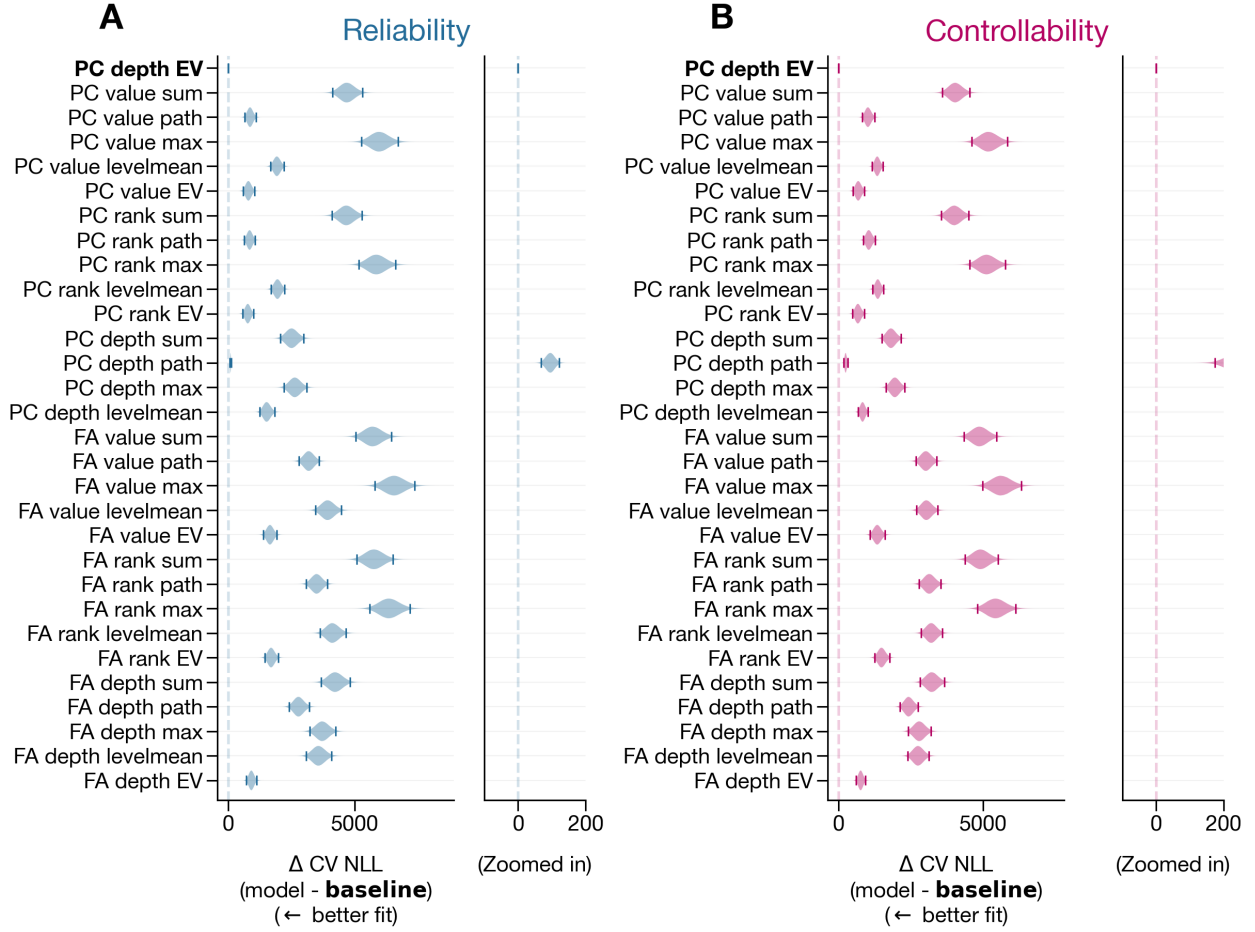


Figure A.8: **Model recovery for the PC Depth-filtering EV model.** Model recovery based on simulated data from the PC Depth-filtering EV (baseline) model. Each plot also has a zoomed-in inset to the right. **A.** In the reliability condition, distributions of bootstrapped ($n = 10^6$) differences between the 5-fold CV negative log likelihoods of alternative models and the baseline. Bars represent 95% confidence intervals. Zoomed-in version of the same plot is to the right. **B.** Same as A, for controllability.

(each alternative minus the generating baseline, with $n = 10^6$ participant resamples) showed that the generating model was substantially favored (using 95% confidence intervals) relative to all alternatives.

We find that under all conditions, for *all* models, the best-fitting model corresponded with the model that actually generated the synthetic dataset, demonstrating that our models are both distinguishable and recoverable from simulated data. For brevity, we show only the model recovery results for the PC Depth-filtering class of models (Fig. A.8–A.12).

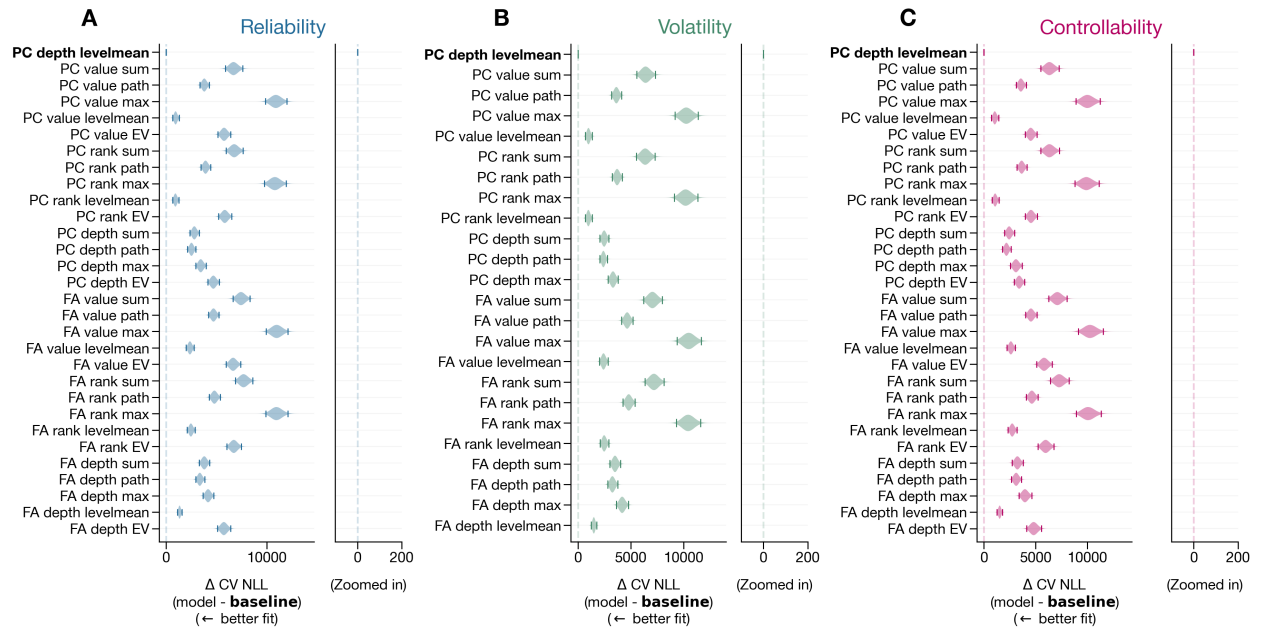


Figure A.9: **Model recovery for the PC Depth-filtering Level-Mean model.** Model recovery based on simulated data from the PC Depth-filtering Level-Mean (baseline) model. Each plot also has a zoomed-in inset to the right. **A.** In the reliability condition, distributions of bootstrapped ($n = 10^6$) differences between the 5-fold CV negative log likelihoods of alternative models and the baseline. Bars represent 95% confidence intervals. Zoomed-in version of the same plot is to the right. **B.** Same as A, for volatility. **C.** Same as A, for controllability.

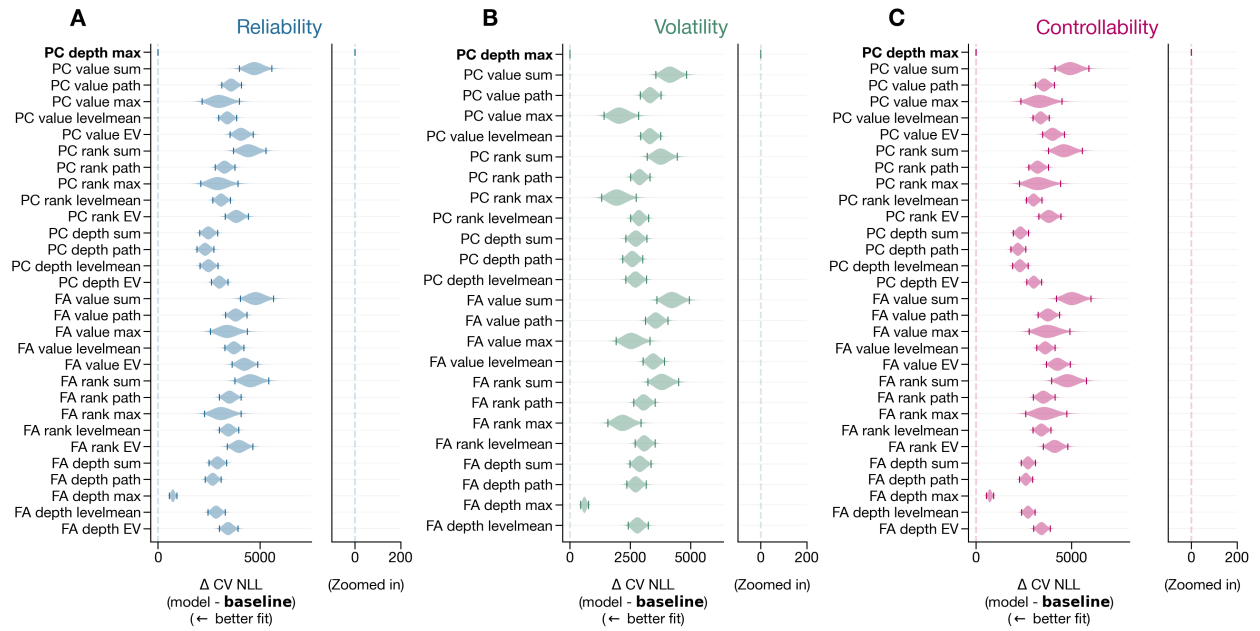


Figure A.10: **Model recovery for the PC Depth-filtering Max model.** Model recovery based on simulated data from the PC Depth-filtering Max (baseline) model. Each plot also has a zoomed-in inset to the right. **A.** In the reliability condition, distributions of bootstrapped ($n = 10^6$) differences between the 5-fold CV negative log likelihoods of alternative models and the baseline. Bars represent 95% confidence intervals. Zoomed-in version of the same plot is to the right. **B.** Same as A, for volatility. **C.** Same as A, for controllability.

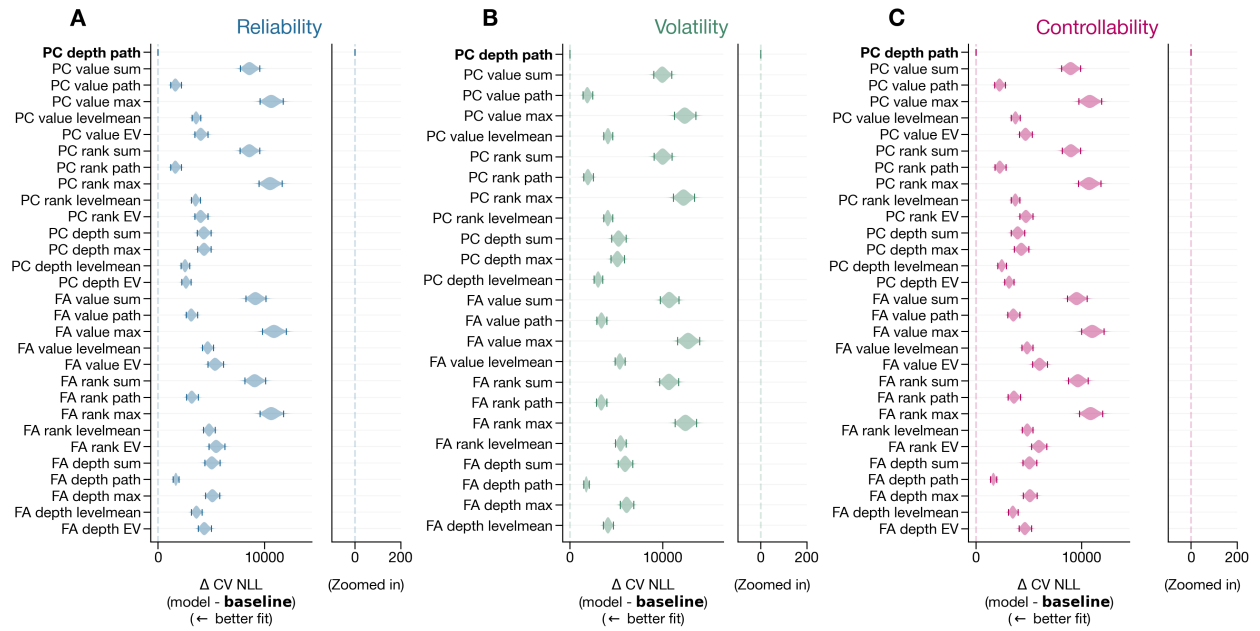


Figure A.11: **Model recovery for the PC Depth-filtering Path model.** Model recovery based on simulated data from the PC Depth-filtering Path (baseline) model. Each plot also has a zoomed-in inset to the right. **A.** In the reliability condition, distributions of bootstrapped ($n = 10^6$) differences between the 5-fold CV negative log likelihoods of alternative models and the baseline. Bars represent 95% confidence intervals. Zoomed-in version of the same plot is to the right. **B.** Same as A, for volatility. **C.** Same as A, for controllability.

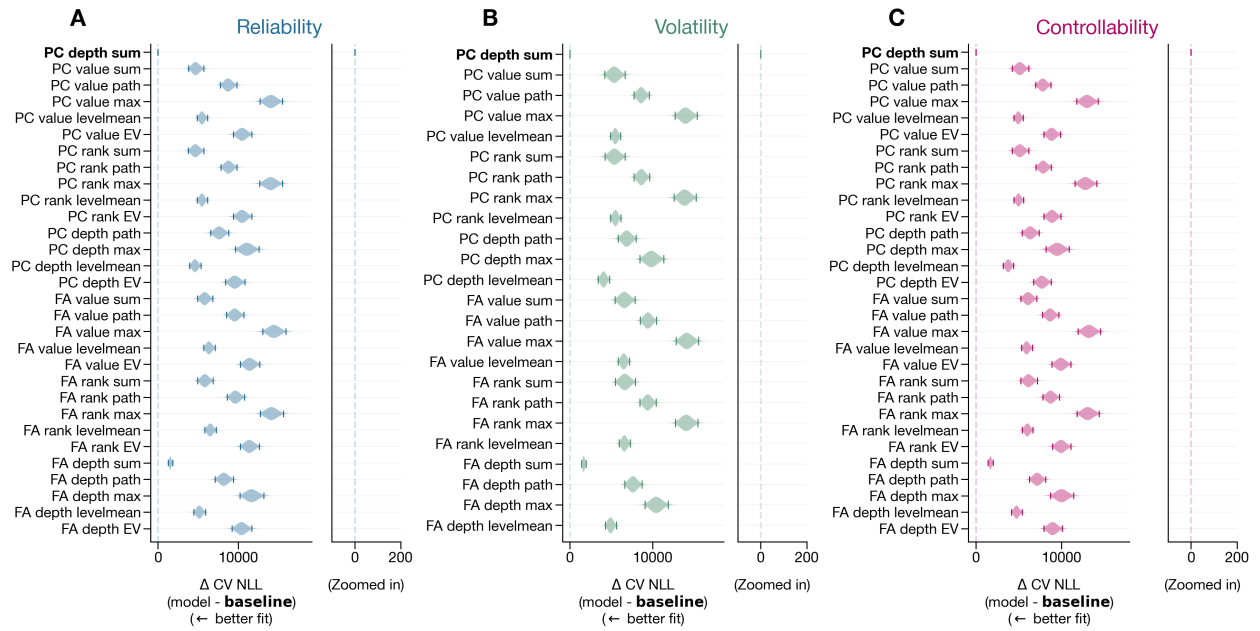


Figure A.12: **Model recovery for the PC Depth-filtering Sum model.** Model recovery based on simulated data from the PC Depth-filtering Sum (baseline) model. Each plot also has a zoomed-in inset to the right. **A.** In the reliability condition, distributions of bootstrapped ($n = 10^6$) differences between the 5-fold CV negative log likelihoods of alternative models and the baseline. Bars represent 95% confidence intervals. Zoomed-in version of the same plot is to the right. **B.** Same as A, for volatility. **C.** Same as A, for controllability.

A.6.3 Parameter recovery

We have now shown that our models are recoverable, but we also would like to show that each of the parameters is individually identifiable. To do this, we took the fitted synthetic data and compared the fitted parameters with the ground-truth parameter values.

Successful recovery of the parameters means that the majority of the points should lie on the diagonal; we find that this is indeed the case for our best-fitting models, the PC Depth-filtering Level-Mean (Fig. A.13) and Path (Fig. A.14), demonstrating that our parameter recovery is sound. We also find that the PC Depth-filtering Max (Fig. A.15) and Sum (Fig. A.16) models, which are also competitive in the Volatility condition, show decent recovery. As expected, parameter recoverability naturally decreases in regions of structural non-identifiability — e.g., at high depth levels where deeper planners behave identically once the horizon is reached, and for extreme $\log(\beta)$ values where behavior becomes either near-random or near-deterministic. These regimes correspond to regions where the likelihood surface is flat by construction.

A.7 Model Heterogeneity

To assess potential heterogeneity in planning strategies across participants, we conducted random-effects Bayesian model comparison (RFX-BMC) across participants. This analysis estimates the probability that each model is the most frequent in the population, explicitly allowing for heterogeneity in strategy use across individuals (Rigoux et al., 2014).

Consistent with our model comparison procedure in the main text, we used cross-validated negative log-likelihood (CV-NLL), aggregated across folds (and across repeated random CV seeds where applicable), as a sample-based approximation to log model evidence for each participant and model. These subject-level estimates were then entered into a RFX-BMC procedure implemented in Python (adapted from the VBA toolbox (Rigoux et al., 2014)). This procedure yields two key summary statistics: (1) *posterior model frequency*, the estimated proportion of participants best explained by each model, and (2) the *protected exceedance probability* (PXP), the posterior

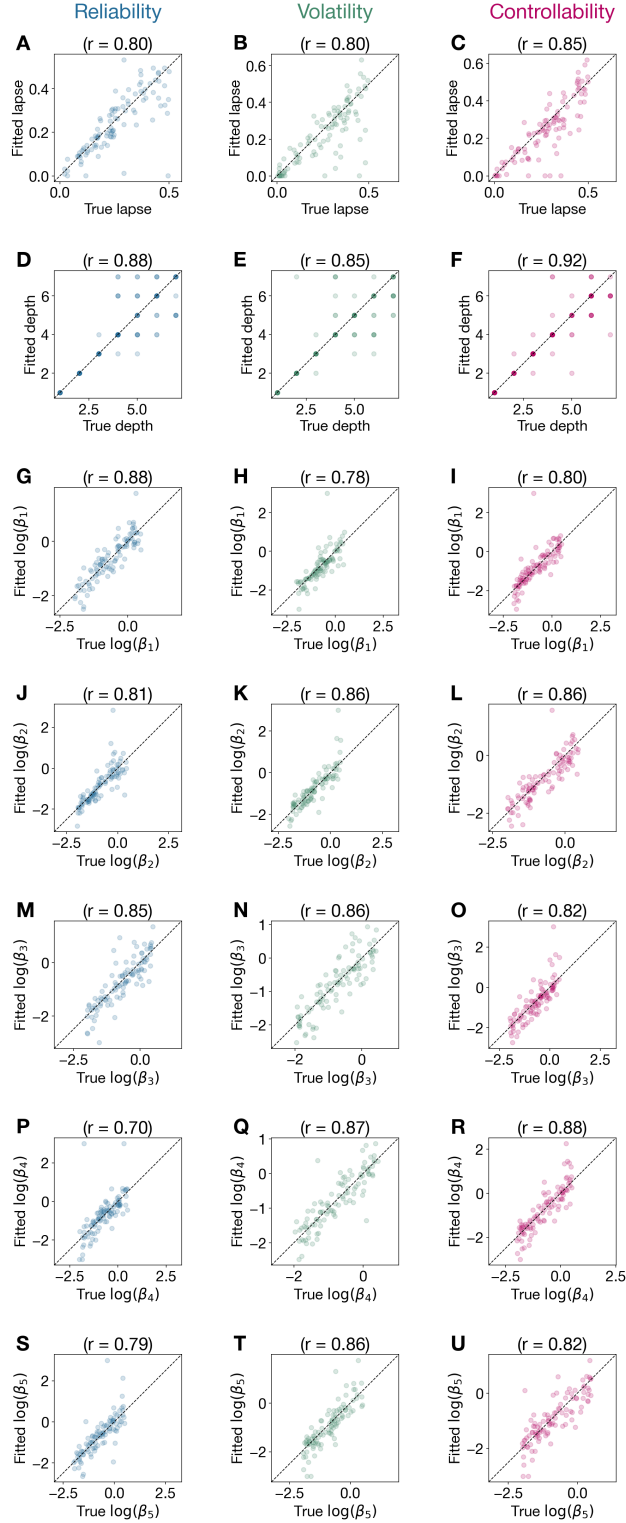


Figure A.13: **Parameter recovery for the PC Depth-filtering Level-Mean model.** Parameter recovery for simulated data from the PC Depth-filtering Level-Mean model. A–C. Ground truth depth versus fitted depth. D–F. Ground truth lapse vs fitted lapse. G–U. Ground truth $\log(\beta)$ versus fitted $\log(\beta)$, for each stochasticity level of the given condition.

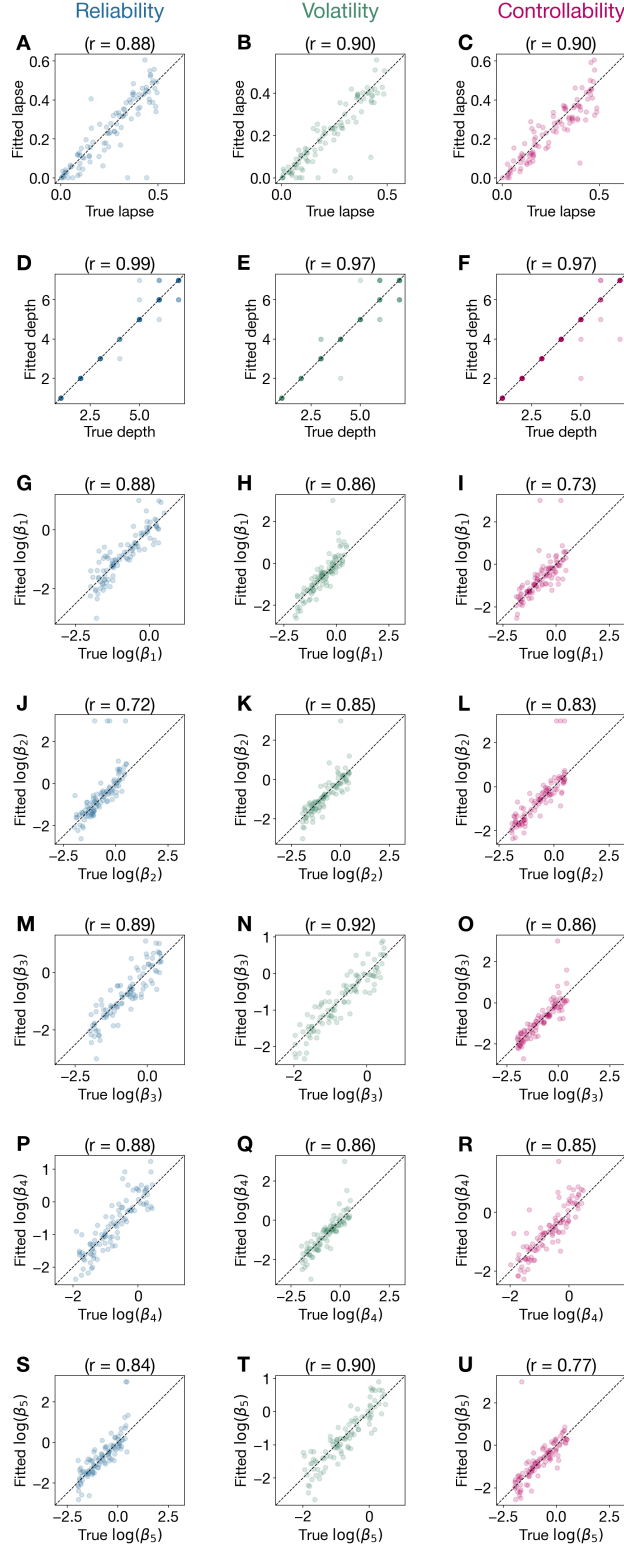


Figure A.14: **Parameter recovery for the PC Depth-filtering Path model.** Parameter recovery for simulated data from the PC Depth-filtering Path model. A–C. Ground truth depth versus fitted depth. D–F. Ground truth lapse vs fitted lapse. G–U. Ground truth $\log(\beta)$ versus fitted $\log(\beta)$, for each stochasticity level of the given condition.

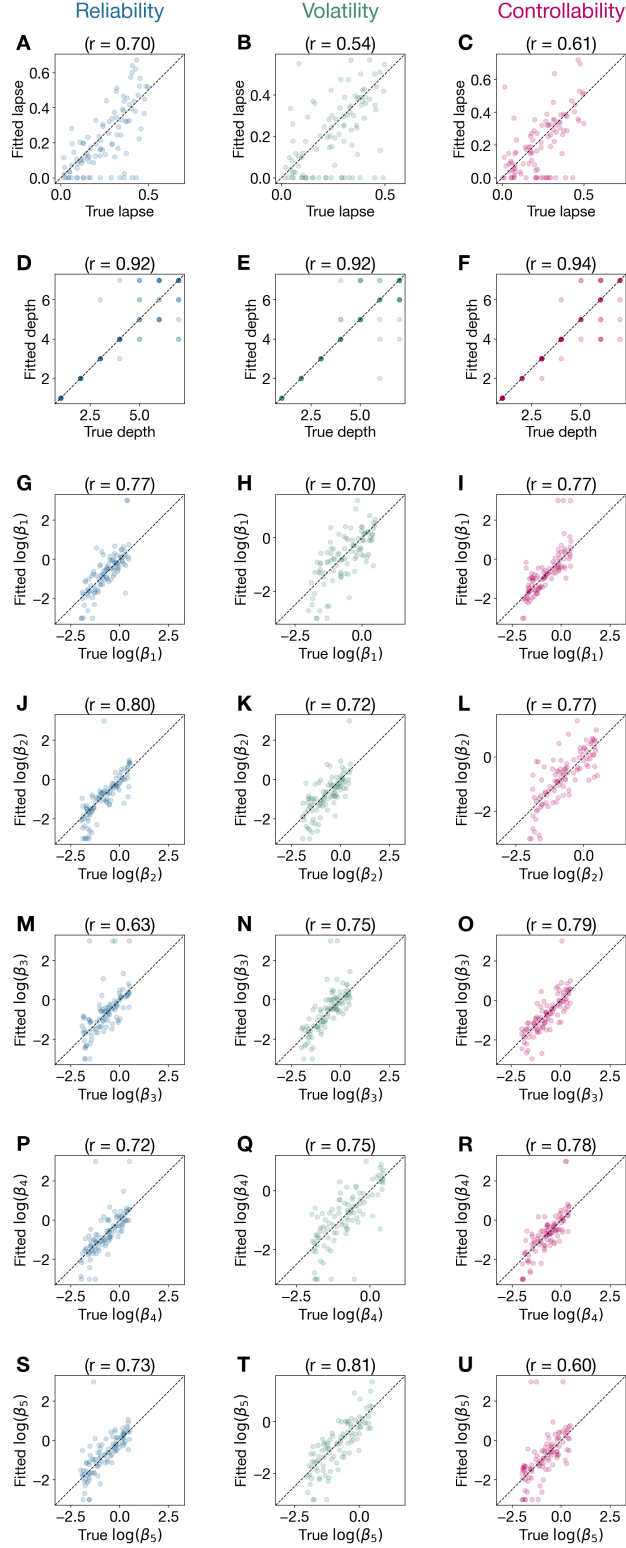


Figure A.15: **Parameter recovery for the PC Depth-filtering Max model.** Parameter recovery for simulated data from the PC Depth-filtering Max model. A–C. Ground truth depth versus fitted depth. D–F. Ground truth lapse vs fitted lapse. G–U. Ground truth $\log(\beta)$ versus fitted $\log(\beta)$, for each stochasticity level of the given condition.

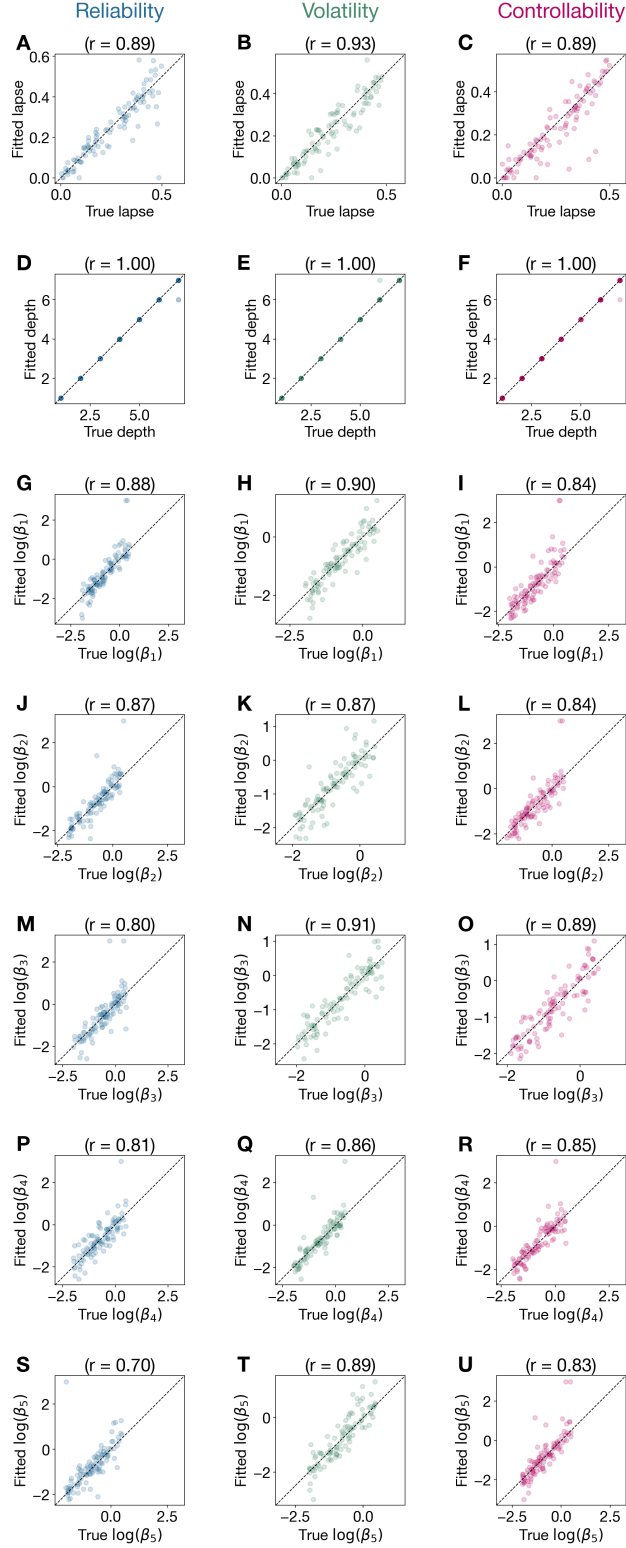


Figure A.16: **Parameter recovery for the PC Depth-filtering Sum model.** Parameter recovery for simulated data from the PC Depth-filtering Sum model. A–C. Ground truth depth versus fitted depth. D–F. Ground truth lapse vs fitted lapse. G–U. Ground truth $\log(\beta)$ versus fitted $\log(\beta)$, for each stochasticity level of the given condition.

probability that a given model is more frequent than all others in the population, accounting for the possibility that observed differences arise by chance.

A.7.1 Robustness of Heterogeneity Analysis

We treat RFX-BMC as a complementary summary of model heterogeneity rather than the primary basis for our main conclusions, which are based on aggregate cross-validated model performance. In particular, we are cautious about over-interpreting the RFX-BMC results because several closely related model classes exhibited similar cross-validated predictive performance. In this regime, RFX-BMC can be sensitive to small variations in subject-level evidence arising from different cross-validation partitions, which can lead to variability in posterior model frequencies and protected exceedance probabilities.

To assess the robustness of the RFX-BMC procedure on our data, we re-split the empirical data with five independent CV seeds, re-fit all models under each split, and applied RFX-BMC separately to each fit cohort, five instances of 5-fold cross-validation in total. This analysis quantifies the stability of RFX-BMC inferences under reasonable perturbations of the data partitioning and allows us to assess whether observed patterns of model heterogeneity are robust to the choice of CV split. Importantly, the additional CV realizations were *not used* in the primary aggregate model-comparison figures or to select among models.

A.7.2 Heterogeneity Results

As anticipated, the RFX-BMC procedure exhibited substantial variability across cross-validation realizations, indicating sensitivity to small differences in subject-level model evidence induced by the CV partitioning procedure. Notably, we *did not* find similar instability in our main model comparison result. Despite the variability in RFX-BMC inferences, a consistent qualitative pattern emerged across all stochasticity conditions: models belonging to the PC Depth-filtering family reliably achieved the highest posterior model frequencies and protected exceedance probabilities, supporting the broader conclusion that this model class best captures participant behavior. By

contrast, finer-grained distinctions between closely related value-function variants within the PC Depth-filtering family (e.g., Level-Mean, Max, Sum, Path) were considerably less stable across CV realizations. In many cases, differences in posterior model frequency between these variants were small, and protected exceedance probabilities remained modest, indicating insufficient evidence to support strong claims about participant-level preference for specific variants. Importantly, non-determinizing expected-value models rarely achieved the largest exceedance probabilities (one of the five robustness-check CV realizations was the sole exception), further supporting the broader conclusion that participants relied primarily on determinizing planning strategies. Together, these analyses suggest that the aggregate computational principles identified in the main text are robust, while finer-grained claims about heterogeneous strategy selection remain underdetermined by the present dataset.

Appendix B | Planning in a Strategy Game in Monkeys

B.1 Data

Chapter 3’s Methods (§3.4) gives an overview of the training curriculum and computer opponent at the level a reader of the Results needs. Here we give the full staged curriculum and the opponent’s heuristic parameters.

B.1.1 Training Protocol

Stage	Task	Days
Initial training	Joystick and choice training	45
Four-in-a-Row game	3-in-a-row and 4-in-a-row from existing board	116
Free play (solo)	Free play without an opponent turn	38
Free play (opponent)	Monkey win only	10
	Both players can win	81
Blocking training	Vertical, diagonal, and horizontal blocking	78
	Combinatorial blocking tasks	36
Adaptive opponent	Play with aggressive/defensive opponent	148
	Play with mixed opponent strategies	18
Total training days		570

Table B.1: **M1 training protocol.** Monkey (M1) training curriculum for Four-in-a-Row.

The monkey was trained in a staged curriculum (Table B.1). Training began with joystick control and discrete choice responses. The monkey then learned Four-in-a-Row on partially completed boards, first completing 3-in-a-row and 4-in-a-row patterns from existing stones. In a solo

free-play phase, the monkey placed pieces without an opponent turn. Opponent play was introduced in two steps: initially only the monkey could win, and only later was the opponent also allowed to win. The monkey was next trained to learn deliberate blocking, including vertical, diagonal, and horizontal block trials followed by combinatorial blocking configurations. Finally, the monkey played full games against an adaptive computer opponent that could shift between aggressive and defensive strategies, with a short additional block of mixed opponent strategies.

Our analyses focus on behavior recorded in this last stage of training, when the monkey played full games against the adaptive opponent without human intervention.

B.1.2 Computer Opponent

Parameter	Value	Description
$\beta_{\text{block_2IAR}}$	10	Weight on blocking 2IAR
$\beta_{\text{block_3IAR}}$	100	Weight on blocking 3IAR
β_{2IAR}	10	Weight on 2IAR
β_{3IAR}	100	Weight on 3IAR
β_{4IAR}	1000	Weight on 4IAR
ρ	varies	Block rate: trial-wise balance of blocking vs. connecting
w	10	History window: recent trials used to update ρ
ϵ	0.1	Exploration in updates to ρ

Table B.2: **Computer opponent heuristic parameters.**

The computer opponent used a simplified version of the Tree Search model used to describe human behavior (van Opheusden et al., 2023), with myopic rather than deep evaluation. At each turn, the opponent evaluated candidate moves using a heuristic value function over board patterns (2-in-a-row, 3-in-a-row, and 4-in-a-row configurations in horizontal, vertical, and diagonal orientations), with feature weights $\beta_{\text{block_2IAR}}$, $\beta_{\text{block_3IAR}}$, β_{2IAR} , β_{3IAR} , and β_{4IAR} (Table B.2). Throughout recording, these feature weights were held fixed. Note that there was no $\beta_{\text{block_4IAR}}$, since a completed 4-in-a-row ends the game. The opponent could additionally shift between of-

fensive and defensive emphasis via a trial-wise block rate ρ , updated from a sliding window of $w = 10$ recent trials with exploration rate $\epsilon = 0.1$; ρ dynamically adjusted to the monkey’s play while the β weights remained constant.

B.1.3 List of Known Issues in Behavioral Data

While preparing this thesis, we identified three implementation bugs affecting data collected under the training protocol described above. Two altered the computer opponent’s play; the third affected the monkey’s reward delivery and did not affect opponent strength:

1. The opponent’s move-selection code sometimes ignored the monkey’s horizontal connections when evaluating threats.
2. The opponent evaluated vertical, diagonal, and horizontal features sequentially rather than jointly, so it failed to assign extra weight to squares where multiple features overlapped.
3. The monkey erroneously received 1/6th of the intended reward for blocking moves.

Issues 1 and 2 weakened the opponent by causing it to undervalue or miss threats the monkey created. Table B.3 gives the date ranges over which each issue was present, by monkey. As of January 2026 all of these issues have been fixed in the training protocol of both monkeys.

Monkey	Date Range	Issues Flagged
M1	02/11/2023 – 03/18/2023	Issue 2
	08/04/2023 – 10/13/2023	Issues 1, 2
	02/09/2024 – 05/01/2024	Issues 1, 2, 3
M2	10/10/2023 – 12/13/2024	Issues 1, 2, 3

Table B.3: **Known issues in behavioral data.** Summary of data issues discovered after collection, by data collection date. All issues resolved as of January 2026.

B.2 Gameplay

The Methods section of Chapter 3 describes the trial structure itself. Here we focus on the data: how sessions are parsed, how malformed trials are cleaned, how board states are extracted, and how the resulting positions are selected and split for cross-validation.

Each recording session produced a behavioral log and, where available, an eye-tracking trace and neural spikes. The behavioral log recorded the task-event timestamps, the positions the computer and monkey played, and the opponent’s parameters for that trial. We parsed each session into this tabular form and then applied the cleaning, reconstruction, and selection steps below. The game-level filter for so-called *nontrivial* games was applied once, upstream of everything else, so that the behavioral, gaze, and neural analyses all inherit an identical set of games.

B.2.1 Behavioral Preprocessing

Each trial involves one computer move with one monkey move, but the logs contain a few deviations that needed to be resolved beforehand. We categorized four trial types: **(1)** *complete* trials, in which both the computer and the monkey move; **(2a)** trials in which the computer completes a 4-in-a-row and wins, so the monkey never moves; **(2b)** trials in which the monkey fails to register its move (an error); and **(3)** trials immediately following a 2b error, in which the monkey moves but the computer does not (it does not re-move after an error). A fifth case, in which neither player moves, is dropped by default. We dropped the no-action trials, any game that never resolves into a win for either side, and a small number of malformed games (those in which the monkey appears to move first, or in which a type-3 trial is not preceded by its 2b partner).

Each 2b error and the type-3 trial that follows it describe a single, interrupted decision, so we merged them into a single *pseudotrial*: the computer’s move and its event timestamps were stitched from the 2b row onto the type-3 row, the 2b row was dropped, and the result read as one (long) trial in which the monkey eventually placed its piece.

With the trial sequence cleaned, we reconstructed the exact board the monkey saw at each

trial. For every complete trial we applied the computer’s move and then recorded the resulting position, the occupied computer and monkey positions, paired with the position the monkey chose from it. This (position, move) pair, with the monkey to move, describes the data and target for the gameplay models. Positions were stored as bitboards: the computer’s and monkey’s pieces each as $\sum_x 2^x$ over their occupied positions x , and the move likewise as 2^x for the chosen position, so that a position’s ply (its piece count) is just the combined sum of the bitboards.

Two selection steps then defined the modeling set. First, the upstream game-level filter keeps only games that reach at least nine pieces, measured over their non-cpu-win positions. This allows us to filter out games where either the monkey or the CPU won immediately. For fitting, we took all surviving ≥ 9 -ply games (games are logged chronologically) and partitioned them into five equal-size temporal cohorts, largely for fitting tractability but also to preserve temporal contiguity. Within each cohort we split games into three cross-validation folds at the level of whole games (a game is never divided across folds) and wrote each fold as a table of (black pieces, white pieces, move, color, trial id, piece count) rows. We trimmed the earliest positions of every game (fewer than three pieces on the board), which carry too few features for the search to fit. The result is the per-cohort, per-fold dataset consumed by the model fitting of §B.2.5. Table B.4 reports the dataset at each of these stages.

Stage	Filter	Games	Trials	Monkey win rate
Unfiltered	—	20,932	85,932	0.42
Total	≥ 9 -ply games	5,764	32,868	0.51
Training	drop < 3 -piece trials	5,760	27,083	0.51

Table B.4: **Monkey Four-in-a-Row dataset summary.** Summary of data at each filtering stage. The full record spans 2023-02-20 to 2024-05-01. “Trials” counts (position, move) pairs; “Total” is the ≥ 9 -ply game set shared by the behavioral, gaze, and neural analyses, and “Training” is the modeling set—the full ≥ 9 -ply games with the oldest few dropped so the count divides evenly across cohorts.

The models we fit to the monkey’s choices in Chapter 3 follow a heuristic-guided best-first tree search developed to describe human Four-in-a-Row gameplay (van Opheusden et al., 2023, 2017).

Rather than evaluate a board by playing it out to the end, which is an intractable state space of 10^{16} positions, the model iteratively builds a tree and evaluates the board states along that tree, backpropagating the best path down to the root. We adopted it with minimal modifications; where deviations exist, we note them in (§B.2.7).

For a full reference of the model specifications, we refer to the Supplementary Information section of van Opheusden et al. (2023). This appendix clarifies where our implementation departs from the existing model, and where appropriate, restates some of the basic procedural scaffolding that gives context to our implementation. We build up the model in the order a single decision unfolds: how a board is valued (§B.2.2), how that value is turned into a move through bounded search (§B.2.3), how we score the resulting stochastic policy against data (§B.2.4), and how its parameters are fit (§B.2.5). We then summarize the model variants we compared and the deviations from the human model (§B.2.6–B.2.7). Throughout we match the parameter names of the source: the stopping probability γ , pruning threshold τ , feature-drop rate δ , lapse rate λ , opponent scale η , and center weight c (Table B.5).

B.2.2 Board and Value Functions

A board state s is fully specified by the occupied positions on a 4×9 grid by player p (the player to move, in this case, the monkey) and opponent $\bar{p}$. The heuristic value function operates on features of the board, which are templates of positions that are pattern-matched to a given board position. For example, a 3-in-a-row (3IAR) corresponds with a template where there exists four contiguous positions, where three of them are filled by the player of interest and one is empty. A player *owns* a position if its pieces match the template; that is, if the player occupies all the positions specified in the template and the remaining positions are empty. Importantly, blocked features are not considered valid; an n -in-a-row feature is therefore equivalently a potential 4-in-a-row feature that is currently n -filled. Features are collected into *groups* based on category, and features of the same group share the same weight; in our case the groups are 4-in-a-row, 3-in-a-row (connected and disconnected patterns sharing a single weight), connected 2-in-a-row,

and disconnected 2-in-a-row.

The value of the board is the weighted sum of the value of each of the features of the board, plus a center bias c . Each feature group for the player g has weight w_g , and each of the opponent's features has *scaled* weight $-\eta * w_g$ (where $\eta = 1$ means that the player weights the opponent's features as equal in magnitude to its own, and $\eta = 0$ means that the player completely ignores its opponent's features). The result of this weighting is a scalar heuristic value that represents the utility of that state. In Algorithm 12, `pieces_self(s)` and `pieces_opp(s)` denote the positions the player and opponent occupy, and `count_self(g, s)` and `count_opp(g, s)` count the enabled group- g features each owns.

Algorithm 12 `EVALUATE(s)` — the heuristic value of a board, from the mover's perspective

Require: state s ; weights $\{w_g\}$, opponent scale η , center weight c ; enabled features F

```

1:  $V \leftarrow 0$ 
2: for  $x \in \text{pieces\_self}(s)$  do
3:    $V \leftarrow V + c v_x$  ▷ reward own pieces near the center
4: end for
5: for  $x \in \text{pieces\_opp}(s)$  do
6:    $V \leftarrow V - c v_x$ 
7: end for
8: for each feature group  $g$  do
9:    $V \leftarrow V + w_g \text{count\_self}(g, s) - \eta w_g \text{count\_opp}(g, s)$  ▷ weighted sum of self and
   opponent features
10: end for
11: return  $V$ 

```

The center bias is fixed per position at $v_x = [(\text{row}(x) - 1.5)^2 + (\text{col}(x) - 4)^2]^{-1/2}$, largest at the center of the board and diminishing toward the edges. Rather than re-evaluate every successor board from scratch when it considers a move, the model evaluates each candidate position by the *change* it would make to this value. When noise is switched on, each move's score also receives an independent Gaussian *value-noise* perturbation $\varepsilon_x \sim \mathcal{N}(0, 1)$. We inherited this move-valuation procedure as given by van Opheusden et al. (2023).

B.2.3 Tree Search Algorithm

Our model is designed to be a process model to capture the deliberation of the monkey during gameplay: the model iteratively builds a small tree by extending the best node and propagating the new information back to the root. Below we describe the parameters in full.

- How long to think, as controlled by a stopping probability γ , $0 \leq \gamma \leq 1$. At each step, the search halts with probability γ ¹. A large γ means the model stops sooner in expectation; at $\gamma = 1$ it expands the root once and behaves myopically. Conversely, a small γ corresponds with deeper search in expectation.
- Which moves to consider and which to prune, as controlled by pruning parameter τ ; any successor whose move falls τ below the best available move is pruned from the tree so the search does not spend time deliberating on bad options. Evidence for this kind of pruning is shown in existing work and appears to follow Pavlovian properties (Huys et al., 2012).
- Attentional lapses over feature groups, which are dropped with probability δ at the start of tree search; this represents the case when a player failed to notice a particular feature group. Note that in our production fits parameter recoverability degraded once δ grew large, so we excluded δ from the search entirely and fixed it at 0 for our analysis (§B.2.8).
- Evaluation noise, which we parameterize by ε_x (see §B.2.2), which adds noise to the move evaluations.
- Lapses over choice, parameterized by λ , which represents the probability that the player makes a random legal move on the board.

Algorithm 13 assembles these pieces. Each iteration walks the path from the root to the best available leaf, i.e. the *principal variation*, expands that leaf into its surviving successors, evaluates them, and propagates the new values back toward the root via minimax (the player

¹In the actual implementation, the total number of expansions is drawn once from a geometric distribution with parameter γ at the start of search; this is an equivalent formulation but avoids repeated computation.

takes the maximum over its options, the opponent the minimum). When the search ends, the model chooses the move at the root with the best backed-up value (unless it lapses).

Algorithm 13 MAKEMOVE(s) — deliberating to a move (after van Opheusden et al.)

Require: state s ; stopping prob. γ , drop rate δ , pruning τ , lapse λ

```

1: if LAPSE( $\lambda$ ) then
2:   return RANDOMMOVE( $s$ )                                 $\triangleright$  lapse: play a random legal move
3: end if
4: if  $\delta > 0$  then
5:   DROPFEATURES( $\delta$ )   $\triangleright$  drop each feature group w.p.  $\delta$  for this search; excluded from the
                        search and skipped ( $\delta = 0$ ) in our fits, §B.2.8
6: end if
7: root  $\leftarrow$  NODE( $s$ )
8: while not STOP( $\gamma$ ) and not DETERMINED(root) do
9:    $n \leftarrow$  SELECTNODE(root)                             $\triangleright$  descend the principal variation to a leaf
10:  EXPANDNODE( $n$ )                                            $\triangleright$  add a child per move within  $\tau$  of the best; evaluate with  $V$ 
11:  BACKPROPAGATE( $n$ )                                        $\triangleright$  propagate values to the root by minimax
12: end while
13:  $a \leftarrow \arg \max_{c \in \text{children}(\text{root})} c.\text{val}$ 
14: return  $a$ 

```

Here LAPSE(λ) and STOP(γ) are stochastic, returning true with probability λ and γ respectively, so the number of expansions before STOP fires is geometric in γ ; DETERMINED tests whether the root’s outcome is already forced; SELECTNODE descends to a leaf by repeatedly taking the best child (the mover maximizing its value, the opponent minimizing it); EXPANDNODE adds one child per legal move that survives the τ prune and evaluates it with V ; and BACKPROPAGATE updates the values of all ancestors of the new nodes. A search also ends early once the root is *determined*: a forced win, loss, or draw has been proven and there is nothing left to learn. The human model also carries a second, convergence-based stopping rule (halt once the preferred root move has gone unchanged for many consecutive expansions), which it uses to predict response times and eye movements (van Opheusden et al., 2023); in the monkey fits we left this rule inactive.

A subtlety worth noting is that this prune operates on the *heuristic* value, which includes the 4-in-a-row term, *before* a node is expanded and a winning move can be recognized as terminal. The 4-in-a-row weight therefore acts less as the value of a win than as the salience that keeps a

winning move from being pruned; we examine downstream implications in §B.2.9.

B.2.4 Negative Log-Likelihood Estimation with Inverse Binomial Sampling

Because our model samples moves stochastically, it can be somewhat awkward to fit to the data. In general, there is no well-defined closed-form solution for the likelihood of the observed move under the model. As in the human work, we used inverse binomial sampling (IBS) (van Opheusden et al., 2020). The idea is fairly simple: to estimate how probable the observed move was, simply run the model over and over on that board and count how many runs it takes to reproduce the actual move the monkey made. Algorithm 14 is an *unbiased* estimator of the log-likelihood—it neither systematically over- nor under-states how well the model fits—which is exactly the property an optimizer needs.

Algorithm 14 $\text{IBS}(s_i, a_i; \theta)$ — an unbiased estimate of one trial’s log-likelihood

Require: observed trial (s_i, a_i) ; model parameters θ

```

1:  $L \leftarrow 0$ ;  $k \leftarrow 0$ 
2: repeat
3:    $k \leftarrow k + 1$ 
4:    $\tilde{a} \leftarrow \text{MAKEMOVE}(s_i; \theta)$  ▷ sample a move from the model
5:   if  $\tilde{a} \neq a_i$  then
6:      $L \leftarrow L - \frac{1}{k}$  ▷ each miss adds a harmonic penalty
7:   end if
8: until  $\tilde{a} = a_i$ 
9: return  $L$ 
```

Where K_i is the number of samples needed on trial i , the estimate is $L_i = 0$ when the first sample already matches and $L_i = -\sum_{k=1}^{K_i-1} \frac{1}{k}$ otherwise. Because each L_i is noisy, we averaged several independent repeated estimates per trial and summed across trials to obtain the objective the optimizer maximizes.

B.2.5 Model Fitting

The fitting objective inherits three inconvenient properties from IBS: it is stochastic (re-evaluations using the same parameters may give different results depending on noise) and expensive (each evaluation requires sampling until the observed move is reproduced), and it has no usable gradient. These are precisely the conditions Bayesian Adaptive Direct Search (BADS) is built for (Acerbi and Ma, 2017), and we used it with uncertainty handling enabled so that it treated the objective as noisy rather than chasing sampling jitter (function tolerance 10^{-7} and an absolute mesh-tolerance gate of 1×10^{-3} , which the search reaches within a few hundred evaluations, well below the 10,000-evaluation safety ceiling). Parameters were held within the hard and plausible bounds of Table B.5. Because the noisy objective can easily result in our optimizer getting stuck in local minima, each fold was fit from five random starts within these bounds; we kept the start with the best training log-likelihood, re-evaluated at high IBS precision, and computed held-out loss only at those winning parameters.

Parameter	Hard bounds	Soft bounds	Description
τ	[0, 10]	[2.0, 9.5]	Pruning threshold
γ	[0.05, 1]	[0.2, 0.7]	Stopping probability
δ	fixed at 0	—	Feature-drop probability
λ	[0.02, 0.5]	[0.02, 0.15]	Lapse rate
η	[0, 5]	[1.0, 3.5]	Opponent scale
c	[−10, 10]	[−2, 3]	Center weight
$w_{4\text{IAR}}$	[−4, 24]	[5, 15]	Weight on 4IAR
$w_{3\text{IAR}}$	[−4, 24]	[1.6, 4.4]	Weight on 3IAR (merged)
$w_{2\text{IAR_CON}}$	[−4, 24]	[0.2, 1.8]	Weight on connected 2IAR
$w_{2\text{IAR_DIS}}$	[−4, 24]	[−0.2, 1.4]	Weight on disconnected 2IAR

Table B.5: **Monkey Tree Search model parameters.** Hard and soft bounds listed for BADS optimization.

We instituted a warm-start procedure for optimization. Since the variance of an IBS estimate falls with the number of repeats, we spent repeats where they mattered: early in optimization, when BADS was still roaming, a coarse estimate from a few repeats per trial was enough to

point it in the right direction, so we began at a minimum number of repeats $n_{repeats} = 5$ and ramped up over the search, reserving the largest repeat counts for the fine polishing near the optimum $n_{repeats} = 40$. A higher precision setting (50 replicates) was used both to re-evaluate the five multistarts’ training likelihood to select the winner (for winner selection, we also averaged over 5 evaluations to reduce noise), and to score the winning parameters on held-out data. Fitting followed the cross-validation scheme described in Chapter 3: we kept the monkey’s games with at least nine pieces placed, divided them into five chronological cohorts to allow for slow changes in play, and within each cohort ran three-fold cross-validation at the level of whole games; a model’s overall score is its cross-validated negative log-likelihood summed across folds and cohorts.

B.2.6 Model Variants

The variants compared in Chapter 3 are all restrictions of Algorithm 13, with varying degrees of flexibility:

- **Tree Search** corresponds with the full model.
- **Myopic** fixes $\gamma = 1$, allowing a single root expansion: the model still evaluates moves with the full heuristic but no longer looks ahead.
- **Self-Only** (Myopic) additionally sets $\eta = 0$, so the model ignores the opponent’s features entirely.
- **Feature lesions** remove one feature group at a time from the Tree Search model—one lesion each for 4-in-a-row, 3-in-a-row, connected 2-in-a-row, and disconnected 2-in-a-row—to gauge how much each pattern contributes.

B.2.7 Changes from Van Opheusden et al. (2023)

While we have tried to keep the model largely unchanged from the original Tree Search algorithm fit to human data, our model differs somewhat from van Opheusden et al. (2023). Three differences

are worth flagging.

- **A simpler opponent term.** The human model balances the two perspectives with an “active scaling constant” that reweights move values through a pair of normalized coefficients. We collapsed that machinery into the single opponent scale η , which multiplies the opponent’s feature weights directly; $\eta = 1$ recovers symmetric weighting. The two schemes serve the same purpose but are not identical, so η should not be read as equivalent to the human “active scaling constant.”
- **No convergence stopping rule.** As noted in §B.2.3, the consecutive-unchanged-move stopping rule was switched off, so deliberation length depends only on γ and determinacy.
- **Excluded feature dropping and adjusted bounds.** We excluded the feature-drop rate δ from the search entirely, fixing it at 0, rather than fitting it as the human model does (§B.2.3), and we constrained the remaining parameters to adjusted, somewhat narrower bounds (Table B.5) chosen to aid identifiability.

B.2.8 Parameter Recovery and Identifiability

The fitting procedure of §B.2.5 was shaped by two failure modes common to fitting this particular Tree Search model: unreliable convergence and weak parameter identifiability. Earlier instantiations of the model showed three related failure modes, each of which made model interpretation too noisy to be meaningful. First, they *underfit*: a fuller model sometimes scored worse on its own lesion’s training data than the lesion did, which is impossible at a true optimum and so betrays incomplete convergence. Second, the optimization was unreliable: the Tree Search likelihood has no closed form and is estimated by sampling (IBS, §B.2.4) and optimized with a noisy, derivative-free search (BADs, §B.2.5), so a difference in score between two models could reflect which model the optimizer happened to fit better rather than a difference in the data. Third, the stopping probability γ , the parameter on which the Myopic-versus-Tree-Search comparison rests, was poorly identified, recovering only weakly from simulated data.

The procedure addresses these by fitting each model from *multiple random restarts* rather than a single optimization: every fold was fit from five starts spread across the plausible parameter range, and the start with the best re-evaluated training likelihood was retained (§B.2.5). Because an individual BADS run can settle in a local optimum of the noisy objective, taking the best of several restarts is what allows the recovered score to reflect the data rather than the initialization of a single run. The remainder of this section is the evidence that, under this procedure, both failure modes are controlled: the fits converge, and the parameters, above all γ , are recoverable.

One further parameter deserves comment here: the feature-drop rate δ . We observed that parameter recoverability degraded once δ was allowed to grow sufficiently large, at which point it traded off against the feature weights and lapse rate rather than settling on a stable value. We therefore excluded δ from the search entirely, fixing it at 0 for all production fits (Table B.5) and when generating the synthetic data below.

Nesting. A full model contains its lesion as a special case, so the full model ought to outperform the lesioned models on the training data; a full model that fits worse points to underconvergence or overfitting. On the production fits this nesting property holds on all 60 of 60 lesion–fold comparisons: the multiple restarts let each fold reach its optimum reliably, so a fuller model no longer scores worse than its own lesion.

Round-trip recovery. To test the procedure end to end, we generated synthetic data from the model at known parameters and refit it blind. We drew 30 ground-truth parameter sets from a low-discrepancy (Sobol) sweep of the parameter space, simulated one move per board over a real training fold (so each synthetic dataset is matched one-to-one with a production fit), and refit each from five fresh starts, none initialized at the true value. A fit *recovers* if its parameters are identifiable from the refitting process. Across the thirty test points, recovered values track their generating values closely for the control parameters and lower-order feature weights—the center weight ($r^2 = 0.97$), disconnected 2-in-a-row (0.91), connected 2-in-a-row (0.81), opponent scale (0.78), and lapse rate (0.74) (Fig. B.1). Crucially, the stopping probability γ —the Myopic-versus-

Tree-Search knob—recovers at $r^2 = 0.86$, confirming that the distinction is identifiable. Recovery is looser for the pruning threshold (0.58) and degrades as a feature becomes rarer on the board, with the 3-in-a-row weight intermediate ($r^2 = 0.55$) and the 4-in-a-row weight the least identified of all ($r^2 = 0.03$); we take up this last case in §B.2.9. Because the model comparison rests on the achieved held-out likelihood rather than on point-identifying any single weight, this residual looseness does not threaten its conclusions.

B.2.9 The Four-in-a-Row Weight

The 4-in-a-row weight occupies a special position in the model. It carries the largest fitted value of any feature, yet it is the least identifiable (§B.2.8) and the cheapest to remove: lesioning it costs less held-out likelihood than lesioning any other feature (Chapter 3). Both anomalies follow from the role the feature plays in the search rather than the evaluation.

A completed 4-in-a-row is a terminal win, assigned the value $+\infty$ directly, so the 4-in-a-row weight never sets the value of a won position. Instead it acts during *pruning*. Before a node is expanded, the search discards every candidate move whose heuristic value falls more than τ below the best (§B.2.3), and this prune operates on the heuristic value, which includes the 4-in-a-row term, *before* the move is expanded and its terminal value discovered. The weight is therefore what lifts a winning move above the pruning threshold so that the search expands and confirms it; with too little weight, a winning move can be discarded before it is ever recognized as a win, producing a *missed win*. The feature acts as the salience of a win rather than its value.

This accounts for both anomalies. Because a 4-in-a-row is relevant only on the rare boards where a win is within reach, most positions carry no information about its weight, and the weight is correspondingly hard to recover. And its behavioral signature, the rate of missed wins, saturates: once the weight is large enough that winning moves are never pruned, further increases have no effect, so the likelihood is flat in the weight above that point and its precise value is unrecoverable. Because missed wins do sometimes occur in the data, the weight is constrained, but only loosely, and it remains the least identified of the feature weights. The same missed-win be-

Parameter Recovery (n=30, n_trials=3611)

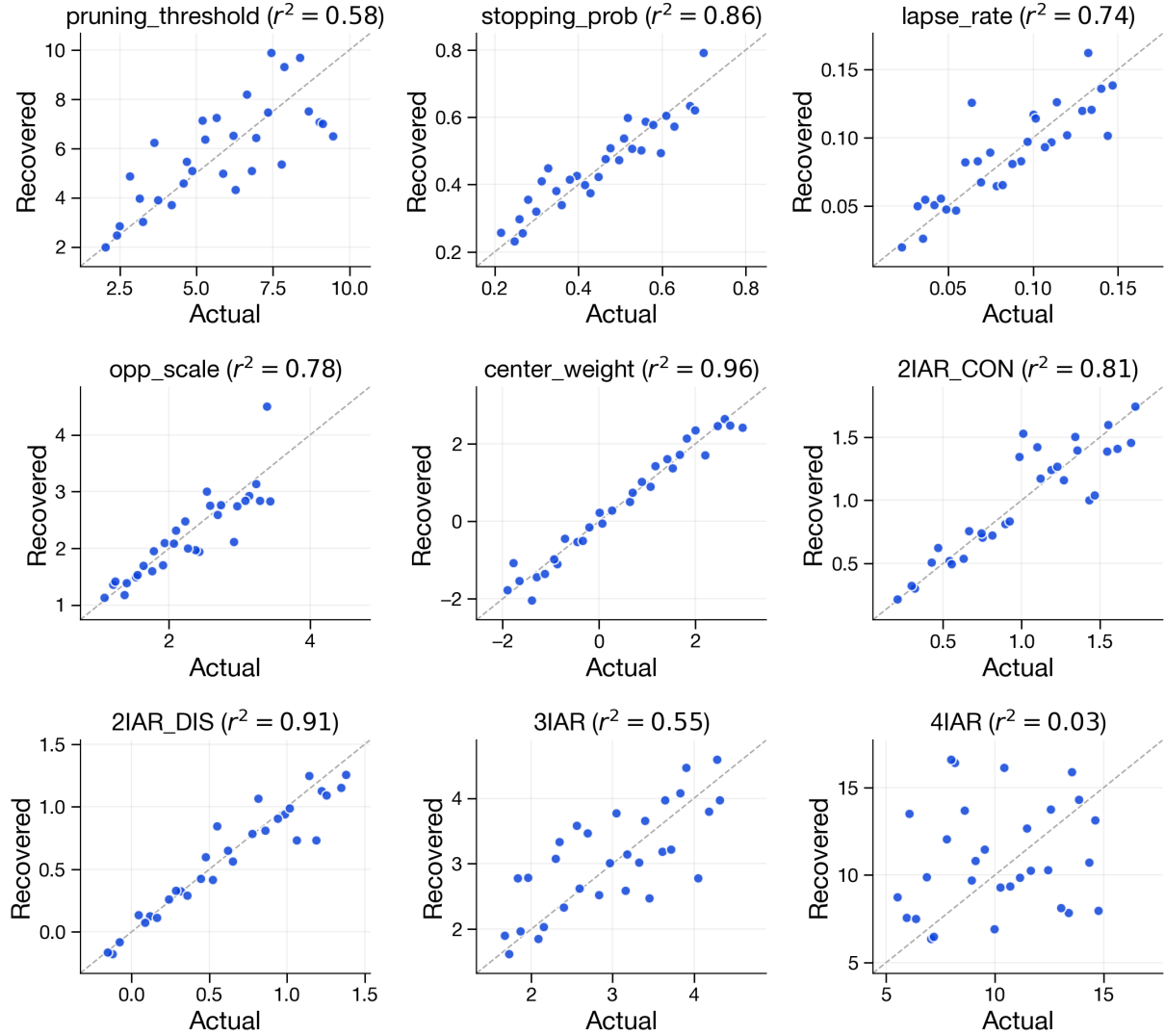


Figure B.1: **Parameter recovery.** Recovered versus ground-truth values across the 30 Sobol test points, one panel per parameter (a perfect fit lies on the dashed diagonal). Most parameters recover well; the stopping probability γ —the Myopic-versus-Tree-Search knob—recovers at $r^2 = 0.86$. The 4-in-a-row weight is the only strongly underconstrained parameter ($r^2 = 0.03$), limited by how rarely the data contain a 4-in-a-row.

havior, and the same outsized but weakly determined 4-in-a-row weight, are reported for human players fit with this model (van Opheusden, 2019).

B.3 Gaze

B.3.1 Gaze Preprocessing

Where a session carries a gaze trace, the raw `.eyedata` file records gaze as a stream of timestamped (x, y) samples in screen pixels, sampled at approximately 500 Hz (one sample every 2 ms) on the same millisecond clock as the behavioral events. Before any sample was used we applied a fixed display calibration—a 10-pixel horizontal offset that corrects a known misalignment between the eye tracker and the 800×600 game display—and marked any sample falling outside the display (beyond a 50-pixel margin) as off-screen, rather than discarding the trial.

Gaze was tied to behavior through its timestamp: for each behavioral event we located the nearest eye sample by binary search on the shared clock. From the aligned stream we built two products. The first is a per-window *fixation histogram*, counting how long gaze dwelt on each board position over a specified interval of the trial; the second is an *event-aligned* trace that keeps the full sample sequence in a fixed window around a chosen event, extracted from 1500 ms before to 500 ms after the alignment event, matching the $[-1.5, +0.5]$ s analysis window quoted in Chapter 3.

Within any window, fixations were identified in three steps. First, the raw x and y traces were smoothed with a median filter spanning 21 samples to suppress tracker jitter. Second, instantaneous eye velocity was estimated from the smoothed trace with a five-point central difference, and any sample whose speed exceeded a fixed threshold (or that was invalid) was labeled a saccade and set aside. Third, the remaining contiguous runs of non-saccadic samples were grouped into candidate fixations; groups shorter than 40 samples (about 80 ms) were discarded as artifacts, and each surviving group was summarized by the mean location of its samples. A window shorter than the median-filter span yields no fixations and contributes an empty histogram.

Each fixation was finally assigned to one of the 36 positions of the 4×9 board, whose on-screen centers lie on a regular grid spanning the play area. We used one of two assignment rules. For fixation histograms, a fixation is assigned to the *nearest* position center. For event-aligned analyses we instead required the fixation to fall within a position’s on-screen target region, a circle of radius 27 pixels drawn during gameplay, so that gaze landing between positions is left unassigned rather than forced onto the closest one; gaze outside the board area is also unassigned. A complementary per-sample classification labeled each sample as falling on the eventual move, on another board position, on the off-board hold cue (within 60 pixels of its fixed location below the board), between positions, or off-screen; this was the labeling behind the gaze time-course analysis of Chapter 3. Throughout, pseudotrials and trials with invalid event codes at the window’s endpoints were excluded.

B.3.2 Trial-Wise Validation via KL Divergence

We conducted a short follow-up to Chapter 3’s zero-shot gaze comparison. In the main text, this is reported using the cross-trial NLL, which weights trials by their number of fixations so that fixation-rich trials count for more. As a trial-wise validation that instead treats every trial equally regardless of fixation count, we also compared the predicted and true gaze distributions using the KL divergence between them. For a trial i , let S_i be its empty board positions, $q_i(s) = n_i(s) / \sum_{s'} n_i(s')$ the normalized observed gaze distribution, and $p_i(s)$ the predictor map (floored and renormalized over S_i),

$$\text{KLD} = \frac{1}{T} \sum_{i=1}^T \sum_{s \in S_i} q_i(s) \log \frac{q_i(s)}{p_i(s)}. \quad (\text{B.1})$$

where:

- $q_i(s)$ is the normalized observed gaze distribution on trial i ,
- $p_i(s)$ is the predictor map’s distribution on trial i ,
- T is the number of trials.

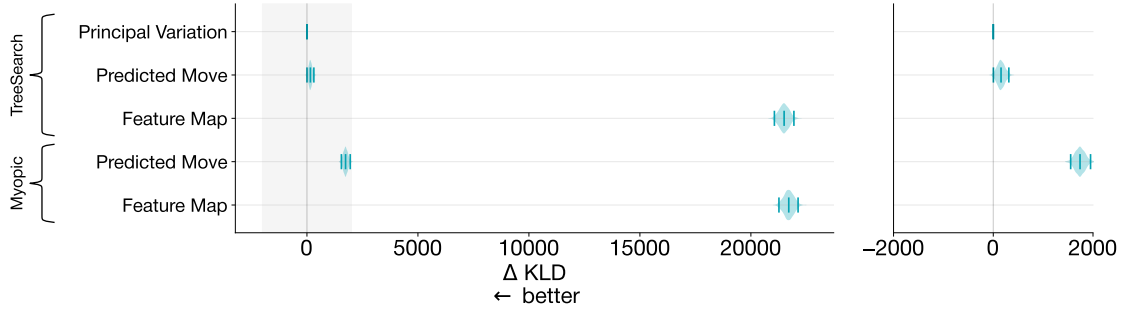


Figure B.2: **Zero-shot gaze KL divergence.** Difference in KL divergence between the predicted gaze distribution and the true gaze distribution for each zero-shot predictor, across two model classes. Note that occupied positions were not considered in this analysis. Error bars represent bootstrap-derived 95% CI.

The lower the KL divergence, the more faithfully the prediction matches the actual distribution. We compared our four classes of contenders across two different models, the full Tree Search and the Myopic model. We found that the Tree Search principal variation is the best predictor of gaze, with the second-best being the Tree Search predicted move, the same ordering found with the NLL (Fig. B.2).

B.3.3 Gaze Regression R^2

Chapter 3 reports the gaze regression (§3.4.3, Eqs. 3.5–3.6) as a paired ΔR^2 difference between models, since this is what isolates the Tree Search advantage from the Myopic baseline. However, this obscures the variance explained by the baseline model and other models in an absolute sense. For reference, we report here the corresponding raw held-out R^2 of each model’s full Composite regression, with and without the nuisance regressors (Fig. B.3). These are the same fits underlying the paired comparisons of the main text; the raw values make explicit how much of each model’s total explanatory power the paired difference represents, and how much variance is explained in absolute terms once the nuisance regressors are added.

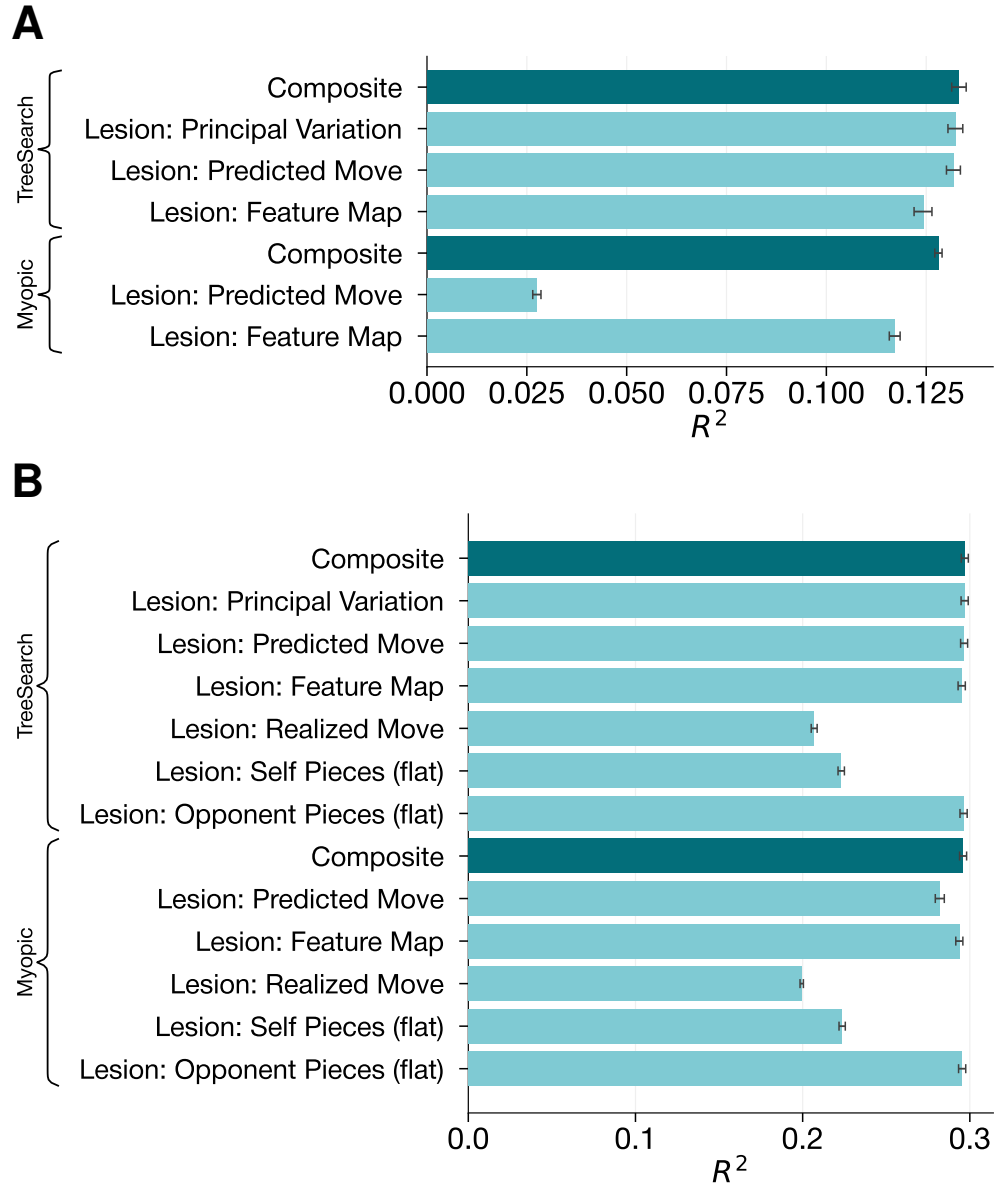


Figure B.3: **Raw gaze regression R^2** . Held-out R^2 of the full Composite regression for each model, (A) without and (B) with nuisance regressors (self pieces, opponent pieces, realized move). Compare with the paired ΔR^2 differences of Figs. 3.7–3.8 in the main text. Error bars represent bootstrap-derived 95% CI.

B.4 Neural

B.4.1 Neural Preprocessing

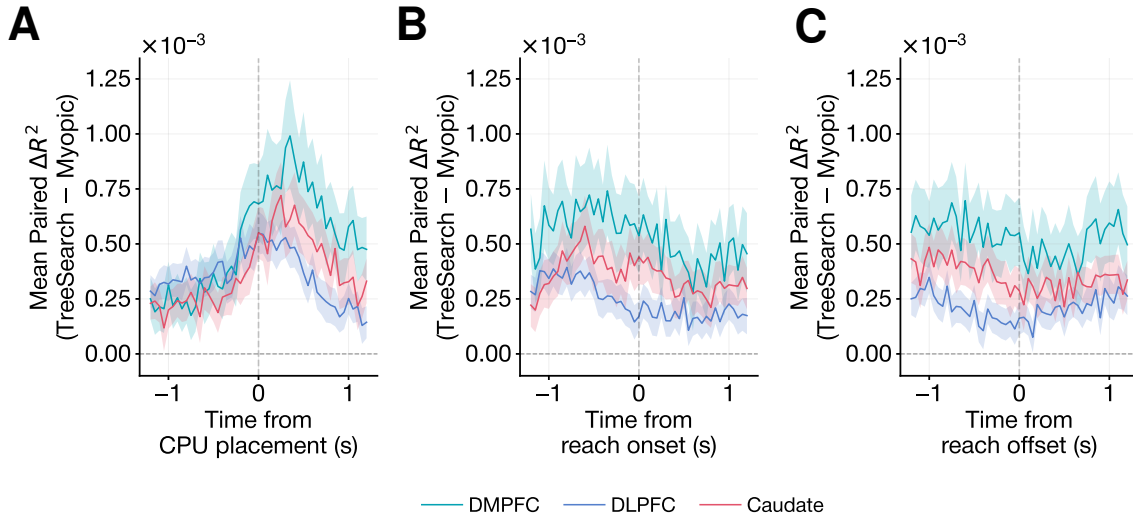


Figure B.4: **Time-resolved mean paired ΔR^2** . Same comparison as Fig. 3.12A, computed per 50 ms bin rather than aggregated over the turn, aligned to the computer move (A), reach onset (B), and reach offset (C). Shaded bands represent bootstrap-derived 95% CI; the dashed line marks the null of no difference.

For sessions with simultaneous recordings, the raw spike file lists every detected spike as a (time, neuron) pair, the product of upstream spike sorting (MountainSort5, with manual curation; Chapter 3). Each neuron was given a session-unique label, and its recording region was read directly from its identifier: the leading (thousands) digit encodes the area—1 for DMPFC, 2 for DLPFC, 3 for caudate, and 4 for hippocampus—of which the hippocampal units were set aside, leaving the three regions analyzed in Chapter 3.

To turn the spike stream into trial-aligned firing, we aligned to one of three task events—the computer’s move (`c_cpu_placement`), reach onset (`e_reach_onset`), or reach offset (`f_reach_offset`)—and counted each neuron’s spikes in consecutive 50 ms bins over a window of $[-1200, +1200]$ ms around that event (49 bins), assigning each spike to a bin by its time relative to the event. A trial enters the count only if its event time is valid and enough recording remains

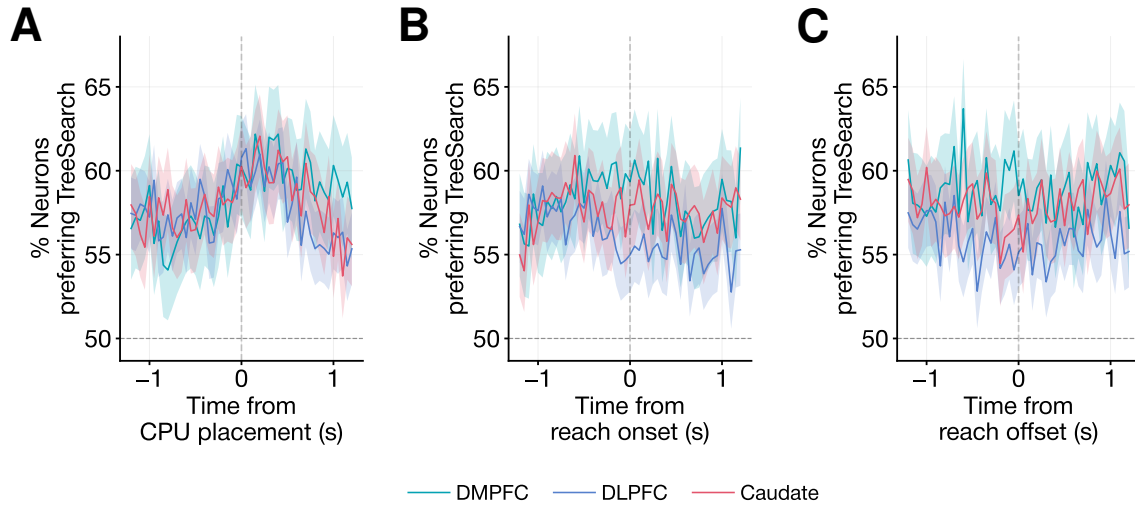


Figure B.5: **Time-resolved proportion of neurons preferring Tree Search.** Same comparison as Fig. 3.12B, computed per 50 ms bin, aligned as in Fig. B.4. The dashed line marks the null of 50%.

after the event to fill the window; trials failing either test are dropped. The result is a (trial $\times$ time bin $\times$ neuron) array of spike counts, one per session and event.

Each spike array was paired with the session’s behavioral table, which carried the reconstructed board (the computer’s and monkey’s pieces and the chosen move), all four task-event times, and per-trial flags for pseudotrials and computer-win trials. The games entering both tables are those surviving the single upstream ≥ 9 -ply game filter (§B.2.1), so the neural set shares its games with the behavioral and gaze analyses. Downstream, the encoding analyses of Chapter 3 left-merged the spikes onto the behavioral trials and dropped any trial with no spikes for that event before regressing, since the slope test propagates missing values; pseudotrials and trials with invalid go-cue or reach-offset codes were excluded.

B.4.2 Time-Resolved Confirmation of the Paired Model Comparison

Chapter 3’s paired model comparison (§3.4.4) is reported in aggregate, over the whole turn. Because the time-resolved analysis must exclude neurons that are flat within a given bin—where R^2 is definitionally undefined ($0/0$)—the set of contributing neurons varies across bins, and the magnitude of the effect cannot be reliably compared across time bins. However, the advantage

clears its null ($\Delta R^2 > 0$) and the proportion (preferring Tree Search > 0.5) at every bin examined, which we report as evidence that the effect is present throughout the deliberation window rather than as a temporal profile. As a robustness check, we computed this time-resolved version by indexing $\Delta R^2_{j,t}$ by 50 ms bin, aligned to the computer's move, reach onset, and reach offset, and bootstrapping per bin exactly as in the aggregate case: both statistics clear their 95% CI at every one of the 49 bins, in every region and at every alignment (Figs. B.4–B.5). This is consistent with, but does not go beyond, the aggregate result in the main text.

Appendix C | Learned Metacontrol of Planning at Scale

C.1 Data

Chapter 4’s Data subsection (§4.4) describes the population at the level a reader of the Results needs: 600+0 games, $Elo \geq 2000$, October–December 2023, with a short list of exclusions. Here we give the filtering pipeline that produces that population, stage by stage, together with the row counts at each stage, in the same spirit as §B.2.1’s dataset table (Table B.4).

Raw games are read from an attached Lichess archive and restricted to games with `initial_clock = 600`, `clock_increment = 0`, and both players’ $Elo \geq 2000$, within the October–December 2023 window. Three further exclusions are then applied at the level of whole games, not individual moves, so that a flagged game is dropped in its entirety rather than trimmed: (1) any move with a negative response time (server-lag artifacts); (2) berserking, detected from the clock at plies 3–4 sitting at essentially half the initial allotment, the signature of a player forfeiting time for a bigger payoff in tournament games; and (3) time-odds games, detected from a clock reading that *rises* from one of a player’s moves to the next under a zero-increment time control, which is only possible if the opponent granted extra time. This is the stage Chapter 4 reports, consisting of 1,971,698 games and 145,142,731 moves.

Moves logged at exactly zero seconds are then dropped (135,482,903 moves survive), since Lichess records response time only to integer-second resolution and a zero-second entry is a pre-move or a response too fast to register any deliberation at all; retaining them would distort the low tail of the log-response-time distribution used throughout the chapter. Finally, restricting to the ply-15–75 window used for every subsequent analysis (§4.4; chosen to exclude opening

memorization and endgame simplification) leaves 89,218,280 moves, the population underlying every board-feature and response-time figure in Chapter 4’s Results. Table C.1 summarizes these stages.

Stage	Filter	Games	Moves
Quality-filtered	600+0, Elo $\geq$ 2000, Oct–Dec 2023; excl. negative RT, berserk, time-odds	1,971,698	145,142,731
Non-zero RT	drop 0-second moves	—	135,482,903
Ply window	restrict to plies 15–75	—	89,218,280

Table C.1: **Chess dataset summary.** Summary of data at each filtering stage. “Moves” counts one player’s response to a single board position, pooled across both colors. Game counts are not tracked once filtering operates at the move level (—); the ply-15–75 window is the analysis population for every board-feature and response-time result in Chapter 4.

Board-structural features (legal moves, captures, checks) are computed once per distinct FEN over this ply-windowed population (84,034,834 distinct FENs over the retained 89 million moves). The following section defines each feature and where it is computed or stored.

C.2 Empirical Analyses

This section documents the variables behind Chapter 4’s model-agnostic response-time analyses: what each one means, how it is computed, and where it lives. We also resolve an apparent non-monotonicity flagged in the chapter’s Results (§C.2.2).

Several figures in Chapter 4 split the aggregate trend by game stage into ply tertiles (Figs. 4.2, 4.3, 4.4, 4.5, 4.6). The ply tertile cutpoints (ply 32 and 49) are the tertiles of the ply distribution computed once, directly over the ply-15–75-windowed population of §C.1, and then fixed as constants and reused identically across every figure, rather than recomputed per analysis; this ensures a given ply always falls in the same tertile regardless of which board feature is being plotted alongside it or how it might be additionally filtered.

C.2.1 Board and Response-Time Variables

Every variable below is defined over the retained ply-15–75 dataset described in §C.1:

- **Response time (RT).** The player-to-move’s time to respond, in seconds, as measured by Lichess; the dependent variable throughout. Lichess only records integer-valued response times. Because raw RTs are heavy-tailed and approximately log-normal, all analyses use log-RT, with axes labeled in seconds on a log scale.
- **Ply.** The move index within the game, stored directly on each move row.
- **Player clock time.** The player-to-move’s remaining clock time in seconds at the moment of the move, stored directly.
- **Self material / material imbalance.** Self material is the weighted piece count for the player to move (pawn = 1, knight = bishop = 3, rook = 5, queen = 9, king excluded); material imbalance is self material minus the opponent’s, so positive values mean the player to move is ahead. Both are parsed from the move’s board-position string at the same filtering stage that builds the ply-windowed table (§C.1).
- **Legal moves.** The number of legal moves available to the player to move.
- **Captures available / checks available.** The number of legal moves that are captures, and the number that give check, respectively, in the current position. Unlike the previous features, these are not attached to the raw move record; they are computed once per distinct FEN with `python-chess` (iterating the position’s legal moves and testing each with `is_capture` / `gives_check`) and stored in a features table keyed by FEN, then joined back onto the ply-windowed move table by FEN string (§C.1).
- **In check.** A boolean flag for whether the player to move is currently in check, stored directly per move (independently re-validated against a fresh `python-chess is_check()` pass over a held-out sample). Used as an exclusion filter in the following section.

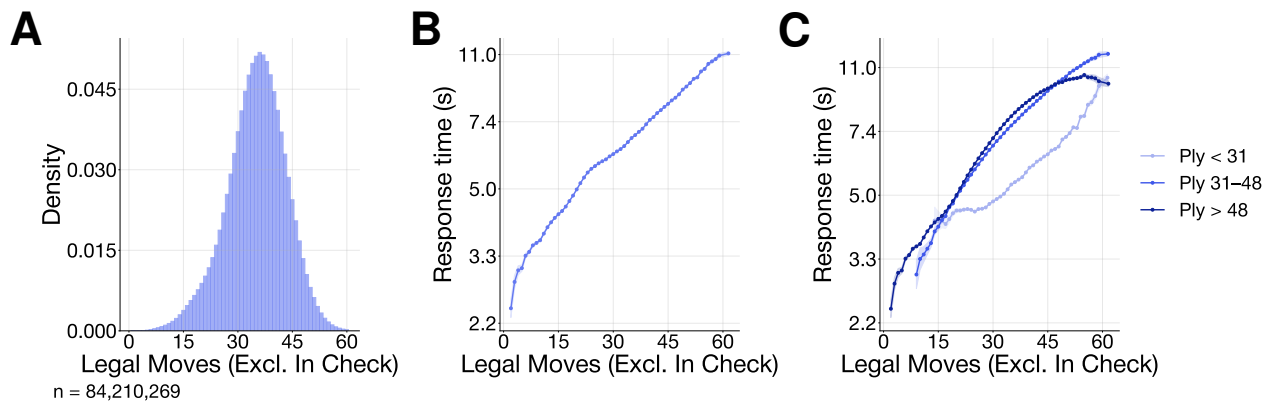


Figure C.1: **Legal moves vs. response time, excluding in-check positions.** Same layout as Fig. 4.5, restricted to moves where the player to move is not in check. RT is plotted on a log-transformed axis. (A) Histogram of legal moves. (B) Legal moves plotted against response time (mean $\pm$ SEM). (C) Same as (B) but separated by ply tertiles. The 1–9-move bump present in the pooled data (Fig. 4.5) is absent once in-check positions are excluded.

C.2.2 Legal Moves: In-Check Positions

Chapter 4 notes a consistent bump in response time at 1–9 legal moves, with the resulting dip afterwards breaking the otherwise monotonic rise of RT with legal moves (§4.4). This bump is attributable to positions where the player is in check. Being in check severely restricts the legal moves available, since only capturing, blocking, or moving the king out of check is permitted, so in-check positions are heavily overrepresented at low legal-move counts; at the same time, in-check moves are often answered *more slowly* than non-check moves with the same number of legal options, at every legal-move count, presumably because escaping check is a forced but still deliberated decision rather than a reflex. Pooling in-check and non-check positions therefore mixes a slow-responding, low-legal-move subpopulation into precisely the range (1–9 legal moves) where that subpopulation’s share happens to fall off sharply, producing the dip in the pooled curve. Excluding in-check positions removes the dip entirely, leaving a clean monotonic relationship between legal moves and response time (Fig. C.1); the dip is therefore an artifact of mixing two subpopulations with different response-time levels, rather than a discontinuity in the underlying relationship between legal moves and response time.

C.3 Metacontroller

The controller of Chapter 4 answers a question the planner itself never asks: not *which* move is best, but whether it is worth thinking any harder to find out. Search is useful precisely because it can change one’s mind, yet each expansion costs time, and at some point the marginal position is no longer worth the marginal computation. This section specifies how the controller makes that halt-or-continue judgment: what the planner it inspects actually does on each step (§C.3.1), how it reads the shape of an ongoing search (§C.3.2), what target it is trained to predict (§C.3.3), and how the two are learned (§C.3.4).

We mirror the notation in main. At expansion step t the planner holds a search tree T_t rooted at the real-world position $S_t = \text{Root}(T_t)$; N is the maximum number of expansions, and T_N is the completed teacher tree used as ground truth once a search has run to completion. Each root child a has a backed-up value $Q(a, T)$, and the greedy move at step t is $a_t^* = \arg \max_a Q(a, T_t)$. The metacontroller inspects T_t and emits a scalar $\hat{A}_t$, positive favoring continuation; it halts at the first step where $\hat{A}_t \leq 0$. §C.3.3 defines the target this scalar is trained toward.

C.3.1 The Planner

The planner is a standard PUCT search (Rosin, 2011). For a given partial tree, it repeatedly selects a child until it reaches a leaf. At every already-expanded node it selects the child maximizing the score

$$\text{score}(v) = Q_v + c_{\text{puct}} P_v \frac{\sqrt{N_u}}{1 + N_v},$$

its own backed-up value Q_v plus an exploration term weighted by its move prior P_v from Leela’s policy head and its visit count N_v relative to the parent’s, N_u ($c_{\text{puct}} = 1$ throughout; an unvisited child starts at $Q_v = 0$), until it reaches a leaf.

If that node is terminal or at the maximum depth, its value is simply re-threaded back to the root and the step is free (these do not count as an expansion). Otherwise, the node is expanded;

every legal move at that node is attached as a child of that node. Move priors for all of a node's children come from a single shared policy query at the parent; the engine (Leela) then evaluates each child's value and win-draw-loss (WDL). Then, values are propagated back to the root, flipping sign at every ply on the way to the root (Algorithm 15).

The consequence worth flagging explicitly: one expansion attaches as many new children, and issues as many per-child value queries, as the position has legal moves, even though it advances the expansion count t by exactly one. A tree's total size therefore grows far faster than the number of expansions it took to build it. In our current formulation, no pruning is applied. Below we describe a single PUCT search step.

Algorithm 15 `policy.STEP( $T$ )` – one PUCT search step

Require: tree T ; PRIORS (one shared query per node; returns move priors $\{P_m\}$); EVAL (one query per legal move; returns a value and a WDL vector); exploration constant c_{puct} ; visit counts $\{N_u\}$

- 1: $u \leftarrow \text{Root}(T)$; $\text{path} \leftarrow \emptyset$
- 2: **while** u has children in T **do** ▷ descend to an unexpanded leaf
- 3: $v^* \leftarrow \arg \max_{v \in \text{Children}(u)} \text{score}(v)$ ▷ $Q_v = 0$ for unvisited children
- 4: $\text{path} \leftarrow \text{path} \parallel (u, v^*)$; $u \leftarrow v^*$
- 5: **end while**
- 6: **if** u is terminal **or** $\text{depth}(u) \geq d_{\text{max}}$ **then**
- 7: `BACKPROPAGATE`(path, V_u) ▷ no new evaluation; free step, does not count as an expansion
- 8: **return** T
- 9: **end if**
- 10: $\{P_m\} \leftarrow \text{PRIORS}(u)$ ▷ one shared query for every legal move at u
- 11: **for** each legal move m at u **do**
- 12: $(V_{u \cdot m}, \omega_{u \cdot m}) \leftarrow \text{EVAL}(u \cdot m)$ ▷ one query per child
- 13: attach child $u \cdot m$ to u with value $V_{u \cdot m}$, prior $P_{u \cdot m}$, and $N_{u \cdot m} \leftarrow 0$ (a new, still-childless leaf)
- 14: **end for**
- 15: `BACKPROPAGATE`(path, V_u) ▷ flip sign at every ply back to the root; increment N_u along the path
- 16: **return** T ▷ counts as one expansion; adds $|\text{legal}(u)|$ leaves

C.3.2 Tree Encoder

The controller ingests a partial tree T_t at each step and decides to continue planning or halt. The design choice that distinguishes this from earlier metacontrollers is that this representation is built from the *topology* of the tree rather than a flat list of summary statistics: the encoder is given only what the engine reports about each node and must infer everything search-relevant from how the nodes are connected and evaluated. The architecture is a TreeBiGRU (Kokkinos and Potamianos, 2017), extending the bottom-up Tree-LSTM (Tai et al., 2015) with a symmetric top-down sweep and a GRU in place of the LSTM cell.

Concretely, each node u carries a small feature tuple,

$$\mathbf{x}_u^{(t)} = (V_u^{(t)}, \omega_u^{(t)}, \text{var}_{\omega,u}^{(t)}),$$

the node’s scalar value at time t , its WDL distribution, and a variance summarizing the engine’s uncertainty, each as currently known at expansion step t . Because these quantities are revised by backpropagation every time a descendant of u is expanded (§C.3.1), the same node encoded at two different steps generally carries two different feature tuples.

A small MLP (one hidden layer, width 32) embeds each tuple to an initial state $h_u^{(0)} \in \mathbb{R}^{32}$, and the tree is then encoded by message passing (Algorithm 16). Because a tree is acyclic, information can be propagated exactly in two sweeps (Kokkinos and Potamianos, 2017): an upward sweep from leaves to root lets every node summarize its entire subtree, and a downward sweep from root to leaves lets every node see its context within the whole search. Running each sweep in topological order means a single round ($L = 1$) already gives every node a tree-wide receptive field, the same guarantee that lets belief propagation resolve exactly on a tree in one pass (Pearl, 1982), with no need for the many rounds a general graph would require.

In the upward sweep, a parent attends over its children with two-head attention (16 dimensions per head, a 32-dimensional message), with each child tagged by a slot encoding $\phi(s_{u,v})$ (children ordered by their UCI move) so the parent can tell them apart; in the downward sweep

each node simply receives a projection of its parent’s state. A shared GRU folds each incoming message into the node’s running state (Algorithm 16):

Algorithm 16 Tree encoder – round $\ell \rightarrow \ell+1$: leaves-to-root then root-to-leaves message passing

Require: tree T_t ; node features $\{\mathbf{x}_u^{(t)}\}$ (only needed at $\ell = 0$); child slot indices $\{s_{u,v}\}$

```

1: if  $\ell = 0$  then
2:    $h_u^{(0)} \leftarrow \text{EMBED}(\mathbf{x}_u^{(t)})$  for every node  $u$ 
3: end if
4: for depth  $d = d_{\max}$  down to 0 do                                 $\triangleright$  upward: summarize each subtree
5:   for node  $u$  at depth  $d$  with children  $C(u)$  do
6:      $\psi_{u,v}^{(\ell)} \leftarrow h_v^{(\ell)} + \phi(s_{u,v})$  for  $v \in C(u)$            $\triangleright$  tag each child by its slot
7:      $m_u^{(\ell)} \leftarrow \text{MHA}(\mathbf{q}=W_q h_u^{(\ell)}, \{\mathbf{k}=W_k \psi_{u,v}^{(\ell)}, \mathbf{v}=W_v \psi_{u,v}^{(\ell)}\}_v)$ 
8:      $h'_u^{(\ell)} \leftarrow \text{GRU}(m_u^{(\ell)}, h_u^{(\ell)})$ 
9:   end for
10: end for
11: for depth  $d = 0$  to  $d_{\max}$  do                                 $\triangleright$  downward: give each node its context
12:   for node  $v$  at depth  $d$  with parent  $u$  do
13:      $h_v^{(\ell+1)} \leftarrow \text{GRU}(W_{\text{down}} h'_u^{(\ell)}, h'_v^{(\ell)})$ 
14:   end for
15: end for
16: return node states  $\{h_u^{(\ell+1)}\}$ ; after the final round ( $\ell+1 = L$ ), root embedding  $z_t = h_r^{(L)}$ 

```

The root embedding $z_t = h_r^{(L)}$ is the controller’s read of the whole search. A small MLP head (2 hidden layers, width 32) uses this embedding to infer the advantage, together with one scalar side input, the number of expansions taken so far, t :

$$\hat{A}_t = \text{MLP}(z_t, t).$$

During controller training the encoder is frozen and its outputs cached, so only the readout head is updated.

C.3.3 Halt/Continue Values

To train the head we need a target: for each step of a real search, was halting or continuing actually the better choice? The difficulty is that the answer depends on the future of the search,

which the controller cannot see at decision time. We resolve this in hindsight, using the completed teacher tree T_N as ground truth.

The value of halting at step t is not how good the chosen move *looked* under the partial tree T_t , but how good it *turns out* to be once the full search has spoken: it is scored against the final tree T_N , not T_t . The value of continuing is the better of stopping one step later or carrying on optimally, minus the cost of the additional planning step:

$$V_{\text{halt}}(T_t) = Q(a_t^*, T_N), \quad (\text{C.1})$$

$$V_{\text{cont}}(T_t) = V^*(T_{t+1}) - c_t, \quad V^*(T_t) = \max(V_{\text{halt}}(T_t), V_{\text{cont}}(T_t)), \quad (\text{C.2})$$

$$A_t = V_{\text{cont}}(T_t) - V_{\text{halt}}(T_t). \quad (\text{C.3})$$

Because $V_{\text{cont}}(T_t)$ depends on $V^*(T_{t+1})$, and so on to the end of the search, this recursion can only be resolved by working backward from the last recorded step. This can be calculated via dynamic programming, which produces the oracle advantage A_t against which the readout head is trained (Algorithm 17; §C.3.4 gives the training objective built on top of it). Ties are broken in favor of halting.

Algorithm 17 HALTCONTINUEORACLE — backward induction for the readout target

Require: halt rewards $r_i = Q(a_i^*, T_N)$; expansion limit N ; cost function C (§C.3.3)

```

1:  $M \leftarrow \min(\text{len}(r), N)$ ;  $\rho_i \leftarrow N - i$  ▷ expansions remaining at step  $i$ 
2: for  $i = M - 1$  down to 0 do
3:   if  $\rho_i \leq \delta_t$  then ▷ too little budget left to finish a step; forced to stop
4:      $v_{\text{next}} \leftarrow V_{\text{timeout}}$ 
5:   else if  $i + 1 < M$  then
6:      $v_{\text{next}} \leftarrow V_{i+1}^*$  ▷ carry the recursion backward
7:   else
8:      $v_{\text{next}} \leftarrow r_i$  ▷ end of the recorded trace
9:   end if
10:   $V_i^{\text{cont}} \leftarrow v_{\text{next}} - C_i$ ;  $A_i \leftarrow V_i^{\text{cont}} - r_i$  ▷ the target:  $A_i > 0 \Rightarrow$  keep thinking
11:   $V_i^* \leftarrow r_i$  if  $r_i \geq V_i^{\text{cont}}$  else  $V_i^{\text{cont}}$  ▷ ties go to halting
12: end for
13: return advantages  $\{A_i\}$ , optimal values  $\{V_i^*\}$ , per-step halt/continue actions
```

The runs reported here use a single fixed expansion limit, $N = 96$, matching the full teacher tree.

Cost Function

A critical component of resource-rational planning is the cost of additional thinking. Here we define the per-expansion cost c_t , which is what makes stopping ever worthwhile. In the runs reported here this cost is constant per step (linear in time), $c_t \equiv \lambda = 0.005$ for each step, equivalently $C_{time} = \lambda t$.

We also explore a richer, two-part cost:

$$C(n, \rho) = C_{\text{maint}}(n) + C_{\text{time}}(\rho),$$

where C_{maint} charges for the size of the tree itself and C_{time} charges for the time taken; the reported regime disables the first term entirely and reduces the second to the constant λ above. Once expansions are exhausted, the value of continuing is instead assigned a fixed timeout value, $V_{\text{timeout}} = -1$.

C.3.4 Training

Training proceeds in two stages, mirroring the two things the controller must learn: first to read a search tree, then to decide when to stop. Both stages draw on the same root-FEN reservoir sample described in Chapter 4 (§C.3.1 gives the tree-generation procedure that turns each FEN into a teacher tree); trees are split 80/20 into training and validation before packing, and packed into shards (2,000 trees per shard for the encoder’s edge-level table, 500 trees per shard for the controller’s episode table).

Table C.2 continues the funnel of Table C.1 from the same quality-filtered corpus (§C.1): distinct board positions are extracted from it as root FENs and used as tree-generation roots. A seeded random subsample of these root FENs is first restricted by an early-discovery exclusion,

which removes trees that churn and revisit an earlier candidate, since a best move that is found, abandoned, and only later re-validated is a poor signal to train an agent to learn when search is worthwhile. Survivors are then restricted to root FENs that also fall in the ply-15–75 window used throughout Chapter 4 (§C.1). The 80/20 training/validation split is drawn by a seeded random shuffle by FEN.

Stage	Filter	Root FENs
Quality-filtered	same corpus as Table C.1	—
Subsampled	seeded random subsample of root FENs	200,000
Early-discovery exclusion	drop churning best-move traces	132,062
Ply window	restrict to plies 15–75	80,011
Training split	80% of the ply-windowed sample, by FEN	64,009
Validation split	20% of the ply-windowed sample, by FEN	16,002

Table C.2: **Metacontroller training-data funnel.** Root-FEN funnel for metacontroller training. Each row restricts the row above it; “Root FENs” counts distinct teacher trees, one per root position. The top row is the same quality-filtered corpus as Table C.1 (1,971,698 games, 145,142,731 moves; counts omitted here to avoid mixing units with the FEN-level rows below). Training and validation splits are disjoint partitions of the ply-windowed sample ($64,009 + 16,002 = 80,011$) drawn by a seeded random shuffle by FEN.

Encoder pretraining. The encoder is pretrained on a self-contained task that forces it to internalize search structure, supervised on partial trees rather than the fully completed tree T_N : for each tree we sample a snapshot step $t \sim U[1, N - k]$ and fix a lookahead length $k = 12$.

From the parent’s step- t encoded state and the slot embedding of one of its children, a small decoder (one hidden layer, width 32) must predict that child’s WDL distribution k expansions later, under T_{t+k} :

$$\hat{\omega}_{u,v} = \text{softmax}(\text{MLP}_{\text{dec}}([h_u^{(L)}; \phi(s_{u,v})])), \quad (\text{C.4})$$

$$\mathcal{L}_{u,v} = \text{CE}(\hat{\omega}_{u,v}, \bar{\omega}_{u,v}), \quad (\text{C.5})$$

the per-edge cross-entropy loss against the teacher target $\bar{\omega}_{u,v}$, a visit-weighted consolidation of

v 's backpropagated WDL over the window $[t, t + k]$ (not a single node's raw evaluation, and not the fully-searched tree's final value).

We minimize the average of $\mathcal{L}_{u,v}$ over all edges in T_t , weighted by each edge's change in visit count $\Delta N_{u,v}$ over $[t, t + k]$, so that edges actually refined by a backup in the lookahead window contribute more to the loss than edges whose subtree merely grew larger without being revisited. We train for 6 epochs with Adam (learning rate 10^{-3}) and keep the checkpoint with the lowest validation cross-entropy loss.

Metacontroller training. With the encoder frozen, we fit the advantage head against the oracle advantage of §C.3.3 by policy-gradient reinforcement learning: the head parameterizes a stochastic halt/continue policy, and training ascends the gradient of that policy's expected return. Two quantities from the oracle recursion enter here, and they play different roles despite both being built from $V_{\text{halt}}(T_t)$: $\hat{A}_t$ is a marginal, one-step comparison (continue vs. halt *from here*), so the cost already spent reaching T_t cancels out of it; the episode-level payoff $g(t) \equiv V_{\text{halt}}(T_t) - \sum_{s < t} c_s$ does depend on that cost, since it scores the whole episode's return under the counterfactual of stopping exactly at t . $\hat{A}_t$ decides whether to keep going; $g(t)$ scores what stopping at t would actually pay out.

Because $\hat{A}_t$ must be trained by gradient descent, it cannot threshold directly into a hard halt/continue choice, which has no gradient to learn from; instead it is passed through a sigmoid to give a continue probability, $p_t = \sigma(\hat{A}_t)$, and these per-step probabilities induce a distribution over the stopping step. Because the oracle already specifies $g(t)$ for every candidate stopping step, the expected return is just the probability-weighted average of $g(t)$ over stopping times, and this stopping-time distribution can be directly computed over the recorded trace without sampling, rather than estimated from sampled rollouts as in REINFORCE:

$$\mathbb{E}[\text{return}] = \sum_t P(a_t = \text{halt}) g(t), \quad P(a_t = \text{halt}) = \left(\prod_{s < t} p_s \right) (1 - p_t).$$

Training minimizes the resulting expected regret against the hindsight-optimal return $V^*(T_0)$,

for 200 epochs with Adam (learning rate 10^{-3}) at a batch size of 512 episodes per gradient step. Model selection is done on validation regret under the deployed hard greedy rule ($\hat{A}_t \leq 0$ halts)—the same criterion the training loss approximates, but evaluated on held-out data.

C.3.5 Baseline Controllers

Chapter 4’s baseline ladder (§4.4.4) compares the trained metacontroller against five halt/continue rules, varying in sophistication: Always Stop, Always Continue, Fixed Stop, Tree Stats, and Action Gap. All six controllers, including the metacontroller itself, share the same general inference loop (Algorithm 18): at each step the controller reads some function of the partial tree T_t and decides whether to halt or request one more expansion from the planner (§C.3.1); they differ only in what they read from T_t and how they map it to a decision.

Algorithm 18 Halt/continue inference loop, generic over the controller decision rule **DECIDE**

Require: root position S_0 ; expansion limit N ; controller decision rule **DECIDE**(T_t)

```

1:  $T_0 \leftarrow$  single-node tree rooted at  $S_0$ ;  $t \leftarrow 0$ 
2: while  $t < N$  do
3:   if DECIDE( $T_t$ ) = HALT then
4:     break
5:   end if
6:    $T_{t+1} \leftarrow$  policy.STEP( $T_t$ ) ▷ one planner expansion (Algorithm 15)
7:    $t \leftarrow t + 1$ 
8: end while
9: return  $a_t^* = \arg \max_a Q(a, T_t)$  ▷ commit to the greedy root move
```

Fixed Stop’s threshold κ^* is fit once on the training set by grid search over regret (§4.4.4) and then applied as a constant to every held-out episode; Tree Stats and Action Gap are learned read-out heads with the same MLP architecture as the metacontroller’s advantage head, but reading hand-crafted features in place of the tree encoder’s embedding z_t . All six controllers are evaluated under the same hard greedy stopping rule at inference. Table C.3 distinguishes the decision head for each of the controllers.

Controller	Reads from T_t	Decide(T_t)
Always Stop	—	Halt at $t = 0$
Always Continue	—	Continue until $t = N$
Fixed Stop	t	Halt iff $t > \kappa^\star$
Tree Stats	(t, n_t, h_t, w_t)	Halt iff $\text{MLP}_{\text{stats}}(t, n_t, h_t, w_t) \leq 0$
Action Gap	(t, Δ_t)	Halt iff $\text{MLP}_{\text{gap}}(t, \Delta_t) \leq 0$
Metacontroller	$z_t = h_r^{(L)}$ (§C.3.2)	Halt iff $\hat{A}_t = \text{MLP}(z_t, t) \leq 0$

Table C.3: **Baseline controller decision rules.** All six controllers share the inference loop of Algorithm 18; n_t, h_t, w_t are the node count, max depth, and max width of T_t , and $\Delta_t = Q(a_t^\star, T_t) - Q(a_t^{(2)}, T_t)$ is the value gap between the current best and second-best root moves. Tree Stats and Action Gap are learned readout heads reading hand-crafted features in place of the tree encoder’s embedding z_t ; Fixed Stop’s $\kappa^\star$ is fit by grid search on the training set (§4.4.4).

References

- Abe, H. and Lee, D. (2011). Distributed Coding of Actual and Hypothetical Outcomes in the Orbital and Dorsolateral Prefrontal Cortex. *Neuron*, 70(4):731–741.
- Acerbi, L. and Ma, W. J. (2017). Practical Bayesian Optimization for Model Fitting with Bayesian Adaptive Direct Search. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.
- Aiton, E. and Shimao, E. (1981). Gorai Kinzō’s study of Leibniz and the I ching hexagrams. *Annals of Science*, 38(1):71–92.
- Akam, T., Costa, R., and Dayan, P. (2015). Simple Plans or Sophisticated Habits? State, Transition and Learning Interactions in the Two-Step Task. *PLOS Computational Biology*, 11(12):e1004648.
- Allais, M. (1953). Le comportement de l’homme rationnel devant le risque: Critique des postulats et axiomes de l’école Americaine. [Rational man’s behavior in the presence of risk: Critique of the postulates and axioms of the American school.]. *Econometrica*, 21:503–546.
- Allen, K., Brändle, F., Botvinick, M., Fan, J. E., Gershman, S. J., Gopnik, A., Griffiths, T. L., Hartshorne, J. K., Hauser, T. U., Ho, M. K., de Leeuw, J. R., Ma, W. J., Murayama, K., Nelson, J. D., van Opheusden, B., Pouncy, T., Rafner, J., Rahwan, I., Rutledge, R. B., Sherson, J., Şimşek, Ö., Spiers, H., Summerfield, C., Thalmann, M., Vélez, N., Watrous, A. J., Tenenbaum, J. B., and Schulz, E. (2024). Using games to understand the mind. *Nature Human Behaviour*, 8(6):1035–1043.
- Allis, L. (1994). *Searching for Solutions in Games and Artificial Intelligence*. PhD thesis.
- Anderson, J. R. (1990). *The Adaptive Character of Thought*. Psychology Press.

- Angelova, D. and Käbel, J. (2019). Weather Volatility and Production Efficiency. *Sustainability*, 11(24):6970.
- Auer, P., Cesa-Bianchi, N., and Fischer, P. (2002). Finite-time Analysis of the Multiarmed Bandit Problem. *Machine Learning*, 47(2):235–256.
- Averbeck, B. B., Sohn, J.-W., and Lee, D. (2006). Activity in prefrontal cortex during dynamic selection of action sequences. *Nature Neuroscience*, 9(2):276–282.
- Balleine, B. W. and Dickinson, A. (1998). Goal-directed instrumental action: Contingency and incentive learning and their cortical substrates. *Neuropharmacology*, 37(4):407–419.
- Barack, Bakkour, Shohamy, and Salzman (2023). Visuospatial information foraging describes search behavior in learning latent environmental features. *Scientific Reports*, 13(1):1126.
- Barr, D. J., Levy, R., Scheepers, C., and Tily, H. J. (2013). Random effects structure for confirmatory hypothesis testing: Keep it maximal. *Journal of Memory and Language*, 68(3):255–278.
- Barracough, D. J., Conroy, M. L., and Lee, D. (2004). Prefrontal cortex and decision making in a mixed-strategy game. *Nature Neuroscience*, 7(4):404–410.
- Bartolo, R. and Averbeck, B. B. (2020). Prefrontal Cortex Predicts State Switches during Reversal Learning. *Neuron*, 106(6):1044–1054.e4.
- Bascom, W. R. (1991). *Ifa Divination: Communication Between Gods and Men in West Africa*. Indiana University Press.
- Bates, D., Mächler, M., Bolker, B., and Walker, S. (2015). Fitting Linear Mixed-Effects Models Using lme4. *Journal of Statistical Software*, 67:1–48.
- Behrens, T. E. J., Woolrich, M. W., Walton, M. E., and Rushworth, M. F. S. (2007). Learning the value of information in an uncertain world. *Nature Neuroscience*, 10(9):1214–1221.
- Bellman, R. (1957). *Dynamic Programming*. Princeton University Press, Princeton, NJ.

- Bellman, R. (1958). On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90.
- Bhui, R., Lai, L., and Gershman, S. J. (2021). Resource-rational decision making. *Current Opinion in Behavioral Sciences*, 41:15–21.
- Black, F. and Scholes, M. (1973). The Pricing of Options and Corporate Liabilities. *Journal of Political Economy*, 81(3):637–654.
- Bland, A. and Schaefer, A. (2012). Different Varieties of Uncertainty in Human Decision-Making. *Frontiers in Neuroscience*, 6.
- Botvinick, M. M., Niv, Y., and Barto, A. G. (2009). Hierarchically organized behavior and its neural foundations: A reinforcement learning perspective. *Cognition*, 113(3):262–280.
- Byrd, R. H., Lu, P., Nocedal, J., and Zhu, C. (1995). A Limited Memory Algorithm for Bound Constrained Optimization. *SIAM Journal on Scientific Computing*, 16(5):1190–1208.
- Cai, X., Kim, S., and Lee, D. (2011). Heterogeneous coding of temporally discounted values in the dorsal and ventral striatum during intertemporal choice. *Neuron*, 69(1):170–182.
- Callaway, F., Lieder, F., Das, P., Gul, S., Krueger, P. M., and Griffiths, T. L. (2018). A resource-rational analysis of human planning. In *Proceedings of the 40th Annual Meeting of the Cognitive Science Society, CogSci 2018*, pages 178–183. The Cognitive Science Society.
- Callaway, F., Lieder, F., Krueger, P., and Griffiths, T. (2017). Mouselab-MDP: A new paradigm for tracing how people plan.
- Callaway, F., Rangel, A., and Griffiths, T. L. (2021). Fixation patterns in simple choice reflect optimal information sampling. *PLOS Computational Biology*, 17(3):e1008863.
- Callaway, F., van Opheusden, B., Gul, S., Das, P., Krueger, P. M., Griffiths, T. L., and Lieder, F. (2022). Rational use of cognitive resources in human planning. *Nature Human Behaviour*, 6(8):1112–1125.

- Campbell, M., Hoane, A. J., and Hsu, F.-h. (2002). Deep Blue. *Artificial Intelligence*, 134(1):57–83.
- Campbell, S. D. and Diebold, F. X. (2005). Weather Forecasting for Weather Derivatives. *Journal of the American Statistical Association*, 100(469):6–16.
- Campitelli, G. and Gobet, F. (2004). Adaptive Expert Decision Making: Skilled Chess Players Search More and Deeper. *ICGA Journal*, 27(4):209–216.
- Charness, N., Reingold, E. M., Pomplun, M., and Stampe, D. M. (2001). The perceptual aspect of skilled performance in chess: Evidence from eye movements. *Memory & Cognition*, 29(8):1146–1152.
- Chase, W. G. and Simon, H. A. (1973). Perception in chess. *Cognitive Psychology*, 4(1):55–81.
- Chater, N. and Oaksford, M. (2000). The Rational Analysis of Mind and Behavior. *Synthese*, 122(1/2):93–131.
- Chevalier, N., Meaney, J. A., Traut, H. J., and Munakata, Y. (2020). Adaptiveness in proactive control engagement in children and adults. *Developmental Cognitive Neuroscience*, 46:100870.
- Chinchalkar, S. (1996). An Upper Bound for the Number of Reachable Positions. *ICGA Journal*, 19(3):181–183.
- Chomsky, N. (1959). On certain formal properties of grammars. *Information and Control*, 2(2):137–167.
- Chomsky, N. (1969). *Aspects of the Theory of Syntax*. MIT Press, Cambridge, MA, USA.
- Chung, S., Anokhin, I., and Krueger, D. (2023). Thinker: Learning to Plan and Act. *Advances in Neural Information Processing Systems*, 36:22896–22933.
- Connors, M. H., Burns, B. D., and Campitelli, G. (2011). Expertise in complex decision making: The role of search in chess 70 years after de Groot. *Cognitive Science*, 35(8):1567–1579.

- Costales, R., Iqbal, S., and Sha, F. (2022). Possibility Before Utility: Learning And Using Hierarchical Affordances.
- Coulom, R. (2007). Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search. In van den Herik, H. J., Ciancarini, P., and Donkers, H. H. L. M. J., editors, *Computers and Games*, pages 72–83, Berlin, Heidelberg. Springer.
- Craik, K. J. W. (1967). *The Nature of Explanation*, volume 445. CUP Archive.
- Daly, K. (2008). Financial volatility: Issues and measuring techniques. *Physica A: Statistical Mechanics and its Applications*, 387(11):2377–2393.
- Damani, M., Shenfeld, I., Peng, A., Bobu, A., and Andreas, J. (2024). Learning How Hard to Think: Input-Adaptive Allocation of LM Computation.
- Davis, E. P., Stout, S. A., Molet, J., Vegetabile, B., Glynn, L. M., Sandman, C. A., Heins, K., Stern, H., and Baram, T. Z. (2017). Exposure to unpredictable maternal sensory signals influences cognitive development across species. *Proceedings of the National Academy of Sciences of the United States of America*, 114(39):10390–10395.
- Daw, N. D., Gershman, S. J., Seymour, B., Dayan, P., and Dolan, R. J. (2011). Model-based influences on humans’ choices and striatal prediction errors. *Neuron*, 69(6):1204–1215.
- Daw, N. D., Niv, Y., and Dayan, P. (2005). Uncertainty-based competition between prefrontal and dorsolateral striatal systems for behavioral control. *Nature neuroscience*, 8(12):1704–1711.
- Daw, N. D., O’Doherty, J. P., Dayan, P., Seymour, B., and Dolan, R. J. (2006). Cortical substrates for exploratory decisions in humans. *Nature*, 441(7095):876–879.
- Dayan, P. (1993). Improving Generalization for Temporal Difference Learning: The Successor Representation. *Neural Computation*, 5(4):613–624.
- de Groot, A. D. (1946). *Het Denken van Den Schaker: Een Experimenteel-Psychologische Studie*. PhD thesis, University of Amsterdam, Amsterdam.

- de Groot, A. D. (2008). *Thought and Choice in Chess*. Amsterdam University Press.
- de Groot, A. D., Gobet, F., and Jongman, R. W. (1996). *Perception and Memory in Chess: Studies in the Heuristics of the Professional Eye*. Perception and Memory in Chess: Studies in the Heuristics of the Professional Eye. Van Gorcum & Co, Assen, Netherlands.
- De Wit, M., de Vries, S., Kamp, J., and Withagen, R. (2017). Affordances and neuroscience: Steps towards a successful marriage. *Neuroscience & Biobehavioral Reviews*, 80.
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271.
- Dolan, R. J. and Dayan, P. (2013). Goals and Habits in the Brain. *Neuron*, 80(2):312–325.
- Doom, J. R., Vanzomeren-Dohm, A. A., and Simpson, J. A. (2016). Early unpredictability predicts increased adolescent externalizing behaviors and substance use: A life history perspective. *Development and Psychopathology*, 28(4pt2):1505–1516.
- Drugowitsch, J., Moreno-Bote, R., Churchland, A. K., Shadlen, M. N., and Pouget, A. (2012). The Cost of Accumulating Evidence in Perceptual Decision Making. *Journal of Neuroscience*, 32(11):3612–3628.
- Dumbalska, T., Yuen, K., Bhatti, A., Ali, I., Zaneva, M., Sandbrink, K., and Summerfield, C. (2026). Curriculum learning for complex tasks.
- Eichenbaum, H. and Cohen, N. J. (1993). *Memory, Amnesia, and the Hippocampal System*. MIT Press, Cambridge, MA, USA.
- Ellis, B. J., Sheridan, M. A., Belsky, J., and McLaughlin, K. A. (2022). Why and how does early adversity influence development? Toward an integrated model of dimensions of environmental experience. *Development and Psychopathology*, 34(2):447–471.
- Eluchans, M., Lancia, G. L., Maselli, A., D'Alessandro, M., Gordon, J., and Pezzulo, G. (2024). Adaptive planning depth in human problem solving.

- Even, S. (2011). *Graph Algorithms*. Cambridge University Press.
- Eysenbach, B., Salakhutdinov, R., and Levine, S. (2021). Robust Predictable Control.
- Felso, V. and Lieder, F. (2022). Measuring individual differences in the depth of planning | Request PDF.
- Fenneman, J. and Frankenhuys, W. E. (2020). Is impulsive behavior adaptive in harsh and unpredictable environments? A formal model. *Evolution and Human Behavior*, 41(4):261–273.
- Fields, A., Bloom, P. A., VanTieghem, M., Harmon, C., Choy, T., Camacho, N. L., Gibson, L., Umbach, R., Heleniak, C., and Tottenham, N. (2021). Adaptation in the face of adversity: Decrements and enhancements in children’s cognitive control behavior following early caregiving instability. *Developmental Science*, 24(6):e13133.
- Ford Jr, L. R. (1956). Network flow theory. Technical report.
- Foster, D. J. and Wilson, M. A. (2006). Reverse replay of behavioural sequences in hippocampal place cells during the awake state. *Nature*, 440(7084):680–683.
- Frankenhuis, W. E., Panchanathan, K., and Nettle, D. (2016). Cognition in harsh and unpredictable environments. *Current Opinion in Psychology*, 7:76–80.
- Frömer, R., Lin, H., Dean Wolf, C. K., Inzlicht, M., and Shenhav, A. (2021). Expectations of reward and efficacy guide cognitive control allocation. *Nature Communications*, 12(1):1030.
- Gershman, S. J., Horvitz, E. J., and Tenenbaum, J. B. (2015). Computational rationality: A converging paradigm for intelligence in brains, minds, and machines. *Science*, 349(6245):273–278.
- Gershman, S. J. and Lak, A. (2025). Policy Complexity Suppresses Dopamine Responses. *The Journal of Neuroscience*, 45(9):e1756242024.
- Gibson, J. J. (1979). *The Ecological Approach to Visual Perception: Classic Edition*. Houghton Mifflin.
- Gigerenzer, G. (2008). Why Heuristics Work. *Perspectives on Psychological Science*, 3(1):20–29.

- Gigerenzer, G. and Goldstein, D. G. (1996). Reasoning the fast and frugal way: Models of bounded rationality. *Psychological Review*, 103(4):650–669.
- Gigerenzer, G. and Hoffrage, U. (1995). How to improve Bayesian reasoning without instruction: Frequency formats. *Psychological Review*, 102(4):684–704.
- Gillan, C. M., Kosinski, M., Whelan, R., Phelps, E. A., and Daw, N. D. (2016). Characterizing a psychiatric symptom dimension related to deficits in goal-directed control. <https://elifesciences.org/articles/11305>.
- Giroto, V., Legrenzi, P., and Rizzo, A. (1991). Event controllability in counterfactual thinking. *Acta Psychologica*, 78(1):111–133.
- Gläscher, J., Daw, N., Dayan, P., and O’Doherty, J. P. (2010). States versus rewards: Dissociable neural prediction error signals underlying model-based and model-free reinforcement learning. *Neuron*, 66(4):585–595.
- Glynn, L. M., Stern, H. S., Howland, M. A., Risbrough, V. B., Baker, D. G., Nievergelt, C. M., Baram, T. Z., and Davis, E. P. (2019). Measuring novel antecedents of mental illness: The Questionnaire of Unpredictability in Childhood. *Neuropsychopharmacology*, 44(5):876–882.
- Graves, A. (2017). Adaptive Computation Time for Recurrent Neural Networks.
- Hafting, T., Fyhn, M., Molden, S., Moser, M.-B., and Moser, E. I. (2005). Microstructure of a spatial map in the entorhinal cortex. *Nature*, 436(7052):801–806.
- Hampton, A. N., Bossaerts, P., and O’Doherty, J. P. (2006). The Role of the Ventromedial Prefrontal Cortex in Abstract State-Based Inference during Decision Making in Humans. *Journal of Neuroscience*, 26(32):8360–8367.
- Hamrick, J. B., Ballard, A. J., Pascanu, R., Vinyals, O., Heess, N., and Battaglia, P. W. (2017). Meta-control for Adaptive Imagination-Based Optimization.

- Harlow, J. M. (1869). *Recovery from the Passage of an Iron Bar Through the Head*. D. Clapp.
- Hart, P. E., Nilsson, N. J., and Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, 4(2):100–107.
- Haruno, M. and Kawato, M. (2006). Different Neural Correlates of Reward Expectation and Reward Expectation Error in the Putamen and Caudate Nucleus During Stimulus-Action-Reward Association Learning. *Journal of Neurophysiology*, 95(2):948–959.
- Hay, N., Russell, S., Tolpin, D., and Shimony, S. E. (2012). Selecting Computations: Theory and Applications.
- Hay, N., Russell, S., Tolpin, D., and Shimony, S. E. (2014). Selecting Computations: Theory and Applications.
- Hay, N. and Russell, S. J. (2011). Metareasoning for monte carlo tree search. Technical report, Technical Report UCB/EECS-2011-119, EECS Department, University of
- Hertwig, R. and Erev, I. (2009). The description–experience gap in risky choice. *Trends in Cognitive Sciences*, 13(12):517–523.
- Hertwig, R., Leuker, C., Pachur, T., Spiliopoulos, L., and Pleskac, T. J. (2022). Studies in Ecological Rationality. *Topics in Cognitive Science*, 14(3):467–491.
- Ho, M. K., Abel, D., Cohen, J. D., Littman, M. L., and Griffiths, T. L. (2020). The Efficiency of Human Cognition Reflects Planned Information Processing.
- Ho, M. K., Abel, D., Griffiths, T. L., and Littman, M. L. (2019). The value of abstraction. *Current Opinion in Behavioral Sciences*, 29:111–116.
- Hobbes, T. (1651). *Leviathan*.
- Holding, D. H. and Reynolds, R. I. (1982). Recall or evaluation of chess positions as determinants of chess skill. *Memory & Cognition*, 10(3):237–242.

- Huang, J., Velarde, I., Ma, W. J., and Baldassano, C. (2023). Schema-based predictive eye movements support sequential memory encoding. *eLife*, 12:e82599.
- Huys, Q., Vogelstein, J., and Dayan, P. (2008). Psychiatry: Insights into depression through normative decision-making models. In *Advances in Neural Information Processing Systems*, volume 21. Curran Associates, Inc.
- Huys, Q. J. M., Eshel, N., O’Nions, E., Sheridan, L., Dayan, P., and Roiser, J. P. (2012). Bonsai Trees in Your Head: How the Pavlovian System Sculpts Goal-Directed Choices by Pruning Decision Trees. *PLOS Computational Biology*, 8(3):e1002410.
- Ierapetritou, M. G., Pistikopoulos, E. N., and Floudas, C. A. (1996). Operational planning under uncertainty. *Computers & Chemical Engineering*, 20(12):1499–1516.
- Ito, M. and Doya, K. (2009). Validation of decision-making models and analysis of decision variables in the rat basal ganglia. *Journal of Neuroscience*, 29(31):9861–9874.
- Jensen, K. T., Hennequin, G., and Mattar, M. G. (2024). A recurrent network model of planning explains hippocampal replay and human behavior. *Nature Neuroscience*, 27(7):1340–1348.
- Johnson, A. and Redish, A. D. (2007). Neural Ensembles in CA3 Transiently Encode Paths Forward of the Animal at a Decision Point. *Journal of Neuroscience*, 27(45):12176–12189.
- Jolly, E. (2018). Pymer4: Connecting R and Python for Linear Mixed Modeling. *Journal of Open Source Software*, 3(31):862.
- Kadvany, J. (2016). Pāṇini’s Grammar and Modern Computation. *History and Philosophy of Logic*, 37(4):325–346.
- Kahneman, D. (2011). *Thinking, Fast and Slow*. macmillan.
- Kahneman, D. and Tversky, A. (1972). Subjective probability: A judgment of representativeness. *Cognitive psychology*, 3(3):430–454.

- Kahneman, D. and Tversky, A. (1979). Prospect Theory: An Analysis of Decision under Risk. *Econometrica*, 47(2):263–291.
- Kahneman, D. and Tversky, A. (1984). Choices, values, and frames. *American Psychologist*, 39(4):341–350.
- Khetarpal, K., Ahmed, Z., Comanici, G., Abel, D., and Precup, D. (2020). What can I do here? A Theory of Affordances in Reinforcement Learning. In *Proceedings of the 37th International Conference on Machine Learning*, pages 5243–5253. PMLR.
- Kim, S., Hwang, J., and Lee, D. (2008). Prefrontal coding of temporally discounted values during intertemporal choice. *Neuron*, 59(1):161–172.
- Kim, S. and Lee, D. (2011). Prefrontal cortex and impulsive decision making. *Biological psychiatry*, 69(12):1140–1146.
- Kingma, D. P. and Ba, J. (2017). Adam: A Method for Stochastic Optimization.
- Knuth, D. E. and Moore, R. W. (1975). An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4):293–326.
- Kocsis, L. and Szepesvári, C. (2006). Bandit Based Monte-Carlo Planning. In Fürnkranz, J., Schaffer, T., and Spiliopoulou, M., editors, *Machine Learning: ECML 2006*, pages 282–293, Berlin, Heidelberg. Springer.
- Kokkinos, F. and Potamianos, A. (2017). Structural Attention Neural Networks for improved sentiment analysis.
- Kool, W., Cushman, F. A., and Gershman, S. J. (2016). When Does Model-Based Control Pay Off? *PLOS Computational Biology*, 12(8):e1005090.
- Krajbich, I., Armel, C., and Rangel, A. (2010). Visual fixations and the computation and comparison of value in simple choice. *Nature Neuroscience*, 13(10):1292–1298.

- Kriegeskorte, N., Mur, M., and Bandettini, P. (2008). Representational similarity analysis - connecting the branches of systems neuroscience. *Frontiers in Systems Neuroscience*, 2.
- Kuperwajs, I., Ho, M. K., and Ma, W. J. (2024). Heuristics for meta-planning from a normative model of information search.
- Lai, L. and Gershman, S. J. (2021). Chapter Five - Policy compression: An information bottleneck in action selection. In Federmeier, K. D., editor, *Psychology of Learning and Motivation*, volume 74 of *The Psychology of Learning and Motivation*, pages 195–232. Academic Press.
- Lai, L. and Gershman, S. J. (2024). Human decision making balances reward maximization and policy compression. *PLOS Computational Biology*, 20(4):e1012057.
- Lau, B. and Glimcher, P. W. (2008). Value representations in the primate striatum during matching behavior. *Neuron*, 58(3):451–463.
- Lebiere, C. and Anderson, J. R. (2011). Cognitive Constraints on Decision Making under Uncertainty. *Frontiers in Psychology*, 2.
- Lee, C. Y. (1961). An Algorithm for Path Connections and Its Applications. *IRE Transactions on Electronic Computers*, EC-10(3):346–365.
- Lee, S. W., Shimojo, S., and O’Doherty, J. P. (2014). Neural computations underlying arbitration between model-based and model-free learning. *Neuron*, 81(3):687–699.
- Leibniz, G. W. (1679). De Progressio Dyadica (On the Binary Progression).
- Leibniz, G. W. (1703). Explication de l’Arithmétique Binaire, qui se sert des seuls caractères 0 et 1, avec des remarques sur son utilité, et sur ce qu’elle donne le sens des anciennes figures chinoises de Fohy. *Histoire de l’Académie Royale des Sciences*, pages 85–89.
- Leon, M. I. and Shadlen, M. N. (1999). Effect of expected reward magnitude on the response of neurons in the dorsolateral prefrontal cortex of the macaque. *Neuron*, 24(2):415–425.

- Levenstein, S., Prantera, C., Varvo, V., Scribano, M. L., Berto, E., Luzi, C., and Andreoli, A. (1993). Development of the perceived stress questionnaire: A new tool for psychosomatic research. *Journal of Psychosomatic Research*, 37(1):19–32.
- Lewis, R. L., Howes, A., and Singh, S. (2014). Computational rationality: Linking mechanism and behavior through bounded utility maximization. *Topics in Cognitive Science*, 6(2):279–311.
- Lieder, F., Callaway, F., and Griffiths, T. (2025). The Rational Use of Cognitive Resources.
- Lieder, F., Goodman, N., and Huys, Q. (2013). Lieder, F., and Goodman, N. D., and Huys, Q. J. M. (2013). Controllability and resource-rational planning. Cosyne Abstracts 2013, Salt Lake City USA.
- Lieder, F. and Griffiths, T. L. (2020). Resource-rational analysis: Understanding human cognition as the optimal use of limited computational resources. *Behavioral and Brain Sciences*, 43:e1.
- Lin, X. D. and Ma, W. J. (2024). The Cognitive Components of Complex Planning. *Proceedings of the Annual Meeting of the Cognitive Science Society*, 46(0).
- Linderman, S. W., Miller, A. C., Adams, R. P., Blei, D. M., Paninski, L., and Johnson, M. J. (2016). Recurrent switching linear dynamical systems.
- Llull, R. (1305). *Ars magna*.
- Llull, R. (1984). *Selected Works of Ramón Llull (1232-1316)*. Princeton University Press.
- Lopes, L. L. (1983). Some thoughts on the psychological concept of risk. *Journal of Experimental Psychology: Human Perception and Performance*, 9(1):137–144.
- Loued-Khenissi, L. and Corradi-Dell’Acqua, C. (2024). Gambling on others’ health: Risky pro-social decision-making in the era of COVID-19. *Frontiers in Psychology*, 15.
- Lovelace, A. A. (1843). Notes by the Translator to Sketch of the Analytical Engine Invented by Charles Babbage, by L.F. Menabrea. *Scientific Memoirs*, 3:666–731.
- Lowerre, B. T. (1976). *The Harpy Speech Recognition System*. Carnegie Mellon University.

- Ma, I., Ma, W. J., and Gureckis, T. M. (2021). Information sampling for contingency planning. *Proceedings of the Annual Meeting of the Cognitive Science Society*, 43(43).
- MacDonald, C. J., Lepage, K. Q., Eden, U. T., and Eichenbaum, H. (2011). Hippocampal "time cells" bridge the gap in memory for discontinuous events. *Neuron*, 71(4):737–749.
- Maitre, M.-J. (2022). The development of binary arithmetic by Leibniz: Influence or independence regarding the Xiantiantu of Shao Yong. *Journal of East-West Thought*, 12(2):17–37.
- Manoury, A., Nguyen, S. M., and Buche, C. (2019a). CHIME: An Adaptive Hierarchical Representation for Continuous Intrinsically Motivated Exploration. In *2019 Third IEEE International Conference on Robotic Computing (IRC)*, pages 167–170.
- Manoury, A., Nguyen, S. M., and Buche, C. (2019b). Hierarchical Affordance Discovery using Intrinsic Motivation. In *Proceedings of the 7th International Conference on Human-Agent Interaction*, pages 186–193.
- Manvi, R., Singh, A., and Ermon, S. (2024). Adaptive Inference-Time Compute: LLMs Can Predict if They Can Do Better, Even Mid-Generation.
- Marr, D. (2010). *Vision: A Computational Investigation into the Human Representation and Processing of Visual Information*. MIT Press, Cambridge, MA, USA.
- Mathys, C., Daunizeau, J., Friston, K. J., and Stephan, K. E. (2011). A Bayesian Foundation for Individual Learning Under Uncertainty. *Frontiers in Human Neuroscience*, 5.
- Matsumoto, K., Suzuki, W., and Tanaka, K. (2003). Neuronal correlates of goal-based motor selection in the prefrontal cortex. *Science*, 301(5630):229–232.
- Mattar, M. G. and Daw, N. D. (2018). Prioritized memory access explains planning and hippocampal replay. *Nature Neuroscience*, 21(11):1609–1617.

- McCarthy, J., Minsky, M. L., Rochester, N., and Shannon, C. E. (2006). A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence: August 31, 1955. *AI Magazine*, 27(4):12–14.
- McCulloch, W. S. et al. (1974). Recollections of the many sources of cybernetics. In *ASC Forum*, volume 6, pages 5–16.
- McIlroy-Young, R., Sen, S., Kleinberg, J., and Anderson, A. (2020). Aligning Superhuman AI with Human Behavior: Chess as a Model System. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1677–1687.
- McIlroy-Young, R., Wang, R., Sen, S., Kleinberg, J., and Anderson, A. (2022). Learning Models of Individual Behavior in Chess. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 1253–1263.
- Mi, Q. and Summerfield, C. (2026). Human curriculum learning of a cue combination task. *Nature Human Behaviour*, pages 1–18.
- Miller, G. A. and Chomsky, N. (1963). Finitary Models of Language Users. In Luce, R. D., Bush, R. R., and Galanter, E., editors, *Handbook of Mathematical Psychology*, volume 2, pages 419–491. John Wiley & Sons, New York.
- Mitra, A., Srivastava, N., and Srinivasan, N. (2023). Unpredictability shortens planning horizons. *Proceedings of the Annual Meeting of the Cognitive Science Society*, 45(45).
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). Playing Atari with Deep Reinforcement Learning.
- Momennejad, I., Russek, E. M., Cheong, J. H., Botvinick, M. M., Daw, N. D., and Gershman, S. J. (2017). The successor representation in human reinforcement learning. *Nature Human Behaviour*, 1(9):680–692.

- Montague, P., Dayan, P., and Sejnowski, T. (1996). A framework for mesencephalic dopamine systems based on predictive Hebbian learning. *The Journal of Neuroscience*, 16(5):1936–1947.
- Moore, E. F. (1959). The shortest path through a maze. In *Proc. of the International Symposium on the Theory of Switching*, pages 285–292. Harvard University Press.
- Munakata, Y., Placido, D., and Zhuang, W. (2023). What’s Next? Advances and Challenges in Understanding How Environmental Predictability Shapes the Development of Cognitive Control. *Current Directions in Psychological Science*, 32(6):431–438.
- Muppidi, A., Darwish, F., Cope, D., Henriques, J. F., and Foerster, J. N. (2026). Finding the Time to Think: Learning Planning Budgets in Real-Time RL.
- Newell, A., Shaw, J. C., and Simon, H. A. (1959). Report on a general problem solving program. In *IFIP Congress*, volume 256, page 1959. Pittsburgh, PA.
- Newell, A. and Simon, H. (1956). The logic theory machine—A complex information processing system. *IRE Transactions on Information Theory*, 2(3):61–79.
- Newell, A. and Simon, H. A. (1976). Computer science as empirical inquiry: Symbols and search. *Communications of the ACM*, 19(3):113–126.
- Nica, A., Khetarpal, K., and Precup, D. (2022). The Paradox of Choice: Using Attention in Hierarchical Reinforcement Learning.
- Niki, H. and Watanabe, M. (1979). Prefrontal and cingulate unit activity during timing behavior in the monkey. *Brain Research*, 171(2):213–224.
- Niv, Y. (2019). Learning task-state representations. *Nature Neuroscience*, 22(10):1544–1553.
- O’Doherty, J., Dayan, P., Schultz, J., Deichmann, R., Friston, K., and Dolan, R. J. (2004). Dissociable roles of ventral and dorsal striatum in instrumental conditioning. *Science (New York, N.Y.)*, 304(5669):452–454.

- O’Keefe, J. and Nadel, L. (1978). *The Hippocampus as a Cognitive Map*. Oxford university press.
- O’Reilly, R. C. and Frank, M. J. (2006). Making Working Memory Work: A Computational Model of Learning in the Prefrontal Cortex and Basal Ganglia. *Neural Computation*, 18(2):283–328.
- Ortega, P. A. and Braun, D. A. (2013). Thermodynamics as a theory of decision-making with information-processing costs. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 469(2153):20120683.
- Owen, A. M. (1997). Cognitive planning in humans: Neuropsychological, neuroanatomical and neuropharmacological perspectives. *Progress in Neurobiology*, 53(4):431–450.
- Owen, A. M., Downes, J. J., Sahakian, B. J., Polkey, C. E., and Robbins, T. W. (1990). Planning and spatial working memory following frontal lobe lesions in man. *Neuropsychologia*, 28(10):1021–1034.
- Owen, A. M., Sahakian, B. J., Hodges, J. R., Summers, B. A., Polkey, C. E., and Robbins, T. W. (1995). Dopamine-dependent frontostriatal planning deficits in early Parkinson’s disease. *Neuropsychology*, 9(1):126–140.
- Pāṇini (1980). *The Ashtādhyāyī of Pāṇini*. Motilal Banarsidass Publishing House.
- Pascanu, R., Li, Y., Vinyals, O., Heess, N., Buesing, L., Racanière, S., Reichert, D., Weber, T., Wierstra, D., and Battaglia, P. (2017). Learning model-based planning from scratch.
- Pavlov, I. P. (1906). The Scientific Investigation of the Psychical Faculties or Processes in the Higher Animals. *Science*, 24(620):613–619.
- Pearl, J. (1982). Reverend bayes on inference engines: A distributed hierarchical approach. In *Proceedings of the Second AAAI Conference on Artificial Intelligence*, AAAI’82, pages 133–136, Pittsburgh, Pennsylvania. AAAI Press.
- Pearl, J. (1983). Heuristics: Intelligent search strategies for computer problem solving.

- Penfield, W. and Evans, J. P. (1935). The frontal lobe in man: A clinical study of maximum removals. *Brain: A Journal of Neurology*, 58:115–133.
- Perkins, F. (2004). *Leibniz and China: A Commerce of Light*. Cambridge University Press.
- Pfeiffer, B. E. (2020). The content of hippocampal ”replay”. *Hippocampus*, 30(1):6–18.
- Pfeiffer, B. E. and Foster, D. J. (2013). Hippocampal place-cell sequences depict future paths to remembered goals. *Nature*, 497(7447):74–79.
- Piray, P. and Daw, N. D. (2020). A simple model for learning in volatile environments. *PLOS Computational Biology*, 16(7):e1007963.
- Piray, P. and Daw, N. D. (2021). A model for learning based on the joint estimation of stochasticity and volatility. *Nature Communications*, 12(1):6587.
- Redish, A. D. (2016). Vicarious trial and error. *Nature Reviews Neuroscience*, 17(3):147–159.
- Rigoux, L., Stephan, K. E., Friston, K. J., and Daunizeau, J. (2014). Bayesian model selection for group studies - revisited. *NeuroImage*, 84:971–985.
- Rosin, C. D. (2011). Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence*, 61(3):203–230.
- Ruoss, A., Delétang, G., Medapati, S., Grau-Moya, J., Wenliang, L. K., Catt, E., Reid, J., and Gnewein, T. (2024). Grandmaster-Level Chess Without Search.
- Russek, E., Acosta-Kane, D., van Opheusden, B., Mattar, M. G., and Griffiths, T. (2022). Time spent thinking in online chess reflects the value of computation.
- Russek, E. M., Momennejad, I., Botvinick, M. M., Gershman, S. J., and Daw, N. D. (2017). Predictive representations can link model-based reinforcement learning to model-free mechanisms. *PLoS computational biology*, 13(9):e1005768.

- Russell, S. and Wefald, E. (1989). On optimal game-tree search using rational meta-reasoning. In *IJCAI*, pages 334–340.
- Russell, S. and Wefald, E. (1991). Principles of metareasoning. *Artificial Intelligence*, 49(1-3):361–395.
- Russell, S. J. and Subramanian, D. (1995). Provably Bounded-Optimal Agents.
- Rutledge, R. B., Dean, M., Caplin, A., and Glimcher, P. W. (2010). Testing the Reward Prediction Error Hypothesis with an Axiomatic Model. *The Journal of Neuroscience*, 30(40):13525–13536.
- Saisubramanian, S., Wray, K. H., Pineda, L., and Zilberstein, S. (2019). Planning in Stochastic Environments with Goal Uncertainty. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1649–1654.
- Saisubramanian, S. and Zilbertsein, S. (2019). Adaptive Outcome Selection for Planning with Reduced Models. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1655–1660.
- Samuelson, P. A. (1938). A note on the pure theory of consumer’s behaviour. *Economica*, 5(17):61–71.
- Savage, L. J. (1954). The foundations of statistics Wiley. New York.
- Schmidhuber, J. (2012). Self-Delimiting Neural Networks.
- Schultz, W., Dayan, P., and Montague, P. R. (1997). A Neural Substrate of Prediction and Reward. *Science*, 275(5306):1593–1599.
- Scoville, W. B. and Milner, B. (1957). Loss of Recent Memory after Bilateral Hippocampal Lesions. *Journal of Neurology, Neurosurgery, and Psychiatry*, 20(1):11–21.
- Sedgewick, R. (2001). *Algorithms in C++ Part 5: Graph Algorithms*. Pearson Education.

- Seo, H., Cai, X., Donahue, C. H., and Lee, D. (2014). Neural correlates of strategic reasoning during competitive games. *Science*, 346(6207):340–343.
- Seow, T. X. F., Benoit, E., Dempsey, C., Jennings, M., Maxwell, A., O’Connell, R., and Gillan, C. M. (2021). Model-Based Planning Deficits in Compulsivity Are Linked to Faulty Neural Representations of Task Structure. *Journal of Neuroscience*, 41(30):6539–6550.
- Sezener, C. E., Dezfouli, A., and Keramati, M. (2019). Optimizing the depth and the direction of prospective planning using information values. *PLOS Computational Biology*, 15(3):e1006827.
- Sezener, E. and Dayan, P. (2020). Static and Dynamic Values of Computation in MCTS. In *Proceedings of the 36th Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 31–40. PMLR.
- Shallice, T. (1982). Specific Impairments of Planning. *Philosophical transactions of the Royal Society of London. Series B, Biological sciences*, 298:199–209.
- Shallice, T. and Burgess, P. W. (1991). Deficits in strategy application following frontal lobe damage in man. *Brain: A Journal of Neurology*, 114 (Pt 2):727–741.
- Shannon, C. E. (1950). XXII. Programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314):256–275.
- Shannon, C. E. (1988). A chess-playing machine. In *Computer Games I*, pages 81–88. Springer.
- Sharp, P. B., Dolan, R. J., and Eldar, E. (2023). Disrupted state transition learning as a computational marker of compulsivity. *Psychological Medicine*, 53(5):2095–2105.
- Sharpe, W. F. (1964). Capital Asset Prices: A Theory of Market Equilibrium Under Conditions of Risk. *The Journal of Finance*, 19(3):425–442.
- Sigman, M., Etchemendy, P., Fernandez Slezak, D., and Cecchi, G. A. (2010). Response Time Distributions in Rapid Chess: A Large-Scale Decision Making Experiment. *Frontiers in Neuroscience*, 4.

- Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., and Hassabis, D. (2017a). Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm.
- Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., Chen, Y., Lillicrap, T., Hui, F., Sifre, L., van den Driessche, G., Graepel, T., and Hassabis, D. (2017b). Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359.
- Simon, H. and Chase, W. (1988). Skill in Chess. In Levy, D., editor, *Computer Chess Compendium*, pages 175–188. Springer, New York, NY.
- Simon, H. A. (1955). A Behavioral Model of Rational Choice. *The Quarterly Journal of Economics*, 69(1):99–118.
- Simon, H. A. and Chase, W. G. (1973). Skill in chess. *American Scientist*, 61(4):394–403.
- Simpson, J. A., Griskevicius, V., Kuo, S. I.-C., Sung, S., and Collins, W. A. (2012). Evolution, stress, and sensitive periods: The influence of unpredictability in early versus late childhood on sex and risky behavior. *Developmental Psychology*, 48(3):674–686.
- Singer, A. C. and Frank, L. M. (2009). Rewarded Outcomes Enhance Reactivation of Experience in the Hippocampus. *Neuron*, 64(6):910–921.
- Skinner, B. F. (1963). Operant behavior. *American Psychologist*, 18(8):503–515.
- Snell, C., Lee, J., Xu, K., and Kumar, A. (2024). Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters.
- Snider, J., Lee, D., Poizner, H., and Gepshtein, S. (2015). Prospective Optimization with Limited Resources. *PLOS Computational Biology*, 11(9):e1004501.
- Stachenfeld, K. L., Botvinick, M. M., and Gershman, S. J. (2017). The hippocampus as a predictive map. *Nature Neuroscience*, 20(11):1643–1653.

- Steinerberger, S. (2015). On the number of positions in chess without promotion. *International Journal of Game Theory*, 44(3):761–767.
- Story, G. W., Kurth-Nelson, Z., Moutoussis, M., Iigaya, K., Will, G. J., Hauser, T. U., Blain, B., Vlaev, I., and Dolan, R. J. (2023). Discounting future reward in an uncertain world. *Decision*.
- Sussillo, D., Jozefowicz, R., Abbott, L. F., and Pandarinath, C. (2016). LFADS - Latent Factor Analysis via Dynamical Systems.
- Sutton, R. S. and Barto, A. G. (1998). *Reinforcement Learning: An Introduction*. Adaptive Computation and Machine Learning. MIT Press, Cambridge, Mass.
- Tai, K. S., Socher, R., and Manning, C. D. (2015). Improved Semantic Representations From Tree-Structured Long Short-Term Memory Networks.
- Tanaka, S. C., Doya, K., Okada, G., Ueda, K., Okamoto, Y., and Yamawaki, S. (2004). Prediction of immediate and future rewards differentially recruits cortico-basal ganglia loops. *Nature Neuroscience*, 7(8):887–893.
- Thorndike, E. L. (1898). Review of Animal Intelligence: An Experimental Study of the Associative Processes in Animals. *Psychological Review*, 5(5):551.
- Thorndike, E. L. (1911). Animal intelligence: Experimental studies.
- Tian, L. Y., Garzón Gupta, K., Hanuska, D. J., Rouse, A. G., Eldridge, M. A. G., Schieber, M. H., Wang, X.-J., Tenenbaum, J. B., and Freiwald, W. A. (2026). Neural representation of action symbols in primate frontal cortex. *Nature*, 654(8117):152–162.
- Todd, P. M. and Brighton, H. (2016). Building the Theory of Ecological Rationality. *Minds and Machines*, 26(1):9–30.
- Todd, P. M. and Gigerenzer, G. (2007). Environments That Make Us Smart: Ecological Rationality. *Current Directions in Psychological Science*, 16(3):167–171.

- Todd, P. M. and Gigerenzer, G., editors (2012). *Ecological Rationality: Intelligence in the World*. Ecological Rationality: Intelligence in the World. Oxford University Press, New York, NY, US.
- Tolman, E. C. (1948). Cognitive maps in rats and men. *Psychological Review*, 55(4):189–208.
- Tromp, J. and Österlund, P. (2021). Chess position ranking: Software suite for ranking chess positions and estimating the number of legal chess positions.
- Tucker, M. and Ellis, R. (1998). On the relations between seen objects and components of potential actions. *Journal of Experimental Psychology: Human Perception and Performance*, 24(3):830–846.
- Turing, A. M. (1936). On Computable Numbers, with an Application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(1):230–265.
- Tversky, A. and Kahneman, D. (1973). Availability: A heuristic for judging frequency and probability. *Cognitive psychology*, 5(2):207–232.
- Tversky, A. and Kahneman, D. (1981). The Framing of Decisions and the Psychology of Choice. *Science*, 211(4481):453–458.
- Tversky, A. and Kahneman, D. (1991). Loss aversion in riskless choice: A reference-dependent model. *The quarterly journal of economics*, 106(4):1039–1061.
- Tversky, A. and Kahneman, D. (1992). Advances in prospect theory: Cumulative representation of uncertainty. *Journal of Risk and Uncertainty*, 5(4):297–323.
- Tversky, A., Kahneman, D., Slovic, P., et al. (1982). *Judgment under Uncertainty: Heuristics and Biases*. Cambridge.
- van Opheusden, B., Acerbi, L., and Ma, W. J. (2020). Unbiased and efficient log-likelihood estimation with inverse binomial sampling. *PLOS Computational Biology*, 16(12):e1008483.
- van Opheusden, B., Galbiati, G., Bnaya, Z., Li, Y., and Ma, W. J. (2017). A computational model for decision tree search. In *CogSci*.

- van Opheusden, B., Kuperwajs, I., Galbiati, G., Bnaya, Z., Li, Y., and Ma, W. J. (2023). Expertise increases planning depth in human gameplay. *Nature*, 618(7967):1000–1005.
- van Opheusden, B. and Ma, W. J. (2019). Tasks for aligning human and machine planning. *Current Opinion in Behavioral Sciences*, 29:127–133.
- van Opheusden, S. (2019). *Complex Decision-Making*. PhD thesis, New York University.
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., Carey, C. J., Polat, İ., Feng, Y., Moore, E. W., VanderPlas, J., Laxalde, D., Perktold, J., Cimrman, R., Henriksen, I., Quintero, E. A., Harris, C. R., Archibald, A. M., Ribeiro, A. H., Pedregosa, F., and van Mulbregt, P. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272.
- Volz, K. G. and Gigerenzer, G. (2012). Cognitive Processes in Decisions Under Risk are not the Same as in Decisions Under Uncertainty. *Frontiers in Neuroscience*, 6.
- von Neumann, J. (1928). Zur theorie der gesellschaftsspiele. *Mathematische annalen*, 100(1):295–320.
- von Neumann, J. and Morgenstern, O. (1947). Theory of games and economic behavior, 2nd rev.
- Wang, J. X., Kurth-Nelson, Z., Kumaran, D., Tirumala, D., Soyer, H., Leibo, J. Z., Hassabis, D., and Botvinick, M. (2018). Prefrontal cortex as a meta-reinforcement learning system. *Nature neuroscience*, 21(6):860–868.
- Wang, K. A., Xia, J., Chung, S., and Greenwald, A. (2025). Dynamic Thinker: Optimizing Decision-Time Planning with Costly Compute. In *The Seventeenth Workshop on Adaptive and Learning Agents*.

- Wiener, J. M., Lafon, M., and Berthoz, A. (2008). Path planning under spatial uncertainty. *Memory & Cognition*, 36(3):495–504.
- Wilhelm, B. (2011). *The I Ching or Book of Changes*. Princeton University Press.
- Wilkinson, M. R., Ball, L. J., and Alford, D. (2015). Counterfactual Reasoning for Regretted Situations Involving Controllable Versus Uncontrollable Events: The Modulating Role of Contingent Self-Esteem. *Advances in Cognitive Psychology*, 11(1):22–30.
- Wilkinson, M. R., Ball, L. J., and Cooper, R. (2010). Arbitrating Between Theory-Theory and Simulation Theory: Evidence from a Think-Aloud Study of Counterfactual Reasoning. *Conference: In S Ohlsson and R Catrambone (Eds.) Proceedings of the Thirty Second Annual Conference of the Cognitive Science Society , Austin, TX: Cognitive Science Society*.
- Wilson, M. A. and McNaughton, B. L. (1994). Reactivation of hippocampal ensemble memories during sleep. *Science*, 265(5172):676–679.
- Wise, T., Emery, K., and Radulescu, A. (2023). Naturalistic reinforcement learning. *Trends in Cognitive Sciences*, page S1364661323002127.
- Woodruff, S. C. (2016). Planning for an unknowable future: Uncertainty in climate change adaptation planning. *Climatic Change*, 139(3):445–459.
- Wunderlich, K., Dayan, P., and Dolan, R. J. (2012). Mapping value based planning and extensively trained choice in the human brain. *Nature Neuroscience*, 15(5):786–791.
- Xu, Y., Harms, M. B., Green, C. S., Wilson, R. C., and Pollak, S. D. (2023). Childhood unpredictability and the development of exploration. *Proceedings of the National Academy of Sciences*, 120(49):e2303869120.
- Yang, Q., Lin, Z., Zhang, W., Li, J., Chen, X., Zhang, J., and Yang, T. (2022). Monkey plays Pac-Man with compositional strategies and hierarchical decision-making. *eLife*, 11:e74500.

- Yin, H. H., Ostlund, S. B., Knowlton, B. J., and Balleine, B. W. (2005). The role of the dorsomedial striatum in instrumental conditioning. *The European Journal of Neuroscience*, 22(2):513–523.
- Yoon, S., Fern, A., and Givan, R. (2007). FF-Replan: A Baseline for Probabilistic Planning. In *International Conference on Automated Planning and Scheduling*.
- Zandvoort, M., Van der Vlist, M. J., Klijn, F., and Van den Brink, A. (2018). Navigating amid uncertainty in spatial planning. *Planning Theory*, 17(1):96–116.
- Zhang, Y., Jacob, A. P., Lai, V., Fried, D., and Ippolito, D. (2024). Human-aligned Chess with a Bit of Search.
- Zhu, C., Byrd, R. H., Lu, P., and Nocedal, J. (1997). Algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound-constrained optimization. *ACM Trans. Math. Softw.*, 23(4):550–560.
- Zylberberg, A., Fetsch, C. R., and Shadlen, M. N. (2016). The influence of evidence volatility on choice, reaction time and confidence in a perceptual decision. *eLife*, 5:e17688.